

算法板子

竞赛代码模板完整大版本

完整覆盖 • 专题详解 • 统一目录

2026 年 9 月 2 日

目录

第一章 总览、复杂度与学习路线	13
1.1 先判断题目属于哪一类	13
1.2 复杂度速查（单次操作约 1 秒时的经验值）	13
1.3 分层路线	13
1.3.1 L0: 实现基本功	13
1.3.2 L1: 线性与排序	14
1.3.3 L2: 基础图和数学	14
1.3.4 L3: 基础 DP 和经典数据结构	14
1.3.5 L4: 字符串、树和提升题	14
1.3.6 L5: 银牌常见进阶	14
1.4 赛场决策树	14
1.5 正确性证明的四种常用模板	14
1.6 提交前检查	15
第二章 竞赛模板	16
2.1 C++ 竞赛实现规范	17
第三章 STL 常用函数速查	18
3.1 容器	18
3.1.1 vector	18
3.1.2 array	18
3.1.3 string	19
3.1.4 set / multiset	19
3.1.5 map	19
3.1.6 unordered_map / unordered_set	20

3.1.7	priority_queue	21
3.1.8	deque	21
3.1.9	stack	21
3.1.10	list	21
3.2	常用算法	22
3.3	pair / tuple	23
3.4	bitset	23
3.4.1	bitset 优化背包 / 子集和可行性	25
3.4.2	bitset 优化传递闭包 (Floyd)	25
3.4.3	bitset 优化字符串匹配 (Shift-And)	26
3.5	__int128	26
3.6	__builtin 系列	27
3.7	C++ 常用函数速查	28
3.7.1	数值函数	28
3.7.2	区间操作函数	29
3.7.3	排列与组合	30
3.7.4	全排列函数详解 (next_permutation / prev_permutation)	31
3.7.5	排列序 (康托展开与逆康托展开)	35
3.7.6	字符串处理函数	36
3.7.7	其他实用工具	38

第四章 位运算 40

4.1	基本性质	40
4.1.1	分配律与吸收律	40
4.1.2	异或的重要性质	40
4.1.3	$O(1)$ 求连续自然数的异或和	41
4.1.4	与/或的单调性	42
4.1.5	常用恒等式速查	42
4.2	竞赛常用性质科普	42
4.2.1	按位独立性 (Bitwise Independence)	42
4.2.2	异或与线性代数	43

4.2.3	位运算与集合论的对应	43
4.2.4	popcount 的性质与应用	44
4.2.5	高位/低位操作	44
4.2.6	位运算与不等式	44
4.2.7	SOS DP (子集和 / 超集和)	45
4.3	常用技巧	45
4.4	位运算与异或	46
4.5	线性基 (XOR Basis)	46
4.6	线性基: 参数、调用与改造	47
4.6.1	它解决什么问题	47
4.6.2	可直接使用的模板	47
4.6.3	例题: P3812 线性基	49
4.6.4	常见错误	50
第五章	基础算法	51
5.1	枚举与剪枝	51
5.1.1	子集枚举	51
5.1.2	回溯剪枝	51
5.1.3	折半搜索 (Meet-in-the-middle)	52
5.2	前缀和、差分与扫描	53
5.2.1	一维前缀和	53
5.2.2	一维差分	53
5.2.3	扫描线思想	54
5.3	排序、二分、双指针	54
5.3.1	二分查找	54
5.3.2	二分答案	55
5.3.3	双指针/滑动窗口	56
5.3.4	三分与凸性	56
5.4	贪心	56
5.4.1	典型证明模式	57
5.4.2	反悔贪心	57

5.4.3	差分约束式贪心	57
5.5	单调栈与单调队列	58
5.5.1	单调栈	58
5.5.2	单调队列	59
5.6	离散化与坐标压缩	59
5.7	常见构造套路	59
5.8	随机化与哈希技巧	60
5.9	位集优化	60
5.10	快速小技巧	60
第六章	数学	62
6.1	模运算与快速幂	62
6.2	最大公约数、扩展欧几里得与同余	62
6.3	质数与筛法	63
6.3.1	试除与质因数分解	63
6.3.2	埃氏筛与线性筛	63
6.3.3	约数枚举与整除分块	64
6.4	欧拉函数、莫比乌斯与积性函数	64
6.5	组合数与阶乘 (Binomial Coefficients)	65
6.5.1	常用推式性质	65
6.5.2	质数模且 n 不大	67
6.5.3	卢卡斯定理 (Lucas Theorem)	67
6.5.4	非质数模	68
6.6	矩阵快速幂与线性递推 (Matrix Exponentiation and Linear Recurrence)	68
6.7	容斥与组合计数	69
6.8	概率与期望	70
6.9	常见数列与技巧	70
6.10	生成函数基础	71
6.11	Burnside 引理与 Pólya 计数定理 (Burnside's Lemma and Pólya Enumeration Theorem)	72
6.12	Prüfer 序列与矩阵树定理 (Prüfer Sequence and Matrix-Tree Theorem)	73

6.13 二次剩余与 Tonelli - Shanks 算法 (Quadratic Residues and Tonelli - Shanks Algorithm)	75
6.14 置换与循环	77
6.15 快速傅里叶变换与数论变换 (了解)	77
6.16 高斯消元	78
6.17 任意精度整数 (纯 C++17)	79
6.18 数学与组合计数: 参数、调用与改造	88
6.18.1 高级容斥与倍数容斥	88
6.18.2 卢卡斯定理 (Lucas Theorem) 与组合数模质数	89
6.18.3 中国剩余定理与扩展中国剩余定理	91
6.18.4 高斯消元、秩、解空间与行列式	93
6.18.5 莫比乌斯反演	95
6.18.6 原根、Baby-step Giant-step 离散对数与扩展算法	96
6.18.7 生成函数、数论变换、Burnside 引理与 Pólya 计数定理	98
6.18.8 二次剩余	101
6.18.9 Prüfer 序列与矩阵树	102
6.18.10 矩阵快速幂与线性递推	103
6.18.11 高斯消元求实数方程组	104

第七章 数据结构 107

7.1 STL 必会组件	107
7.2 并查集 (Disjoint Set Union, DSU)	108
7.3 树状数组 (Fenwick Tree)	108
7.4 线段树	109
7.4.1 常见维护	109
7.4.2 线段树上的二分	110
7.5 ST 表 (Sparse Table, 静态区间查询)	111
7.6 堆与可并堆思想	112
7.7 单调结构与差分约束	112
7.8 字典树 (Trie, Prefix Tree)	113
7.9 可持久化数据结构	114
7.10 分块与莫队算法 (Mo's Algorithm)	115

7.10.1 块状数组	115
7.10.2 莫队	116
7.11 平衡树思想（了解 Treap）	117
7.12 撤销与可回滚结构	117
7.13 李超线段树（Li Chao Segment Tree）	118
7.14 区间最值势能线段树（Segment Tree Beats）	120
7.15 高级数据结构：参数、调用与改造	123
7.15.1 高阶前缀和与多阶差分	123
7.15.2 多懒标记与仿射标记线段树	124
7.15.3 线段树上二分	126
7.15.4 莫队、带修改莫队与回滚莫队	127
7.15.5 三维偏序的 CDQ 分治与动态规划优化	129
7.15.6 可持久化字典树（Persistent Trie）	132
7.15.7 李超线段树：整条直线与线段插入	134
7.15.8 区间最值势能线段树（Segment Tree Beats）	136
7.15.9 可持久化并查集	139

第八章 字符串算法 142

8.1 选择算法	142
8.2 Knuth–Morris–Pratt 字符串匹配算法（KMP）	142
8.3 Z 函数	143
8.4 字符串哈希	144
8.5 Manacher	144
8.6 字典树与 Aho–Corasick 多模式匹配自动机（AC 自动机）	145
8.6.1 普通字符串字典树（Trie）	145
8.6.2 Aho–Corasick 多模式匹配自动机	146
8.7 后缀数组与最长公共前缀（Suffix Array and Longest Common Prefix, LCP）	148
8.8 后缀自动机（Suffix Automaton, SAM）	149
8.9 回文自动机（Palindromic Automaton, PAM, 了解）	150
8.10 字符串常见组合	151
8.11 字符串高级算法：参数、调用与改造	152

8.11.1 Booth 最小表示法	152
8.11.2 回文自动机 (Palindromic Automaton)	153
8.11.3 后缀数组、最长公共前缀、后缀自动机与 Aho–Corasick 失配树	156
8.11.4 后缀数组与 LCP	158
8.11.5 AC 自动机与 fail 树	160

第九章 图论 163

9.1 图的表示与基础遍历	163
9.1.1 多源 BFS	163
9.2 有向无环图 (Directed Acyclic Graph, DAG) 与拓扑排序	164
9.3 最短路	165
9.4 最小生成树	166
9.4.1 Kruskal	166
9.4.2 Prim	167
9.4.3 Kruskal 重构树	168
9.5 强连通分量 (Strongly Connected Components, SCC) 与缩点	171
9.6 割点、桥、双连通	172
9.7 二分图	172
9.7.1 染色判定	172
9.7.2 最大匹配	173
9.7.3 带权匹配	174
9.8 网络流	174
9.8.1 建模	174
9.8.2 Dinic	174
9.8.3 费用流	175
9.9 树上算法	177
9.9.1 DFS 序与子树	177
9.9.2 最近公共祖先 (Lowest Common Ancestor, LCA) 倍增	177
9.9.3 树上差分	178
9.9.4 直径与重心	179
9.9.5 树形 DP	179

9.9.6 重链剖分 (Heavy-Light Decomposition, HLD)	180
9.9.7 点分治 (入门)	180
9.9.8 树上启发式合并 (Disjoint Set Union on Tree, DSU on Tree)	181
9.9.9 长链剖分	182
9.9.10 Link-Cut Tree 动态树 (LCT)	183
9.10 欧拉路、欧拉回路与哈密顿回路	186
9.11 图论组合套路	187
第十章 树上算法与高级结构	188
10.1 树上高级算法: 参数、调用与改造	188
10.1.1 树上启发式合并 (Disjoint Set Union on Tree)	188
10.1.2 长链剖分	190
10.1.3 点分治与点分树	192
10.1.4 伸展树、文艺平衡树与 Link-Cut Tree 动态树	193
第十一章 动态规划	196
11.1 DP 四问法	196
11.2 线性 DP	196
11.3 背包	197
11.4 区间 DP	198
11.5 树形 DP 与换根 DP	198
11.6 有向无环图动态规划与路径计数 (Directed Acyclic Graph Dynamic Programming)	199
11.7 状态压缩动态规划	199
11.8 数位动态规划 (Digit Dynamic Programming)	200
11.9 概率动态规划与期望动态规划 (Probability and Expected-Value Dynamic Programming)	201
11.10 记忆化搜索	201
11.11 动态规划优化 (Dynamic Programming Optimization)	201
11.11.1 滚动数组	202
11.11.2 单调队列优化	202
11.11.3 斜率优化 (Convex Hull Trick, CHT)	203

11.11.4 分治优化	203
11.12 组合游戏动态规划 (Combinatorial Game Dynamic Programming)	204
11.13 常见 DP 误区	204
11.14 动态规划进阶: 参数、调用与改造	204
11.14.1 复杂数位动态规划与自动机数位动态规划	205
11.14.2 所有子集和动态规划、轮廓线动态规划与状态压缩	206
11.14.3 单调队列、斜率优化与李超树优化动态规划	207
11.14.4 分治优化、四边形不等式、树形背包与 Steiner 树状压动态规划	209
11.14.5 动态动态规划与矩阵线段树	210
11.14.6 概率动态规划的自环方程	211

第十二章 博弈专题 213

12.1 Nim、Bash 与反常 Nim 取石子游戏	213
12.2 通用 Sprague-Grundy 函数、最小未出现值与有向无环图博弈	214
12.3 Nim 变体: 取法集合与 Sprague-Grundy 周期	215
12.4 极大极小搜索 (Minimax) 与 Alpha-Beta 剪枝	216
12.5 博弈题检查清单	217
12.6 博弈论: 参数、调用与改造	217
12.6.1 必胜态/必败态动态规划 (Win/Lose Dynamic Programming)	217
12.6.2 有向无环图上的 Sprague-Grundy 函数与最小未出现值	219
12.6.3 多个独立子游戏为什么异或	221
12.6.4 普通 Nim: 不只判断, 还要恢复必胜操作	221
12.6.5 Bash、反常 Nim 与阶梯 Nim	222
12.6.6 Sprague-Grundy 函数打表、周期与大范围取石子	225
12.6.7 图游戏建模、状压博弈和记忆化	226
12.6.8 极大极小搜索 (Minimax) 与 Alpha-Beta 剪枝	228
12.6.9 状态去重、哈希与有环博弈	229

第十三章 计算几何与计算技巧 231

13.1 浮点规范	231
13.2 向量与叉积	232
13.3 直线、线段与交点	232

13.3.1 圆几何	233
13.4 多边形	234
13.5 凸包 (Andrew)	234
13.6 旋转卡壳	235
13.7 扫描线	236
13.8 最近点对	237
13.9 半平面交 (Half-Plane Intersection, HPI, 了解)	238
13.10 几何中的随机化与哈希技巧	240
13.11 几何中的位集优化	240
13.12 几何快速小技巧	240
13.13 计算几何高级算法: 参数、调用与改造	241
13.13.1 极角排序与凸包	242
13.13.2 旋转卡壳、凸多边形内点与最近点对	242
13.13.3 半平面交 (Half-Plane Intersection)	244
13.13.4 圆几何与扫描线	244
13.13.5 圆与直线、最小圆覆盖	246
13.13.6 扫描线求矩形并面积	248
13.14 超过十行模板的针对性自测	250
13.14.1 基础、枚举、差分与二分	251
13.14.2 数学、筛法、生成函数与树计数	252
13.14.3 数据结构: 下标、懒标记、版本与回滚	254
13.14.4 图论: 路径、连通性、流和树上边界	256
13.14.5 动态规划和字符串	259
13.14.6 进阶专题: 整体二分、时间分治、可满足性与匹配	260

第十四章 完整例题、接口注释与可执行测试 262

14.1 统一卡片接口注释	262
14.2 测试矩阵与正式题例	262
14.3 基础与数学: 完整小题例	263
14.3.1 子集和、区间加与二分答案	263
14.3.2 树状数组、线段树与回滚并查集	264

14.3.3 高精度整数四则运算	265
14.4 图论与树：完整小题例	265
14.4.1 最短路、拓扑与最大流	265
14.4.2 SCC、桥、匹配与 LCA	266
14.5 字符串、动态规划与几何：完整小题例	266
14.6 一键验收	266
第十五章 对拍	267
15.1 对拍脚本	267
15.1.1 Linux / macOS (Bash)	267
15.1.2 Windows (BAT)	268
15.2 随机数据生成器 (gen.cpp)	269
15.3 暴力解法 (brute.cpp)	272
15.4 交互题对拍	273
第十六章 竞赛环境速查	275
16.1 Windows 命令行 (CMD / PowerShell)	275
16.1.1 编译与运行	275
16.1.2 文件操作	276
16.1.3 进程与调试	276
16.1.4 常用技巧	276
16.2 VS Code 快捷键速查	277
16.2.1 文件与编辑	277
16.2.2 查找与替换	278
16.2.3 多光标与选择	278
16.2.4 终端	278
16.2.5 调试	278
16.2.6 其他实用操作	279
16.3 VS Code 竞赛配置	279
16.3.1 一键编译运行 (tasks.json)	279
16.3.2 调试配置 (launch.json)	280
16.3.3 推荐插件	280

16.3.4 推荐用户设置 (settings.json 片段) 280

第十七章 题型/关键词 → 算法索引 282

17.1 一、博弈论 282

17.2 二、高级数据结构 283

17.3 三、数学与组合计数 285

17.4 四、字符串算法 288

17.5 五、计算几何 289

17.6 六、动态规划进阶 290

17.7 七、按题面快速查找 292

17.8 八、常见算法组合 293

1 总览、复杂度与学习路线

1.1 先判断题目属于哪一类

1. 静态数组/区间：前缀和、差分、排序、双指针、二分、树状数组、线段树。
2. 图/网格可达性：BFS、DFS、连通分量、拓扑、最短路、并查集。
3. 选择最优方案：贪心（交换论证/单调性）或 DP（状态有重叠）。
4. 计数/取模：组合数、容斥、DP、生成式递推、矩阵快速幂。
5. 字符串模式：KMP/Z/哈希；多模式匹配用 AC，回文用 Manacher。
6. 树上问题：先想 DFS 序、子树、倍增、LCA、树形 DP，再考虑重链/点分治。
7. 双方对抗/取石子：先判无偏性与终止规则，再选 Win/Lose、SG 或 Minimax。

1.2 复杂度速查（单次操作约 1 秒时的经验值）

规模	通常可接受	典型算法
$n \leq 20$	$O(2^{nn})$	状压、子集枚举、折半搜索
$n \leq 40$	$O(2^{(n/2)})$	Meet-in-the-middle
$n \leq 200$	$O(n^3)$	Floyd、区间 DP、矩阵转移
$n \leq 2000$	$O(n^2)$	二维 DP、朴素 LIS、部分图算法
$n \leq 2e5$	$O(n \log n)$	排序、堆、树状数组、线段树、Dijkstra
$n \leq 1e6$	$O(n)$ 或 $O(n \log \log n)$	线性扫描、线性筛、哈希
$n \leq 1e18$	$O(\log n)$ / $O(\log^2 n)$	快速幂、矩阵快速幂、数论分块

若题目有 T 组数据，先把所有规模相加；不要只看单组上限。

1.3 分层路线

1.3.1 L0：实现基本功

C++ 输入输出、引用/指针、`vector/string/pair/tuple`、排序比较器、`lambda`、`lower_bound`、位运算、递归栈、`longlong`、模运算规范。

1.3.2 L1: 线性与排序

前缀和/差分 → 双指针/滑动窗口 → 二分查找/二分答案 → 离散化 → 排序贪心 → 单调栈/单调队列。

1.3.3 L2: 基础图和数学

BFS/DFS/连通分量 → 拓扑 → 并查集 → Dijkstra/MST → 模运算、筛法、组合数、逆元。

1.3.4 L3: 基础 DP 和经典数据结构

线性 DP、背包、区间 DP、树形 DP → 树状数组、线段树、ST 表、堆、Trie。

1.3.5 L4: 字符串、树和提升题

KMP/Z/哈希/Manacher → LCA/树上差分/直径 → SCC/割点桥/二分图匹配 → 0-1 BFS、差分约束、网络流。

1.3.6 L5: 银牌常见进阶

CDQ/整体二分、点分治、虚树、LCT/DSU on Tree、凸包与旋转卡壳、后缀数组/SAM/PAM、矩阵快速幂、状压与数位 DP 优化、基础博弈。

1.4 赛场决策树

先读约束

- └ n 很小 ($\leq 20/40$) ? → 子集枚举 / 折半搜索 / 状压 DP
- └ 需要连续区间 ? → 前缀和 / 差分 / 双指针 / 单调队列
- └ “最小的最大值/最大的最小值” ? → 二分答案 + 可行性检查
- └ 图上可达/层数 ? → BFS; 有权最短路 ? → 0-1 BFS / Dijkstra
- └ 边权全为 1 且有状态 ? → 分层 BFS / 乘积图
- └ 选取若干对象最优 ? → 先尝试贪心交换论证, 再写 DP
- └ 多次区间修改/查询 ? → 差分 / 树状数组 / 线段树
- └ 树上路径/子树 ? → DFS 序、LCA、树上差分、重链或点分治
- └ 字符串匹配/回文/多模式 ? → KMP/Z、Manacher、AC 自动机

1.5 正确性证明的四种常用模板

- **不变量**: 循环/数据结构每一步都维持同一个性质, 如 Dijkstra 已确定点的最短路不再改变。

- 交换论证：把任意最优解改造成贪心解而不变差，如按结束时间排序的区间调度。
- 归纳/状态转移：证明 DP 状态覆盖所有合法前缀，转移恰好枚举最后一步。
- 势能/单调性：证明每次操作让某个量单调变化，从而保证双指针、并查集、单调栈的线性复杂度。

1.6 提交前检查

- 是否把 `int` 乘法提升成 `1LL*a*b`?
- 模数不是质数时是否误用了逆元?
- `vector` 是否提前 `reserve`，递归深度是否可能爆栈?
- 图是否 0/1 下标一致，是否忘记反向边或重边?
- 二分边界是否满足“左闭右闭/左闭右开”约定?
- 多测是否清空邻接表、全局答案、标记数组、缓存?
- 几何是否统一使用 `longdouble` 与 `eps`，叉积是否溢出?

2 竞赛模板

- `#define int long long` 之后 `main` 必须写 `signed main`, 否则编译错误。
- 位运算中常数 1 默认是 32 位, 当 $i \geq 31$ 时 `1 << i` 溢出变负数, 必须写 `1LL << i`。
- `inf = 2e18` 做加法时可能溢出 (如 `inf + x` 爆 `long long`), 松弛前先判 `if(dist[u] != inf)`。

```
#include <bits/stdc++.h>
using namespace std;
#define int long long
#define endl '\n'

// 变量: inf=表示不可达状态的正无穷哨兵值 inf; 参与加法前先判断当前值不是 inf。
const int inf = 2e18;
// 变量: N=当前代码允许使用的最大下标上界 N。
const int N = 1e6 + 10;

// 接口: solve(): 读取一个测试用例的规模 n, 并作为题目核心逻辑的单测入口; 无返回值。
void solve() {
    // 变量: n=当前测试用例读入的元素个数 n。
    int n;
    cin >> n;
}

// 接口: main(): 开启快速 I/O, 读入测试数 t, 并为每个测试调用一次 solve; 入口函数; 每组测试前按注释清空全局状态。
signed main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    // 变量: t=当前程序要执行的测试用例数量 t。
    int t = 1;
    cin >> t;
    while (t--) {
        solve();
    }
}
```

```

    }
    return 0;
}

```

- 不需要多测时把 `cin >> t` 注释掉即可。
- 多测时注意：solve() 里不要中途 `return`（会导致后续输入串流）；全局数组/容器必须清空干净。
- 局部变量必须初始化！`int sum`；在 OJ 上可能是乱码，必须写 `int sum = 0`；。

2.1 C++ 竞赛实现规范

```

#include <bits/stdc++.h>
using namespace std;
using i64 = long long;
using u64 = unsigned long long;
using i128 = __int128_t;
using u128 = __uint128_t;
// 变量：INF=表示不可达状态的正无穷哨兵值 INF；参与加法前先判断当前值不是 INF。
constexpr int INF = 0x3f3f3f3f;
constexpr i64 LINF = (1LL << 62);

// 接口：print_i128(x)：十进制输出一个 __int128；不返回值。
// 参数：待十进制输出的 __int128 整数 x。
void print_i128(i128 x) { // 需要输出超出 long long 时使用
    // 先转成无符号绝对值，避免对 INT128_MIN 直接取反产生有符号溢出。
    // 变量：value=x 的无符号绝对值，可完整容纳 2^127。
    u128 value = static_cast<u128>(x);
    if (x < 0) { cout << '-'; value = 0 - value; }
    // 变量：s=从标准输入读取、供当前字符串算法处理的字符串 s。
    string s;
    do { s.push_back('0' + value % 10); value /= 10; } while (value);
    reverse(s.begin(), s.end()); cout << s;
}

```

- `ios::sync_with_stdio(false);cin.tie(nullptr)`；放在 main 开头。
- 需要修改容器元素时使用引用：`for(auto&x:a)`。
- 大对象传参用 `constvector<int>&`；避免递归中反复复制。
- `sort` 比较器必须是严格弱序，不能写 `<=`。
- 计数、乘积、距离默认 `longlong`；乘法可能达到 `1e18` 时显式 `1LL` 或 `__int128`。

3 STL 常用函数速查

3.1 容器

3.1.1 vector

```
vector<int> a(n, 0);           // n 个元素初始化为 0
a.push_back(x);               // 尾部插入
a.pop_back();                 // 尾部删除
a.size();                     // 元素个数（返回 size_t，比较时注意）
a.empty();                    // 是否为空
a.clear();                    // 清空
a.resize(m, val);              // 重设大小
a.erase(a.begin() + i);       // 删除第 i 个元素 O(n)
// 去重：
sort(a.begin(), a.end());
a.erase(unique(a.begin(), a.end()), a.end());
```

3.1.2 array

```
array<int, N> a;                // 固定大小数组，N 必须是编译期常量
array<int, 5> a = {1, 2, 3, 4, 5}; // 初始化
array<int, 5> a = {};            // 全部初始化为 0
a.size();                       // 元素个数（编译期确定，始终为 N）
a.fill(val);                    // 全部填充为 val
a.front(); a.back();            // 首尾元素
a.data();                       // 底层数组指针
a[i];                           // 随机访问 O(1)
// 支持比较运算符（按字典序），可以直接当 map 的 key
// 与 C 数组的区别：可以赋值、比较、作为函数返回值
```

- array 不能用变量做大小，array<int, n> 编译错误（n 必须是 constexpr）。
- 局部 array 不会自动初始化为 0！必须写 array<int, 5> a = {}; 或 a.fill(0);。

- 竞赛中常用场景：需要把固定长度的数组当 map / set 的 key 时，用 array 比手写 struct 方便。

3.1.3 string

```
// 变量：s=从标准输入读取、供当前字符串算法处理的字符串 s。
string s;
getline(cin, s);           // 读一整行（注意前面 cin>>n 后要 cin.ignore()）
s.substr(pos, len);        // 从 pos 开始取 len 个字符
s.find("abc");             // 查找子串，找不到返回 string::npos
s += "xyz";               // 拼接
to_string(123);           // int -> string
stoi("123");              // string -> int
stoll("123456789012");    // string -> long long
```

3.1.4 set / multiset

```
// 变量：s=用于演示有序集合接口的 set 容器 s。
set<int> s;
s.insert(x);              // 插入
s.erase(x);              // 删除所有值为 x 的元素
s.count(x);              // 0 或 1
s.lower_bound(x);        // >= x 的第一个迭代器
s.upper_bound(x);        // > x 的第一个迭代器
// multiset 删除单个元素：
// 变量：it=容器 ms 中当前查找结果的迭代器 it。
auto it = ms.find(x);
if (it != ms.end()) ms.erase(it); // 只删一个！
// 如果写 ms.erase(x) 会删掉所有等于 x 的元素！
```

3.1.5 map

```
// 变量：mp=把 int 键映射到 int 值并保持键有序的 map 容器 mp。
map<int, int> mp;
mp[key] = val;            // 插入/修改
mp.count(key);            // 是否存在
mp.erase(key);           // 删除
// 遍历：
for (auto& [k, v] : mp) { ... }
// 注意：mp[key] 如果 key 不存在会自动插入默认值 0！
// 只查询用 mp.count(key) 或 mp.find(key)
```

3.1.6 unordered_map / unordered_set

```
unordered_map<int, int> mp; // 哈希表, 平均 O(1) 查找/插入/删除
mp[key] = val;           // 插入/修改 (同 map)
mp.count(key);           // 是否存在
mp.erase(key);           // 删除
for (auto& [k, v] : mp) { ... } // 遍历 (无序!)

// 变量: us=当前无序集合 us。
unordered_set<int> us;
us.insert(x);
us.count(x);             // 0 或 1
us.erase(x);

// 与 map/set 的区别:
// 1. 无序, 不支持 lower_bound / upper_bound
// 2. 平均 O(1), 但最坏 O(n) (哈希冲突)
// 3. 不能直接用 pair/vector 做 key (需要自定义哈希)
```

- 被卡哈希: Codeforces 等平台可以构造数据卡 unordered_map 的默认哈希, 导致退化为 $O(n^2)$ 。
- 防卡方法: 自定义哈希函数加随机种子:

```
struct custom_hash {
    // 接口: operator(): 返回键值的 size_t 哈希值, 供 unordered_map/unordered_set 去重;
    // 返回类型 size_t。
    size_t operator()(int x) const {
        static const size_t FIXED_RANDOM =
            chrono::steady_clock::now().time_since_epoch().count();
        x ^= FIXED_RANDOM;
        return x ^ (x >> 16);
    }
};

// 变量: mp=使用 custom_hash 把 int 键映射到 int 值的哈希表 mp。
unordered_map<int, int, custom_hash> mp;
```

- 用 mp.reserve(n) 预分配空间、mp.max_load_factor(0.25) 降低负载因子, 可以显著提速。
- 需要有序遍历或 lower_bound 时, 必须用 map / set。
- unordered_map 的 [] 操作同样会自动插入默认值, 注意与 map 一样的坑。

3.1.7 priority_queue

```
// 大根堆（默认）
// 变量：pq=当前优先队列 pq；堆顶含义见比较器。
priority_queue<int> pq;
// 小根堆
// 变量：pq=当前优先队列 pq；堆顶含义见比较器。
priority_queue<int, vector<int>, greater<int>> pq;
pq.push(x);
pq.top();    // 堆顶（不弹出）
pq.pop();    // 弹出堆顶
```

3.1.8 deque

```
// 变量：dq=当前双端队列 dq。
deque<int> dq;
dq.push_front(x); dq.push_back(x);
dq.pop_front();   dq.pop_back();
dq.front();       dq.back();
dq[i];            // 支持随机访问
```

3.1.9 stack

```
// 变量：stk=当前栈 stk。
stack<int> stk;
stk.push(x);      // 压栈
stk.pop();        // 弹栈（无返回值！）
stk.top();        // 取栈顶元素
stk.empty();      // 是否为空
stk.size();       // 元素个数
// 注意：stack 没有 clear()，清空方式：
// while (!stk.empty()) stk.pop();
// 或直接 stk = stack<int>();
```

3.1.10 list

```
list<int> lst;
lst.push_back(x); lst.push_front(x); // 头尾插入 O(1)
lst.pop_back();   lst.pop_front();   // 头尾删除 O(1)
lst.front();      lst.back();        // 访问头尾
lst.insert(it, x); // 在迭代器 it 前插入 O(1)
lst.erase(it);    // 删除迭代器指向的元素 O(1)
```

```

lst.size();          lst.empty();
lst.remove(val);      // 删除所有值为 val 的元素
lst.sort();           // 链表专用排序（归并） $O(n \log n)$ 
lst.unique();         // 去除相邻重复元素（需先排序）
lst.merge(lst2);      // 合并两个有序链表
lst.reverse();        // 反转链表
lst.splice(it, lst2); // 将 lst2 全部移接到 it 前面  $O(1)$ 
// 注意：list 不支持随机访问（没有 []），遍历只能用迭代器
// 适用场景：频繁在中间插入/删除，且不需要随机访问

```

3.2 常用算法

```

// 排序
sort(a.begin(), a.end());           // 升序
sort(a.begin(), a.end(), greater<int>()); // 降序

// 二分查找（要求有序）
lower_bound(a.begin(), a.end(), x); //  $\geq x$  的第一个位置
upper_bound(a.begin(), a.end(), x); //  $> x$  的第一个位置

// 去重
unique(a.begin(), a.end()); // 返回去重后的尾迭代器

// 全排列
next_permutation(a.begin(), a.end()); // 下一个排列，到最后返回 false

// 最值
*max_element(a.begin(), a.end());
*min_element(a.begin(), a.end());

// 累加
accumulate(a.begin(), a.end(), 0LL); // 注意初始值类型！

```

- 自定义 sort 的比较函数必须用严格小于 $<$ ，不能用 $\leq$ ，否则破坏严格弱序，导致随机 RE/WA。
- accumulate 初始值写 0 会按 int 累加溢出，必须写 0LL。
- vector.size() 返回无符号数，a.size() - 1 当 size 为 0 时会下溢变成极大值！

3.3 pair / tuple

```
// pair
// 变量: p=用于演示 pair 接口的整数对 p。
pair<int, int> p = {1, 2};
auto [x, y] = p; // C++17 结构化绑定
p.first; p.second; // 访问元素
// pair 支持比较运算符 (先比 first, 再比 second)
// 可以直接作为 map 的 key、放入 set、sort

// make_pair
auto p2 = make_pair(3, 4); // 自动推导类型

// tuple (三元及以上)
tuple<int, int, int> t = {1, 2, 3};
auto [a, b, c] = t; // C++17 结构化绑定
get<0>(t); get<1>(t); get<2>(t); // 访问元素
// 变量: t2=用于演示 tuple 构造与字典序比较的三元组 t2。
auto t2 = make_tuple(1, 2, 3);
// tuple 也支持比较运算符 (按字典序)
// 竞赛中三元排序直接用 tuple 最方便
```

- 多关键字排序用 `vector<tuple<int,int,int>>` 比写结构体更快。
- `tie(a, b, c) = make_tuple(x, y, z)` 可以一行赋值多个变量。
- C++17 的结构化绑定 `auto [a, b, c] = t` 最简洁, 推荐使用。

3.4 bitset

`bitset<N>` 是固定大小的位集合, `N` 必须是编译期常量。空间 $N/8$ 字节, 位运算操作是 $O(N/64)$ (64 位压缩)。

```
bitset<1000> bs; // 全部初始化为 0
bitset<8> bs2("10101010"); // 从字符串初始化 (注意: 字符串最右对应第 0 位)
bitset<8> bs3(0xAA); // 从整数初始化

// 单位操作
bs.set(i); // 第 i 位置 1
bs.set(i, 0); // 第 i 位置 0
bs.reset(i); // 第 i 位置 0
bs.flip(i); // 第 i 位取反
```



```

bs.test(i);          // 返回第 i 位的值 (bool)
bs[i];               // 同 test(i)，但不做越界检查

// 全局操作
bs.set();            // 全部置 1
bs.reset();          // 全部置 0
bs.flip();           // 全部取反
bs.count();          // 1 的个数 (popcount)
bs.size();           // 总位数 N
bs.any();            // 是否存在至少一个 1
bs.none();           // 是否全为 0
bs.all();            // 是否全为 1 (C++11)

// 位运算 (O(N/64)，比手写快很多)
bitset<1000> a, b;
a & b; a | b; a ^ b; ~a; // 返回新 bitset
a &= b; a |= b; a ^= b; // 原地修改
a <<= k; a >>= k;       // 左移/右移 k 位
(a & b).count();        // 两个集合的交集大小

// 转换
bs.to_ulong();        // 转为 unsigned long (N <= 32 时)
bs.to_ullong();       // 转为 unsigned long long (N <= 64 时)
bs.to_string();       // 转为 "010101..." 字符串

// 查找 (非标准但 GCC 支持)
bs._Find_first();     // 第一个 1 的位置 (没有则返回 N)
bs._Find_next(i);     // i 之后第一个 1 的位置 (没有则返回 N)
// 枚举所有 1 的位置:
// 变量: i=动态 bitset 中当前找到的置位下标 i。
for (int i = bs._Find_first(); i < (int)bs.size(); i = bs._Find_next(i)) {
    // 处理位置 i
}

```

- `bitset` 的大小 `N` 必须是编译期常量，不能用变量！
- `bs[i]` 的 i 从 0 开始，第 0 位是最低位。
- 从字符串构造时，字符串的最后一个字符对应第 0 位（最低位），容易搞反。
- `_Find_first` 和 `_Find_next` 是 GCC 扩展，非标准，但竞赛中可用。

- **bitset 优化 DP**: 将布尔 DP 数组改用 bitset, 转移用位移 + 或运算, 复杂度除以 64。
经典例题: 背包可行性判断 $bs = (bs \ll w[i]) |$ 。
- **bitset 求交集/汉明距离**: $(a \oplus b).count()$ 即汉明距离, $(a \& b).count()$ 即共同元素个数。
- **bitset 优化图算法**: 传递闭包 $O(n^3/64)$, 二部图匹配加速等。
- **bitset 加速暴力**: $n \leq 40000$ 的 $O(n^2)$ 布尔运算用 bitset 可以压到 $O(n^2/64)$, 约等于 $n \leq 5000$ 的常规暴力速度。

3.4.1 bitset 优化背包 / 子集和可行性

判断若干物品能凑出哪些重量 (布尔 DP), 把 $dp[j]$ 改为 bitset 的第 j 位, 转移用左移 + 或, 复杂度 $O(nV/64)$ 。

```
// 数值域:  $0 \leq w[i] < \text{MAXV}$ ; 左移超出 MAXV 的位按 bitset 规则丢弃。
//  $bs[j] = 1$  表示重量  $j$  可以被凑出
bitset<MAXV> bs;
bs[0] = 1; // 重量 0 总能凑出 (什么都不选)
// 变量:  $i = \text{bitset } 0/1$  背包当前加入的物品下标  $i$ 。
for (int i = 0; i < n; i++)
    bs |= bs << w[i]; // 01 背包: 每个物品做一次整体左移再或
//  $bs[j]$  为 1 即容量  $j$  可达;  $bs.count()$  为可达重量种类数

// 完全背包可行性: 物品可重复选, 按位逐位递推
// for (int i = 0; i < n; i++)
//     for (int j = w[i]; j < MAXV; j++) if (bs[j-w[i]]) bs[j] = 1;
```

- MAXV 必须 $\geq$ 最大可能重量 + 1, 且是编译期常量。
- $bs = bs \ll w[i]$ 一次只把每个物品计入一次, 是 01 背包语义; 完全背包需另写逐位递推。

3.4.2 bitset 优化传递闭包 (Floyd)

求有向图可达性矩阵, $O(n^3)$ 的 Floyd 用 bitset 整行或运算压到 $O(n^3/64)$ 。

```
// 接口: transitive_closure(reach, n): 原地计算前  $n$  个顶点的传递闭包。
// 参数: reach 初始含自环和原图边;  $n$  为实际顶点数且  $n \leq \text{MAXN}$ 。
template<size_t MAXN>
void transitive_closure(array<bitset<MAXN>, MAXN>& reach, int n) {
```

```
// 变量: k=当前允许作为中转点的顶点 k。
for (int k = 0; k < n; k++)
    // 变量: i=当前起点 i。
    for (int i = 0; i < n; i++)
        if (reach[i][k])
            reach[i] |= reach[k];
}
```

3.4.3 bitset 优化字符串匹配 (Shift-And)

在文本 t 中查找模式串 p (长 m)，用 bitset 维护”当前已匹配前缀”集合， $O(nm/64)$ 。

```
// B[c] 第 j 位为 1 表示 p[j]==c
bitset<MAXM> B[256], D;
// 变量: matches=模式串在文本中的全部 0-based 起始位置。
vector<int> matches;
// 变量: j=Shift-And 模式串 p 当前写入位掩码的字符位置 j。
for (int j = 0; j < m; j++) B[(unsigned char)p[j]][j] = 1;
// 变量: i=Shift-And 扫描主串 t 的当前字符位置 i。
for (int i = 0; i < (int)t.size(); i++) {
    D = ((D << 1) | bitset<MAXM>(1)) & B[(unsigned char)t[i]];
    if (D[m - 1]) {
        matches.push_back(i - m + 1);
    }
}
}
```

Shift-And 的状态 $D[j]=1$ 表示 $p[0..j]$ 是 t 当前结尾的后缀。把 $|$ 换成”或非”、判 $D[m-1]==0$ 即为 Shift-Or 变体，原理相同。

3.5 `__int128`

GCC 扩展的 128 位整数，范围约 $[-1.7 \times 10^{38}, 1.7 \times 10^{38}]$ 。用于中间计算结果超过 long long 但最终结果不超过的情况。

```
// 变量: a=演示 __int128 算术与模乘的 128 位整数 a。
__int128 a = 1;
// 不能直接 cin/cout, 需要手写 IO
// 接口: print128(x): 十进制输出一个 __int128; 不返回值。
// 参数: 待十进制输出的 __int128 整数 x。
void print128(__int128 x) {
```

```

    if (x < 0) { putchar('-'); x = -x; }
    if (x > 9) print128(x / 10);
    putchar(x % 10 + '0');
}

// 接口: read128(): 从标准输入读取并返回一个 __int128; 返回类型 __int128。
__int128 read128() {
    // 变量: x=十进制读入过程中累计绝对值的 __int128 结果 x; f=读入 __int128 时记录正负号
    // 的符号 f; c=当前从输入读取的字符 c。
    __int128 x = 0; int f = 1; char c = getchar();
    while (c < '0' || c > '9') { if (c == '-') f = -1; c = getchar(); }
    while (c >= '0' && c <= '9') { x = x * 10 + c - '0'; c = getchar(); }
    return x * f;
}

// 典型应用: 大数乘法取模, 避免 long long 乘法溢出
// 接口: mul_mod(a, b, mod): 计算 a*b mod mod, 并避免中间乘法溢出; 返回类型 int。
// 参数: 模乘的第一个整数因子 a; 模乘的第二个整数因子 b; 正模数 mod。
int mul_mod(int a, int b, int mod) {
    return (__int128)a * b % mod;
}

```

- __int128 不能直接用 cin/cout/printf, 必须手写 IO。
- __int128 是 GCC 扩展, MSVC 不支持。竞赛 OJ 一般都支持。
- 不能用于 bitset 的下标或 vector 的大小。

3.6 __builtin 系列

```

// 二进制中 1 的个数
__builtin_popcount(x);    // unsigned int 版
__builtin_popcountll(x);  // unsigned long long 版

// 末尾 0 的个数 (即最低位 1 的位置)
__builtin_ctz(x);         // x != 0 时才有意义
__builtin_ctzll(x);

// 前导 0 的个数
__builtin_clz(x);         // x != 0 时才有意义
__builtin_clzll(x);

// 奇偶性 (1 的个数是否为奇数)

```

```

__builtin_parity(x);          // 奇数个 1 返回 1
__builtin_parityll(x);

// 常用推导
// 最高位 1 的位置 = 63 - __builtin_clzll(x)    (x > 0)
// 最低位 1 的位置 = __builtin_ctzll(x)         (x > 0)
// 向上取 2 的幂 = 1LL << (64 - __builtin_clzll(x - 1)) (x > 1)
// 向下取 2 的幂 = 1LL << (63 - __builtin_clzll(x))    (x > 0)
// log2(x) 下取整 = 63 - __builtin_clzll(x)    (x > 0)

```

- `__builtin_clz(0)` 和 `__builtin_ctz(0)` 是未定义行为！使用前必须判断 $x \neq 0$ 。
- `int` 类型用不带 `ll` 的版本，`long long` 类型必须用 `ll` 版本，否则结果错误。

3.7 C++ 常用函数速查

3.7.1 数值函数

```

// <algorithm> 头文件
min(a, b); max(a, b);          // 最小值/最大值
min({a, b, c, d});             // 多个值取最小 (C++11, 初始化列表)
max({a, b, c, d});

swap(a, b);                    // 交换两个值
clamp(x, lo, hi);              // 限制 x 在 [lo, hi] 范围内 (C++17)

// <numeric> 头文件
accumulate(a.begin(), a.end(), 0LL); // 累加 (注意初始值类型!)
accumulate(a.begin(), a.end(), 1LL, multiplies<>()); // 累乘
iota(a.begin(), a.end(), 0);        // 填充 0, 1, 2, ...
partial_sum(a.begin(), a.end(), pre.begin()); // 前缀和
adjacent_difference(a.begin(), a.end(), d.begin()); // 差分
inner_product(a.begin(), a.end(), b.begin(), 0LL); // 内积
gcd(a, b); lcm(a, b);            // 最大公约数/最小公倍数 (C++17)
midpoint(a, b);                  // (a+b)/2 不溢出 (C++20)
reduce(a.begin(), a.end());       // 并行累加 (C++17, 无序)

// <cmath> 头文件
abs(x);                         // 绝对值 (整数用 abs, 浮点也可以)
ceil(x); floor(x); round(x);    // 上取整/下取整/四舍五入
sqrt(x); cbrt(x);               // 平方根/立方根
log(x); log2(x); log10(x);      // 自然对数/以2为底/以10为底
pow(x, y);                      // x 的 y 次方 (浮点, 整数慎用!)

```

```
fmod(x, y);           // 浮点取模
hypot(x, y);          // sqrt(x*x + y*y), 不溢出
```

- `pow(x, y)` 返回浮点数, `pow(5, 2)` 可能返回 24.999..., 赋给 `int` 变成 24! 整数幂次用快速幂。
- 整数除法上取整: $\lceil a/b \rceil = (a + b - 1) / b$ ($a, b > 0$ 时)。不要用 `ceil((double)a / b)`, 浮点不精确。
- `abs(INT_MIN)` 溢出! `INT_MIN` 的绝对值超过 `INT_MAX`。

3.7.2 区间操作函数

```
// 排序
sort(a.begin(), a.end());           // 升序
sort(a.begin(), a.end(), greater<int>()); // 降序
stable_sort(a.begin(), a.end());     // 稳定排序
partial_sort(a.begin(), a.begin()+k, a.end()); // 只排前 k 小
nth_element(a.begin(), a.begin()+k, a.end()); // 第 k 小放到位置 k, O(n)

// 二分查找 (要求有序)
lower_bound(a.begin(), a.end(), x); // >= x 的第一个位置
upper_bound(a.begin(), a.end(), x); // > x 的第一个位置
binary_search(a.begin(), a.end(), x); // 是否存在 x (bool)
equal_range(a.begin(), a.end(), x); // 返回 {lower_bound, upper_bound} 的 pair

// 查找
find(a.begin(), a.end(), x);         // 找第一个 x, 返回迭代器
count(a.begin(), a.end(), x);        // 统计 x 的个数
// 接口: 判定 lambda(v): 判断当前数组元素 v 是否大于 0。
// 参数: 当前被 count_if 判断的数组元素 v。
count_if(a.begin(), a.end(), [](int v){ return v > 0; }); // 条件统计
any_of(a.begin(), a.end(), pred);    // 是否存在满足条件的
all_of(a.begin(), a.end(), pred);    // 是否全部满足条件
none_of(a.begin(), a.end(), pred);   // 是否全不满足条件

// 最值
*max_element(a.begin(), a.end());    // 最大值
*min_element(a.begin(), a.end());    // 最小值
auto [mi, ma] = minmax_element(a.begin(), a.end()); // 同时求最小和最大的迭代器

// 去重 (要求有序)
auto it = unique(a.begin(), a.end()); // 返回去重后的尾迭代器
```

```

a.erase(it, a.end()); // 真正删除多余元素

// 反转、旋转、填充
reverse(a.begin(), a.end()); // 反转
rotate(a.begin(), a.begin()+k, a.end()); // 左旋 k 位
fill(a.begin(), a.end(), val); // 填充
fill_n(a.begin(), k, val); // 填充前 k 个
// 接口：生成 lambda(): 调用 rng() 生成当前要写入数组的随机值。
generate(a.begin(), a.end(), [&]() { return rng(); }); // 用函数生成

// 拷贝、变换
copy(a.begin(), a.end(), b.begin()); // 拷贝到 b
// 接口：变换 lambda(x): 返回输入元素 x 的两倍并写入目标数组。
// 参数：当前被 transform 变换的输入元素 x。
transform(a.begin(), a.end(), b.begin(), [](int x) { return x*2; }); // 变换

```

- nth_element 是 $O(n)$ 的，比 sort 后取第 k 个快很多。常用于求中位数。
- partial_sort 只排前 k 小， $O(n \log k)$ ，当 $k \ll n$ 时比完整排序快。
- rotate 实现数组左旋非常方便， $O(n)$ 。

3.7.3 排列与组合

```

// 全排列
// next_permutation: 生成下一个字典序排列，到最后返回 false
// prev_permutation: 生成上一个字典序排列，到最前返回 false
// 变量：a=用于演示排列算法的整数序列 a。
vector<int> a = {1, 2, 3};
do {
    // 处理当前排列 a
} while (next_permutation(a.begin(), a.end()));
// 注意：要枚举所有排列，a 必须先排序为最小排列！

// 枚举子集（用 bitmask）
// 变量：mask=用二进制位编码的当前状态 mask。
for (int mask = 0; mask < (1 << n); mask++) {
    // mask 的第 i 位为 1 表示选第 i 个元素
    // 变量：i=当前子集掩码中正在检查是否被选中的元素位 i。
    for (int i = 0; i < n; i++)
        if (mask >> i & 1) { /* 选了第 i 个 */ }
}

```

```
// 枚举大小为 k 的子集（用 bitmask）
// 见位运算章节

// is_permutation: 判断两个序列是否互为排列
is_permutation(a.begin(), a.end(), b.begin()); // bool
```

- `next_permutation` 会修改原数组！如果要枚举所有排列，初始必须是最小排列（升序）。
- 全排列总数是 $n!$ ， $n \geq 12$ 时 $n! > 10^8$ ，注意复杂度。
- `prev_permutation` 生成上一个排列，初始必须是最大排列（降序）才能枚举所有排列。

3.7.4 全排列函数详解（`next_permutation` / `prev_permutation`）

C++ STL 提供的 `next_permutation` 和 `prev_permutation` 是生成排列的核心工具。它们原地修改序列，将其变换为字典序的下一个/上一个排列，返回 `bool` 表示是否成功（到达最后/最前时返回 `false` 并回绕为最小/最大排列）。时间复杂度单次 $O(n)$ ，枚举所有排列总计 $O(n! \cdot n)$ 。

```
// 函数签名
bool next_permutation(BidirIt first, BidirIt last);
bool next_permutation(BidirIt first, BidirIt last, Compare comp);
bool prev_permutation(BidirIt first, BidirIt last);
bool prev_permutation(BidirIt first, BidirIt last, Compare comp);
// 返回值：成功生成下一个/上一个排列返回 true
//          已经是最大/最小排列时回绕并返回 false

// ===== 用法一：枚举所有排列 =====
// 必须先排序为最小排列（升序），然后用 do-while 循环
// 变量：a=用于演示排列算法的整数序列 a。
vector<int> a = {1, 2, 3};
sort(a.begin(), a.end()); // 必须先升序排列！
do {
    // 处理当前排列，例如输出
    // 变量：x=从 a 枚举的元素值 x。
    for (int x : a) cout << x << " ";
    cout << endl;
} while (next_permutation(a.begin(), a.end()));
// 输出：1 2 3 / 1 3 2 / 2 1 3 / 2 3 1 / 3 1 2 / 3 2 1

// ===== 用法二：有重复元素的排列（自动去重） =====
// next_permutation 天然支持重复元素，不会生成重复排列
// 变量：a=用于演示排列算法的整数序列 a。
```



```

vector<int> a = {1, 1, 2};
sort(a.begin(), a.end());
do {
    // 变量: x=从 a 枚举的元素值 x。
    for (int x : a) cout << x << " ";
    cout << endl;
} while (next_permutation(a.begin(), a.end()));
// 输出: 1 1 2 / 1 2 1 / 2 1 1 (只有 3 种, 不是 3!=6 种)

// ===== 用法三: 只排列部分元素 (前 k 个的全排列) =====
// 枚举 n 个元素中选 k 个的全排列 (P(n,k))
// 变量: n=被选元素全集的大小 n; k=需要从 n 个元素中选取的元素个数 k。
int n = 4, k = 2;
// 变量: a=用于演示排列算法的整数序列 a。
vector<int> a = {1, 2, 3, 4};
sort(a.begin(), a.end());
do {
    // 只看前 k 个元素, 它们是一种选法的排列
    // 变量: i=当前输出所选组合中第 i 个元素的下标 i。
    for (int i = 0; i < k; i++) cout << a[i] << " ";
    cout << endl;
    // 将后面的部分反转, 使 next_permutation 跳到下一组前 k 个
    reverse(a.begin() + k, a.end());
} while (next_permutation(a.begin(), a.end()));

// ===== 用法四: 求当前排列之后的第 k 个排列 =====
// 变量: a=用于演示排列算法的整数序列 a。
vector<int> a = {1, 2, 3, 4};
// 变量: k=限制输出的前 k 个排列数量 k。
int k = 5;
while (k-- && next_permutation(a.begin(), a.end()));
// a 变成第 5 个后继排列

// ===== 用法五: 对字符串使用 =====
// 变量: s=从标准输入读取、供当前字符串算法处理的字符串 s。
string s = "abc";
sort(s.begin(), s.end());
do {
    cout << s << endl;
} while (next_permutation(s.begin(), s.end()));
// 输出: abc / acb / bac / bca / cab / cba

// ===== 用法六: 自定义比较函数 (降序排列枚举) =====
vector<int> a = {3, 2, 1}; // 按 greater 的最小排列

```

```
do {
    // 变量: x=从 a 枚举的元素值 x。
    for (int x : a) cout << x << " ";
    cout << endl;
} while (next_permutation(a.begin(), a.end(), greater<int>()));
// 枚举所有"降序字典序"的排列
```

- **初始状态很重要:** 用 `next_permutation` 枚举所有排列, 初始必须是升序 (字典序最小); 用 `prev_permutation` 则初始必须是降序 (字典序最大)。否则只能枚举到从当前位置到末尾的部分排列。
- **回绕行为:** 当 `next_permutation` 在最大排列上调用时, 会将数组变回最小排列并返回 `false`。这就是 `do-while` 的终止条件——回绕后 `false` 退出循环。
- **复杂度:** $n!$ 增长极快, $n = 8$ 时 $n! = 40320$, $n = 10$ 时 $n! = 3628800$, $n = 12$ 时 $n! \approx 4.8 \times 10^8$ 。竞赛中 $n \leq 8$ 比较安全, $n \leq 10$ 勉强可行。
- **不要用 while:** 写 `while (next_permutation(...))` 会跳过初始排列! 必须用 `do { ... } while (next_permutation(...))`。

- **判断是否为排列:** 用 `is_permutation(a.begin(), a.end(), b.begin())` 判断 a 和 b 是否互为排列。
- **`next_permutation` 的原理:** 从右往左找第一个 $a[i] < a[i + 1]$ 的位置, 再从右往左找第一个 $a[j] > a[i]$ 的位置, 交换 $a[i]$ 和 $a[j]$, 最后反转 $a[i + 1 \dots]$ 。
- **求排列数/排名:** 需要知道某排列是第几个, 用康托展开 (见下节)。
- **竞赛经典场景:** 暴力枚举所有排列检查是否满足条件 (如 N 皇后的暴力解、TSP 暴力等)。

DFS 递归生成全排列

当需要在生成过程中剪枝 (如 N 皇后), 或需要更灵活的控制 (如只要部分元素的排列), 用 DFS 递归比 `next_permutation` 更方便。

```
// 生成 {1, 2, ..., n} 的全排列 (不重复元素)
// 变量: path=排列 DFS 当前已经选定的前缀 path。
vector<int> path;
// 变量: used=记录每个对象是否已被使用的标记 used。
vector<bool> used;

// 接口: dfs(n): 枚举并逐个输出 1..n 的全部排列; 不返回值。
```

```
// 参数：待生成排列的长度 n。
void dfs(int n) {
    if ((int)path.size() == n) {
        // 得到一个排列 path，处理它
        // 变量：x=从 path 枚举的元素值 x。
        for (int x : path) cout << x << " ";
        cout << endl;
        return;
    }
    // 变量：i=无重复全排列 DFS 当前尝试放入路径的候选值 i。
    for (int i = 1; i <= n; i++) {
        if (used[i]) continue;
        path.push_back(i);
        used[i] = true;
        dfs(n);
        path.pop_back();
        used[i] = false;
    }
}

// 调用：used.assign(n + 1, false); dfs(n);

// 有重复元素的 DFS 全排列（去重版）
// nums 必须先排序！
vector<int> nums; // 已排序
// 变量：path2=含重复元素时的当前 DFS 排列 path2。
vector<int> path2;
// 变量：used2=含重复元素时每个下标是否已被当前排列使用。
vector<bool> used2;

// 接口：dfs2(n)：枚举已排序 nums 的所有不重复全排列并逐个输出；不返回值。
// 参数：待生成排列的长度 n。
void dfs2(int n) {
    if ((int)path2.size() == n) {
        // 变量：x=从 path2 枚举的元素值 x。
        for (int x : path2) cout << x << " ";
        cout << endl;
        return;
    }
    // 变量：i=含重复元素全排列 DFS 当前尝试使用的 nums 下标 i。
    for (int i = 0; i < n; i++) {
        if (used2[i]) continue;
        // 去重：同一层中，相同值只选第一个未用的
        if (i > 0 && nums[i] == nums[i-1] && !used2[i-1]) continue;
        path2.push_back(nums[i]);
    }
}
```

```

        used2[i] = true;
        dfs2(n);
        path2.pop_back();
        used2[i] = false;
    }
}
// 调用: sort(nums.begin(), nums.end()); used2.assign(n, false); dfs2(n);

```

- DFS 版本可以方便地在递归中**剪枝**，例如 N 皇后问题在放置每个皇后时检查冲突，大幅减少搜索量。
- DFS 生成**部分排列**（选 k 个）：只需把终止条件改为 `path.size() == k`。
- 去重的关键条件！`used2[i-1]` 表示：如果前一个相同元素没有被选（说明是同层回溯回来的），则跳过当前元素，避免重复。

3.7.5 排列序（康托展开与逆康托展开）

康托展开：将一个排列映射为它在所有排列中的字典序排名（0-indexed）。逆康托展开：由排名恢复排列。时间复杂度均为 $O(n^2)$ （用树状数组可优化到 $O(n \log n)$ ）。

```

// 康托展开：求排列 p 在所有 n! 个排列中的排名（0-indexed）
// p 是 {0, 1, ..., n-1} 的排列
// 接口：cantor(p)：返回排列 p 的 0-based Cantor 排名；返回类型 int。
// 参数：待计算排名的 0-based 排列 p。
int cantor(vector<int>& p) {
    // 变量：n=容器 p 的元素数量 n。
    int n = p.size();
    // 变量：rank=当前排列的 0-based Cantor 排名 rank。
    int rank = 0;
    // 变量：fact=阶乘表 fact; fact[i]=i!。
    vector<int> fact(n, 1);
    // 变量：i=Cantor 排名使用的阶乘表下标 i。
    for (int i = 1; i < n; i++) fact[i] = fact[i-1] * i;
    // 变量：i=Cantor 排名中当前计算较小后继数量的排列位置 i。
    for (int i = 0; i < n; i++) {
        int cnt = 0; // 统计 p[i] 后面比 p[i] 小的个数
        // 变量：j=Cantor 排名中位于 i 之后、用于统计较小元素的位置 j。
        for (int j = i + 1; j < n; j++)
            if (p[j] < p[i]) cnt++;
        rank += cnt * fact[n - 1 - i];
    }
    return rank;
}

```

```

}

// 逆康托展开：由排名 rank 恢复排列 (0-indexed)
// 返回 {0, 1, ..., n-1} 的第 rank 个排列
// 接口：inv_cantor(rank, n)：由 0-based 排名还原 1..n 的排列；返回类型 vector<int>。
// 参数：待还原的 0-based Cantor 排名 rank；待还原排列的长度 n。
vector<int> inv_cantor(int rank, int n) {
    // 变量：fact=阶乘表 fact; fact[i]=i!。
    vector<int> fact(n, 1);
    // 变量：i=Cantor 排名使用的阶乘表下标 i。
    for (int i = 1; i < n; i++) fact[i] = fact[i-1] * i;
    vector<int> avail; // 可用元素
    // 变量：i=逆 Cantor 展开初始化可用值集合的元素值 i。
    for (int i = 0; i < n; i++) avail.push_back(i);
    // 变量：res=逆 Cantor 展开逐位构造的排列 res。
    vector<int> res;
    // 变量：i=逆 Cantor 展开中当前确定输出值的排列位置 i。
    for (int i = 0; i < n; i++) {
        // 变量：idx=当前访问容器的 0-based 下标 idx。
        int idx = rank / fact[n - 1 - i];
        rank %= fact[n - 1 - i];
        res.push_back(avail[idx]);
        avail.erase(avail.begin() + idx);
    }
    return res;
}

```

- 如果排列是 $\{1, 2, \dots, n\}$ (1-indexed)，只需将输入减 1 再调用，或将 avail 改为 $1 \sim n$ 。
- 用树状数组替代内层循环可以优化到 $O(n \log n)$ (统计前缀中比当前小的个数)。
- 康托展开在排列哈希、状态压缩 (如 8-puzzle) 中非常有用。
- $n \leq 20$ 时 $n!$ 超过 long long 范围，需要用高精度或只处理小规模。 $n \leq 12$ 时 int 足够， $n \leq 20$ 用 long long。

3.7.6 字符串处理函数

```

// <string> 头文件
s.substr(pos, len);           // 子串
s.find("abc");                 // 查找子串位置，找不到返回 string::npos
s.rfind("abc");               // 从右往左找
s.find_first_of("aeiou");      // 找第一个属于给定字符集的字符
s.find_last_of("aeiou");       // 找最后一个属于给定字符集的字符

```

```

s.find_first_not_of("01");    // 找第一个不在给定字符集中的字符
s.replace(pos, len, "new");   // 替换子串
s.insert(pos, "abc");         // 在 pos 处插入
s.erase(pos, len);           // 删除从 pos 开始的 len 个字符
s.starts_with("pre");         // 是否以 "pre" 开头 (C++20)
s.ends_with("suf");          // 是否以 "suf" 结尾 (C++20)
s.contains("sub");            // 是否包含子串 (C++23)

// 类型转换
to_string(123);               // int -> string
to_string(3.14);              // double -> string
stoi("123");                  // string -> int
stoll("123456789012");       // string -> long long
stod("3.14");                 // string -> double

// <cctype> 字符分类 (传入 char)
isalpha(c);   // 是否是字母
isdigit(c);   // 是否是数字
isalnum(c);   // 是否是字母或数字
isupper(c);   // 是否是大写字母
islower(c);   // 是否是小写字母
isspace(c);   // 是否是空白字符
toupper(c);   // 转大写
tolower(c);   // 转小写

// 整行读入
getline(cin, s); // 读一整行 (注意前面 cin>>n 后要 cin.ignore())

// 字符串流 (<sstream>)
stringstream ss("1 2 3");
// 变量: a=字符串解析得到的第一个整数 a; b=字符串解析得到的第二个整数 b; c=字符串解析得到的
//       第三个整数 c。
int a, b, c;
ss >> a >> b >> c; // 按空格分割解析

```

- `s.find()` 找不到时返回 `string::npos` (一个极大的无符号数), 不是 `-1`! 判断时用 `if (s.find(x) != string::npos)`。
- `stoi` 对超出 `int` 范围的字符串会抛异常, 大数用 `stoll`。
- `getline` 前如果用了 `cin >>`, 必须先 `cin.ignore()` 消耗掉残留的换行符, 否则读到空行。

3.7.7 其他实用工具

```
// 二分答案的 lambda 写法
// 接口: check(mid): 示范二分答案判定接口; 调用者必须把函数体替换为题目的 mid 可行性条件。
// 参数: 当前待判定的二分答案 mid。
auto check = [&](int mid) -> bool {
    // 判断 mid 是否可行
    return true;
};

// 自定义哈希 (pair 做 key)
struct pair_hash {
    // 接口: operator(): 返回键值的 size_t 哈希值, 供 unordered_map/unordered_set 去重;
    // 返回类型 size_t。
    size_t operator()(const pair<int,int>& p) const {
        return hash<long long>()(((long long)p.first << 32) | p.second);
    }
};

// 变量: mp=使用 pair_hash 把整数对键映射到 int 值的哈希表 mp。
unordered_map<pair<int,int>, int, pair_hash> mp;
// 变量: st=用于按 pair 键去重的无序集合 st。
unordered_set<pair<int,int>, pair_hash> st;

// 离散化
vector<int> vals(a.begin(), a.end()); // 复制原数组
sort(vals.begin(), vals.end());
vals.erase(unique(vals.begin(), vals.end()), vals.end());
// 映射: 原值 -> 离散化后的编号
// 变量: i=坐标离散化时当前回写压缩编号的数组下标 i。
for (int i = 0; i < n; i++)
    a[i] = lower_bound(vals.begin(), vals.end(), a[i]) - vals.begin();
// 反映射: vals[a[i]] 即为原值

// iota 快速填充连续序列
// 变量: idx=当前访问容器的 0-based 下标 idx。
vector<int> idx(n);
iota(idx.begin(), idx.end(), 0); // idx = {0, 1, 2, ..., n-1}
// 按其他数组的值排序下标 (间接排序)
// 接口: 比较器 lambda(i, j): 按 a[i]、a[j] 的值升序排列下标。
// 参数: 第一个候选下标 i; 第二个候选下标 j。
sort(idx.begin(), idx.end(), [&](int i, int j) {
    return a[i] < a[j];
});
```

```
// merge 合并两个有序序列
merge(a.begin(), a.end(), b.begin(), b.end(), c.begin());

// set_intersection / set_union / set_difference (要求有序)
// 变量: res=当前筛选操作收集的整数结果 res。
vector<int> res;
set_intersection(a.begin(), a.end(), b.begin(), b.end(), back_inserter(res));
// 类似地: set_union, set_difference, set_symmetric_difference
```

- 离散化是竞赛高频操作，用于将大值域压缩到 $[0, n)$ ，配合树状数组/线段树使用。
- 间接排序（按下标排序）可以保留原数组不变，同时知道排序后每个元素的原始位置。
- `merge` 和集合运算函数要求输入有序，输出也有序。

4 位运算

4.1 基本性质

- 与 $\&$: 同 1 为 1。交换律、结合律。 $a \& 0 = 0$, $a \& a = a$, $a \& (\sim a) = 0$ 。
- 或 $|$: 有 1 为 1。交换律、结合律。 $a | 0 = a$, $a | a = a$, $a | (\sim a) = -1$ (全 1)。
- 异或 $\oplus$: 不同为 1。交换律、结合律。 $a \oplus 0 = a$, $a \oplus a = 0$, $a \oplus (\sim a) = -1$ 。
- 取反 $\sim$: $\sim a = -a - 1$ (补码性质)。

4.1.1 分配律与吸收律

- 与对或的分配律: $a \& (b | c) = (a \& b) | (a \& c)$
- 或对与的分配律: $a | (b \& c) = (a | b) \& (a | c)$
- 异或对与的分配律: $a \& (b \oplus c) = (a \& b) \oplus (a \& c)$
- 吸收律: $a \& (a | b) = a$, $a | (a \& b) = a$
- 德摩根律: $\sim (a \& b) = (\sim a) | (\sim b)$, $\sim (a | b) = (\sim a) \& (\sim b)$

4.1.2 异或的重要性质

- 自逆性: $a \oplus b \oplus b = a$ (异或同一个数两次等于没异或, 用于加密/交换)。
- 无进位加法: 异或等价于不考虑进位的二进制加法。
- 前缀异或: $a_l \oplus a_{l+1} \oplus \dots \oplus a_r = \text{pre}[r] \oplus \text{pre}[l-1]$, 类比前缀和。
- 交换两数 (不用临时变量): $a \hat{=} b$; $b \hat{=} a$; $a \hat{=} b$; (但实际竞赛中直接用 swap 更安全)。
- 线性基: n 个数的所有异或组合可以用至多 $\log V$ 个基底表示。

4.1.3 $O(1)$ 求连续自然数的异或和

求 $1 \oplus 2 \oplus \dots \oplus m$ 是一个经典结论。令 $S(m) = \bigoplus_{i=1}^m i$ ，则结果只取决于 $m \bmod 4$ ：

- $m \bmod 4 = 0$ 时， $S(m) = m$
- $m \bmod 4 = 1$ 时， $S(m) = 1$
- $m \bmod 4 = 2$ 时， $S(m) = m + 1$
- $m \bmod 4 = 3$ 时， $S(m) = 0$

为什么有这个结论：核心引理是每 4 个对齐到 4 的倍数的连续整数，异或起来等于 0。即对任意 $k \equiv 0 \pmod{4}$ ：

$$k \oplus (k+1) \oplus (k+2) \oplus (k+3) = 0.$$

原因：这 4 个数的低 2 位恰好取遍 00, 01, 10, 11，异或为 0；而高位（第 2 位及以上）四个数完全相同，相同的数异或偶数次也为 0。所以整块异或为 0。

把 $S(m)$ 写成 $0 \oplus 1 \oplus \dots \oplus m$ （多异或一个 0 不影响结果），再从 0 开始每 4 个分一组：(0 1 2 3)(4 5 6 7)⋯，每个完整块都异或为 0，于是只剩下结尾不满一块的零头：

- $m \equiv 3$ ：恰好若干完整块，全抵消， $S(m) = 0$ 。
- $m \equiv 0$ ：到 $m-1$ ($\equiv 3$) 为完整块抵消，再异或上多出来的 m ， $S(m) = m$ 。
- $m \equiv 1$ ： $S(m) = S(m-1) \oplus m = (m-1) \oplus m$ 。此时 m 为奇数， $m-1$ 与 m 只差最低位，故 $(m-1) \oplus m = 1$ 。
- $m \equiv 2$ ： $S(m) = S(m-1) \oplus m = 1 \oplus m$ 。此时 m 为偶数（最低位为 0），故 $1 \oplus m = m+1$ 。

推论（区间异或和）：利用前缀异或，任意区间 $\bigoplus_{i=l}^r i = S(r) \oplus S(l-1)$ ，同样 $O(1)$ 。

```
// 1 ^ 2 ^ ... ^ m, O(1)
// 接口：xor_n(m)：返回整数闭区间 [1,m] 的异或和；返回类型 int。
// 参数：异或前缀的右端点 m。
int xor_n(int m) {
    switch (m & 3) {          // m & 3 等价于 m % 4
        case 0: return m;
        case 1: return 1;
        case 2: return m + 1;
        default: return 0;    // case 3
    }
}

// 区间异或和 1 ^ (l+1) ^ ... ^ r, O(1)
// 接口：xor_range(l, r)：返回闭区间 [l,r] 内所有整数的异或和；返回类型 int。
// 参数：闭区间左端点 l；闭区间右端点 r。
int xor_range(int l, int r) {
    return xor_n(r) ^ xor_n(l - 1);
}
```

```
}
```

4.1.4 与/或的单调性

- 与运算越与越小： $a \& b \leq \min(a, b)$ 。连续与一串数，结果单调不增。
- 或运算越或越大： $a | b \geq \max(a, b)$ 。连续或一串数，结果单调不减。
- 推论：子数组的 AND 值最多有 $O(\log V)$ 种（每次变化至少少一个 1）；OR 同理。
- 应用：枚举子数组的 AND/OR 值时，可以利用单调性 + 双指针/二分优化。

4.1.5 常用恒等式速查

- $a \& (a - 1)$ ：去掉 a 最低位的 1。应用：统计 1 的个数、判断 2 的幂。
- $a \& (-a)$ ：取出 a 最低位的 1 (lowbit)。应用：树状数组。
- $a | (a + 1)$ ：将 a 最低位的 0 置为 1。
- $a \& (a + 1)$ ：将 a 末尾连续的 1 全部清零。
- $a | (a - 1)$ ：将 a 末尾连续的 0 全部置 1。
- $(a \oplus b) | (a \& b) = a | b$
- $(a \oplus b) \& (a \& b) = 0$ （异或和与互斥）
- $a + b = (a \oplus b) + 2 \cdot (a \& b)$ （加法 = 无进位和 + 进位）
- $a + b = (a | b) + (a \& b)$ （另一个加法分解）

4.2 竞赛常用性质科普

4.2.1 按位独立性 (Bitwise Independence)

位运算中，AND / OR / XOR 对每一位的运算是独立的——第 i 位的结果只取决于所有操作数的第 i 位。

竞赛意义：遇到“对一组数做位运算后求最优值”的题目，可以逐位贪心或逐位 DP，把一个 $\log V$ 位的问题拆成 $\log V$ 个独立的 0/1 问题。

典型应用：

- 求“选若干数使 OR 最大”——从高位到低位贪心，每一位独立决策。
- 求“所有子数组 AND 之和”——对每一位分别统计有多少子数组该位为 1。
- 位运算卷积（AND/OR/XOR 卷积）的正确性也依赖于按位独立性。

4.2.2 异或与线性代数

异或运算构成 GF(2)（二元域）上的加法，AND 构成乘法。因此：

- n 个数的所有异或组合构成一个线性空间，维度 $\leq \log V$ 。
- 线性基（Gaussian Elimination on GF(2)）可以在 $O(n \log V)$ 时间内求出基底。
- 线性基支持：求最大/最小异或值、判断某个值能否被异或出、求第 k 小异或值。
- 异或方程组可以用高斯消元求解（开关灯问题、翻转问题）。

4.2.3 位运算与集合论的对应

将整数看作集合（第 i 位为 1 表示元素 i 在集合中），位运算对应集合运算：

- $a \& b = A \cap B$ （交集）
- $a | b = A \cup B$ （并集）
- $a \oplus b = A \triangle B$ （对称差）
- $\sim a = \overline{A}$ （补集）
- $a \& (\sim b) = A \setminus B$ （差集）
- $(a \& b) == a$ 等价于 $A \subseteq B$ （子集判断）

竞赛意义：状压 DP 中，集合操作全部用位运算实现，理解这层对应关系可以快速写出状态转移。

4.2.4 popcount 的性质与应用

$\text{popcount}(x)$ 表示 x 二进制中 1 的个数。常用性质：

- $\text{popcount}(a \oplus b) = a$ 和 b 不同位的个数（汉明距离）。
- $\text{popcount}(a \& b) = a$ 和 b 同时为 1 的位数。
- $\text{popcount}(a | b) = \text{popcount}(a) + \text{popcount}(b) - \text{popcount}(a \& b)$ （容斥）。
- $a + b = 2 \cdot (a \& b) + (a \oplus b)$ ，因此 $\text{popcount}(a) + \text{popcount}(b) = \text{popcount}(a \oplus b) + 2 \cdot \text{popcount}(a \& b)$ 。

竞赛应用：

- 状压 DP 中按 popcount 分层枚举（先枚举选 1 个的状态，再选 2 个……）。
- 判断两个状态的“距离”用汉明距离。
- 用 `__builtin_popcountll(x)` 获取， $O(1)$ 。

4.2.5 高位/低位操作

- 最高位位置： $\lfloor \log_2 x \rfloor = 63 - \text{__builtin_clzll}(x)$ ($x > 0$)。
- 最低位位置： $\text{__builtin_ctzll}(x)$ ($x > 0$)。
- 只保留最高位的 1： $1LL \ll (63 - \text{__builtin_clzll}(x))$ 。
- 只保留最低位的 1： $x \& (-x)$ (lowbit)。
- 将最高位以下全部填 1：先求最高位位置 k ，然后 $(1LL \ll (k+1)) - 1$ 。

竞赛意义：

- 贪心从高位到低位确定答案时，需要快速定位最高位。
- 树状数组的核心操作就是 lowbit。
- 求“不超过 x 的最大 2 的幂”等问题直接用最高位操作。

4.2.6 位运算与不等式

- $a \& b \leq \min(a, b)$ ， $a | b \geq \max(a, b)$ 。
- $a \oplus b \leq a + b$ （异或不会产生进位，加法会）。
- $a + b = (a | b) + (a \& b) \geq a | b$ 。
- $(a \oplus b) + (a \& b) \cdot 2 = a + b$ 。
- 若 $a \& b = 0$ （无公共位），则 $a + b = a | b = a \oplus b$ 。

竞赛意义：

- ”选两个数使 AND 最大”——从高位贪心，尽量让高位同时为 1。
- ”选两个数使 XOR 最大”——用异或字典树从高位贪心取不同位。
- $a \& b = 0$ 时三种运算等价，常用于状压 DP 中”两个集合不相交”的条件。

4.2.7 SOS DP（子集和 / 超集和）

给定数组 $f[mask]$ ，求所有子集的和： $g[mask] = \sum_{sub \subseteq mask} f[sub]$ 。

暴力枚举子集是 $O(3^n)$ ，SOS DP 可以做到 $O(n \cdot 2^n)$ ：

// 数值域： $0 \leq n < 31$ ，且每个子集和都必须可由 f 的元素类型表示；否则把 f 改为 `__int128`。
 // 变量： i =SOS DP 当前向所有掩码传播贡献的二进制位 i 。

```
for (int i = 0; i < n; i++)
    // 变量：mask=用二进制位编码的当前状态 mask。
    for (int mask = 0; mask < (1 << n); mask++)
        if (mask >> i & 1)
            f[mask] += f[mask ^ (1 << i)];
```

类似地，超集和 ($g[mask] = \sum_{mask \subseteq sup} f[sup]$) 只需把条件改为 `if (!(mask >> i & 1))`。

竞赛应用：

- 求”有多少对 (i, j) 满足 $a_i \& a_j = a_i$ （即 a_i 是 a_j 的子集）”。
- 位运算卷积（AND/OR 卷积）的核心变换。
- 容斥原理在位运算上的高效实现。

4.3 常用技巧

```
// lowbit: 取出最低位的 1
// 接口: lowbit(x): 返回 x 的二进制最低位 1 所代表的权值  $x \& -x$ ; 返回类型 int。
// 参数: 要求最低位 1 权值的整数 x。
int lowbit(int x) { return x & (-x); }

// 判断是否是 2 的幂
// 接口: is_pow2(x): 判断正整数 x 是否恰好是 2 的幂;  $x > 0$  且二进制中仅有一个 1 时返回 true
// 参数: 待判定是否为 2 的幂的正整数 x。
bool is_pow2(int x) { return x > 0 && (x & (x - 1)) == 0; }

// 枚举子集（枚举 mask 的所有非空子集）
// 变量: sub=当前枚举的子掩码 sub。
for (int sub = mask; sub; sub = (sub - 1) & mask) {
    // sub 是 mask 的一个子集
```

```

}

// 枚举大小为 k 的子集（从 n 个元素中选 k 个）
// 变量: s=从前 k 个二进制位全为 1 开始枚举的子集掩码 s。
int s = (1 << k) - 1;
while (s < (1 << n)) {
    // 处理 s
    // 变量: lb=Gosper 枚举中当前掩码 s 的最低位 1 对应权值 lb。
    int lb = s & (-s);
    // 变量: r=Gosper 枚举中大于 s 的下一个同 popcount 掩码构造量 r。
    int r = s + lb;
    s = ((r ^ s) >> 2) / lb | r;
}

```

- 位运算优先级极低！`if (x & 1 == 1)` 实际是 `x & (1 == 1)`，必须加括号 `if ((x & 1) == 1)`。
- 左移 `1 << i` 当 $i \geq 31$ 时溢出，必须写 `1LL << i`。

4.4 位运算与异或

- `x&-x`：最低位的 1；Fenwick 跳跃依赖此性质。
- `__builtin_popcountll(x)`：1 的个数。
- 枚举子集：`sub=(sub-1)&mask`。
- 异或满足交换律、结合律、自反性；“除一个数外都出现两次”直接全体异或。
- 线性基：维护若干最高位不同的向量，支持最大异或、判断可表示、计数不同异或值；见 08-进阶专题。

4.5 线性基（XOR Basis）

维护最高位不同的向量。插入时从高位向下消元；最大异或值从高位贪心尝试。可扩展为求第 k 小异或、判断异或方程组是否有解、求线性空间维数。

```

struct LinearBasis {
    long long b[61]{}; // b[i] 的最高位是 i；每个主元位只保留一个向量。
    // 接口: insert(x): 把数值 x 消元后插入线性基；线性相关时保持基不变；会修改当前结构；多测时重新构造或显式清空成员。
    // 参数: 待消元并插入线性基的整数 x。
    bool insert(long long x) {
        // 变量: i=线性基插入时从高到低检查的二进制位编号 i。
        for (int i = 60; i >= 0; --i) if (x >> i & 1) {

```

```

        // 从高位消元；如果该位已有主元，就异或消掉这一位。
        if (!b[i]) { b[i] = x; return true; }
        x ^= b[i];
    }
    // 完全消成 0: x 属于已有线性空间，插入不会增加秩。
    return false;
}
// 接口: maximum(x): 返回把 x 与线性基中若干向量异或后可得到的最大值；返回类型 long long。
// 参数: 求最大异或值时使用的初始值 x。
long long maximum(long long x = 0) const {
    // 从高位向低位贪心；异或后变大才保留，保证最高位最优。
    // 变量: i=线性基从高到低贪心检查的二进制位 i。
    for (int i = 60; i >= 0; --i) x = max(x, x ^ b[i]);
    return x;
}
};

```

4.6 线性基：参数、调用与改造

本节紧跟前面的基础模板。先看每个接口传什么，再看哪些不变量不能动，最后看如何替换维护信息或转移。

4.6.1 它解决什么问题

线性基处理的不是普通加法，而是异或意义下的线性空间。典型题面包括：

- 从若干数中选子集，使异或和最大/最小；
- 判断一个数能否由给定数异或得到；
- 在线加入数，查询当前所有数能得到的最大异或值；
- 两个集合的异或空间是否相交，或求异或值的第 k 小。

把每个整数看成二进制向量。异或就是在 $\mathbb{F}_2$ 上相加，所以每一位只有 0/1，消元时只需要“最高位主元”。若数据小于 2^{60} ，开 **LOG=61**；若题目给到 2^{100} ，必须改用 **bitset** 或高精度实现。

4.6.2 可直接使用的模板

```

template<int LOG = 61>
struct LinearBasis {
    using U = unsigned long long;
    U p[LOG]{}; // p[i] 的最高位是 i

```



```

// 接口: clear(): 把全部基向量清零, 使当前 LinearBasis 恢复为空线性基; 无返回值。
void clear() { memset(p, 0, sizeof(p)); }

// 接口: insert(x): 把数值 x 消元后插入线性基; 线性相关时保持基不变; 会修改当前结构; 多测时重新构造或显式清空成员。
// 参数: 待消元并插入当前线性基的整数 x。
void insert(U x) {
    // 变量: i=线性基插入时从高到低检查的二进制位编号 i。
    for (int i = LOG - 1; i >= 0; --i) {
        if ((x >> i) & 1ULL) continue;
        if (!p[i]) {
            p[i] = x;
            return;
        }
        x ^= p[i];
    }
}

// 接口: can(x): 判断 x 是否能由当前线性基中的向量异或得到; x 消元为 0 时返回 true。
// 参数: 待判断能否被线性基表示的数值 x。
bool can(U x) const {
    // 变量: i=判断 x 可表示性时从高到低消元的二进制位编号 i。
    for (int i = LOG - 1; i >= 0; --i) {
        if (!((x >> i) & 1ULL)) continue;
        if (!p[i]) return false;
        x ^= p[i];
    }
    return true;
}

// 接口: max_xor(x): 返回把 x 与当前线性基中若干向量异或后的最大值; 返回类型 U。
// 参数: 求最大异或值时使用的初始值 x。
U max_xor(U x = 0) const {
    // 变量: i=求最大异或值时从高到低贪心检查的二进制位编号 i。
    for (int i = LOG - 1; i >= 0; --i)
        x = max(x, x ^ p[i]);
    return x;
}
};

```

`insert` 是异或版高斯消元; `can` 是消元判定; `max_xor` 从高位到低位贪心, 若异或后变大就保留。这个顺序不能反过来, 否则低位的选择可能破坏高位最优性。

4.6.3 例题：P3812 线性基

题目：P3812 〔模板〕线性基。题意是给出 n 个整数，选择任意子集，使它们的异或和最大。

套用步骤：

1. 每个输入数都代表一个可以选择或不选择的向量；
2. 不需要记录“选了哪些数”，因为目标只依赖最终异或值；
3. 逐个 `insert(x)`；
4. 输入结束后输出 `basis.max_xor()`。

例如输入集合为 1,5,9,4：插入后可以得到 $1^4^8=13$ ，模板从最高位开始尝试，得到最大值 13。完整调用如下：

```
// 接口：main()：读入 n 个整数建立线性基，并输出可取得的最大子集异或值；入口函数；每组测试前按注释清空全局状态。
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    // 变量：n=待插入线性基的整数个数 n。
    int n;
    cin >> n;
    LinearBasis<61> basis;
    while (n--) {
        // 变量：x=当前读入并待插入线性基的整数 x。
        unsigned long long x;
        cin >> x;
        basis.insert(x);
    }
    cout << basis.max_xor() << '\n';
}
```

测试样例：线性基必须按 `void solve()` 包装

下面约定每组输入为 n 个非负整数，输出任意子集异或和的最大值；第一行是测试组数 T 。前两组检查零向量、重复向量和多测重建，后两组检查独立主元以及高位贪心。

输入

```
4
1
0
3
5 5 5
4
1 2 4 8
```

```
5
8 7 3 10 5
```

输出

```
0
5
15
15
```

如果第二组大于 5，通常是消元时把重复向量当成了新主元；如果第三、四组小于 15，优先检查插入方向和 `max_xor` 是否从最高位向下贪心。

练习升级：P4301 新 Nim 游戏。它把“异或空间”放进博弈/删数语境中，看到“剩下的数不能异或出 0”时，应该想到线性基的秩和线性相关，而不是暴力枚举子集。

4.6.4 常见错误

- 把线性基当成普通的异或前缀；它维护的是一组向量，插入顺序不影响空间，但主元排列会影响某些“第 k 小”模板。
- `longlong` 的符号位参与比较时容易出错。纯非负数据优先使用 `unsignedlonglong`。
- 求最大值时要从高位到低位；求最小非零值时通常要先把基化成更适合贪心的形式。
- 线性基只能描述“每个数至多选一次”的异或组合；若某个数可以无限次使用，问题已经不是这个模板。

5 基础算法

5.1 枚举与剪枝

5.1.1 子集枚举

`for(int s=mask; s; s=(s-1)&mask)` 枚举 `mask` 的所有非空子集，总复杂度是 $O(3^n)$ （对所有 `mask` 求子集时）。

```
// 接口: enumerate_submasks(n): 按 mask 从小到大枚举每个 mask 的全部子掩码（含空
// 集）；返回 {mask, sub} 序列。
// 参数: 二进制集合使用的位数 n; 要求  $0 \leq n < 31$ 。
vector<pair<int, int>> enumerate_submasks(int n) {
    // 变量: result=按枚举顺序保存的 {mask, sub} 对。
    vector<pair<int, int>> result;
    // 变量: mask=用二进制位编码的当前状态 mask。
    for (int mask = 0; mask < (1 << n); ++mask) {
        // 变量: sub=当前枚举的子掩码 sub。
        for (int sub = mask; sub = (sub - 1) & mask) {
            result.push_back({mask, sub});
            if (sub == 0) break;
        }
    }
    return result;
}
```

5.1.2 回溯剪枝

固定顺序、提前判非法、记录当前最优上界、相同状态记忆化。对排列/组合先排序再跳过相同元素：`if(i>start&&a[i]==a[i-1])continue;`。

```
// 接口: dfs(pos, sum): 枚举非负数组 a 中互不重复的组合并统计和为 target 的方案；累计和
// 使用 __int128 避免加法溢出。
// 参数: 当前 0-based 位置 pos; 当前精确累计值 sum; 要求 a 已排序且元素非负、answer 的
// 方案计数可由 long long 表示。
void dfs(int pos, __int128 sum) {
```

```

    if (sum == target) { ++answer; return; }
    if (pos == n || sum > target) return;
    // 变量: i=组合 DFS 当前可选择的数组元素下标 i。
    for (int i = pos; i < n; ++i) {
        if (i > pos && a[i] == a[i - 1]) continue;
        dfs(i + 1, sum + a[i]);
    }
}

```

5.1.3 折半搜索 (Meet-in-the-middle)

$n \leq 40$ 、每个元素选/不选且目标为和/异或/最值时，把数组分两半枚举，排序一侧后二分或双指针，复杂度 $O(2^{n/2} \log 2^{n/2})$ 。

```

// 接口: gen(a, l, r, limit): 枚举 a 的半开区间 [l,r) 中不超过 limit 的子集和。
// 参数: 非负数组 a; 合法半开区间 [l,r); 非负上界 limit。超过上界的和在加法前剪枝。
vector<long long> gen(const vector<long long>& a, int l, int r,
                     long long limit) {
    // 变量: res=当前半区枚举得到的全部子集和 res。
    vector<long long> res{0};
    // 变量: i=当前半开区间 [l,r) 内顺序枚举的元素下标 i。
    for (int i = l; i < r; ++i) {
        // 变量: old=加入当前元素 a[i] 生成新子集和之前 res 中已有的元素数量 old。
        int old = (int)res.size();
        // 变量: j=组合生成中复制上一轮已有结果的下标 j。
        for (int j = 0; j < old; ++j)
            if (a[i] <= limit - res[j]) res.push_back(res[j] + a[i]);
    }
    return res;
}

// 接口: mitm_max_subset_sum(a, limit): 返回不超过 limit 的最大子集和; 要求 a 中元素
// 非负且 limit >= 0。
// 参数: 非负整数数组 a; 子集和上界 limit。
long long mitm_max_subset_sum(const vector<long long>& a, long long limit) {
    // 变量: n=输入数组元素个数。
    int n = (int)a.size();
    // 变量: L=左半部分的全部子集和; R=右半部分的全部子集和。
    auto L = gen(a, 0, n / 2, limit), R = gen(a, n / 2, n, limit);
    sort(R.begin(), R.end());
    // 变量: best=目前找到的不超过 limit 的最大子集和。
    long long best = 0;
    // 变量: x=当前左半部分子集和。
    for (long long x : L) {

```

```

    if (x > limit) continue;
    // 变量: it=右半部分中第一个使 x+*it 超过 limit 的位置。
    auto it = upper_bound(R.begin(), R.end(), limit - x);
    if (it != R.begin()) best = max(best, x + *prev(it));
}
return best;
}

```

5.2 前缀和、差分与扫描

5.2.1 一维前缀和

$pre[i+1]=pre[i]+a[i]$, 区间 $[l,r]$ 和为 $pre[r+1]-pre[l]$ 。二维前缀和使用容斥:
 $S(x2,y2)-S(x1-1,y2)-S(x2,y1-1)+S(x1-1,y1-1)$ 。

5.2.2 一维差分

对 $[l,r]$ 加 v : $d[l]+=v; d[r+1]-=v$; 最后累加 d 恢复数组。二维矩形加法在四个角标记, 二维前缀恢复。

```

// 数值域: 要求所有前缀和、区间修改累计量和修改后的 a[i] 均可由 long long 表示。
// 变量: pre=原数组 a 的 1-based 前缀和数组 pre; diff=记录区间增量端点标记的一维差分数组 diff。
vector<long long> pre(n + 1), diff(n + 2);
// 变量: i=正在计算前缀和 pre[i] 的 1-based 数组位置 i。
for (int i = 1; i <= n; ++i) pre[i] = pre[i - 1] + a[i];
// 接口: range_sum(l, r): 返回 1-based 闭区间 [l,r] 的前缀和差。
// 参数: 查询闭区间左端点 l; 查询闭区间右端点 r。
// 变量: range_sum=返回指定区间和的局部 lambda range_sum。
auto range_sum = [&](int l, int r) { return pre[r] - pre[l - 1]; };

// 接口: range_add(l, r, v): 在差分数组中记录 1-based 闭区间 [l,r] 增加 v。
// 参数: 修改闭区间左端点 l; 修改闭区间右端点 r; 区间增加量 v。
auto range_add = [&](int l, int r, long long v) {
    diff[l] += v;
    diff[r + 1] -= v;
};
// 变量: l=差分更新区间左端点 l; r=差分更新区间右端点 r; v=区间增加量 v。
for (auto [l, r, v] : updates) range_add(l, r, v);
// 变量: cur=扫描到当前位置时由差分数组累加得到的区间增量 cur。
long long cur = 0;
// 变量: i=差分数组还原时当前累加并写回的 1-based 位置 i。
for (int i = 1; i <= n; ++i) {

```

```

    cur += diff[i];
    a[i] += cur;
}

```

5.2.3 扫描线思想

把“开始/结束”事件排序，按坐标推进并维护当前集合。矩形面积、区间覆盖、会议室数量都可转成事件；需要动态维护覆盖长度时配合线段树。

```

vector<pair<int, int>> event; // {坐标, +1 开始 / -1 结束}
// 变量: l=当前扫描线区间的左端点 l; r=当前扫描线区间的右端点 r。
for (auto [l, r] : intervals)
    event.push_back({l, 1}), event.push_back({r, -1});
sort(event.begin(), event.end());
// 变量: active=扫描到当前位置时仍活跃的区间数 active; peak=扫描过程中出现过的最大活跃数
// 量 peak。
int active = 0, peak = 0;
// 变量: x=扫描线事件坐标 x; delta=该事件带来的活跃计数增量 delta。
for (auto [x, delta] : event) {
    active += delta;
    peak = max(peak, active);
}

```

5.3 排序、二分、双指针

5.3.1 二分查找

`lower_bound` 找第一个 $\geq x$, `upper_bound` 找第一个 $> x$ 。手写时推荐半开区间 $[l, r)$:

```

// 变量: l=当前区间左端点 l; r=当前区间右端点 r。
int l = 0, r = n;
while (l < r) {
    // 变量: m=当前递归区间 [l,r] 的中点 m。
    int m = l + (r - l) / 2;
    if (a[m] < x) l = m + 1;
    else r = m;
}
// l 是第一个 a[l] >= x 的位置, 可能等于 n

```

5.3.2 二分答案

当答案具有单调可行性 `check(x)` 时，在答案范围二分。先明确求“最小可行”还是“最大可行”，并把 `check` 写成无副作用函数。典型：最小最大分段和、最小速度、最大最小距离、满足容量的最少天数。

```
// 接口: minimize_max_segment_sum(a, k): 把非负数组分成至多 k 个非空连续段; 答案可由
// long long 表示时返回该值, 否则返回 nullopt。
// 参数: 非空非负数组 a; 允许的最大分段数 k, 要求 k>=1。
optional<long long> minimize_max_segment_sum(const vector<long long>& a, int k)
{
    // 接口: check(limit): 判断能否把 a 分成至多 k 段且每段和不超过 limit。
    auto check = [&](__int128 limit) {
        // 变量: groups=当前贪心划出的连续段数。
        int groups = 1;
        // 变量: sum=当前连续段的元素和。
        __int128 sum = 0;
        // 变量: x=当前扫描的非负数组元素。
        for (long long x : a) {
            if (x > limit) return false;
            if (sum + x > limit) sum = x, ++groups;
            else sum += x;
        }
        return groups <= k;
    };
    // 变量: l=最大单元素值, 亦为二分答案下界。
    __int128 l = *max_element(a.begin(), a.end());
    // 变量: r=全部元素之和, 亦为二分答案上界。
    __int128 r = 0;
    // 变量: x=当前累加到宽整数总和中的非负数组元素。
    for (long long x : a) r += x;
    while (l < r) {
        // 变量: mid=当前二分判定的最大允许段和。
        __int128 mid = l + (r - l) / 2;
        if (check(mid)) r = mid;
        else l = mid + 1;
    }
    if (l > numeric_limits<long long>::max()) return nullopt;
    return (long long)l;
}
```


5.3.3 双指针/滑动窗口

窗口右端只增不减，维护“当前窗口合法”不变量；若加入 r 使其非法，就移动 l 直到恢复。适用于正数和、至多/至少 k 个不同值、最长合法子串。含负数的和通常不能直接用双指针。

```
// 变量: cnt=当前计数 cnt。
vector<int> cnt(alphabet);
// 变量: distinct=当前窗口内不同值的数量 distinct; left=滑动窗口左端点 left; answer=
    当前维护的答案 answer。
int distinct = 0, left = 0, answer = 0;
// 变量: right=滑动窗口右端点 right。
for (int right = 0; right < n; ++right) {
    if (cnt[s[right]]++ == 0) ++distinct;
    while (distinct > k) {
        if (--cnt[s[left++]] == 0) --distinct;
    }
    answer = max(answer, right - left + 1);
}
```

5.3.4 三分与凸性

连续/离散单峰函数可三分；整数域更稳妥的做法是三分到很小区间后暴力。很多题可通过凸性改写为二分导数或斜率，不要盲目使用浮点三分。

```
// 数值域: 要求  $l \leq r$ 、 $r-l$  可由 long long 表示且  $r < \text{LLONG\_MAX}$ ； $f(x)$  的返回值也须可由
    long long 表示。
while (r - l > 3) {
    // 变量: m1=整数三分区间内靠左的三等分点 m1。
    long long m1 = l + (r - l) / 3;
    // 变量: m2=整数三分区间内靠右的三等分点 m2。
    long long m2 = r - (r - l) / 3;
    if (f(m1) < f(m2)) l = m1 + 1; // 求最大值
    else r = m2 - 1;
}
// 变量: ans=整数三分结束后在剩余区间暴力检查得到的最大函数值 ans。
long long ans = LLONG_MIN;
// 变量: x=暴力检查凹凸函数值的当前整数横坐标 x。
for (long long x = l; x <= r; ++x) ans = max(ans, f(x));
```

5.4 贪心

5.4.1 典型证明模式

- 区间调度：按右端点升序，选最早结束的可行区间；交换任意最优解的第一个区间即可证明。
- 最小生成树：割性质/环性质。
- Huffman：每次合并最小的两个权值（最优前缀码、合并果子）。
- 任务排序：按截止时间、比值、边际收益排序，必须写出交换论证，不要凭直觉。

```
sort(intervals.begin(), intervals.end(),
    // 接口：匿名 lambda(x, y): 按区间右端点从小到大排序，供不相交区间贪心选择。
    // 参数：第一个候选区间 x; 第二个候选区间 y。
    [](auto &x, auto &y) { return x.second < y.second; });
// 变量：last=上一个被选区间的右端点 last，初值表示尚未选择；chosen=当前已选择的两两不相
// 交区间数量 chosen。
int last = numeric_limits<int>::min(), chosen = 0;
// 变量：l=当前候选区间的左端点 l; r=当前候选区间的右端点 r。
for (auto [l, r] : intervals) {
    if (l >= last) last = r, ++chosen;
}
```

5.4.2 反悔贪心

先按某个顺序加入方案，用堆保存可撤销元素；若当前超限，撤掉代价最大/收益最小者。常见于“最多完成多少任务”“带截止时间的作业选择”。

```
sort(jobs.begin(), jobs.end(),
    // 接口：匿名 lambda(x, y): 按任务截止时间从小到大排序。
    // 参数：第一个候选任务 x; 第二个候选任务 y。
    [](auto &x, auto &y) { return x.deadline < y.deadline; });
// 变量：chosen=维护当前已选任务收益的最小堆 chosen。
priority_queue<int> chosen;
// 变量：job=当前按贪心顺序处理的任务 job。
for (auto job : jobs) {
    chosen.push(job.cost);
    if ((int)chosen.size() > job.deadline) chosen.pop();
}
```

5.4.3 差分约束式贪心

“每个位置至少/至多”“相邻差不超过”等限制有时可从左到右维护最小需求，再从右到左修正；若约束包含图上的环，转为最短路/最长路更可靠。

```
// x[v] <= x[u] + w; 无解等价于存在可达负环
```

```

struct Edge { int v; long long w; }; // v=有向边终点, w=x[v]-x[u] 的 long long
    上界。
// 变量: g=当前有向图的邻接表 g。
vector<vector<Edge>> g(n);
// 变量: d=从指定源点到各顶点的距离数组 d。
vector<__int128> d(n, 0); // 宽整数势函数避免多条 long long 边连续松弛时溢出。
// 变量: inq=顶点当前是否在队列中的标记 inq; path_len=当前最短路所含边数, 用于判负环。
vector<int> inq(n), path_len(n);
// 变量: q=差分约束 SPFA 中保存待松弛顶点的队列 q。
queue<int> q;
// 变量: s=差分约束初始入队的顶点编号 s。
for (int s = 0; s < n; ++s) q.push(s), inq[s] = 1;
// 变量: ok=当前条件是否仍成立 ok。
bool ok = true;
while (!q.empty() && ok) {
    // 变量: u=差分约束 SPFA 当前出队并准备松弛出边的顶点编号 u。
    int u = q.front(); q.pop(); inq[u] = 0;
    // 变量: v=当前边的终点 v; w=当前边权 w。
    for (auto [v, w] : g[u]) if (d[v] > d[u] + (__int128)w) {
        d[v] = d[u] + (__int128)w;
        path_len[v] = path_len[u] + 1;
        if (path_len[v] >= n) { ok = false; break; }
        if (!inq[v]) q.push(v), inq[v] = 1;
    }
}
}

```

5.5 单调栈与单调队列

5.5.1 单调栈

维护栈内下标对应值单调。遇到更小/更大元素时弹栈, 弹出元素的右侧第一个更小/更大位置被确定。应用: 下一个更大元素、柱状图最大矩形、最大子矩形、贡献法统计子数组最小值。

```

// 变量: nextGreater=每个位置右侧第一个更大元素下标 nextGreater; st=维护下标且对应值单
    调递减的栈 st。
vector<int> nextGreater(n, -1), st;
// 变量: i=单调栈正在处理并确定右侧更大元素的数组位置 i。
for (int i = 0; i < n; ++i) {
    while (!st.empty() && a[st.back()] < a[i])
        nextGreater[st.back()] = i, st.pop_back();
    st.push_back(i);
}

```

5.5.2 单调队列

队首始终是窗口最优值，队尾删除不可能成为最优的元素。应用：滑动窗口最值、DP 转移 $dp[i] = \min(dp[j]) + w(i)$ 、有长度限制的最短/最长子数组。

```
// 变量: q=按数组值单调递减维护候选下标的双端队列 q。
deque<int> q;
// 变量: i=滑动窗口右端新加入单调队列的数组位置 i。
for (int i = 0; i < n; ++i) {
    while (!q.empty() && q.front() <= i - k) q.pop_front();
    while (!q.empty() && a[q.back()] <= a[i]) q.pop_back();
    q.push_back(i);
    if (i >= k - 1) answer.push_back(a[q.front()]);
}
```

5.6 离散化与坐标压缩

收集所有会出现的坐标，排序去重，用 `lower_bound` 映射。若区间端点表示连续长度，通常还要把 x 与 $x+1$ 或相邻端点间距保留，避免把长区间压成一个点。值域大、操作数小的线段树/树状数组必须先离散化。

```
// 变量: xs=离散化后排序去重的坐标 xs。
vector<long long> xs = coordinates;
sort(xs.begin(), xs.end());
xs.erase(unique(xs.begin(), xs.end()), xs.end());
// 接口: id(x): 返回原坐标 x 在有序离散数组 xs 中的 0-based 下标。
// 参数: 待离散化的原坐标 x。
auto id = [&](long long x) {
    return int(lower_bound(xs.begin(), xs.end(), x) - xs.begin()) + 1;
};
```

5.7 常见构造套路

- 先处理奇偶、余数、最大值/最小值等不变量。
- 需要构造排列时，优先考虑循环移位、两端放置、按奇偶分组。
- 证明“无解”时找模数、颜色、度数、连通性或总和不变量。
- 证明“总能构造”时给出明确操作顺序，并说明每一步都保持前置条件。

```
// 常见构造: 把 1..n 按奇偶分组输出
// 变量: ans=按奇数在前、偶数在后生成的构造序列 ans。
vector<int> ans;
// 变量: x=当前追加到构造序列的奇数 x。
```

```
for (int x = 1; x <= n; x += 2) ans.push_back(x);
// 变量: x=当前追加到构造序列的偶数 x。
for (int x = 2; x <= n; x += 2) ans.push_back(x);
```

5.8 随机化与哈希技巧

- 随机 `shuffle` + 快速选择求期望线性第 k 小。
- 64 位随机哈希表示集合，支持多重集合差异的高概率比较；严谨题目用排序/Trie/计数。
- Treap 随机优先级避免退化；不要把 `rand()` 作为高质量随机源，使用 `mt19937_64`。

```
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
shuffle(a.begin(), a.end(), rng);
nth_element(a.begin(), a.begin() + k, a.end());
// 变量: answer=当前维护的答案 answer。
int answer = a[k];
uint64_t random_hash = rng();
```

5.9 位集优化

`std::bitset` 将 64/机器字并行，适合传递闭包（布尔矩阵）、集合交并、子集计数；`bitset` 大小需编译期常量，动态场景使用 `vector<unsignedlonglong>`。

```
// 变量: N=当前代码允许使用的最大下标上界 N。
const int N = 2000;
// 接口: bitset_closure(reach, n): 原地计算 reach 前 n 行表示的传递闭包。
// 参数: reach 初始含自环和原图边; n 为实际顶点数且 n<=N。
void bitset_closure(array<bitset<N>, N>& reach, int n) {
    // 变量: k=当前允许作为中转点的顶点 k。
    for (int k = 0; k < n; ++k)
        // 变量: i=当前起点 i。
        for (int i = 0; i < n; ++i)
            if (reach[i][k]) reach[i] |= reach[k];
}
```

5.10 快速小技巧

- `std::gcd`、`std::lcm` (C++17)；后者仅在结果可由返回类型表示时使用。
- `__builtin_ctzll` 只对非零数调用。
- 排序后用 `unique` 去重，差异数组用 `adjacent_difference`。
- 大量模乘、叉积、斜率比较时先判断上界，必要时统一 `__int128`。

```

// 接口: checked_lcm(a, b): 非负最小公倍数可由 long long 表示时返回该值, 否则返回
//      nullopt; 0 与任意数返回 0。
// 参数: 任意 long long 整数 a、b, 包括 LLONG_MIN 与 LLONG_MAX。
auto checked_lcm = [](long long a, long long b) -> optional<long long> {
    if (a == 0 || b == 0) return 0;
    // 先提升再取绝对值, 避免对 LLONG_MIN 求负产生未定义行为。
    // 变量: x、y=提升到宽整数域后的两个操作数。
    __int128 x = a, y = b;
    if (x < 0) x = -x;
    if (y < 0) y = -y;
    // 变量: u、v=欧几里得算法的当前两项; g=绝对值最大公约数; value=精确最小公倍数。
    __int128 u = x, v = y;
    while (v != 0) {
        // 变量: r=本轮欧几里得算法的余数。
        __int128 r = u % v; u = v; v = r;
    }
    // 变量: g=绝对值最大公约数; value=宽整数域中的精确最小公倍数。
    __int128 g = u, value = x / g * y;
    if (value > numeric_limits<long long>::max()) return nullopt;
    return (long long)value;
};
// 变量: l=a 与 b 的非负最小公倍数; 溢出时为 nullopt。
optional<long long> l = checked_lcm(a, b);
sort(v.begin(), v.end());
v.erase(unique(v.begin(), v.end()), v.end());
long long lowbit = x & -x; // x != 0
// 变量: safe_product=用 __int128 计算并取模后安全转回 long long 的乘积
//      safe_product。
long long safe_product = (long long)((__int128)x * y % MOD);

```

6 数学

6.1 模运算与快速幂

对质数/任意模数都成立： $(a+b)\%p$ 、 $(a-b+p)\%p$ 、 $a \times b \% p$ 。除法不能直接取模，必须证明逆元存在。

// 接口: `qpow(a, e, mod)`: 计算 $a^e \bmod \text{mod}$; 返回类型 `long long`。

// 参数: 待求模幂的底数 `a`; 非负指数 `e`; 正模数 `mod`。

```
long long qpow(long long a, long long e, long long mod) {
    a %= mod;
    if (a < 0) a += mod;
    // 变量: r=快速幂当前累计答案 r。
    long long r = 1 % mod;
    while (e) {
        if (e & 1) r = (__int128)r * a % mod;
        a = (__int128)a * a % mod; e >>= 1;
    }
    return r;
}
```

6.2 最大公约数、扩展欧几里得与同余

// 接口: `exgcd(a, b, x, y)`: 求 $\text{gcd}(a,b)$, 并通过 `x,y` 返回 $ax+by=\text{gcd}(a,b)$ 的一组系数; 返回类型 `long long`。

// 参数: 扩展欧几里得的第一个整数输入 `a`; 扩展欧几里得的第二个整数输入 `b`; 输出引用 `x`, 满足 $a \times x + b \times y = \text{gcd}(a,b)$; 输出引用 `y`, 满足 $a \times x + b \times y = \text{gcd}(a,b)$ 。

```
long long exgcd(long long a, long long b, long long &x, long long &y) {
    if (!b) { x = 1; y = 0; return a; }
    // 变量: x1=递归子问题返回的 Bézout 系数 x1; y1=递归子问题返回的 Bézout 系数 y1;
    // g=递归返回的 gcd(a,b) 值 g。
    long long x1, y1, g = exgcd(b, a % b, x1, y1);
    x = y1; y = x1 - (a / b) * y1; return g;
}
```

解 $ax \equiv b \pmod{m}$: 令 $g = \gcd(a, m)$, 若 $b \% g \neq 0$ 无解; 否则约去 g , 用扩展欧几里得求一个解, 通解模 m/g 。

中国剩余定理 (互质模数): $x = \sum a_i \cdot M_i \cdot \text{inv}(M_i \bmod m_i) \bmod M$ 。非互质模数用“合并两个同余方程”: 检查差值能否被 $\gcd$ 整除, 再扩展欧几里得求增量。

6.3 质数与筛法

6.3.1 试除与质因数分解

试除到 $i \cdot i \leq n$, 每次除尽; 单个 $n \leq 1e12$ 适用。若只需判质数, 复杂度 $O(\sqrt{n})$ 。

```
// 接口: factor(n): 试除分解 n, 返回 (质因子, 指数); 返回类型 vector<pair<long long, int>>。
// 参数: 待试除分解的正整数 n。
vector<pair<long long, int>> factor(long long n) {
    // 变量: res=按试除顺序收集的质因子及指数列表 res。
    vector<pair<long long, int>> res;
    // 变量: p=当前试除的候选质因子 p。
    for (long long p = 2; p * p <= n; ++p) {
        if (n % p) continue;
        // 变量: e=当前质因子 p 在原数中的指数 e。
        int e = 0;
        while (n % p == 0) n /= p, ++e;
        res.push_back({p, e});
    }
    if (n > 1) res.push_back({n, 1});
    return res;
}
```

6.3.2 埃氏筛与线性筛

埃氏筛复杂度 $O(n \log \log n)$; 线性筛保证每个合数只被最小质因子筛一次, 复杂度 $O(n)$, 可同时求 ϕ 、莫比乌斯等积性函数。

```
// N: 数组长度; 本模板筛 [1, N), 所以最大可查询 N-1。
vector<int> primes; // primes: 目前找到的全部质数, 严格递增。
vector<int> lp(N); // lp[x]: x 的最小质因子; 0 表示 x 还没被筛到。
vector<int> phi(N); // phi[x]: 1..x 中与 x 互质的整数个数。
phi[1] = 1; // 定义 phi(1)=1。
for (int i = 2; i < N; ++i) { // i: 当前正在确定的整数。
    // lp[i]==0 说明 i 没被更小质数筛过, 因此 i 是质数。
    if (!lp[i]) lp[i] = i, primes.push_back(i), phi[i] = i - 1;
    for (int p : primes) { // p: 从小到大枚举的质数因子。
        if (i * p > N) break;
        lp[i * p] = p;
        if (i % p == 0) phi[i * p] = phi[i] * p;
        else phi[i * p] = phi[i] * (p - 1);
    }
}
```



```

        if (p > lp[i] || 1LL * i * p >= N) break;
        lp[i * p] = p;      // i*p: 本次被筛掉的合数。
        if (p == lp[i]) phi[i * p] = phi[i] * p;
        else phi[i * p] = phi[i] * (p - 1);
    }
}
// 多测：若每组都重新筛，先 primes.clear(), 并把 lp、phi 填 0。

```

6.3.3 约数枚举与整除分块

枚举约数成对出现， $O(\sqrt{n})$ 。对 $\text{floor}(n/i)$ 的求和，商相同的 i 区间为 $[i, n/(n/i)]$ ，可在 $O(\sqrt{n})$ 个块内完成（数论分块）。

```

// 数值域：要求  $1 \leq n \leq \text{LLONG\_MAX}$ ，且  $\sum_{i=1}^n \text{floor}(n/i)$  可由 answer 的类型表示；
// 超出时把 answer 改为 __int128。
// 变量：l=当前区间左端点 l。
for (long long l = 1, r; l <= n; l = r + 1) {
    // 变量：quotient=当前整除分块内恒定的商 quotient=n/l。
    long long quotient = n / l;
    r = n / quotient;
    // [l, r] 内 n / i 都等于 quotient
    answer += (r - l + 1) * quotient;
}

```

6.4 欧拉函数、莫比乌斯与积性函数

$\phi(n)=n\prod(1-1/p)$ 。求单点先分解；求前缀用线性筛。

莫比乌斯函数： $\mu(1)=1$ ，含平方因子为 0，否则质因数个数奇偶决定 ± 1 。常见反演：若 $f(n)=\sum_{d|n} g(d)$ ，则 $g(n)=\sum_{d|n} \mu(d)f(n/d)$ 。

```

// N: 数组长度；合法查询下标为 [1,N]。
vector<int> primes; // primes: 已经找到的全部质数，严格递增。
vector<int> lp(N);  // lp[x]: x 的最小质因子；0 表示尚未筛到。
vector<int> phi(N); // phi[x]: 1..x 中与 x 互质的整数个数。
vector<int> mu(N);  // mu[x]: 莫比乌斯函数值，只可能是 -1、0、1。
phi[1] = mu[1] = 1; // phi(1)=mu(1)=1。
for (int i = 2; i < N; ++i) { // i: 当前正在确定的整数。
    // i 没被筛过，所以 i 是质数；质数只有一个不同质因子，mu=-1。
    if (!lp[i]) lp[i] = i, primes.push_back(i), phi[i] = i - 1, mu[i] = -1;
    for (int p : primes) { // p: 从小到大枚举的质数因子。
        if (1LL * i * p >= N) break;
        lp[i * p] = p;      // i*p: 本次被筛掉的合数。
    }
}

```

```

    if (p == lp[i]) { // i*p 含 p^2, 因此 mu[i*p]=0。
        phi[i * p] = phi[i] * p;
        mu[i * p] = 0;
        break;
    } else { // p 不整除 i: 新增一个不同质因子。
        phi[i * p] = phi[i] * (p - 1);
        mu[i * p] = -mu[i];
    }
}
}
// mu[x]=0: x 含平方因子; 否则 mu[x]=(-1)^(不同质因子个数)。
// 多测: 若每组都重新筛, 先 primes.clear(), 并把 lp/phi/mu 填 0。

```

6.5 组合数与阶乘 (Binomial Coefficients)

6.5.1 常用推式性质

以下默认 $n \geq 0$, 并约定当 $k < 0$ 或 $k > n$ 时 $\binom{n}{k} = 0$ 。有了这个约定, 卷积公式不用反复裁剪求和上下界。

对称、递推与吸收

$$\binom{n}{k} = \binom{n}{n-k}, \quad \binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}.$$

第一式表示“选出 k 个”等价于“排除 $n-k$ 个”; 第二式按某个指定元素选或不选分类。

$$k \binom{n}{k} = n \binom{n-1}{k-1}, \quad (n-k) \binom{n}{k} = n \binom{n-1}{k}.$$

这两条吸收恒等式用来消掉求和项前面的 k 或 $n-k$ 。若枚举相邻的 k , 可用

$$\binom{n}{k+1} = \binom{n}{k} \frac{n-k}{k+1}.$$

整数计算时先乘再除; 模意义下的除法必须乘 $(k+1)^{-1}$, 只有分母在当前模数下可逆时才能直接使用。

嵌套选取可以交换先后顺序:

$$\binom{n}{k} \binom{k}{r} = \binom{n}{r} \binom{n-r}{k-r} \quad (0 \leq r \leq k \leq n).$$

左边先选 k 个再从中标记 r 个; 右边先选被标记的 r 个, 再补齐其余 $k-r$ 个。

二项式展开与带权和

$$\sum_{k=0}^n \binom{n}{k} x^k = (1+x)^n.$$

代入 $x = 1, -1$, 或对 x 求导后再代入 $x = 1$, 可快速得到:

$$\sum_{k=0}^n \binom{n}{k} = 2^n, \quad \sum_{k=0}^n (-1)^k \binom{n}{k} = 0 \quad (n \geq 1),$$

$$\sum_{k=0}^n k \binom{n}{k} = n \cdot 2^{n-1}, \quad \sum_{k=0}^n k(k-1) \binom{n}{k} = n(n-1) \cdot 2^{n-2}.$$

因此 $\sum k^2 \binom{n}{k} = n(n+1) \cdot 2^{n-2}$ ($n \geq 2$)。看到 k^2 时先拆成 $k(k-1) + k$, 通常比直接处理平方更容易。

曲棍球杆、交错前缀与卷积

$$\sum_{i=r}^n \binom{i}{r} = \binom{n+1}{r+1}.$$

这是曲棍球杆恒等式, 适合把“上标连续变化、下标固定”的前缀和直接合并成一个组合数。对应的交错前缀为

$$\sum_{i=0}^m (-1)^i \binom{n}{i} = (-1)^m \binom{n-1}{m} \quad (0 \leq m < n).$$

范德蒙德卷积为

$$\sum_{i \in \mathbb{Z}} \binom{a}{i} \binom{b}{r-i} = \binom{a+b}{r}.$$

含义是从大小分别为 a, b 的两个集合中总共选 r 个, 按第一个集合选了多少个分类。令 $a = b = n, r = n$, 结合对称性可得常用平方和:

$$\sum_{i=0}^n \binom{n}{i}^2 = \binom{2n}{n}.$$

推式时优先尝试三种视角: 按某个元素是否被选分类得到 Pascal 递推; 交换“先选谁、再选谁”的顺序得到吸收或嵌套选取; 把求和看成二项式乘积的某一项系数得到卷积。

6.5.2 质数模且 n 不大

预处理 $\text{fac}[i]$ 、 $\text{ifac}[i]$ ， $C(n,k)=\text{fac}[n]\text{ifac}[k]\text{ifac}[n-k]$ ，预处理 $O(N)$ ，查询 $O(1)$ 。

```
// 变量: MOD=当前取模运算使用的正模数。
const long long MOD = 998244353;
// 变量: fac=阶乘数组 fac; ifac=逆阶乘数组 ifac。
vector<long long> fac(N), ifac(N);
fac[0] = 1;
// 变量: i=阶乘表正在预处理的整数下标 i。
for (int i = 1; i < N; ++i) fac[i] = fac[i - 1] * i % MOD;
ifac[N - 1] = qpow(fac[N - 1], MOD - 2, MOD);
// 变量: i=逆阶乘表自后向前递推的整数下标 i。
for (int i = N - 1; i; --i) ifac[i - 1] = ifac[i] * i % MOD;
// 接口: C(n, k): 返回预处理阶乘下的组合数  $C(n,k) \bmod \text{MOD}$ ，越界时返回 0。
// 参数: 组合数上标 n; 组合数下标 k。
auto C = [&](int n, int k) -> long long {
    if (k < 0 || k > n) return 0;
    return fac[n] * ifac[k] % MOD * ifac[n - k] % MOD;
};
```

6.5.3 卢卡斯定理 (Lucas Theorem)

质数 p 下: $C(n,k) \equiv \prod C(n_i, k_i) \pmod{p}$ ，其中 n_i, k_i 是 p 进制位。适用于 n 很大、 p 较小。

```
// 接口: Csmall(n, k, p): 计算当前质数模数下的小组合数  $C(n,k)$ ；返回类型 long long。
// 参数: 组合数  $C(n,k)$  的上标 n; 组合数  $C(n,k)$  的下标 k; 组合数运算使用的质数模数 p。
long long Csmall(long long n, long long k, long long p) {
    if (k < 0 || k > n) return 0;
    // 变量: ans=逐项乘除计算组合数  $C(n,k)$  的累积结果 ans。
    long long ans = 1;
    // 变量: i=小组合数  $C(n,k)$  逐项乘除时的因子下标 i。
    for (long long i = 1; i <= k; ++i) {
        ans = ans * (n - i + 1) % p;
        ans = ans * qpow(i, p - 2, p) % p;
    }
    return ans;
}
// 接口: lucas(n, k, p): 用 Lucas 定理计算  $C(n,k) \bmod p$ ；返回类型 long long。
// 参数: 组合数  $C(n,k)$  的上标 n; 组合数  $C(n,k)$  的下标 k; Lucas 定理使用的质数模数 p。
long long lucas(long long n, long long k, long long p) {
    if (!k) return 1;
    return Csmall(n % p, k % p, p) * lucas(n / p, k / p, p) % p;
}
```

6.5.4 非质数模

分解模数的质数幂，计算每个质数幂下的阶乘（跳过 p 因子），再用 CRT 合并。实现复杂且容易错，只有题目明确需要时使用。

```
// 合并  $x = a1 \pmod{m1}$ ,  $x = a2 \pmod{m2}$ , 允许模数不互质
// 接口: merge_congruence(a1, m1, a2, m2, a, m): 合并两个同余方程, 结果写入 a,m; 方程相容并完成合并时返回 true。
// 参数: 第一个同余式的余数 a1; 第一个同余式的正模数 m1; 第二个同余式的余数 a2; 第二个同余式的正模数 m2; 合并后余数的输入输出引用 a; 合并后正模数的输入输出引用 m。
bool merge_congruence(long long a1, long long m1,
                      long long a2, long long m2,
                      long long &a, long long &m) {
    // 变量: x=exgcd 返回、满足  $m1 \cdot x + m2 \cdot y = \gcd(m1, m2)$  的 Bézout 系数 x; y=exgcd 返回、满足  $m1 \cdot x + m2 \cdot y = \gcd(m1, m2)$  的 Bézout 系数 y; g=递归返回的 gcd(a,b) 值 g。
    long long x, y, g = exgcd(m1, m2, x, y);
    // 变量: d=两个同余方程余数之差 d=a2-a1。
    long long d = a2 - a1;
    if (d % g) return false;
    // 变量: mod=当前正模数 mod。
    long long mod = m2 / g;
    // 变量: t=CRT 合并时由 Bézout 系数求出的第二个模数倍数 t。
    long long t = (__int128)(d / g) * x % mod;
    if (t < 0) t += mod;
    m = m1 / g * m2;
    a = (a1 + (__int128)m1 * t) % m;
    if (a < 0) a += m;
    return true;
}
```

6.6 矩阵快速幂与线性递推 (Matrix Exponentiation and Linear Recurrence)

线性递推 $f(n) = c_1 f(n-1) + \dots + c_k f(n-k)$ 可转为 $k \times k$ 矩阵快速幂，复杂度 $O(k^3 \log n)$ ；Fibonacci 为 2×2 。矩阵乘法注意模乘溢出。

```
struct Mat {
    int n; // 矩阵阶数; 合法下标均为  $[0, n)$ 。
    vector<vector<long long>> a; // a[i][j]: 第 i 行第 j 列, 已对 MOD 取模。
    // 接口: Mat(n, ident): 构造 n 阶零矩阵; ident=true 时构造单位矩阵; 构造并初始化对象; 每组测试建议重新构造。
};
```

```

// 参数: 矩阵阶数 n; 是否构造单位矩阵 ident。
Mat(int n, bool ident = false): n(n), a(n, vector<long long>(n)) {
    // 变量: i=单位矩阵当前写入对角元的行列下标 i。
    if (ident) for (int i = 0; i < n; ++i) a[i][i] = 1;
}
};

// 接口: operator*(A, B): 返回输入矩阵 A 与 B 的矩阵乘积; 返回类型 Mat。
// 参数: 矩阵乘法的第一个输入矩阵 A; 矩阵乘法的第二个输入矩阵 B。
Mat operator*(const Mat& A, const Mat& B) {
    // 变量: C=矩阵乘法正在累加构造的结果矩阵 C。
    Mat C(A.n);
    // 变量: i=矩阵乘法结果矩阵的行下标 i。
    for (int i = 0; i < A.n; ++i)
        // 变量: k=矩阵乘法中连接行 i 与列 j 的公共维下标 k。
        for (int k = 0; k < A.n; ++k) if (A.a[i][k])
            // 变量: j=矩阵乘法结果矩阵的列下标 j。
            for (int j = 0; j < A.n; ++j)
                C.a[i][j] = (C.a[i][j] + (__int128)A.a[i][k] * B.a[k][j]) %
MOD;
    return C;
}

// 接口: mpow(a, e): 返回矩阵 a 的 e 次幂; 返回类型 Mat。
// 参数: 待求幂的方阵 a; 非负指数 e。
Mat mpow(Mat a, long long e) {
    // 变量: r=当前区间右端点 r。
    Mat r(a.n, true);
    while (e) { if (e & 1) r = r * a; a = a * a; e >>= 1; }
    return r;
}

```

6.7 容斥与组合计数

全集计数减去至少一个坏条件。 $n \leq 20$ 时枚举子集:

$ans = \sum (-1)^{|S|} count(\text{同时满足 } S \text{ 中条件})$ 。

条件独立/按质因子分解时可用容斥; 条件数量大时优先找补集、莫比乌斯反演或 DP。

```

// 数值域: 要求  $0 \leq m < 31$ ; count_satisfying(mask) 返回 long long, 累加改用 __int128
以容纳全部  $2^m$  项。
// 变量: ans=容斥枚举所有非空集合后累计的带符号计数 ans。
__int128 ans = 0;
// 变量: mask=用二进制位编码的当前状态 mask。
for (int mask = 0; mask < (1 << m); ++mask) {
    // 变量: bits=当前值的有效二进制位数 bits。

```

```

int bits = __builtin_popcount((unsigned)mask);
__int128 ways = count_satisfying(mask); // 同时满足 mask 中条件
ans += (bits & 1) ? -ways : ways;
}

```

6.8 概率与期望

- 期望线性： $E[X+Y]=E[X]+E[Y]$ ，即使不独立也成立。
- 终止状态递推： $E[state]=1+\sum p_i E[next_i]$ ；若有自环移项并检查系数非零。
- 取模概率需要模逆；分母与模数不互质时不能直接除。

```

// 接口: expected_dag(u): 返回 DAG 上从指定状态出发的期望值; 返回类型 double。
// 参数: 期望过程的起始状态编号 u。
double expected_dag(int u) {
    if (vis[u]) return E[u];
    vis[u] = true;
    E[u] = 1.0; // 走一步
    // 变量: v=转移到的后继状态编号 v; p=采用该转移的概率 p。
    for (auto [v, p] : transitions[u]) E[u] += p * expected_dag(v);
    return E[u];
}

```

6.9 常见数列与技巧

- 等差/等比求和、前缀异或（周期 4）、 $1 \text{ xor } \dots \text{ xor } n$ 。
- $\text{gcd}(a, b) = \text{gcd}(a, b - a)$; $\text{lcm}(a, b) = a / \text{gcd}(a, b) * b$ 。
- 斐波那契 gcd: $\text{gcd}(F_m, F_n) = F_{\text{gcd}(m, n)}$ （用于识别构造题）。
- 反复应用 $x \rightarrow x / \text{gcd}(x, a)$ 时，质因数分解通常比模拟更快。

```

// 接口: xor_prefix(n): 返回 0 xor 1 xor ... xor n; 返回类型 long long。
// 参数: 异或前缀的非负右端点 n。
long long xor_prefix(long long n) {
    switch (n & 3) {
        case 0: return n;
        case 1: return 1;
        case 2: return n + 1;
        default: return 0;
    }
}

```

```

// 接口: fib(n): 返回 (F_n, F_{n+1}); 返回类型 pair<long long, long long>。

```

```
// 参数: Fibonacci 序列下标 n。
pair<long long, long long> fib(long long n) { // (F_n, F_{n+1})
    if (!n) return {0, 1};
    auto [a, b] = fib(n >> 1);
    // 变量: c=快速倍增公式得到的 F(2k) 值 c。
    long long c = a * (2 * b - a);
    // 变量: d=快速倍增公式得到的 F(2k+1) 值 d。
    long long d = a * a + b * b;
    return (n & 1) ? pair{d, c + d} : pair{c, d};
}
```

6.10 生成函数基础

把数列 a_n 视为形式幂级数 $A(x) = \sum a_n x^n$ 。加法对应逐项相加，乘法对应卷积； $1/(1-x)$ 是全 1 数列， $1/(1-x)^k$ 的系数是 $C(n+k-1, k-1)$ 。小规模直接卷积，规模大时换 NTT。

```
using Poly = vector<long long>;
// 接口: multiply(a, b, MOD): 返回两个多项式的卷积并按本节模数取模; 返回类型 Poly。
// 参数: 卷积的左系数数组 a; 卷积的右系数数组 b; 运算使用的正模数 MOD。
Poly multiply(const Poly& a, const Poly& b, long long MOD) {
    // 变量: c=存放卷积系数的结果多项式 c。
    Poly c(a.size() + b.size() - 1);
    // 变量: i=多项式 a 当前参与卷积的系数下标 i。
    for (int i = 0; i < (int)a.size(); ++i)
        // 变量: j=多项式 b 当前参与卷积的系数下标 j。
        for (int j = 0; j < (int)b.size(); ++j)
            c[i + j] = (c[i + j] + a[i] * b[j]) % MOD;
    return c;
}

// (1 + x + ... + x^m)^k 的系数: 重复卷积或用 NTT 加速
// 接口: ways(base, k, MOD): 返回当前计数模型下的方案数; 返回类型 Poly。
// 参数: 幂运算的底多项式 base; 计数模型中需要选择的元素数 k; 运算使用的正模数 MOD。
Poly ways(Poly base, int k, long long MOD) {
    // 变量: ans=多项式快速幂中当前累计乘积 ans。
    Poly ans{1};
    while (k) {
        if (k & 1) ans = multiply(ans, base, MOD);
        k >>= 1;
        if (k) base = multiply(base, base, MOD);
    }
    return ans;
}
```


6.11 Burnside 引理与 Pólya 计数定理 (Burnside's Lemma and Pólya Enumeration Theorem)

Burnside 引理：轨道数等于所有群作用下不动点数的平均值。对“颜色数为 c 、置换 p 作用于位置”的染色问题，不动点数为 $c^{\text{cycles}(p)}$ ；取模时必须保证模数与群大小互质，代码用扩展欧几里得求一般逆元，不要求模数为质数。

```
// 接口: mod_pow(a, e, mod): 计算  $a^e \bmod p$ ; 返回类型 long long。
// 参数: 待求模幂的底数 a; 非负指数 e; 正模数 mod。
long long mod_pow(long long a, long long e, long long mod) {
    a %= mod;
    if (a < 0) a += mod;
    // 变量: r=快速幂当前累计答案 r。
    long long r = 1 % mod;
    for (; e; e >>= 1, a = (__int128)a * a % mod)
        if (e & 1) r = (__int128)r * a % mod;
    return r;
}

// 接口: inverse_coprime(value, mod): 求 value 在模 mod 下的逆元; 不存在时返回 -1。
// 参数: 待求逆元的整数 value; 正模数 mod。
long long inverse_coprime(long long value, long long mod) {
    // 变量: r0=扩展欧几里得当前余数; r1=下一余数。
    long long r0 = value, r1 = mod;
    // 变量: x0=当前余数对应 value 的系数; x1=下一余数对应 value 的系数。
    __int128 x0 = 1, x1 = 0;
    while (r1) {
        // 变量: q=欧几里得除法的当前商。
        long long q = r0 / r1;
        // 变量: next_r=欧几里得除法得到的下一余数。
        long long next_r = r0 - q * r1;
        // 变量: next_x=下一余数对应 value 的系数。
        __int128 next_x = x0 - (__int128)q * x1;
        r0 = r1; r1 = next_r;
        x0 = x1; x1 = next_x;
    }
    if (r0 != 1) return -1;
    // 变量: result=value 在模 mod 下的标准非负逆元。
    long long result = (long long)(x0 % mod);
    if (result < 0) result += mod;
    return result;
}

// 接口: burnside(group, colors, MOD): 按 Burnside 引理计算本质不同方案数; 返回类型 long long。
// 参数: 非空群作用置换列表 group; 可用颜色数 colors; 正模数 MOD, 须与 group.size()
```

互质。

```
long long burnside(const vector<vector<int>>& group,
                  int colors, long long MOD) {
    // 变量: sum=Burnside 引理对每个群元素固定染色数的累加和 sum。
    long long sum = 0;
    for (const auto& perm : group) {
        // 变量: n=容器 perm 的元素数量 n; cycles=当前置换的环数量 cycles。
        int n = perm.size(), cycles = 0;
        // 变量: vis=分解当前群作用置换时记录位置是否已进入某个环的标记 vis。
        vector<char> vis(n);
        // 变量: i=当前群作用置换中尚未归入置换环的位置 i。
        for (int i = 0; i < n; ++i) if (!vis[i]) {
            ++cycles;
            // 变量: u=沿当前置换环移动的元素编号 u。
            for (int u = i; !vis[u]; u = perm[u]) vis[u] = true;
        }
        sum = (sum + (__int128)mod_pow(colors, cycles, MOD)) % MOD;
    }
    // 变量: inv_group_size=群大小在模 MOD 下的逆元。
    long long inv_group_size = inverse_coprime((long long)group.size(), MOD);
    if (inv_group_size == -1) return -1;
    return (__int128)sum * inv_group_size % MOD;
}
```

6.12 Prüfer 序列与矩阵树定理 (Prüfer Sequence and Matrix-Tree Theorem)

Prüfer 序列长度为 $n-2$ ，每棵标号树与一个序列一一对应；度数满足 $\deg[v]=\text{出现次数}+1$ 。矩阵树定理把拉普拉斯矩阵删去一行一列后的行列式作为生成树数量。

```
// 接口: prufer_encode(g): 把 1..n 编号树编码为长度 n-2 的 Prüfer 序列; 返回类型
//      vector<int>。
// 参数: 1-based 标号树的无向邻接表 g。
vector<int> prufer_encode(const vector<vector<int>>& g) {
    // 变量: n=容器 g 的元素数量 n。
    int n = g.size() - 1;
    // 变量: deg=顶点度数数组 deg; code=Prüfer 序列 code。
    vector<int> deg(n + 1), code;
    // 变量: leaves=当前度数为 1 的叶子最小堆 leaves。
    priority_queue<int, vector<int>, greater<int>> leaves;
    // 变量: u=当前枚举的树顶点编号 u。
    for (int u = 1; u <= n; ++u) {
        deg[u] = g[u].size();
```

```

        if (deg[u] == 1) leaves.push(u);
    }
    // 变量: step=Prüfer 编码当前删除第几个叶子的步骤下标 step。
    for (int step = 0; step < n - 2; ++step) {
        // 变量: leaf=当前取出的最小叶子 leaf。
        int leaf = leaves.top(); leaves.pop();
        // 变量: v=Prüfer 编码中与当前叶子相邻的未删除顶点 v。
        int v = 0;
        // 变量: x=从 g[leaf] 枚举的相邻顶点 x。
        for (int x : g[leaf]) if (deg[x]) { v = x; break; }
        code.push_back(v);
        if (--deg[v] == 1) leaves.push(v);
        deg[leaf] = 0;
    }
    return code;
}

// 接口: prufer_decode(code): 把 Prüfer 序列还原为无向树边集; 返回类型 vector<pair<
// int, int>>。
// 参数: Prüfer 序列 code。
vector<pair<int, int>> prufer_decode(const vector<int>& code) {
    // 变量: n=容器 code 的元素数量 n。
    int n = code.size() + 2;
    // 变量: deg=顶点度数数组 deg。
    vector<int> deg(n + 1, 1);
    // 变量: x=Prüfer 序列当前编码值 x。
    for (int x : code) ++deg[x];
    // 变量: leaves=当前度数为 1 的叶子最小堆 leaves。
    priority_queue<int, vector<int>, greater<int>> leaves;
    // 变量: i=Prüfer 解码初始化时检查度数的顶点编号 i。
    for (int i = 1; i <= n; ++i) if (deg[i] == 1) leaves.push(i);
    // 变量: edges=由 Prüfer 序列逐步还原得到的树边集合 edges。
    vector<pair<int, int>> edges;
    // 变量: x=Prüfer 序列当前编码值 x。
    for (int x : code) {
        // 变量: u=Prüfer 解码当前从叶子最小堆取出的顶点编号 u。
        int u = leaves.top(); leaves.pop(); edges.push_back({u, x});
        if (--deg[x] == 1) leaves.push(x);
    }
    edges.push_back({leaves.top(), leaves.top()});
    // 变量: u=Prüfer 解码结束时剩余的第一个叶子顶点 u; v=Prüfer 解码结束时剩余的第二个
    // 叶子顶点 v。
    int u = leaves.top(); leaves.pop(); int v = leaves.top();
    edges.back() = {u, v};
}

```

```
return edges;
}
```

// 接口: `tree_count(lap, MOD)`: 对删去一行一列的拉普拉斯矩阵求行列式, 返回生成树数量; 返回类型 `long long`。

// 参数: 删去一行一列后的拉普拉斯矩阵 `lap`; 运算使用的正模数 `MOD`。

```
long long tree_count(vector<vector<long long>> lap, long long MOD) {
    // 变量: n=容器 lap 的元素数量 n; det=高斯消元过程中累计的行列式 det。
    int n = lap.size(); long long det = 1;
    // 变量: col=当前高斯消元处理的列 col。
    for (int col = 0; col < n; ++col) {
        // 变量: pivot=当前高斯消元选择的主元行 pivot。
        int pivot = col;
        while (pivot < n && lap[pivot][col] % MOD == 0) ++pivot;
        if (pivot == n) return 0;
        if (pivot != col) swap(lap[pivot], lap[col]), det = (MOD - det) % MOD;
        det = det * (lap[col][col] % MOD + MOD) % MOD;
        // 变量: inv=当前运算使用的逆元 inv。
        long long inv = mod_pow((lap[col][col] % MOD + MOD) % MOD, MOD - 2,
MOD);
        // 变量: i=模高斯消元中在主元行下方寻找非零元的候选行 i。
        for (int i = col + 1; i < n; ++i) {
            // 变量: factor=当前步骤使用的乘数 factor。
            long long factor = lap[i][col] % MOD * inv % MOD;
            // 变量: j=模高斯消元中从主元列 col 开始更新的列下标 j。
            for (int j = col; j < n; ++j)
                lap[i][j] = (lap[i][j] - factor * lap[col][j]) % MOD;
        }
    }
    return (det + MOD) % MOD;
}
// 调用时传入删除一行一列后的拉普拉斯矩阵。
```

6.13 二次剩余与 Tonelli - Shanks 算法 (Quadratic Residues and Tonelli-Shanks Algorithm)

Legendre 符号判断 a 是否为模奇素数 p 的二次剩余; Tonelli-Shanks 在存在平方根时以 $O(\log^2 p)$ 求出一个根。

```
// 接口: legendre(a, p): 返回 Legendre 符号 (a/p): 1 为非零二次剩余, -1 为非剩余, 0
表示 a=0; 返回类型 long long。
// 参数: 待判断二次剩余性的整数 a; Legendre 符号使用的奇质数模数 p。
long long legendre(long long a, long long p) {
```

```

    a %= p; if (a < 0) a += p;
    if (!a) return 0;
    // 变量: x=Euler 判别得到的  $a^{((p-1)/2)} \bmod p$  值 x。
    long long x = mod_pow(a, (p - 1) / 2, p);
    return x == 1 ? 1 : -1;
}

// 接口: tonelli_shanks(n, p): 求  $x^2 \equiv n \pmod{p}$  的一个根, 无解返回 -1; 返回类型
// long long。
// 参数: 需要求模平方根的剩余 n; 需要求平方根的奇质数模数 p。
long long tonelli_shanks(long long n, long long p) {
    n %= p; if (!n) return 0;
    if (p == 2) return n;
    if (legendre(n, p) != 1) return -1;
    if (p % 4 == 3) return mod_pow(n, (p + 1) / 4, p);
    // 变量: q=分解  $p-1=q \times 2^s$  后得到的奇数因子 q; s=分解  $p-1=q \times 2^s$  时因子 2 的指数 s。
    long long q = p - 1, s = 0;
    while (!(q & 1)) q >>= 1, ++s;
    // 变量: z=Tonelli-Shanks 正在试探的最小二次非剩余候选 z。
    long long z = 2;
    while (legendre(z, p) != -1) ++z;
    // 变量: c=Tonelli-Shanks 当前 2 幂阶生成元 c; x=Tonelli-Shanks 当前平方根候选 x。
    long long c = mod_pow(z, q, p), x = mod_pow(n, (q + 1) / 2, p);
    // 变量: t=Tonelli-Shanks 当前误差项 t; m=Tonelli-Shanks 当前误差项的 2 幂阶上界 m。
    long long t = mod_pow(n, q, p), m = s;
    while (t != 1) {
        // 变量: i=Tonelli-Shanks 中寻找  $t^{(2^i)} \equiv 1$  的最小指数 i; tt=当前平方根修正量 t 的平方 tt。
        long long i = 1, tt = t * t % p;
        while (tt != 1) tt = tt * tt % p, ++i;
        // 变量: b=Tonelli-Shanks 本轮把 x 调整到下一阶所乘的修正因子 b。
        long long b = mod_pow(c, 1LL << (m - i - 1), p);
        x = x * b % p; c = b * b % p; t = t * c % p; m = i;
    }
    return x;
}

```

费马小定理: $a^{(p-1)} \equiv 1 \pmod{p}$ (p 质数且 $p \nmid a$), 逆元为 $a^{(p-2)}$ 。欧拉定理推广到 $\gcd(a, m) = 1$: $a^{\phi(m)} \equiv 1 \pmod{m}$ 。

6.14 置换与循环

置换分解成不相交环；重复应用 k 次时每个位置在所属环内移动 $k \bmod \text{len}$ 。求置换逆、循环节、最少交换次数都从环分解出发。

```
// 变量: vis=置换环分解中记录每个位置是否已经处理的标记 vis; result=原置换重复应用 k 次
// 后每个位置的映射结果 result。
vector<int> vis(n), result(n);
// 变量: i=置换环分解中当前尚未访问的位置 i。
for (int i = 0; i < n; ++i) if (!vis[i]) {
    // 变量: cyc=当前置换环按遍历顺序保存的全部元素编号 cyc。
    vector<int> cyc;
    // 变量: u=沿置换 p 当前环移动的元素编号 u。
    for (int u = i; !vis[u]; u = p[u]) vis[u] = 1, cyc.push_back(u);
    // 变量: m=当前置换环的长度 m。
    int m = cyc.size();
    // 变量: j=当前环内下标; 先对环长取模, 避免 j+k 在 k=LLONG_MAX 时溢出; 要求 k>=0。
    for (int j = 0; j < m; ++j) result[cyc[j]] = cyc[(j + k % m) % m];
}
```

6.15 快速傅里叶变换与数论变换（了解）

多项式卷积、计数合并、字符串匹配。NTT 要求模数为 $c \cdot 2^k + 1$ （常用 998244353），每层蝶形乘原根；逆变换乘 n^{-1} 。只在卷积规模大于 $O(n^2)$ 时值得使用。

```
// 变量: NTT_MOD=NTT 使用的质数模数; NTT_G=该模数下的原根。
constexpr int NTT_MOD = 998244353, NTT_G = 3;
// 接口: ntt(a, inv): 对数组 a 做模 998244353 的 NTT; invert=true 时做逆变换; 不返回值。
// 参数: 原地执行正变换或逆变换的系数数组 a; 是否执行逆变换 inv。
void ntt(vector<int>& a, bool inv) {
    // 变量: n=容器 a 的元素数量 n。
    int n = a.size();
    // 变量: i=NTT 位逆序置换中当前待重排的系数下标 i。
    for (int i = 1, j = 0; i < n; ++i) {
        // 变量: bit=NTT 位逆序置换中从最高有效位开始移动的掩码 bit。
        int bit = n >> 1; for (; j & bit; bit >>= 1) j ^= bit; j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    // 变量: len=NTT 当前蝶形合并块的长度 len。
    for (int len = 2; len <= n; len <<= 1) {
        // 变量: wlen=当前 NTT 蝶形层使用的单位根 wlen。
        int wlen = qpow(NTT_G, (NTT_MOD - 1) / len, NTT_MOD);
        if (inv) wlen = qpow(wlen, NTT_MOD - 2, NTT_MOD);
```

```

// 变量: i=NTT 长度为 len 的蝶形块起始下标 i。
for (int i = 0; i < n; i += len) {
    // 变量: w=NTT 当前蝶形位置使用的单位根幂 w。
    long long w = 1;
    // 变量: j=NTT 长度为 len 的蝶形块内偏移下标 j。
    for (int j = 0; j < len / 2; ++j) {
        // 变量: u=NTT 蝶形运算左半位置的旧系数 u; v=NTT 蝶形运算右半位置乘单
        // 位根后的系数 v。
        int u = a[i + j], v = w * a[i + j + len / 2] % NTT_MOD;
        a[i + j] = (u + v) % NTT_MOD;
        a[i + j + len / 2] = (u - v + NTT_MOD) % NTT_MOD;
        w = w * wlen % NTT_MOD;
    }
}
}
if (inv) {
    // 变量: ni=变换长度 n 在模 NTT_MOD 下的逆元。
    long long ni = qpow(n, NTT_MOD - 2, NTT_MOD);
    for (int &x : a) x = x * ni % NTT_MOD;
}
}

```

6.16 高斯消元

解线性方程组、求秩、行列式。整数模数下用模逆；浮点版本选主元避免除以接近 0 的数。方程有自由变量时记录主元列并构造一组解。

```

// 接口: gauss(a): 在固定质数模 MOD 下对非空系数矩阵消元并返回秩; 返回类型 int。
// 参数: 非空系数矩阵 a; 若传入增广矩阵, 常数列不能参与本函数的秩计算。
int gauss(vector<vector<long long>> a) {
    // 变量: n=增广矩阵的行数 n; m=未知量列数 m; rank=高斯消元已经确定主元的行数, 即当前
    // 矩阵秩 rank。
    int n = a.size(), m = a[0].size(), rank = 0;
    for (auto& row : a)
        for (long long& value : row) value = (value % MOD + MOD) % MOD;
    // 变量: col=当前高斯消元处理的列 col。
    for (int col = 0; col < m && rank < n; ++col) {
        // 变量: pivot=当前高斯消元选择的主元行 pivot。
        int pivot = rank;
        while (pivot < n && a[pivot][col] == 0) ++pivot;
        if (pivot == n) continue;
        swap(a[pivot], a[rank]);
        // 变量: inv=当前运算使用的逆元 inv。
    }
}

```

```

    long long inv = qpow(a[rank][col], MOD - 2, MOD);
    // 变量: j=高斯消元中从主元列 col 开始更新的增广矩阵列下标 j。
    for (int j = col; j < m; ++j)
        a[rank][j] = (__int128)a[rank][j] * inv % MOD;
    // 变量: i=模高斯消元中当前用主元行消去的其它行下标 i。
    for (int i = 0; i < n; ++i) if (i != rank && a[i][col]) {
        // 变量: t=高斯消元当前行在主元列上的消元系数 t。
        long long t = a[i][col];
        // 变量: j=高斯消元中从主元列 col 开始更新的增广矩阵列下标 j。
        for (int j = col; j < m; ++j)
            a[i][j] = (a[i][j] - (__int128)t * a[rank][j] % MOD + MOD) %
MOD;
    }
    ++rank;
}
return rank;
}

```

6.17 任意精度整数（纯 C++17）

当整数可能超过 `__int128`，且评测环境没有 Boost 时，使用下面的有符号十进制高精度模板。内部以 10^9 为一位，支持输入输出、比较、正负数加减乘除、取模、最大公约数和模快速幂，不依赖第三方库。

Python 3.11 及以上默认限制十进制字符串与大整数互转的位数；需要处理超长整数时，可在程序开头关闭该限制：

```

import sys
sys.set_int_max_str_digits(0)

```

```

struct BigInteger {
    // 变量: BASE=每个数组块的进制。
    static constexpr uint32_t BASE = 1000000000;
    // 变量: WIDTH=输出低位块时补齐的十进制位数。
    static constexpr int WIDTH = 9;
    // 变量: digit=以 BASE 为进制的小端绝对值。
    vector<uint32_t> digit;
    // 变量: negative=true 表示负数，零始终非负。
    bool negative = false;

    // 接口: BigInteger(value): 由 long long 构造高精度整数，默认构造零。
    BigInteger(long long value = 0) { *this = value; }
    // 接口: BigInteger(text): 由带可选正负号的十进制字符串构造高精度整数。
    explicit BigInteger(const string& text) { read(text); }
}

```



```

// 接口: operator=(value): 用 long long (含 LLONG_MIN) 重置当前高精度整数。
BigInteger& operator=(long long value) {
    digit.clear();
    negative = value < 0;
    // 变量: magnitude=value 的无符号绝对值; 写法可安全处理 LLONG_MIN。
    unsigned long long magnitude = negative
        ? 0ULL - static_cast<unsigned long long>(value)
        : static_cast<unsigned long long>(value);
    while (magnitude != 0) {
        digit.push_back(magnitude % BASE);
        magnitude /= BASE;
    }
    trim();
    return *this;
}

// 接口: read(text): 从带可选正负号的十进制字符串重置当前值; 非法格式抛异常。
void read(const string& text) {
    digit.clear();
    negative = false;
    if (text.empty()) throw invalid_argument("empty BigInteger");
    // 变量: pos=跳过可选正负号后的首个数字下标。
    size_t pos = 0;
    if (text[pos] == '+' || text[pos] == '-') {
        negative = text[pos] == '-';
        ++pos;
    }
    if (pos == text.size()) throw invalid_argument("invalid BigInteger");
    // 变量: i=检查十进制输入格式的当前字符下标。
    for (size_t i = pos; i < text.size(); ++i) {
        if (!isdigit(static_cast<unsigned char>(text[i])))
            throw invalid_argument("invalid BigInteger");
    }
    // 变量: end=当前尚未拆分部分的右端开区间下标。
    for (size_t end = text.size(); end > pos;) {
        // 变量: begin=当前九位十进制块的左端下标。
        size_t begin = end >= pos + WIDTH ? end - WIDTH : pos;
        digit.push_back(static_cast<uint32_t>(stoul(
            text.substr(begin, end - begin))));
        end = begin;
    }
    trim();
}

```

```

// 接口: str(): 返回不含前导零的十进制表示, 零返回 "0"。
string str() const {
    if (digit.empty()) return "0";
    stringstream out;
    if (negative) out << '-';
    out << digit.back();
    // 变量: i=从高到低输出的内部数位块下标。
    for (int i = static_cast<int>(digit.size()) - 2; i >= 0; --i)
        out << setw(WIDTH) << setfill('0') << digit[i];
    return out.str();
}

// 接口: is_zero(): 返回当前值是否为零。
bool is_zero() const { return digit.empty(); }

// 接口: is_odd(): 返回当前值的绝对值是否为奇数。
bool is_odd() const { return !digit.empty() && (digit[0] & 1); }

// 接口: divide_by_two(): 把非负当前值原地除以 2, 供快速幂缩小指数。
void divide_by_two() {
    assert(!negative);
    uint64_t carry = 0;
    // 变量: i=从高到低执行短除法的内部数位块下标。
    for (int i = static_cast<int>(digit.size()) - 1; i >= 0; --i) {
        // 变量: current=本块加上高位余数后的被除数。
        uint64_t current = digit[i] + carry * BASE;
        digit[i] = current / 2;
        carry = current % 2;
    }
    trim();
}

// 接口: trim(): 删除绝对值最高端的零块, 并把零的符号归一为正。
void trim() {
    while (!digit.empty() && digit.back() == 0) digit.pop_back();
    if (digit.empty()) negative = false;
}

// 接口: abs_compare(a,b): 比较绝对值; 小于、等于、大于分别返回 -1、0、1。
static int abs_compare(const BigInteger& a, const BigInteger& b) {
    if (a.digit.size() != b.digit.size())
        return a.digit.size() < b.digit.size() ? -1 : 1;
    // 变量: i=从最高块向最低块比较绝对值的下标。

```

```

    for (int i = static_cast<int>(a.digit.size()) - 1; i >= 0; --i) {
        if (a.digit[i] != b.digit[i])
            return a.digit[i] < b.digit[i] ? -1 : 1;
    }
    return 0;
}

```

// 接口: `abs_add(a,b)`: 返回两个绝对值之和, 结果非负。

```

static BigInteger abs_add(const BigInteger& a, const BigInteger& b) {
    // 变量: result=准备返回的非负绝对值之和。
    BigInteger result;
    uint64_t carry = 0;
    // 变量: size=两个操作数内部数位块数量的较大值。
    size_t size = max(a.digit.size(), b.digit.size());
    // 变量: i=当前相加的内部数位块下标。
    for (size_t i = 0; i < size || carry != 0; ++i) {
        uint64_t current = carry;
        if (i < a.digit.size()) current += a.digit[i];
        if (i < b.digit.size()) current += b.digit[i];
        result.digit.push_back(current % BASE);
        carry = current / BASE;
    }
    return result;
}

```

// 接口: `abs_sub(a,b)`: 返回 $|a|-|b|$; 前置条件为 $|a|\geq|b|$ 。

```

static BigInteger abs_sub(const BigInteger& a, const BigInteger& b) {
    assert(abs_compare(a, b) >= 0);
    // 变量: result=准备返回的非负绝对值之差。
    BigInteger result;
    int64_t borrow = 0;
    // 变量: i=当前相减的内部数位块下标。
    for (size_t i = 0; i < a.digit.size(); ++i) {
        int64_t current = static_cast<int64_t>(a.digit[i]) - borrow
            - (i < b.digit.size() ? b.digit[i] : 0);
        if (current < 0) current += BASE, borrow = 1;
        else borrow = 0;
        result.digit.push_back(static_cast<uint32_t>(current));
    }
    result.trim();
    return result;
}

```

// 接口: `multiply_small(factor)`: 返回当前绝对值乘 $[0, \text{BASE})$ 内整数 `factor`, 结果

非负。

```

BigInteger multiply_small(uint32_t factor) const {
    // 变量: result=准备返回的非负短乘结果。
    BigInteger result;
    if (factor == 0 || is_zero()) return result;
    uint64_t carry = 0;
    for (uint32_t block : digit) {
        uint64_t current = static_cast<uint64_t>(block) * factor + carry;
        result.digit.push_back(current % BASE);
        carry = current / BASE;
    }
    if (carry != 0) result.digit.push_back(static_cast<uint32_t>(carry));
    return result;
}

// 接口: shift_base_add(block): 原地执行当前非负值乘 BASE 再加一个低位块。
void shift_base_add(uint32_t block) {
    assert(!negative && block < BASE);
    digit.insert(digit.begin(), block);
    trim();
}

// 接口: divmod(a,b): 返回向零截断的商和余数; 前置条件 b!=0。
static pair<BigInteger, BigInteger> divmod(
    const BigInteger& a, const BigInteger& b) {
    if (b.is_zero()) throw domain_error("BigInteger division by zero");
    // 变量: dividend=被除数 a 的绝对值。
    BigInteger dividend = a.negative ? -a : a;
    // 变量: divisor=除数 b 的绝对值。
    BigInteger divisor = b.negative ? -b : b;
    // 变量: quotient=逐块确定的非负商; remainder=当前非负余数。
    BigInteger quotient, remainder;
    quotient.digit.assign(dividend.digit.size(), 0);
    // 变量: i=从高到低移入长除法的被除数块下标。
    for (int i = static_cast<int>(dividend.digit.size()) - 1;
        i >= 0; --i) {
        remainder.shift_base_add(dividend.digit[i]);
        uint32_t low = 0, high = BASE - 1, best = 0;
        while (low <= high) {
            uint32_t middle = low + (high - low) / 2;
            if (abs_compare(divisor.multiply_small(middle), remainder) <=
0) {
                best = middle;
                low = middle + 1;
            }
        }
    }
}

```

```

        } else {
            if (middle == 0) break;
            high = middle - 1;
        }
    }
    quotient.digit[i] = best;
    remainder = abs_sub(remainder, divisor.multiply_small(best));
}
quotient.negative = a.negative != b.negative;
remainder.negative = a.negative;
quotient.trim();
remainder.trim();
return {quotient, remainder};
}

// 接口: operator-(): 返回当前高精度整数的相反数, -0 仍为 0。
BigInteger operator-() const {
    // 变量: result=当前值的副本, 随后只翻转其符号。
    BigInteger result = *this;
    if (!result.is_zero()) result.negative = !result.negative;
    return result;
}

// 接口: operator+(a,b): 返回两个有符号高精度整数之和。
friend BigInteger operator+(const BigInteger& a, const BigInteger& b) {
    if (a.negative == b.negative) {
        // 变量: result=同号操作数绝对值相加后的结果。
        BigInteger result = abs_add(a, b);
        result.negative = a.negative;
        result.trim();
        return result;
    }
    // 变量: order=操作数绝对值的比较结果。
    int order = abs_compare(a, b);
    if (order == 0) return BigInteger(0);
    // 变量: result=异号操作数绝对值相减后的结果。
    BigInteger result = order > 0 ? abs_sub(a, b) : abs_sub(b, a);
    result.negative = order > 0 ? a.negative : b.negative;
    return result;
}

// 接口: operator-(a,b): 返回两个有符号高精度整数之差 a-b。
friend BigInteger operator-(const BigInteger& a, const BigInteger& b) {
    return a + (-b);
}

```

```

}

// 接口: operator*(a,b): 返回两个有符号高精度整数之积。
friend BigInteger operator*(const BigInteger& a, const BigInteger& b) {
    // 变量: result=按块累加的高精度乘积。
    BigInteger result;
    if (a.is_zero() || b.is_zero()) return result;
    result.digit.assign(a.digit.size() + b.digit.size(), 0);
    // 变量: i=第一个高精度乘数 a 当前参与乘法的内部块下标 i。
    for (size_t i = 0; i < a.digit.size(); ++i) {
        uint64_t carry = 0;
        // 变量: j=第二个高精度乘数 b 当前参与乘法的内部块下标 j。
        for (size_t j = 0; j < b.digit.size() || carry != 0; ++j) {
            uint64_t current = result.digit[i + j] + carry;
            if (j < b.digit.size())
                current += static_cast<uint64_t>(a.digit[i]) * b.digit[j];
            result.digit[i + j] = current % BASE;
            carry = current / BASE;
        }
    }
    result.negative = a.negative != b.negative;
    result.trim();
    return result;
}

// 接口: operator/(a,b): 返回 a/b 向零截断的商; 前置条件 b!=0。
friend BigInteger operator/(const BigInteger& a, const BigInteger& b) {
    return divmod(a, b).first;
}

// 接口: operator%(a,b): 返回 a/b 的余数, 符号与 a 相同; 前置条件 b!=0。
friend BigInteger operator%(const BigInteger& a, const BigInteger& b) {
    return divmod(a, b).second;
}

// 接口: operator==(a,b): 返回 a 与 b 是否数值相等。
friend bool operator==(const BigInteger& a, const BigInteger& b) {
    return a.negative == b.negative && a.digit == b.digit;
}

// 接口: operator!=(a,b): 返回 a 与 b 是否数值不等。
friend bool operator!=(const BigInteger& a, const BigInteger& b) {
    return !(a == b);
}

```

```

// 接口: operator<(a,b): 返回 a 是否严格小于 b。
friend bool operator<(const BigInteger& a, const BigInteger& b) {
    if (a.negative != b.negative) return a.negative;
    // 变量: order=操作数绝对值的比较结果。
    int order = abs_compare(a, b);
    return a.negative ? order > 0 : order < 0;
}

// 接口: operator<=(a,b): 返回 a 是否小于等于 b。
friend bool operator<=(const BigInteger& a, const BigInteger& b) {
    return !(b < a);
}

// 接口: operator>(a,b): 返回 a 是否严格大于 b。
friend bool operator>(const BigInteger& a, const BigInteger& b) {
    return b < a;
}

// 接口: operator>=(a,b): 返回 a 是否大于等于 b。
friend bool operator>=(const BigInteger& a, const BigInteger& b) {
    return !(a < b);
}

// 接口: operator>>(in,value): 从输入流读取一个十进制高精度整数。
friend istream& operator>>(istream& in, BigInteger& value) {
    // 变量: text=从输入流读取的带符号十进制字符串。
    string text;
    if (in >> text) value.read(text);
    return in;
}

// 接口: operator<<(out,value): 向输出流写出一个十进制高精度整数。
friend ostream& operator<<(ostream& out, const BigInteger& value) {
    return out << value.str();
}
};

// 接口: abs_big(x): 返回任意精度整数 x 的绝对值。
BigInteger abs_big(BigInteger x) {
    return x < 0 ? -x : x;
}

// 接口: gcd_big(a,b): 返回任意精度整数 a,b 的非负最大公约数; gcd(0,0)=0。

```

```

BigInteger gcd_big(BigInteger a, BigInteger b) {
    a = abs_big(a);
    b = abs_big(b);
    while (!b.is_zero()) {
        // 变量: remainder=本轮欧几里得算法的非负余数。
        BigInteger remainder = a % b;
        a = b;
        b = remainder;
    }
    return a;
}

// 接口: pow_mod_big(base,exponent,mod): 返回  $\text{base}^{\text{exponent}} \bmod \text{mod}$ 。
// 参数: 任意精度底数 base; 非负任意精度指数 exponent; 正模数 mod。
BigInteger pow_mod_big(
    BigInteger base, BigInteger exponent, const BigInteger& mod) {
    assert(!(exponent < 0) && 0 < mod);
    base = base % mod;
    if (base < 0) base = base + mod;
    // 变量: result=快速幂当前累计答案, 始终位于  $[0, \text{mod})$ 。
    BigInteger result = BigInteger(1) % mod;
    while (!exponent.is_zero()) {
        if (exponent.is_odd()) result = result * base % mod;
        base = base * base % mod;
        exponent.divide_by_two();
    }
    return result;
}

```

- 除法向零截断, 余数与被除数同号, 并满足 $a == (a/b) * b + a \% b$ 。若题目要求非负模且 $b > 0$, 用 $r = (a \% b + b) \% b$ 归一化。
- 除数为零会抛出 `domain_error`; 字符串为空、只有符号或含非数字字符会抛出 `invalid_argument`。
- 加减为 $O(n)$, 朴素乘法为 $O(nm)$, 本模板长除法约为 $O(nm \log 10^9)$ 。几万位乘除应改用 FFT/NTT 或专用大数库。
- 本模板只实现整数算术, 不支持小数和按位运算。若评测环境确认提供 Boost, 且需要任意宽位运算, 可改用 `boost::multiprecision::cpp_int`。

只超过 64 位但不超过约 38 位十进制时，优先使用 `__int128`；只需要高精度加法或高精度除以低精度时，也可以从本结构中裁掉乘法和完整长除法，缩短赛时代码。

6.18 数学与组合计数：参数、调用与改造

本节紧跟前面的基础模板。先看每个接口传什么，再看哪些不变量不能动，最后看如何替换维护信息或转移。

6.18.1 高级容斥与倍数容斥

集合容斥是“至少一个条件满足”：

$$|\cup A_i| = \sum |A_i| - \sum |A_i \cap A_j| + \dots$$

当条件是“能被 d 整除”，交集对应 `lcm`；当条件是“所有因子贡献汇总”，常用莫比乌斯函数把“倍数和”反演为“恰好值”。

小范围条件最多二十个时可以枚举子集：

```
// 数值域：要求 0<=m<31、n>=1、a[i]>0；lcm_limit 必须在乘法前检测超过 n 的最小公倍数，bad 的结果位于 [0,n]。
// 变量：bad=容斥累计得到的、至少被一个给定除数整除的整数个数 bad。
long long bad = 0;
// 变量：mask=用二进制位编码的当前状态 mask。
for (int mask = 1; mask < (1 << m); ++mask) {
    // 变量：l=当前子集内全部除数的最小公倍数 l。
    long long l = 1;
    // 变量：bits=当前容斥子集中被选除数的数量 bits。
    int bits = 0;
    // 变量：overflow=当前子集最小公倍数是否已超过安全上界的标记 overflow。
    bool overflow = false;
    // 变量：i=当前容斥子集 mask 中正在检查是否选中除数 a[i] 的位编号 i。
    for (int i = 0; i < m; ++i) if (mask >> i & 1) {
        ++bits;
        l = lcm_limit(l, a[i], n, overflow);
    }
    if (overflow || l > n) continue;
    // 变量：ways=1..n 中能被当前子集最小公倍数 l 整除的整数个数 ways。
    long long ways = n / l;
    if (bits & 1) bad += ways;
    else bad -= ways;
}
```

测试样例：容斥的重复覆盖和最小公倍数溢出分支

下面约定每组输入 nm ，后面给出 m 个除数，输出 $1..n$ 中不能被任何除数整除的数。

输入

```
4
1 1
2
10 2
2 3
30 3
2 3 5
100 2
4 6
```

输出

```
1
3
8
67
```

第二、三组检查交集是否按 lcm 加回；第四组的交集是 $\text{lcm}(4, 6)=12$ ，答案为 $100-25-16+8=67$ ，可以抓住把乘积当最小公倍数或符号写反的问题。

例题映射：题目问 $1..n$ 中不能被任何给定数整除的数，先求“至少被一个整除”的数量，再用 $n-\text{bad}$ 。若 m 很大，子集容斥不可用，检查数值范围后换莫比乌斯或筛法。

6.18.2 卢卡斯定理 (Lucas Theorem) 与组合数模质数

当模数 p 是质数、 n, m 很大但 p 不大时：

$$C(n, m) \equiv C(\lfloor n/p \rfloor, \lfloor m/p \rfloor) \cdot C(n \bmod p, m \bmod p) \pmod{p}.$$

// 接口：mod_pow(a, e, p)：计算 $a^e \bmod p$ ；返回类型 long long。

// 参数：待求模幂的底数 a；非负指数 e；幂运算使用的正模数 p。

```
long long mod_pow(long long a, long long e, long long p) {
    a %= p;
    if (a < 0) a += p;
    long long r = 1 % p; // p=1 时返回 0，避免写死为 1。
    for (; e; e >>= 1, a = (__int128)a * a % p) {
        // e 的最低位为 1，说明当前 a 是答案乘积的一部分。
        if (e & 1) r = (__int128)r * a % p;
    }
    return r;
}
```

```

// 接口: Csmall(n, k, p, fac, ifac): 计算当前质数模数下的小组合数  $C(n, k)$ ; 返回类型 long long。
// 参数: 组合数  $C(n, k)$  的上标 n; 组合数  $C(n, k)$  的下标 k; 组合数运算使用的质数模数 p; 阶乘表 fac; 逆阶乘表 ifac。
long long Csmall(long long n, long long k, long long p,
                 const vector<long long>& fac,
                 const vector<long long>& ifac) {
    // 这里 n、k 必须小于 p; Lucas 会把大数拆成 p 进制位后调用本函数。
    if (k < 0 || k > n) return 0;

    //  $C(n, k) = n! / (k!(n-k)!)$ , ifac 是阶乘逆元。
    return fac[n] * ifac[k] % p * ifac[n-k] % p;
}

// 接口: Lucas(n, k, p, fac, ifac): 用 Lucas 定理计算  $C(n, k) \bmod p$ ; 返回类型 long long。
// 参数: 组合数  $C(n, k)$  的上标 n; 组合数  $C(n, k)$  的下标 k; Lucas 定理使用的质数模数 p; 阶乘表 fac; 逆阶乘表 ifac。
long long Lucas(long long n, long long k, long long p,
                const vector<long long>& fac,
                const vector<long long>& ifac) {
    // 递归处理 n、k 的最低一位 p 进制数字。
    if (k == 0) return 1;

    // 当前位组合数乘上更高位的组合数; 模 p 的进位不合法时 Csmall 返回 0。
    return Csmall(n % p, k % p, p, fac, ifac)
        * Lucas(n / p, k / p, p, fac, ifac) % p;
}

```

测试样例: Lucas 的低位进位、 $k=0$ 和组合数为零

每组输入 nmp, 其中 p 是质数, 输出 $C(n, m) \bmod p$ 。

输入

4

0 0 2

5 2 3

7 1 5

10 3 5

输出

1

1

2

0

$C(10, 3)$ 在模 5 下因为五进制低位发生不合法进位而为 0；如果只计算 $n\%p$ 和 $m\%p$ 而不递归高位，最后一组会错误。

例题：P3807 Lucas 定理。预处理 $fac[0..p-1]$ 和逆阶乘；每组输入 (n, m, p) 调 $Lucas(n, m, p, fac, ifac)$ 。注意这份模板默认 p 不大且是质数；模合数或 p 很大时要用 exLucas/质因数分解，不能盲套。

6.18.3 中国剩余定理与扩展中国剩余定理

要合并

$$x \equiv a \pmod{m}, \quad x \equiv b \pmod{n},$$

令 $x=a+m \times t$ ，得到 $m \times t \equiv b-a \pmod{n}$ 。只有 $\gcd(m, n)$ 整除 $b-a$ 时有解；用扩展欧几里得求 m/g 的逆元。

```
// 接口: exgcd(a, b, x, y): 求 gcd(a,b), 并通过 x,y 返回 ax+by=gcd(a,b) 的一组系数; 返回类型 long long。
// 参数: 扩展欧几里得的第一个整数输入 a; 扩展欧几里得的第二个整数输入 b; 输出引用 x, 满足 a*x+b*y=gcd(a,b); 输出引用 y, 满足 a*x+b*y=gcd(a,b)。
long long exgcd(long long a, long long b,
                long long& x, long long& y) {
    // 返回 gcd(a,b), 同时求出 a*x+b*y=g。
    if (!b) { x = 1; y = 0; return a; }
    // 变量: x1=递归子问题返回的 Bézout 系数 x1; y1=递归子问题返回的 Bézout 系数 y1。
    long long x1, y1;
    // 变量: g=递归返回的 gcd(a,b) 值 g。
    long long g = exgcd(b, a % b, x1, y1);
    // 由 b*x1+(a%b)*y1=g 反推 a*y1+b*(x1-a/b*y1)=g。
    x = y1;
    y = x1 - (a / b) * y1;
    return g;
}

// 接口: merge(a, m, b, n): 合并 x≡a(mod m) 与 x≡b(mod n), 不相容时返回 false; 方程相容且 a,m 已更新时返回 true。
// 参数: 第一个同余式余数及合并后余数的输入输出引用 a; 第一个同余式模数及合并后模数的输入输出引用 m; 第二个同余式的余数 b; 第二个同余式的正模数 n。
bool merge(long long& a, long long& m,
           long long b, long long n) {
    // 旧方程: x ≡ a (mod m); 新方程: x ≡ b (mod n)。
    // 合并后仍用 a,m 表示一个最小非负解和新的最小公倍数模数。
    // 变量: x=exgcd 写回的 Bézout 系数 x; y=exgcd 写回的 Bézout 系数 y。
    long long x, y;
    // 变量: g=递归返回的 gcd(a,b) 值 g。
```

```

long long g = exgcd(m, n, x, y);
// 变量: d=从点 a 指向点 b 的方向向量 d。
long long d = b - a;
// 两同余式相容的充要条件: gcd(m,n) | (b-a)。
if (d % g) return false;
// 令  $x=a+m \cdot t$ , 得到  $(m/g) \cdot t \equiv (b-a)/g \pmod{n/g}$ 。
// exgcd 返回的 x 正是 m/g 在模 n/g 意义下的逆元。
// 变量: t=CRT 合并时由 Bézout 系数求出的第二个模数倍数 t。
__int128 t = (__int128)(d / g) * x;
// 变量: mod=当前正模数 mod。
long long mod = n / g;
t %= mod;
if (t < 0) t += mod;
// 中间乘法可能超过 long long, 所以 na/nm 使用 __int128。
// 变量: na=多项式 a 补齐后的 NTT 长度 na。
__int128 na = (__int128)a + (__int128)m * t;
// 变量: nm=合并后需要的最小结果长度 nm。
__int128 nm = (__int128)m / g * n;
a = (long long)(na % nm);
if (a < 0) a += (long long)nm;
m = (long long)nm;
return true;
}

```

测试样例：扩展中国剩余定理的互质、非互质和无解

下面每组先输入方程数量 k ，随后输入 $bmod$ ，表示 $x \equiv b(modmod)$ ；输出最小非负解，无解输出 No。

输入

```

4
1
3 5
2
2 3
3 5
2
1 4
5 6
2
0 2
1 4

```

输出

3
8
5
No

第三组模数不互质但相容，答案是 5；第四组 x 必须同时为偶数和模 4 余 1，无解。连续多组还要检查合并后的 a, m 是否在下一组重新初始化为 0, 1。

例题：P4777 扩展中国剩余定理。初始化 $a=0, m=1$ ，依次读入每组 $x \equiv b \pmod{mod}$ ，调用 `merge(a, m, b, mod)`，最后输出最小非负 a 。本题明确提醒中间乘法可能溢出，`__int128` 不能省；若题目保证有解也仍然建议保留无解判断。

6.18.4 高斯消元、秩、解空间与行列式

高斯消元的现场顺序：选主元 $\rightarrow$ 交换到当前行 $\rightarrow$ 消去其他行/下面的行 $\rightarrow$ 判断矛盾或自由元。模意义下每次除以主元要乘逆元；普通实数消元则要用 `longdouble` 并带 `eps`。

```
// 接口: gauss_mod(a, mod): 在质数模 mod 下对非空系数矩阵消元并返回秩; 返回类型 int。
// 参数: 非空系数矩阵 a; 题目给定的质数 mod。
int gauss_mod(vector<vector<long long>> a, long long mod) {
    // a 是系数矩阵; 如果是增广矩阵, 最后一列还要单独解释为常数项。
    // 这里采用 Gauss-Jordan 消元: 每个主元列都会把其他行一起消成 0。
    // 变量: n=容器 a 的元素数量 n; m=增广矩阵 a 的列数 m; row=当前高斯消元主元行 row。
    int n = a.size(), m = a[0].size(), row = 0;
    for (auto& values : a)
        for (long long& value : values) value = (value % mod + mod) % mod;
    // 变量: col=当前高斯消元处理的列 col。
    for (int col = 0; col < m && row < n; ++col) {
        int p = -1; // 在当前列寻找主元行。
        // 变量: i=高斯消元中为当前主元列寻找非零元的候选行 i。
        for (int i = row; i < n; ++i) if (a[i][col] != 0) {
            p = i;
            break;
        }
        // 没有非零主元, 说明这一列是自由列, 换下一列。
        if (p == -1) continue;
        swap(a[p], a[row]);
        // 模意义下的除法必须乘逆元; 这里默认 mod 是质数。
        // 变量: inv=当前运算使用的逆元 inv。
        long long inv = mod_pow(a[row][col], mod - 2, mod);
        // 变量: j=高斯消元中从主元列 col 开始更新的增广矩阵列下标 j。
        for (int j = col; j < m; ++j)
            a[row][j] = (__int128)a[row][j] * inv % mod;
        // 变量: i=高斯消元中当前用主元行消去的其它方程行 i。
    }
}
```

```

    for (int i = 0; i < n; ++i) if (i != row) {
        // 变量: k=高斯消元用当前行消去主元列的倍数 k。
        long long k = a[i][col];
        // 用当前主元行消掉其它行的这一列。
        // 变量: j=高斯消元中从主元列 col 开始更新的增广矩阵列下标 j。
        for (int j = col; j < m; ++j)
            a[i][j] = (a[i][j] - (__int128)k * a[row][j] % mod + mod) %
mod;
        }
        ++row;
    }
    return row; // 秩; 增广矩阵要额外判断矛盾行
}

```

行列式在消元时记录交换次数，答案乘所有主元；若模数不是质数，不能用费马逆元，可用整数版消元或先保证主元可逆。矩阵树定理则是构造拉普拉斯矩阵，删去任意一行一列，求余下行列式。

测试样例：模高斯消元的零秩、满秩和相关行

下面约定每组输入 nm 和一个 $n \times m$ 的系数矩阵，输出模意义下的秩；模数固定为 7。

输入

```

4
1 1
0
1 1
1
2 2
1 0
0 1
2 3
1 2 3
2 4 6

```

输出

```

0
1
2
1

```

第一组检查没有主元时不能强行增加秩；最后一组两行线性相关，能抓住没有取模、主元交换和消元方向错误。若把它接到增广矩阵上，还要另外加入“系数全零但常数非零”的无解行。

例题：P6178 Matrix-Tree 定理。无向图中 $L[i][i]$ 加边数， $L[i][j]$ 减边数；删掉最后一行和最后一列后求模行列式。不要把邻接矩阵直接拿来求行列式，也不要将有向图和无向图的拉普拉斯构造混用。

6.18.5 莫比乌斯反演

若题目给出

$$F(n) = \sum_{d|n} f(d),$$

则

$$f(n) = \sum_{d|n} \mu(d) F(n/d).$$

竞赛中常见等价形式是“统计 gcd 恰好为 d ”：先统计所有 gcd 是 d 的倍数，再从大到小用 $f[d] = f[k]$ 反演。

```
vector<int> mu;          // mu[x]=莫比乌斯函数值，求完后合法下标为 [1,n]。
vector<int> primes;      // 不超过 n 的全部质数，严格递增。
vector<char> is_comp;     // is_comp[x]=1 表示 x 是合数；0 还可能是 0/1。
// 接口：linear_sieve_mu(n)：线性筛出 1..n 的莫比乌斯函数 mu；不返回值。
// 参数：筛法计算的正整数上界 n。
void linear_sieve_mu(int n) {
    // mu[1]=1; mu[p]=-1; 有平方因子时 mu=0; 不同质因子乘积时符号交替。
    mu.assign(n + 1, 0);
    is_comp.assign(n + 1, false);
    mu[1] = 1;
    // 变量：i=线性筛当前处理的整数 i。
    for (int i = 2; i <= n; ++i) {
        // i 没被标记说明是质数。
        if (!is_comp[i]) primes.push_back(i), mu[i] = -1;
        // 变量：p=当前从质数表枚举出的质数 p。
        for (int p : primes) {
            if (1LL * i * p > n) break;
            is_comp[i * p] = true;
            if (i % p == 0) {
                // p 已经在 i 中出现，再乘一次就有平方因子。
                mu[i * p] = 0;
                break;
            }
            // p 与 i 互质：mu(i*p)=-mu(i)。
            mu[i * p] = -mu[i];
        }
    }
}
```



```

    }
}
}

```

测试样例：莫比乌斯函数的平方因子和多测重建

下面约定每组输入 n ，输出 $\text{mu}[1..n]$ 。

输入

```

3
1
5
10

```

输出

```

1
1 -1 -1 0 -1
1 -1 -1 0 -1 1 -1 0 0 1

```

$\text{mu}(4)=0$ 和 $\text{mu}(8)=0$ 是平方因子分支；连续调用时 `primes` 必须清空，否则线性筛的全局状态会残留。若后续用它统计互质对，还要单独测试全相同数组和全为质数的数组。

例如统计数对 (i, j) 的 gcd 恰为 1：先用频率数组求 $\text{cnt}[d]=$ 两数都是 d 的倍数的对数，再从大到小令 $\text{exact}[d]=\text{cnt}[d]-\text{sum}(\text{exact}[2d], \text{exact}[3d], \dots)$ ；也可按 mu 直接加权。看到“所有倍数/恰好 gcd /互质对”就优先想反演。

6.18.6 原根、Baby-step Giant-step 离散对数与扩展算法

离散对数题的目标是求最小 x 使 $a^x \equiv b \pmod{p}$ 。当模数是质数且 a 可逆，用 Baby-step Giant-step：设 $m=\text{ceil}(\sqrt{p})$ ，枚举 a^j 存表，再枚举 $b \cdot a^{-im}$ 查表，复杂度 $O(\sqrt{p})$ 。

// 接口：`bsgs(a, b, mod)`：求最小非负 x 使 $a^x \equiv b \pmod{\text{mod}}$ ，无解返回 -1 ；返回类型 `long long`。

// 参数：离散对数方程 $a^x \equiv b \pmod{\text{mod}}$ 的底数 a ；目标剩余 b ；正模数 mod 。

```

long long bsgs(long long a, long long b, long long mod) {
    // 求最小非负 x，使 a^x = b (mod mod)。
    // 基础版要求 gcd(a, mod)=1；下面用费马小定理求逆元，
    // 因而最安全的使用场景是 mod 为质数且 a!=0。
    a %= mod; b %= mod;
    // 变量：m=BSGS 大步与小步采用的步长 m=ceil(sqrt(mod))。
    long long m = sqrtl(mod) + 1;
    // 变量：baby=BSGS 中小步值到指数的哈希表 baby。
    unordered_map<long long, long long> baby;
    // 变量：cur=BSGS 小步枚举中当前的幂值 a^j mod mod。

```

```

long long cur = 1;
// Baby step: 保存  $a^j \rightarrow j$ ,  $j=0..m-1$ 。
// 变量:  $j$ =BSGS 小步表中指数  $j$ , 对应  $a^j \bmod \text{mod}$ 。
for (long long j = 0; j < m; ++j) {
    // 保留最小  $j$ , 保证找到的是更小的答案。
    baby.emplace(cur, j);
    cur = (__int128)cur * a % mod;
}
// Giant step 每次乘  $a^{-m}$ , 所以第  $i$  次对应  $b * a^{-im}$ 。
// 变量: factor=当前步骤使用的乘数 factor。
long long factor = mod_pow(mod_pow(a, m, mod), mod - 2, mod);
cur = b;
// 变量:  $i$ =BSGS 大步枚举次数  $i$ , 对应指数块  $i * m$ 。
for (long long i = 0; i <= m; ++i) {
    // 变量: it=容器 baby 中当前查找结果的迭代器 it。
    auto it = baby.find(cur);
    if (it != baby.end()) {
        //  $\text{cur}=a^j$  时,  $b * a^{-im}=a^j$ , 即  $x=i * m + j$ 。
        return i * m + it->second;
    }
    cur = (__int128)cur * factor % mod;
}
return -1;
}

```

测试样例：离散对数的零次幂、最小解和无解

下面每组输入 abp , 输出最小非负 x 使 $a^x \equiv b \pmod{p}$; 这里的基础 BSGS 只使用质数模数且 a 可逆。

输入

```

4
2 1 5
2 3 5
3 1 7
2 3 7

```

输出

```

0
3
0
-1

```

第一、三组检查 $x=0$ 不能被漏掉；最后一组 3 不在 2 的幂循环中，必须输出 -1。如果把扩

展 BSGS 也接入，再另加 $\gcd(a, p) \neq 1$ 的样例，不能把基础模板直接套过去。

原根判定先分解 $\phi(\text{mod})$ 的质因子，候选 g 满足 $g^{(\phi/q)} \neq 1$ 对所有质因子 q 成立。若 $\gcd(a, \text{mod}) \neq 1$ ，普通 BSGS 不能直接用，需要扩展 BSGS 先剥离 $\gcd$ 。练习可用原根/离散对数专题中的对应题目；题目若要求模 10^9 级，先确认是否为质数，不能看见“指数很大”就自动套 BSGS。

6.18.7 生成函数、数论变换、Burnside 引理与 Pólya 计数定理

普通生成函数把数列 a_n 写成 $A(x) = \sum a_n x^n$ 。卷积就是多项式乘法：

```
// 接口: multiply(a, b): 返回两个多项式的卷积并按本节模数取模; 返回类型 vector<long long>。
// 参数: 卷积的左系数数组 a; 卷积的右系数数组 b。
vector<long long> multiply(const vector<long long>& a,
                           const vector<long long>& b) {
    // a[i] / b[j] 分别是  $x^i / x^j$  的系数, 卷积后下标一定是  $i+j$ 。
    // 变量: c=存放卷积系数的结果多项式 c。
    vector<long long> c(a.size() + b.size() - 1);
    // 变量: i=多项式 a 当前参与卷积的系数下标 i。
    for (int i = 0; i < (int)a.size(); ++i)
        // 变量: j=多项式 b 当前参与卷积的系数下标 j。
        for (int j = 0; j < (int)b.size(); ++j)
            // 先乘后加; 每次取模避免系数过大。
            c[i + j] = (c[i + j] + a[i] * b[j]) % MOD;
    return c;
}
```

测试样例：卷积下标、单位多项式和重复系数

下面约定输入两段系数，输出普通卷积结果。

输入

```
3
1
1
1
2
1 1
2
1 1
1 2 3
2
4 5
```

输出

1

1 2 1

4 13 22 15

第一组检查单位元；第二组检查中间系数累加；第三组检查卷积长度是 `a.size()+b.size()-1`，不能把下标写成 `i+j+1`。

当次数达到 `1e5`，把 `multiply` 换成 NTT；模板题是 P3803 多项式乘法。生成函数题的调用重点是先确定“指数代表什么”：物品数量、总和还是选取个数；卷积下标错一位会导致整个答案错。

Burnside 计算群作用下的不同染色数：

$$A = \frac{1}{|G|} \sum_{g \in G} k^{\text{cycles}(g)}.$$

模意义下的除法只要求 `gcd(|G|, mod)=1`；下面用扩展欧几里得求逆元，因此合数模也可用。若群大小不可逆，函数返回 `-1`。

// 接口: `burnside(perm, colors, mod)`: 按 Burnside 引理计算本质不同方案数；返回类型 `long long`。

// 参数: 非空群作用置换列表 `perm`；可用颜色数 `colors`；正模数 `mod`，须与 `perm.size()` 互质。

```
long long burnside(const vector<vector<int>>& perm,
                  long long colors, long long mod) {
    // 变量: sum=Burnside 引理对每个群元素固定染色数的累加和 sum。
    long long sum = 0;
    for (auto& p : perm) {
        // p[i] 表示置换把位置 i 送到 p[i]。
        // 变量: vis=分解当前群作用置换时记录位置是否已进入某个环的标记 vis。
        vector<char> vis(p.size());
        // 变量: cycles=当前置换的环数量 cycles。
        int cycles = 0;
        // 变量: i=当前群作用置换中尚未归入置换环的位置 i。
        for (int i = 0; i < (int)p.size(); ++i) if (!vis[i]) {
            ++cycles;
            // 沿置换反复跳转，走一圈就是一个循环。
            // 变量: u=沿置换 p 当前环移动的元素编号 u。
            for (int u = i; !vis[u]; u = p[u]) vis[u] = 1;
        }
        // 一个置换固定的染色要求同一循环内颜色相同，故有 colors^cycles 种。
        sum = (sum + (__int128)mod_pow(colors, cycles, mod)) % mod;
    }
    // Burnside: 不动点总数除以群大小；用扩展欧几里得支持任意互质模数。
    // 变量: r0=扩展欧几里得当前余数；r1=下一余数。
```

```

long long r0 = (long long)perm.size(), r1 = mod;
// 变量: x0=当前余数对应群大小的系数; x1=下一余数对应群大小的系数。
__int128 x0 = 1, x1 = 0;
while (r1) {
    // 变量: q=欧几里得除法的当前商。
    long long q = r0 / r1;
    // 变量: next_r=欧几里得除法得到的下一余数。
    long long next_r = r0 - q * r1;
    // 变量: next_x=下一余数对应群大小的系数。
    __int128 next_x = x0 - (__int128)q * x1;
    r0 = r1; r1 = next_r;
    x0 = x1; x1 = next_x;
}
if (r0 != 1) return -1;
// 变量: inv_group_size=群大小在模 mod 下的逆元。
long long inv_group_size = (long long)(x0 % mod);
if (inv_group_size < 0) inv_group_size += mod;
return (__int128)sum * inv_group_size % mod;
}

```

测试样例：Burnside 的单位群、交换群和旋转群

下面用置换数组直接调用 `burnside`，颜色数取 2，模数取 1000000007。

置换组一

```
perm = {{0}}
```

应输出: 2

置换组二

```
perm = {{0,1}, {1,0}}
```

应输出: 3

置换组三

```
perm = {{0,1,2}, {1,2,0}, {2,0,1}}
```

应输出: 4

三组分别对应一个位置、长度二的交换、长度三的旋转；如果循环数统计错，结果会分别偏离 2、3、4。多测调用时还要确认 `vis` 是每个置换重新创建，而不是沿用上一次。

Pólya 是把大量具体置换按循环类型分组并求和。若旋转/翻转群很小，直接枚举置换最稳；若有 n 边正多边形，旋转的循环数是 $\gcd(n, i)$ ，翻转要分奇偶单独数循环。先把“允许的等价变换”写清楚，不要把颜色置换也默认算进群。

6.18.8 二次剩余

题目问 $x^2 \equiv n \pmod{p}$ 且 p 为奇质数时，先用欧拉判别： $n^{(p-1)/2} \equiv 1$ 才可能有解； $n=0$ 单独处理。Tonelli-Shanks 在 $O(\log^2 p)$ 内求平方根。

```
// 接口: tonelli_shanks(n, p): 求  $x^2 \equiv n \pmod{p}$  的一个根，无解返回 -1；返回类型 long long。
// 参数: 需要求模平方根的剩余 n；需要求平方根的奇质数模数 p。
long long tonelli_shanks(long long n, long long p) {
    // 返回一个 r, 使  $r^2 \equiv n \pmod{p}$ ；无解返回 -1。
    // 使用前确认 p 是奇素数；p=2 要单独写分支。
    n %= p;
    if (n == 0) return 0;

    // 欧拉判别: 非零二次剩余的  $(p-1)/2$  次幂必须为 1。
    if (mod_pow(n, (p - 1) / 2, p) != 1) return -1;

    //  $p \equiv 3 \pmod{4}$  时有直接公式，优先走短分支。
    if (p % 4 == 3) return mod_pow(n, (p + 1) / 4, p);

    // 把  $p-1$  写成  $q \times 2^s$ , 其中 q 为奇数。
    // 变量: q=分解  $p-1=q \times 2^s$  后得到的奇数因子 q; s=分解  $p-1=q \times 2^s$  时因子 2 的指数 s。
    long long q = p - 1; int s = 0;
    while ((q & 1) == 0) q >>= 1, ++s;

    // 找一个二次非剩余 z, 使  $z^{(p-1)/2} \equiv -1$ 。
    // 变量: z=Tonelli-Shanks 正在试探的最小二次非剩余候选 z。
    long long z = 2;
    while (mod_pow(z, (p - 1) / 2, p) != p - 1) ++z;
    // 变量: c=Tonelli-Shanks 当前  $2$  幂阶生成元 c。
    long long c = mod_pow(z, q, p);
    // 变量: x=Tonelli-Shanks 当前平方根候选 x。
    long long x = mod_pow(n, (q + 1) / 2, p);
    // 变量: t=Tonelli-Shanks 当前误差项 t。
    long long t = mod_pow(n, q, p);
    // 变量: m=Tonelli-Shanks 当前误差项的  $2$  幂阶上界 m。
    int m = s;
    while (t != 1) {
        // 变量: i=寻找满足  $t^{(2^i)} \equiv 1$  时的最小指数 i; tt=当前平方根修正量 t 的平方 tt。
        int i = 1; long long tt = t * t % p;
        // 找最小 i, 使  $t^{(2^i)} \equiv 1$ ; 这一步保证下一轮 m 变小。
        while (tt != 1) tt = tt * tt % p, ++i;
    }
}
```

```

// 这里的指数是  $2^{(m-i-1)}$ ，不是  $m-i-1$ 。
// p 为 long long 时 m 很小，移位安全；若改用更大整数，
// 请改写为专门的幂指数计算，不能直接套 1LL 左移。
// 变量: b=Tonelli-Shanks 本轮把 x 调整到下一阶所乘的修正因子 b。
long long b = mod_pow(c, 1LL << (m - i - 1), p);
x = x * b % p;
t = t * b % p * b % p;
c = b * b % p; m = i;
}
return min(x, p - x);
}

```

测试样例：二次剩余的零、平方剩余、非剩余

下面每组输入 np，输出模板规定的较小根；无根输出 -1。

输入

4

0 7

1 7

2 7

3 7

输出

0

1

3

-1

2 在模 7 下的根是 3 和 4，模板若输出较小根应为 3；3 是非二次剩余，必须在 Tonelli-Shanks 主循环前返回 -1。若题目要求两个根，则分别输出 r 和 p-r，不要只输出一个根。

例题：P5491 〔模板〕二次剩余。输出两个根时输出 r 和 p-r，并按题目要求排序；没有根输出 -1。 $1LL \ll (m-i-1)$ 在指数大时可能溢出，实际模板应改成 `mod_pow(c, mod_pow(2, m-i-1, p-1), p)` 或用安全的幂次表示。

6.18.9 Prüfer 序列与矩阵树

Prüfer 序列把一棵 n 个点的标号树编码成长度 n-2 的序列。编码每次删除最小叶子并记录它的邻点；解码维护度数和最小叶子堆。最常用结论是：度数为 d_i 的树的数量为

$$\frac{(n-2)!}{\prod_i (d_i - 1)!}$$

矩阵树定理的用法在上面的高斯小节已经给出：从图构造拉普拉斯矩阵，删一行一列求行列式。例题：P6178 Matrix-Tree 定理；若题目给定度数序列求树数，直接用 Prüfer 度数公式，不要真的枚举树。

6.18.10 矩阵快速幂与线性递推

看到“初值固定、每项只依赖前几项、询问第 n 项且 n 很大”，把状态向量写成矩阵乘法。以 Fibonacci 为例：

$$\begin{pmatrix} F_{i+1} \\ F_i \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} F_i \\ F_{i-1} \end{pmatrix}.$$

```
struct Mat {
    long long a[2][2]{}; // a[i][j]: 第 i 个新状态对第 j 个旧状态的系数。
};
// 接口: operator*(A, B): 返回输入矩阵 A 与 B 的矩阵乘积; 返回类型 Mat。
// 参数: 矩阵乘法的第一个输入矩阵 A; 矩阵乘法的第二个输入矩阵 B。
Mat operator*(const Mat& A, const Mat& B) {
    Mat C{}; // 矩阵乘法是累加, 必须从全零矩阵开始。
    // 变量: i=2x2 转移矩阵的行下标 i。
    for (int i = 0; i < 2; ++i)
        // 变量: k=二阶转移矩阵乘法的公共维下标 k。
        for (int k = 0; k < 2; ++k)
            // 变量: j=2x2 转移矩阵的列下标 j。
            for (int j = 0; j < 2; ++j)
                // C=A*B; 对列向量来说先应用 B, 再应用 A。
                C.a[i][j] = (C.a[i][j] + A.a[i][k] * B.a[k][j]) % MOD;
    return C;
}
// 接口: mpow(a, e): 返回矩阵 a 的 e 次幂; 返回类型 Mat。
// 参数: 待求幂的方阵 a; 非负指数 e。
Mat mpow(Mat a, long long e) {
    // r 是单位矩阵, 代表“不做任何转移”。
    // 变量: r=当前区间右端点 r。
    Mat r{{{1, 0}, {0, 1}}};
    for (; e; e >>= 1, a = a * a)
        // 当前二进制位为 1, 就把这一段转移乘进答案。
        if (e & 1) r = r * a;
    return r;
}
// 接口: fibonacci(n): 返回第 n 项 Fibonacci 数 (按本节矩阵定义); 返回类型 long long。
// 参数: Fibonacci 序列下标 n。
long long fibonacci(long long n) {
```



```

// F_0=0 单独处理；n>=1 时 base^(n-1) 的左上角就是 F_n。
if (n == 0) return 0;
// 变量：base=当前幂运算使用的底数 base。
Mat base{{{1, 1}, {1, 0}}};
return (mpow(base, n - 1).a[0][0]) % MOD;
}

```

测试样例：矩阵快速幂的零次幂、初值和普通项

下面直接调用 Fibonacci 示例，输出 F_n ，其中 $F_0=0, F_1=1$ 。

输入

5
0
1
2
10
20

输出

0
1
1
55
6765

第一组检查 $n=0$ 的特判； $n=1$ 检查幂指数为零时单位矩阵不能写成零矩阵； $n=20$ 检查快速幂多次平方和乘法顺序。

例题：P1939 矩阵加速（数列）。先把题目的递推式写成“新状态由旧状态线性组合”，矩阵第一行就是递推系数；不要看到数列就默认 Fibonacci，初值和下标要按题目重新代入。矩阵乘法有方向，**left*right** 表示先做右边转移、再做左边转移。

6.18.11 高斯消元求实数方程组

如果题目要求概率、坐标或实数解，增广矩阵每一行是一条方程。选绝对值最大的主元可以减少误差；没有主元的行是自由变量，最后要区分唯一解、无解和多解。

```

// 接口：gauss_real(a, ans): 对实数增广矩阵消元并返回解的状态；返回类型 int。
// 参数：待做实数高斯消元的增广矩阵 a；接收唯一解的输出数组 ans。
int gauss_real(vector<vector<long double>> a,
               vector<long double>& ans) {
    // a[i][0..m-1] 是系数，a[i][m] 是常数项；这里默认方程数=未知数。
    // 返回 0=无解，1=唯一解，2=多解。
}

```

```

// 变量: n=容器 a 的元素数量 n; m=方阵 a 的列数 m; row=当前高斯消元主元行 row。
int n = a.size(), m = n, row = 0;
// 变量: col=当前高斯消元处理的列 col。
for (int col = 0; col < m && row < n; ++col) {
    // 变量: p=高斯消元为当前列搜索到的主元行 p。
    int p = row;
    // 变量: i=实数高斯消元中在主元行下方寻找最大绝对值元的候选行 i。
    for (int i = row + 1; i < n; ++i)
        if (fabsl(a[i][col]) > fabsl(a[p][col])) p = i;
    // 选绝对值最大的主元可以减小浮点误差。
    if (fabsl(a[p][col]) < 1e-12L) continue;
    swap(a[p], a[row]);
    // 变量: i=高斯消元中当前用主元行消去的其它方程行 i。
    for (int i = 0; i < n; ++i) if (i != row) {
        // 变量: k=高斯消元用当前行消去主元列的倍数 k。
        long double k = a[i][col] / a[row][col];
        // Gauss-Jordan: 把主元列在所有其它行都消成 0。
        // 变量: j=实数高斯消元中从主元列到常数列的更新下标 j。
        for (int j = col; j <= m; ++j)
            a[i][j] -= k * a[row][j];
    }
    ++row;
}
// 变量: i=实数高斯消元结束后检查一致性的剩余方程行 i。
for (int i = row; i < n; ++i) {
    // 变量: all_zero=方程当前行系数是否全为 0 的标记 all_zero。
    bool all_zero = true;
    // 变量: j=实数高斯消元中检查剩余方程行是否全零的系数列下标 j。
    for (int j = 0; j < m; ++j)
        all_zero &= fabsl(a[i][j]) < 1e-12L;
    // 0=0 是多解的一部分; 0=c(c!=0) 是矛盾, 直接无解。
    if (all_zero && fabsl(a[i][m]) > 1e-12L) return 0; // 无解
}
// 主元个数小于未知数个数, 存在自由变量。
if (row < m) return 2; // 多解; 题目若保证唯一可忽略
ans.assign(m, 0);
// 变量: i=实数高斯消元中当前回填唯一解的方程行 i。
for (int i = 0; i < n; ++i) {
    // 变量: pivot=当前高斯消元选择的主元行 pivot。
    int pivot = 0;
    while (pivot < m && fabsl(a[i][pivot]) < 1e-12L) ++pivot;
    if (pivot < m) ans[pivot] = a[i][m] / a[i][pivot];
}
return 1; // 唯一解

```

```
}
```

测试样例：实数高斯消元的唯一解、无解和多解

每组输入两个未知数的两个方程，输出解的类型；唯一解时再输出 x, y 。

测试一

$$x+y=2$$

$$x-y=0$$

应为：唯一解， $x=1, y=1$

测试二

$$x+y=1$$

$$2x+2y=3$$

应为：无解

测试三

$$x+y=2$$

$$2x+2y=4$$

应为：多解

第二组检查“系数全零但常数非零”的矛盾行，第三组检查秩小于未知数个数。不能只测试唯一解，否则 $row < m$ 和无解判断很容易被抄掉。

例题：P3389 高斯消元法。输入的是 n 个方程和 n 个未知数，读入 $a[i][0..n]$ 后直接调用；输出保留题目要求的小数位。模意义下则把除法换成逆元，且只能在主元与模数互质时做。

7 数据结构

7.1 STL 必会组件

需求	容器/算法	复杂度
动态数组	<code>vector</code>	末尾均摊 $O(1)$ ，随机访问 $O(1)$
双端队列	<code>deque</code>	两端 $O(1)$
栈/队列	<code>stack</code> / <code>queue</code>	$O(1)$
优先队列	<code>priority_queue</code>	插入/删除顶 $O(\log n)$
有序集合	<code>set</code> / <code>multiset</code>	增删查 $O(\log n)$
有序键值	<code>map</code>	$O(\log n)$
哈希键值	<code>unordered_map</code>	期望 $O(1)$ ，最坏可能退化
去重/计数	<code>sort</code> + <code>unique</code> / <code>map</code>	$O(n \log n)$

`set` 的 `lower_bound`、`priority_queue<pair<...>>` 的字典序比较必须熟练。

`multiset::erase(iterator)` 与 `erase(value)` 的差别也要注意。

```
// 变量: a=输入数组 input 的工作副本 a。
vector<int> a = input;
sort(a.begin(), a.end());
a.erase(unique(a.begin(), a.end()), a.end());
// 变量: pq=当前优先队列 pq; 堆顶含义见比较器。
priority_queue<pair<int, int>> pq;
// 变量: i=在线中位数初始化时加入优先队列的数组位置 i。
for (int i = 0; i < n; ++i) pq.push({value[i], i});
while (!pq.empty()) {
    auto [value, id] = pq.top(); pq.pop();
    // 处理当前最大值; 必要时用 version[id] 判断是否过期
}
```

7.2 并查集 (Disjoint Set Union, DSU)

维护若干不相交集合并，支持合并和查询连通性。路径压缩 + 按大小合并后，单次摊销 $O(\alpha(n))$ 。

可扩展：维护集合大小、权值和、到父亲的距离/奇偶、撤销栈。带权并查集常用于维护 $\text{dist}[x] - \text{dist}[y] = w$ 或敌友关系。

```
struct DSU {
    vector<int> p, sz; // p[x]=父节点; sz[root]=该集合大小; 点编号 1..n。
    // 接口: DSU(n): 构造管理点 1..n 的并查集, 每个点初始自成集合; 构造并初始化对象; 每组
    // 测试建议重新构造。
    // 参数: 并查集管理的元素编号上界 n。
    DSU(int n): p(n + 1), sz(n + 1, 1) { iota(p.begin(), p.end(), 0); }
    // 接口: find(x): 返回 x 当前所在并查集的代表元; 返回类型 int。
    // 参数: 待查询代表元的元素编号 x。
    int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); }
    // 接口: unite(a, b): 合并两个元素所在集合; 原本同集合时返回 false; 实际合并两个集
    // 合时返回 true。
    // 参数: 第一个待合并元素编号 a; 第二个待合并元素编号 b。
    bool unite(int a, int b) {
        a = find(a); b = find(b); if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b);
        p[b] = a; sz[a] += sz[b]; return true;
    }
};
```

Kruskal、离线加边、连通块统计、最小生成树、带限制的图连通性都优先考虑 DSU。

7.3 树状数组 (Fenwick Tree)

支持单点加、前缀和、区间和， $O(\log n)$ ；空间 $O(n)$ 。通过差分可支持区间加/单点查；两棵树可支持区间加/区间和。下标必须从 1 开始。

```
struct BIT {
    int n; // 可访问下标为 1..n; 禁止用 0 调 add。
    vector<long long> t; // Fenwick 内部树状数组, 不是原数组值。
    // 接口: BIT(n): 构造管理 1..n 下标的空树状数组; 构造并初始化对象; 每组测试建议重新构造。
    // 参数: 树状数组管理的最大下标 n。
    BIT(int n): n(n), t(n + 1) {}
    // 接口: add(x, v): 把增量 v 加到树状数组的 1-based 位置 x; 会修改当前结构; 多测时
    // 重新构造或显式清空成员。
    // 参数: 树状数组的 1-based 修改位置 x; 位置 x 的点增加量 v。
    void add(int x, long long v) { for (; x <= n; x += x & -x) t[x] += v; }
    // 接口: sum(x): 返回树状数组闭前缀 [1, x] 内已存数值的总和; 返回类型 long long。
};
```

```

// 参数：树状数组前缀查询的 1-based 右端点 x。
// 变量：r=树状数组前缀 [1,x] 的累计和 r。
long long sum(int x) const { long long r = 0; for (; x; x -= x & -x) r += t[x]; return r; }
// 接口：range(l, r)：返回 1-based 闭区间 [l,r] 的元素和；返回类型 long long。
// 参数：闭区间左端点 l；闭区间右端点 r。
long long range(int l, int r) const { return sum(r) - sum(l - 1); }
};

```

应用：逆序对、动态排名、离线矩形计数、二维偏序（配合 CDQ/排序）、第 k 个值（二进制提升）。

7.4 线段树

支持任意可合并区间信息。节点存区间答案，`push_up` 合并左右儿子；区间修改要有懒标记并在访问子节点前 `push_down`。

7.4.1 常见维护

- 区间和 + 区间加：标记加法，子节点和增加 $\text{tag} \times \text{len}$ 。
- 区间和 + 区间乘/加：标记为仿射变换 $x \rightarrow \text{mul} \times x + \text{add}$ ，下传时 $\text{add} = \text{add} \times \text{mul_parent} + \text{add_parent}$ 。
- 区间最值 + 区间加：维护最大/最小值及懒加。
- 区间赋值：赋值标记覆盖旧加法；标记优先级必须明确。
- 扫描线：维护每个离散小区间是否被覆盖，保存 `cover` 与 `len`。

复杂度：建树 $O(n)$ ，单次查询/修改 $O(\log n)$ 。只要操作满足结合律，可把节点信息设计成结构体。

```

struct SegTree {
    int n; // 原数组下标为闭区间 [1,n]。
    vector<long long> sum; // sum[p]=节点 p 所管区间的元素和。
    vector<long long> lazy; // lazy[p]=尚未下传的“区间每项增加量”。
    // 接口：SegTree(n)：构造管理 1..n 闭区间的空线段树；构造并初始化对象；每组测试建议重新构造。
    // 参数：线段树管理的 1-based 数组长度及最大下标 n。
    SegTree(int n): n(n), sum(4 * n + 4), lazy(4 * n + 4) {}
    // 接口：apply(p, l, r, v)：把本次懒操作直接作用于节点 p，并同步节点值与标记；会修改当前结构；多测时重新构造或显式清空成员。
    // 参数：当前线段树节点编号 p（根通常为 1）；闭区间左端点 l；闭区间右端点 r；目标区间内每个元素的统一增加量 v。
    void apply(int p, int l, int r, long long v) {
        sum[p] += v * (r - l + 1); lazy[p] += v;
    }
};

```

```

}
// 接口: push(p, l, r): 把线段树节点 p 的懒标记下传到左右孩子并清空本节点标记; 会修改当前结构; 多测时重新构造或显式清空成员。
// 参数: 当前线段树节点编号 p (根通常为 1); 闭区间左端点 l; 闭区间右端点 r。
void push(int p, int l, int r) {
    if (!lazy[p] || l == r) return;
    // 变量: m=当前递归区间 [l,r] 的中点 m。
    int m = (l + r) / 2;
    apply(p * 2, l, m, lazy[p]);
    apply(p * 2 + 1, m + 1, r, lazy[p]);
    lazy[p] = 0;
}
// 接口: add(p, l, r, ql, qr, v): 把目标闭区间 [ql,qr] 的元素统一增加 v; 会修改当前结构; 多测时重新构造或显式清空成员。
// 参数: 当前线段树节点编号 p (根通常为 1); 闭区间左端点 l; 闭区间右端点 r; 目标闭区间左端 ql; 目标闭区间右端 qr; 目标区间内每个元素的统一增加量 v。
void add(int p, int l, int r, int ql, int qr, long long v) {
    if (ql <= l && r <= qr) return apply(p, l, r, v);
    // 变量: m=当前递归区间 [l,r] 的中点 m。
    push(p, l, r); int m = (l + r) / 2;
    if (ql <= m) add(p * 2, l, m, ql, qr, v);
    if (qr > m) add(p * 2 + 1, m + 1, r, ql, qr, v);
    sum[p] = sum[p * 2] + sum[p * 2 + 1];
}
// 接口: query(p, l, r, ql, qr): 返回目标闭区间 [ql,qr] 的元素和; 返回类型 long long。
// 参数: 当前线段树节点编号 p (根通常为 1); 闭区间左端点 l; 闭区间右端点 r; 目标闭区间左端 ql; 目标闭区间右端 qr。
long long query(int p, int l, int r, int ql, int qr) {
    if (ql <= l && r <= qr) return sum[p];
    // 变量: m=当前递归区间 [l,r] 的中点 m; ans=线段树查询目标区间时从左右子树累加的区间和 ans。
    push(p, l, r); int m = (l + r) / 2; long long ans = 0;
    if (ql <= m) ans += query(p * 2, l, m, ql, qr);
    if (qr > m) ans += query(p * 2 + 1, m + 1, r, ql, qr);
    return ans;
}
};

```

7.4.2 线段树上的二分

维护区间和/最大值时, 可从根向下找“第一个满足条件的位置”, 每层只走一个孩子, $O(\log n)$, 比外层二分 + 查询更快。

```
// 接口: first_at_least(p, l, r, x): 返回查询范围内第一个值至少为 x 的位置, 不存在返回
//      -1; 返回类型 int。
// 参数: 当前线段树节点编号 p (根通常为 1); 闭区间左端点 l; 闭区间右端点 r; 查询要求达到
//      的值下界 x。
int first_at_least(int p, int l, int r, long long x) {
    if (tree[p] < x) return -1;
    if (l == r) return l;
    // 变量: m=当前递归区间 [l,r] 的中点 m。
    int m = (l + r) / 2;
    if (tree[p * 2] >= x) return first_at_least(p * 2, l, m, x);
    return first_at_least(p * 2 + 1, m + 1, r, x);
}
```

7.5 ST 表 (Sparse Table, 静态区间查询)

静态数组的幂等运算 (min/max/gcd/and/or) 预处理 $O(n \log n)$, 查询 $O(1)$:

$k = \text{floor}(\log_2(\text{len}))$, 答案为 `merge(st[k][l], st[k][r - 2^k + 1])`。不可用于 sum 这类非幂等运算 (重叠会重复计数)。下面的实现约定输入数组非空, 数组下标和查询端点均为 0-based, 查询区间为闭区间 $[l, r]$ 。

```
struct SparseTable {
    // 变量: st[k][i]=从 i 开始、长度为 2^k 的区间最小值。
    vector<vector<int>>> st;
    // 变量: lg[len]=floor(log2(len)), 用于确定查询所需层数。
    vector<int> lg;

    // 接口: SparseTable(a): 用非空数组 a 构建区间最小值 ST 表; 预处理  $O(n \log n)$ 。
    explicit SparseTable(const vector<int>& a) {
        // 变量: n=容器 a 的元素数量 n。
        int n = a.size();
        lg.assign(n + 1, 0);
        // 变量: i=当前计算二进制对数的区间长度。
        for (int i = 2; i <= n; ++i)
            lg[i] = lg[i / 2] + 1;

        st.assign(lg[n] + 1, vector<int>(n));
        st[0] = a;
        // 变量: k=当前预处理区间长度的二进制指数。
        for (int k = 1; k < (int)st.size(); ++k)
            // 变量: i=稀疏表第 k 层当前预处理区间的 0-based 左端点 i。
            for (int i = 0; i + (1 << k) <= n; ++i)
                st[k][i] = min(st[k - 1][i],
                               st[k - 1][i + (1 << (k - 1))]);
    }
};
```



```

}

// 接口: query(l,r): 返回 0-based 闭区间 [l,r] 的最小值; 要求 0<=l<=r<n。
int query(int l, int r) const {
    // 变量: k=floor(log2(r-l+1)), 两段长度 2^k 的区间可覆盖查询范围。
    int k = lg[r - l + 1];
    return min(st[k][l], st[k][r - (1 << k) + 1]);
}
};

```

7.6 堆与可并堆思想

`priority_queue` 用于每步取最小/最大。双堆维护中位数（小根维护大半边，大根维护小半边）。懒删除：堆中保留旧值，弹出时与当前版本/计数比对。

左偏树/可并堆支持两个堆合并，合并复杂度 $O(\log n)$ ；题目要求“合并集合并取最值”时考虑。斜堆实现更短但无严格最坏保证。

```

priority_queue<int> low; // 小的一半，堆顶是最大值
// 变量: high=双堆中保存较大一半元素的小根堆 high。
priority_queue<int, vector<int>, greater<int>> high;
// 接口: add_number(x): 把整数 x 加入双堆中位数结构并重新平衡两堆大小; 不返回值。
// 参数: 待加入统计结构的整数 x。
void add_number(int x) {
    if (low.empty() || x <= low.top()) low.push(x); else high.push(x);
    if (low.size() > high.size() + 1) high.push(low.top()), low.pop();
    if (high.size() > low.size()) low.push(high.top()), high.pop();
}
// 接口: median(): 返回当前两个堆维护的中位数（奇数个元素时为正中间）; 返回类型 int。
int median() { return low.top(); }

```

7.7 单调结构与差分约束

单调栈/队列见 01-基础与实现规范。差分约束 $x[v] \leq x[u] + w$ 建边 $u \rightarrow v$ ；把全部点作为源点可检查任意连通分量中的负环。

```

// 接口: difference_constraints(n, edges): 在 __int128 域求一组满足  $x[v] \leq x[u] + w$ 
// 的势函数; 存在负环时返回 nullopt。
// 参数: 变量数 n; 约束边 (u,v,w), 顶点使用 0-based 编号。
optional<vector<__int128>> difference_constraints(
    int n, const vector<tuple<int, int, long long>>& edges) {
    // 变量: g=由差分约束构成的有向带权邻接表。
    vector<vector<pair<int, long long>>> g(n);

```

```

// 变量: u=约束边起点; v=约束边终点; w=x[v]-x[u] 的上界。
for (auto [u, v, w] : edges) g[u].push_back({v, w});
// 变量: dist=当前找到的各变量可行上界势函数。
vector<__int128> dist(n, 0);
// 变量: path_edges=产生当前 dist 值的松弛路径边数, 用于识别负环。
vector<int> path_edges(n, 0);
// 变量: in_queue=顶点当前是否已经在待松弛队列中。
vector<char> in_queue(n, true);
// 变量: q=等待扫描出边的顶点队列。
queue<int> q;
// 变量: u=初始化入队的顶点编号。
for (int u = 0; u < n; ++u) q.push(u);
while (!q.empty()) {
    // 变量: u=本轮从队首取出的约束边起点。
    int u = q.front(); q.pop(); in_queue[u] = false;
    // 变量: v=约束边终点; w=x[v]-x[u] 的上界。
    for (auto [v, w] : g[u]) {
        // 变量: candidate=经约束边 u->v 得到的宽整数候选势函数值。
        __int128 candidate = dist[u] + (__int128)w;
        if (dist[v] <= candidate) continue;
        dist[v] = candidate;
        path_edges[v] = path_edges[u] + 1;
        if (path_edges[v] >= n) return nullopt;
        if (!in_queue[v]) in_queue[v] = true, q.push(v);
    }
}
return dist;
}

```

7.8 字典树 (Trie, Prefix Tree)

字符集固定时数组儿子最快; 动态字符集用 `map`。插入/查询长度 `L` 的字符串均为 $O(L)$ 。二进制 Trie 支持最大异或、区间内最大异或 (结合持久化或离线前缀)。

```

struct BinaryTrie {
    int ch[200000][2]; // ch[u][bit]=下一节点编号; 0 表示不存在。
    int cnt[200000]; // cnt[u]=经过节点 u 的已插入数字个数。
    int nodes = 1; // 已分配节点数; 根节点固定为 0。
    // 接口: insert(x): 把整数 x 的各二进制位插入 Trie; 会修改当前结构; 多测时重新构造或显式清空成员。
    // 参数: 待按二进制位插入 Trie 的非负整数 x。
    void insert(int x) {
        // 变量: u=二进制 Trie 根节点编号 u。

```

```

    int u = 1; ++cnt[u];
    // 变量: b=二进制 Trie 当前处理的位编号 b。
    for (int b = 30; b >= 0; --b) {
        // 变量: bit=二进制 Trie 插入时从高到低处理的位编号 bit。
        int bit = (x >> b) & 1;
        if (!ch[u][bit]) ch[u][bit] = ++nodes;
        u = ch[u][bit]; ++cnt[u];
    }
}
// 接口: max_xor(x): 沿二进制 Trie 贪心返回与 x 的最大异或值; 返回类型 int。
// 参数: 求最大异或值时使用的初始值 x。
int max_xor(int x) {
    // 变量: u=二进制 Trie 根节点编号 u; ans=二进制 Trie 贪心过程中已经确定的最大
    异或值 ans。
    int u = 1, ans = 0;
    // 变量: b=二进制 Trie 当前处理的位编号 b。
    for (int b = 30; b >= 0; --b) {
        // 变量: bit=二进制 Trie 最大异或查询中从高到低处理的位编号 bit。
        int bit = (x >> b) & 1;
        if (ch[u][bit ^ 1] && cnt[ch[u][bit ^ 1]])
            ans |= 1 << b, u = ch[u][bit ^ 1];
        else u = ch[u][bit];
    }
    return ans;
}
};

```

7.9 可持久化数据结构

每次修改只复制从根到叶子的 $O(\log n)$ 个节点, 旧版本仍可查询。主席树(可持久化权值线段树)用于静态区间第 k 小: $root[r]-root[l-1]$ 在树上同步下行, $O(\log V)$ 。注意节点池容量约为版本数 $\times \log V$ 。

```

struct Node { int l = 0, r = 0, sum = 0; } tr[4000000]; // 左右儿子编号及该值域节点计数。
// 变量: tot=当前已经创建的数据结构节点数量 tot。
int tot = 0;
// 接口: update(old, l, r, pos): 从旧版本根 old 复制到 pos 的路径、把该值域计数加一并返回新版本根; 返回类型 int。
// 参数: 旧版本根节点编号 old; 闭区间左端点 l; 闭区间右端点 r; 持久化线段树中要修改的 0-based 位置 pos。
int update(int old, int l, int r, int pos) {
    // 变量: now=本次路径复制新建的持久化权值线段树节点编号 now。

```

```

int now = ++tot; tr[now] = tr[old]; ++tr[now].sum;
if (l != r) {
    // 变量: m=当前递归区间 [l,r] 的中点 m。
    int m = (l + r) / 2;
    if (pos <= m) tr[now].l = update(tr[old].l, l, m, pos);
    else tr[now].r = update(tr[old].r, m + 1, r, pos);
}
return now;
}

// 接口: kth(u, v, l, r, k): 返回前缀版本差 v-u 所表示区间内第 k 小元素的离散值下标;
// 返回类型 int。
// 参数: 左端前缀版本根 u; 右端前缀版本根 v; 闭区间左端点 l; 闭区间右端点 r; 查询的 1-
// based 排名 k。
int kth(int u, int v, int l, int r, int k) {
    if (l == r) return l;
    // 变量: left_count=当前节点左子树中的元素数量 left_count。
    int left_count = tr[tr[v].l].sum - tr[tr[u].l].sum;
    // 变量: m=当前递归区间 [l,r] 的中点 m。
    int m = (l + r) / 2;
    if (k <= left_count) return kth(tr[u].l, tr[v].l, l, m, k);
    return kth(tr[u].r, tr[v].r, m + 1, r, k - left_count);
}

```

7.10 分块与莫队算法 (Mo's Algorithm)

7.10.1 块状数组

把数组分成 $\sqrt{n}$ 块, 块内暴力、块间整块处理; 区间加/区间和约 $O(\sqrt{n})$ 。适合操作复杂、难以线段树维护的信息。

```

// 变量: B=块状数组采用的块长 B。
int B = max(1, (int)sqrt(n));
// 变量: a=块状数组逐点保存的原始元素数组 a; tag=当前节点保存的懒标记 tag。
vector<long long> a(n), tag((n + B - 1) / B);
// 接口: add(l, r, v): 用分块懒标记把闭区间 [l,r] 的元素统一增加 v; 会修改当前结构; 多
// 测时重新构造或显式清空成员。
// 参数: 闭区间左端点 l; 闭区间右端点 r; 闭区间 [l,r] 内每个元素的统一增加量 v。
void add(int l, int r, long long v) {
    // 变量: bl=当前块左端编号 bl; br=当前块右端编号 br。
    int bl = l / B, br = r / B;
    // 变量: i=分块区间修改中直接更新的数组位置 i。
    if (bl == br) { for (int i = l; i <= r; ++i) a[i] += v; return; }
    // 变量: i=分块修改左侧不完整块中直接更新的数组位置 i。
}

```

```

for (int i = l; i < (bl + 1) * B; ++i) a[i] += v;
// 变量: b=块状数组当前整块更新的块编号 b。
for (int b = bl + 1; b < br; ++b) tag[b] += v;
// 变量: i=分块修改右侧不完整块中直接更新的数组位置 i。
for (int i = br * B; i <= r; ++i) a[i] += v;
}

```

7.10.2 莫队

离线排序询问的左右端点，维护当前区间并增删元素；移动总复杂度约 $O((n+q)\sqrt{n})$ 。需要可逆的 `add/remove`，如频次、不同数个数、答案贡献。带修改莫队多一个时间维，块大小约 $n^{2/3}$ 。

```

// 变量: block=普通莫队按查询左端点分块时使用的块长 block。
int block;
struct Query { int l, r, id; }; // 1-based 闭区间 [l,r] 与原始询问编号 id。
// 接口: 比较器 lambda(x, y): 按莫队左端块排序，并在同块内按奇偶交替排列右端点。
// 参数: 第一个莫队查询 x; 第二个莫队查询 y。
sort(qs.begin(), qs.end(), [&](const Query& x, const Query& y) {
    // 变量: bx=查询 x 所在块编号 bx; by=查询 y 所在块编号 by。
    int bx = x.l / block, by = y.l / block;
    if (bx != by) return bx < by;
    return (bx & 1) ? x.r > y.r : x.r < y.r;
});
// 变量: L=普通莫队当前 1-based 窗口的左端点 L; R=普通莫队当前 1-based 窗口的右端点 R
// ; 空窗口时 R=0; distinct=当前窗口内不同值的数量 distinct。
int L = 1, R = 0, distinct = 0; // 1-based 空窗口 [1,0]。
// 接口: add(p): 把位置 p 加入莫队窗口，并在该值首次出现时增加 distinct。
// 参数: 要加入窗口的数组位置 p。
// 变量: add=把数组位置加入当前莫队窗口的局部 lambda add。
auto add = [&](int p) { if (++cnt[a[p]] == 1) ++distinct; };
// 接口: remove(p): 把位置 p 移出莫队窗口，并在该值不再出现时减少 distinct。
// 参数: 要移出窗口的数组位置 p。
// 变量: remove=把位置移出当前窗口的局部 lambda remove。
auto remove = [&](int p) { if (--cnt[a[p]] == 0) --distinct; };
// 变量: qu=当前离线查询 qu。
for (auto qu : qs) {
    while (L > qu.l) add(--L); while (R < qu.r) add(++R);
    while (L < qu.l) remove(L++); while (R > qu.r) remove(R--);
    answer[qu.id] = distinct;
}

```

7.11 平衡树思想（了解 Treap）

随机优先级的 BST，支持按键插入、删除、排名、第 k 小、分裂/合并，期望 $O(\log n)$ 。无序序列维护时使用 FHQ Treap，节点存子树大小、区间反转标记、区间和/最大值。需要严格稳定最坏复杂度时优先线段树或 `std::set`。

```
struct Node { int l, r, key, pri, sz; } tr[MAXN]; // 儿子、键、随机优先级、子树大小。
// 变量: root=当前树结构的根节点编号 root; tot=当前已经创建的数据结构节点数量 tot。
int root, tot;
// 接口: size(u): 返回节点 u 所代表子树的节点数，空节点 0 的大小为 0; 返回类型 int。
// 参数: 待统计大小的 Treap 子树根 u。
int size(int u) { return u ? tr[u].sz : 0; }
// 接口: pull(u): 由左右孩子重算 Treap 节点 u 的子树大小; 会修改当前结构; 多测时重新构造或显式清空成员。
// 参数: 待重算大小的 Treap 节点 u。
void pull(int u) { tr[u].sz = size(tr[u].l) + size(tr[u].r) + 1; }
// 接口: split(u, key, x, y): 按键值 key 把 Treap u 拆成键值不大于 key 的左树 x 与其余右树 y; 不返回值。
// 参数: 待拆分的 Treap 根 u; 拆分键 key; 接收拆分后左 Treap 根的引用 x; 接收拆分后右 Treap 根的引用 y。
void split(int u, int key, int &x, int &y) {
    if (!u) return x = y = 0, void();
    if (tr[u].key <= key) split(tr[u].r, key, tr[u].r, y), x = u;
    else split(tr[u].l, key, x, tr[u].l), y = u;
    pull(u);
}
// 接口: merge(x, y): 按优先级合并键值有序的左右 Treap，并返回新根; 返回类型 int。
// 参数: 待合并的左 Treap 根 x; 待合并的右 Treap 根 y。
int merge(int x, int y) {
    if (!x || !y) return x ? x : y;
    if (tr[x].pri > tr[y].pri) return tr[x].r = merge(tr[x].r, y), pull(x), x;
    return tr[y].l = merge(x, tr[y].l), pull(y), y;
}
```

7.12 撤销与可回滚结构

不做路径压缩的 DSU 可用栈记录每次修改，支持回滚到快照 $O(\log n)$ 合并、 $O(1)$ 回滚；配合线段树分治时间解决“边在一段时间内存在”的动态连通性。

```
struct RollbackDSU {
    vector<int> p, sz; // p[x]=父节点; sz[root]=集合大小; 不做路径压缩。
    vector<pair<int, int>> hist; // 每次成功合并的撤销记录; 长度就是快照。
}
```

```

// 接口: RollbackDSU(n): 构造支持快照和回滚、且不做路径压缩的并查集; 构造并初始化对象; 每组测试建议重新构造。
// 参数: 可回滚并查集管理的元素编号上界 n。
RollbackDSU(int n): p(n + 1), sz(n + 1, 1) { iota(p.begin(), p.end(), 0); }
// 接口: find(x): 返回 x 当前所在并查集的代表元; 返回类型 int。
// 参数: 待查询代表元的元素编号 x。
int find(int x) { while (x != p[x]) x = p[x]; return x; }
// 接口: snapshot(): 返回当前撤销栈长度, 供 rollback 恢复; 返回类型 int。
int snapshot() { return hist.size(); }
// 接口: unite(a, b): 合并两个元素所在集合; 原本同集合时返回 false; 实际合并两个集合时返回 true。
// 参数: 第一个待合并元素编号 a; 第二个待合并元素编号 b。
bool unite(int a, int b) {
    a = find(a); b = find(b); if (a == b) return false;
    if (sz[a] < sz[b]) swap(a, b);
    hist.push_back({b, sz[a]}); p[b] = a; sz[a] += sz[b]; return true;
}
// 接口: rollback(snap): 把可撤销数据结构恢复到指定快照; 会修改当前结构; 多测时重新构造或显式清空成员。
// 参数: 此前 snapshot() 返回的撤销栈长度 snap。
void rollback(int snap) {
    while ((int)hist.size() > snap) {
        auto [b, old] = hist.back(); hist.pop_back();
        sz[p[b]] = old; p[b] = b;
    }
}
};

```

7.13 李超线段树 (Li Chao Segment Tree)

维护直线集合在离散/整数横坐标上的最值。每个节点保存当前区间中在中点最优的直线, 递归把另一条直线送入可能胜出的子区间; 插入和查询均为 $O(\log V)$ 。斜率、截距和查询坐标使用 `longlong`, 求值与查询结果统一使用 `__int128`, 因此不会在乘加处发生窄整数溢出。

```

struct LiChao {
    using ll = long long;
    struct Line {
        // 这份代码维护最小值; 显式 empty 标记不会与任何合法直线值冲突。
        // 变量: k=空李超树节点使用的直线斜率 k; b=空李超树节点使用的直线截距 b。
        ll k = 0, b = 0;
        bool empty = true; // 当前节点是否尚未保存真实直线。
        // 接口: Line(): 构造一条空直线; 构造并初始化对象。

```



```

Line() = default;
// 接口: Line(k, b): 构造斜率 k、截距 b 的真实直线; 构造并初始化对象。
// 参数: 直线斜率 k; 直线截距 b。
Line(ll k, ll b): k(k), b(b), empty(false) {}
// 接口: get(x): 返回直线在横坐标 x 处的函数值; 返回类型 __int128。
// 参数: 计算直线函数值的横坐标 x。
__int128 get(ll x) const {
    return empty ? ((__int128)1 << 126) : (__int128)k * x + b;
}
};
int L, R;           // 允许查询的整数横坐标闭区间 [L,R]。
vector<Line> tr;      // tr[p]: 节点区间内当前保留下来的候选直线。
// 接口: storage_size(L, R): 返回整数闭区间 [L,R] 的静态李超树数组长度。
// 参数: 闭区间左端点 L; 闭区间右端点 R, 要求 L<=R。
static size_t storage_size(int L, int R) {
    if (L > R) throw std::invalid_argument("empty Li Chao domain");
    return 4ULL * ((long long)R - L + 1);
}
// 接口: LiChao(L, R): 构造整数横坐标闭区间 [L,R] 上的李超树; 构造并初始化对象; 每组
// 测试建议重新构造。
// 参数: 闭区间左端点 L; 闭区间右端点 R。
LiChao(int L, int R): L(L), R(R), tr(storage_size(L, R)) {}

// 接口: add_line(nw, p, l, r): 把候选直线 nw 插入整数横坐标闭区间 [l,r] 的李超树
// 节点 p; 会修改当前结构; 多测时重新构造或显式清空成员。
// 参数: 待插入李超树的直线 nw; 当前节点 u 的父节点 p; 闭区间左端点 l; 闭区间右端点
// r。
void add_line(Line nw, int p, int l, int r) {
    // 先比较新线和当前线在端点/中点的优劣。
    // 变量: m=当前递归区间 [l,r] 的中点 m。
    int m = l + (int)((long long)r - l) / 2;
    // 变量: lef=新直线在区间左端是否更优 lef。
    bool lef = nw.get(l) < tr[p].get(l);
    // 变量: mid=当前区间中点 mid。
    bool mid = nw.get(m) < tr[p].get(m);

    // 中点更优的线留在当前节点, 旧线递归到可能赢的半边。
    if (mid) swap(nw, tr[p]);
    if (l == r) return;

    // 两线在左端和中点的相对优劣发生变化, 交点在左半。
    if (lef != mid) add_line(nw, p * 2, l, m);
    else add_line(nw, p * 2 + 1, m + 1, r);
}

```



```

// 对外接口：只需要传入斜率 k 和截距 b，不要手动传根节点参数。
// 接口：add_line(k, b)：把斜率 k、截距 b 组成的直线插入李超树；会修改当前结构；多
// 测时重新构造或显式清空成员。
// 参数：待插入直线的斜率 k；待插入直线的截距 b。
void add_line(ll k, ll b) { add_line({k, b}, 1, L, R); }

// 接口：query(x, p, l, r)：返回横坐标 x 处沿李超树查询路径得到的最优直线函数值；返
// 回类型 __int128。
// 参数：计算候选直线最优值的横坐标 x；当前线段树节点编号 p（根通常为 1）；闭区间左端
// 点 l；闭区间右端点 r。
__int128 query(ll x, int p, int l, int r) const {
    // x 所在路径上的每个节点都可能保存一条更优直线，必须全部比较。
    // 变量：ans=李超树当前节点及递归路径在横坐标 x 处的最优函数值 ans。
    __int128 ans = tr[p].get(x);
    if (l == r) return ans;
    // 变量：m=当前递归区间 [l,r] 的中点 m。
    int m = l + (int)((long long)r - l) / 2;
    if (x <= m) return min(ans, query(x, p * 2, l, m));
    return min(ans, query(x, p * 2 + 1, m + 1, r));
}
// x 必须落在构造时给出的 [L,R] 内；大坐标范围应改用动态开点版本。
// 接口：query(x)：返回李超树中全部候选直线在横坐标 x 处的最优函数值；返回类型
__int128。
// 参数：计算候选直线最优值的横坐标 x。
__int128 query(ll x) const { return query(x, 1, L, R); }
};

```

求最大值时把比较符号整体反向。若只会在若干离散横坐标上查询，可以另存有序真实坐标 xs ，递归用下标分区，但直线求值必须代入 $xs[mid]$ 与 $xs[index]$ ，不能把压缩编号当成真实横坐标。

7.14 区间最值势能线段树 (Segment Tree Beats)

“区间取 $\min$ + 区间和/最大值”不能用普通懒标记，因为只有当前最大值会被压低。维护最大值、次大值、最大值出现次数和区间和；当 x 严格大于次大值时可以整段打标记。

```

struct SegBeats {
    using ll = long long;
    // 变量：NEG=不存在次大值时使用的 __int128 负无穷哨兵 NEG。
    static constexpr __int128 NEG = -((__int128)1 << 126);
    int n; // 原数组及所有公开区间接口均使用 1-based 闭区间 [1,n]。
    vector<__int128> sum; // 精确区间和，允许超过 long long 范围。
    vector<ll> mx; // 区间最大值。

```

```
vector<__int128> second_mx; // 严格小于最大值的次大值；宽类型使哨兵不与 ll 合法值冲突。
```

```
vector<int> cnt_mx; // 最大值出现次数。
```

// 接口: `SegBeats(a)`: 由 1-based 数组 `a` 构造区间 `chmin` + 区间和线段树；构造并初始化对象；每组测试建议重新构造。

// 参数: 用于构造线段树的 1-based 初始数组 `a`。

```
SegBeats(const vector<ll>& a): n((int)a.size() - 1),
    sum(4 * n + 4), mx(4 * n + 4), second_mx(4 * n + 4),
    cnt_mx(4 * n + 4) { build(1, 1, n, a); }
```

// 接口: `pull(p)`: 由两个孩子重算线段树节点 `p` 的聚合信息；会修改当前结构；多测时重新构造或显式清空成员。

// 参数: 当前线段树节点编号 `p` (根通常为 1)。

```
void pull(int p) {
    // 父节点只保存“最大值这一类”和“次大值这一类”，
    // 不需要保存整个儿子数组，因此 chmin 才能整段快速处理。
    sum[p] = sum[p * 2] + sum[p * 2 + 1];
    if (mx[p * 2] == mx[p * 2 + 1]) {
        mx[p] = mx[p * 2]; cnt_mx[p] = cnt_mx[p * 2] + cnt_mx[p * 2 + 1];
        second_mx[p] = max(second_mx[p * 2], second_mx[p * 2 + 1]);
    } else {
        // 变量: x=线段树节点 p 的左儿子编号 x; y=线段树节点 p 的右儿子编号 y。
        int x = p * 2, y = p * 2 + 1;
        if (mx[x] < mx[y]) swap(x, y);
        mx[p] = mx[x]; cnt_mx[p] = cnt_mx[x];
        second_mx[p] = max(second_mx[x], (__int128)mx[y]);
    }
}
```

// 接口: `build(p, l, r, a)`: 由 1-based 数组 `a` 建立区间 `chmin` 与区间和线段树；完成初始化；多测时每组重新调用。

// 参数: 当前线段树节点编号 `p` (根通常为 1)；闭区间左端点 `l`；闭区间右端点 `r`；用于构造当前数据结构的初始数组 `a`。

```
void build(int p, int l, int r, const vector<ll>& a) {
    if (l == r) {
        // 叶子只有一个最大值，次大值设为负无穷。
        sum[p] = mx[p] = a[l]; second_mx[p] = NEG; cnt_mx[p] = 1;
        return;
    }
    // 变量: m=当前递归区间 [l,r] 的中点 m。
    int m = (l + r) / 2;
    build(p * 2, l, m, a); build(p * 2 + 1, m + 1, r, a); pull(p);
}
```

// 接口: `apply_chmin(p, x)`: 给当前线段树节点应用 `chmin` 标记；不返回值。

// 参数: 当前线段树节点编号 `p` (根通常为 1)；节点最大值要压低到的上界 `x`。

```

void apply_chmin(int p, ll x) {
    // 调用者保证 second_mx[p] < x < mx[p],
    // 因而只有 cnt_mx 个最大值会下降, sum 可以 O(1) 更新。
    if (x >= mx[p]) return;
    sum[p] -= ((__int128)mx[p] - x) * cnt_mx[p];
    mx[p] = x;
}

// 接口: push(p): 把 Segment Tree Beats 节点 p 的最大值上界约束下传给两个孩子; 会
// 修改当前结构; 多测时重新构造或显式清空成员。
// 参数: 当前线段树节点编号 p (根通常为 1)。
void push(int p) {
    // 父节点的 mx 变小后, 儿子的最大值可能仍比父亲大,
    // 把父亲的上界传给儿子即可, 不需要递归到底。
    if (mx[p * 2] > mx[p]) apply_chmin(p * 2, mx[p]);
    if (mx[p * 2 + 1] > mx[p]) apply_chmin(p * 2 + 1, mx[p]);
}

// 接口: chmin(p, l, r, ql, qr, x): 把目标闭区间内每个值更新为 min(原值, x); 不返回
// 值。
// 参数: 当前节点 u 的父节点 p; 闭区间左端点 l; 闭区间右端点 r; 目标闭区间左端 ql;
// 目标闭区间右端 qr; 目标区间元素要压低到的上界 x。
void chmin(int p, int l, int r, int ql, int qr, ll x) {
    // 无交集或当前最大值已经不超过 x 时, 操作没有影响。
    if (qr < l || r < ql || x >= mx[p]) return;
    // x 大于次大值时只有最大值受影响, 触发势能线段树的核心优化。
    if (ql <= l && r <= qr && x > second_mx[p]) return apply_chmin(p, x);
    // 否则必须下传并继续递归; 删掉这个条件会退化甚至超时。
    // 变量: m=当前递归区间 [l, r] 的中点 m。
    push(p); int m = (l + r) / 2;
    chmin(p * 2, l, m, ql, qr, x);
    chmin(p * 2 + 1, m + 1, r, ql, qr, x); pull(p);
}

// 接口: query_sum(p, l, r, ql, qr): 返回闭区间查询的元素和; 返回类型 __int128。
// 参数: 当前节点 u 的父节点 p; 闭区间左端点 l; 闭区间右端点 r; 目标闭区间左端 ql;
// 目标闭区间右端 qr。
__int128 query_sum(int p, int l, int r, int ql, int qr) {
    if (qr < l || r < ql) return 0;
    if (ql <= l && r <= qr) return sum[p];
    // 变量: m=当前递归区间 [l, r] 的中点 m。
    push(p); int m = (l + r) / 2;
    return query_sum(p * 2, l, m, ql, qr)
        + query_sum(p * 2 + 1, m + 1, r, ql, qr);
}

// 接口: chmin(l, r, x): 把目标闭区间内每个值更新为 min(原值, x); 不返回值。
// 参数: 闭区间左端点 l; 闭区间右端点 r; 目标区间元素要压低到的上界 x。

```

```

void chmin(int l, int r, ll x) { chmin(1, 1, n, l, r, x); }
// 接口: query_sum(l, r): 返回闭区间查询的精确元素和; 返回类型 __int128。
// 参数: 闭区间左端点 l; 闭区间右端点 r。
__int128 query_sum(int l, int r) { return query_sum(1, 1, n, l, r); }
};

```

7.15 高级数据结构：参数、调用与改造

本节紧跟前面的基础模板。先看每个接口传什么，再看哪些不变量不能动，最后看如何替换维护信息或转移。

7.15.1 高阶前缀和与多阶差分

看到“每次给区间加一个一次/二次多项式，最后问点值或前缀”时，普通差分不够。若对差分数组再差分，区间加常数会变成两个端点；加一次函数需要更高阶差分或维护多个系数。

最实用的现场做法是先把更新写成基函数，再分别维护系数的差分。比如区间 $[l, r]$ 加 $a \times i + b$ ，维护两个差分数组 da, db ：

```

// 变量: da=一次项系数的差分数组 da; db=常数项系数的差分数组 db。
vector<long long> da(n + 2), db(n + 2);
// 接口: range_add_linear(l, r, a, b): 在闭区间 [l,r] 给位置 i 加上  $a \times i + b$ ; 不返回值。
// 参数: 闭区间左端点 l; 闭区间右端点 r; 位置 i 增量  $a \times i + b$  的一次项系数 a; 位置 i 增量  $a \times i + b$  的常数项 b。
void range_add_linear(int l, int r, long long a, long long b) {
    da[l] += a; da[r + 1] -= a;
    db[l] += b; db[r + 1] -= b;
}
// 接口: build_linear_values(): 在所有更新结束后还原 1..n 的点值; 返回类型 vector<long long>。
vector<long long> build_linear_values() {
    // 变量: value=还原后的点值数组 value; ca=当前位置的一次项系数; cb=当前位置的常数项系数。
    vector<long long> value(n + 1);
    // 变量: ca=当前位置的一次项系数; cb=当前位置的常数项系数。
    long long ca = 0, cb = 0;
    // 变量: i=还原线性区间更新后点值的 1-based 位置 i。
    for (int i = 1; i <= n; ++i) {
        ca += da[i]; cb += db[i];
        value[i] = ca * i + cb;
    }
    return value;
}

```

```
}
```

若询问的是区间和，则要再维护系数的前缀贡献，或者直接使用带懒标记的线段树。不要在考场临时猜公式：先写出单点贡献 $a \times i + b$ ，再求 $l..r$ 的和 $a \times (l+r) \times (r-l+1)/2 + b \times (r-l+1)$ ，检查端点和取模。

7.15.2 多懒标记与仿射标记线段树

区间操作同时有“乘法”和“加法”时，标记不是两个互相独立的数，而是函数：

$$x \mapsto ax + b.$$

先执行旧标记 (a_1, b_1) ，再执行新标记 (a_2, b_2) ，合成结果为：

$$a = a_2 a_1, \quad b = a_2 b_1 + b_2.$$

```
struct Tag {
    // 一个懒标记表示：把区间内每个 x 变成 mul * x + add。
    // 初始值必须是恒等变换 x -> x，而不是 0。
    // 变量：mul=全局仿射懒标记中作用于每个元素的乘法系数 mul。
    long long mul = 1;
    long long add = 0; // 仿射变换的常数项；恒等标记为 {mul=1, add=0}。
};

struct Node {
    long long sum = 0; // 当前区间模 MOD 的和，规范化到 [0, MOD)。
    Tag tag;           // 尚未下传给儿子的变换。
};

vector<Node> tr;       // 调用前按线段树所需节点数分配；多测时重新初始化。
// 接口：apply(p, l, r, mul, add)：把本次懒操作直接作用于节点 p，并同步节点值与标记；
// 会修改当前结构；多测时重新构造或显式清空成员。
// 参数：当前线段树节点编号 p（根通常为 1）；该节点对应闭区间的左端点 l；右端点 r；新仿射
// 变换 x->mul*x+add 的乘法系数 mul；该变换的加法常数 add；要求 0<MOD 且各量已规范化
// 到 [0, MOD)。
void apply(int p, int l, int r, long long mul, long long add) {
    // 先更新“区间答案”：区间和乘 mul，再额外加 add * 区间长度。
    tr[p].sum = ((__int128)tr[p].sum * mul
        + (__int128)add * (r - l + 1)) % MOD;

    // 原来的标记是 f(x)=old_mul*x+old_add,
    // 新操作是 g(x)=mul*x+add，实际顺序是 g(f(x))。
    // 合成后：mul'=mul*old_mul, add'=mul*old_add+add。
    tr[p].tag.mul = ((__int128)tr[p].tag.mul * mul % MOD;
    tr[p].tag.add = ((__int128)tr[p].tag.add * mul + add) % MOD;
```

```

}
// 接口: push(p, l, r): 把线段树节点 p 的懒标记下传到左右孩子并清空本节点标记; 会修改当前结构; 多测时重新构造或显式清空成员。
// 参数: 当前线段树节点编号 p (根通常为 1); 闭区间左端点 l; 闭区间右端点 r。
void push(int p, int l, int r) {
    // 叶子没有儿子, 标记留在自己这里即可。
    if (l == r) return;
    // 变量: mid=当前区间中点 mid。
    int mid = (l + r) >> 1;
    auto [m, a] = tr[p].tag;

    // 只有非恒等标记才需要下传。
    if (m != 1 || a != 0) {
        apply(p << 1, l, mid, m, a);
        apply(p << 1 | 1, mid + 1, r, m, a);

        // 父亲的变换已经交给两个儿子, 父亲恢复为恒等变换。
        tr[p].tag = {1, 0};
    }
}
}

```

例题可用 P3373 模板线段树 2: 题目有区间乘、区间加、区间和。题目中的“乘法操作”调用 `apply(...,k,0)`, “加法操作”调用 `apply(...,1,k)`。最容易错的是标记合成顺序, 以及 `add` 作用在整个区间长度上。

测试样例: 仿射标记的合成顺序

下面采用 P3373 风格: `1lrk` 表示乘法, `2lrk` 表示加法, `3lr` 表示区间和; 每组第一行是 `nmMOD`, 随后是数组和 `m` 个操作。

输入

```

4
1 4 100
5
3 1 1
1 1 1 3
2 1 1 4
3 1 1
3 5 1000000007
1 2 3
2 1 2 5
3 1 3
1 2 3 2
3 2 2

```

```

2 4 1000
1 1
1 1 1 2
2 1 1 3
1 1 1 4
3 1 1
3 3 1000
100 200 300
1 1 3 2
2 2 3 900
3 1 3

```

输出

```

5
19
16
14
20
1000
0

```

第三组必须按 $((x \times 2) + 3) \times 4$ 计算，结果为 20；如果把标记合成顺序写反，或者把加法只加一次而不是乘区间长度，边界组和第三组都会暴露问题。

7.15.3 线段树上二分

当每个节点维护一个可合并的“容量/最大值/最小值”，并且可以判断“左子树是否存在答案”，就可以在线段树上二分，复杂度 $O(\log n)$ 。

```

// 接口: first_at_least(p, l, r, x): 返回查询范围内第一个值至少为 x 的位置，不存在返回
//      -1; 返回类型 int。
// 参数: 当前线段树节点编号 p (根通常为 1); 闭区间左端点 l; 闭区间右端点 r; 查询要求达到
//      的值下界 x。
int first_at_least(int p, int l, int r, long long x) {
    if (tr[p].mx < x) return -1;
    if (l == r) return l;
    push(p, l, r);
    // 变量: mid=当前区间中点 mid。
    int mid = (l + r) >> 1;
    // 变量: ans=左子树中第一个值至少为 x 的位置; 不存在时为 -1。
    int ans = first_at_least(p << 1, l, mid, x);
    if (ans != -1) return ans;
    return first_at_least(p << 1 | 1, mid + 1, r, x);
}

```


如果要找“第一个前缀和至少为 k ”，节点维护区间和且所有值非负；如果存在负数，不能用前缀和单调性，必须换树套树、平衡树或题目特定结构。例题可练 P3372 模板线段树 1 的改造版：每次单点加、询问第一个达到阈值的位置。

7.15.4 莫队、带修改莫队与回滚莫队

莫队适用于离线区间询问：移动 L/R 一格的代价很小，答案可以增量维护。普通莫队排序 $(l/block, r)$ ；带修改时再加时间 t ，状态是 (L, R, T) 。

```
// t 是时间维：这个询问发生以前，数组已经执行了多少次修改。
struct Query { int l, r, t, id; }; // 查询闭区间、需应用的修改次数、原始编号。

// oldv/newv 必须在读入时保存；处理询问时不能再从当前数组猜 oldv。
struct Change { int pos, oldv, newv; }; // 修改位置及修改前/后的离散化值。

// 当前窗口是 [L,R]，当前已经应用到第 T 次修改。
// 初始设置为 L=1,R=0，表示窗口为空。
// 变量：L=带修改莫队当前 1-based 窗口的左端点 L；R=带修改莫队当前 1-based 窗口的右端点
// R：空窗口时 R=0；T=带修改莫队当前已经应用的修改次数 T。
int L = 1, R = 0, T = 0;

// 这两个函数是“题目答案维护器”，不是莫队本身。
// 例如统计不同颜色：add 时 freq[a[p]] 从 0 变 1 就 ++answer,
// del 时从 1 变 0 就 --answer；统计频率平方和则要先减旧贡献再加新贡献。
// 接口：add_pos(p)：把数组位置 p 加入当前莫队窗口并更新答案；不返回值。
// 参数：加入莫队窗口的数组位置 p。
void add_pos(int p) { /* 把 a[p] 加入当前答案 */ }
// 接口：del_pos(p)：把数组位置 p 移出当前莫队窗口并更新答案；不返回值。
// 参数：移出莫队窗口的数组位置 p。
void del_pos(int p) { /* 从当前答案删除 a[p] */ }

// 接口：apply_change(c, forward)：正向应用或反向撤销一次带修改莫队更新；不返回值。
// 参数：待应用或撤销的离线修改记录 c；true=应用修改，false=撤销修改。
void apply_change(const Change& c, bool forward) {
    // forward=true: 把 oldv 改成 newv; false: 撤销为 oldv。
    // 变量：v=正向修改采用新值、撤销修改采用旧值的实际写入值 v。
    int v = forward ? c.newv : c.oldv;

    // 只有修改点位于当前区间时，它才会影响答案。
    // 顺序必须是“删除旧值 -> 改数组 -> 加入新值”。
    if (L <= c.pos && c.pos <= R) del_pos(c.pos);
    a[c.pos] = v;
    if (L <= c.pos && c.pos <= R) add_pos(c.pos);
}
```



```
// 典型的带修改莫队处理流程：
// 1. 先按 (l 所在块, r 所在块, t) 排序；
// 2. 把当前 L/R/T 一格一格移动到询问要求的位置；
// 3. 每次移动都调用同一套 add/del/apply_change，答案逻辑无需重复编写。
// 接口：process_query(q)：把莫队窗口和时间移动到查询 q，随后记录答案；不返回值。
// 参数：待移动莫队窗口并回答的离线询问 q。
void process_query(Query q) {
    while (T < q.t) apply_change(changes[++T], true);
    while (T > q.t) apply_change(changes[T--], false);
    while (L > q.l) add_pos(--L);
    while (R < q.r) add_pos(++R);
    while (L < q.l) del_pos(L++);
    while (R > q.r) del_pos(R--);
    answer[q.id] = current_answer;
}
```

例题：P1903 数颜色。操作 **RPC** 是修改，**QLR** 是询问。读入时给每个询问记录“它之前发生了多少个修改”作为 **t**；处理某个询问前先移动 **T**，修改覆盖当前区间时先删除旧值再加入新值。若只会加不支持删除，普通莫队不适用；若要求在线，也不能直接套莫队。

测试样例：带修改莫队的区间、时间双向移动

下面采用 P1903 的输入格式，每组第一行是 **nm**，数组后是 **qlr** 或 **rpc**；查询输出区间内不同颜色数。

```
输入
2
1 4
5
Q 1 1
R 1 7
Q 1 1
Q 1 1
5 7
1 2 1 3 2
Q 1 5
R 2 3
Q 1 3
R 1 2
Q 1 2
Q 2 5
Q 1 5
```

输出

```
1
1
1
3
2
2
3
3
```

第一组检查修改点正好在当前区间内且同一查询重复；第二组要求时间前进到 2，并且查询区间来回变化。最容易抄错的是没有按“删除旧值、修改数组、加入新值”的顺序处理修改。回滚莫队适合“左端点块内、右端点可移动，修改只需撤销”的场景。核心是保存操作日志，不要真的把整个数组复制一份；每次离开一个块时回滚到块开始的快照。

7.15.5 三维偏序的 CDQ 分治与动态规划优化

CDQ 解决的不是“任意三维查询”，而是一个维度分治、另一个维度排序、剩余维度用树状数组/线段树合并。经典三维偏序：统计满足 $x_j \leq x_i, y_j \leq y_i, z_j \leq z_i$ 的点数。

```
// 这里给的是“统计三维偏序”的可改造骨架：
// 对每个点 i，统计有多少个点 j 满足  $x_j \leq x_i, y_j \leq y_i, z_j \leq z_i$ 。
// 调用前：按  $(x, y, z)$  排序、把完全相同的点合并，z 压缩成 1..Z。
struct Point {
    int x, y, z;          // 三个偏序坐标；进入 CDQ 前按  $(x, y, z)$  排序。
    int cnt = 1;          // 完全相同点的个数；合并重复点后使用。
    long long ans = 0;    // 不含自己这一组的答案。
};

struct Fenwick {
    vector<int> t; // 1-based 频次数组；大小 n+1, t[0] 不使用。
    // 接口：Fenwick(n)：构造管理 1..n 下标的空树状数组；构造并初始化对象；每组测试建议重新构造。
    // 参数：树状数组管理的最大下标 n。
    Fenwick(int n): t(n + 1) {}

    // 在压缩坐标 x 上加入 v 个点。
    // 接口：add(x, v)：把增量 v 加到树状数组的 1-based 位置 x；会修改当前结构；多测时重新构造或显式清空成员。
    // 参数：树状数组的 1-based 修改位置 x；位置 x 的点增加量 v。
    void add(int x, int v) {
        for (; x < (int)t.size(); x += x & -x) t[x] += v;
    }
}
```

```

// 返回  $z \leq x$  的点数。
// 接口: sum(x): 返回树状数组前缀  $[1, x]$  内已插入元素的计数; 返回类型 int。
// 参数: 树状数组前缀查询的右端点 x。
int sum(int x) const {
    // 变量: res=当前 CDQ 子问题累计的逆序贡献数 res。
    int res = 0;
    for (; x > 0; x -= x & -x) res += t[x];
    return res;
}

};

// 变量: bit=当前处理的二进制位 bit。
Fenwick bit(MAX_Z);

// 接口: cdq(a, l, r): 处理下标闭区间  $[l, r]$  的 CDQ 分治贡献; 不返回值。
// 参数: 按 x 排序并在 CDQ 中累计二维偏序贡献的数组 a; 闭区间左端点 l; 闭区间右端点 r。
void cdq(vector<Point>& a, int l, int r) {
    // a 的“分治下标”仍然按 x 分组; 递归返回时区间内部会按 y 有序。
    if (l == r) return;
    // 变量: m=当前递归区间  $[l, r]$  的中点 m。
    int m = l + (int)((long long)r - l) / 2;

    // 先完成左半内部和右半内部的贡献。
    cdq(a, l, m);
    cdq(a, m + 1, r);

    // 此时 a[l..m]、a[m+1..r] 各自已经按 y 排序。
    // 左半只能贡献给右半, 不能把右半反过来影响左半。
    // 变量: i=CDQ 合并中扫描左半区间  $[l, m]$  的点下标 i。
    int i = l;
    // 变量: j=CDQ 合并中扫描右半区间  $[m+1, r]$  的点下标 j。
    for (int j = m + 1; j <= r; ++j) {
        while (i <= m && a[i].y <= a[j].y) {
            // BIT 的下标是 z; 加入左半中同时满足 x、y 限制的点。
            bit.add(a[i].z, a[i].cnt);
            ++i;
        }
        // BIT 前缀和再筛选  $z \leq a[j].z$ , 得到三维都不超过当前点的数量。
        a[j].ans += bit.sum(a[j].z);
    }

    // 必须撤销本次加入, 否则别的分治区间会错误继承左半贡献。
    // 变量: k=当前循环枚举的整数下标 k。

```

```

for (int k = l; k < i; ++k) bit.add(a[k].z, -a[k].cnt);

// 合并为按 y 有序，供父层用双指针处理第二维。
inplace_merge(a.begin() + l, a.begin() + m + 1,
              a.begin() + r + 1,
              // 接口：匿名 lambda(A, B): 按点的 y 坐标升序归并 CDQ 子区间。
              // 参数：第一个候选点 A; 第二个候选点 B。
              [](const Point& A, const Point& B) {
                  return A.y < B.y;
              });
}

```

使用时先离散化 z ，按 x 排序，把相同 (x, y, z) 合并并记录权值。例题：P3810 三维偏序。题目中的每个点就是模板的 `Point`，查询“左下后方点数量”就是 `bit.sum(z)`；重复点必须先合并，否则会漏算相等坐标。

测试样例：三维偏序的相等坐标与重复点

下面约定每组输入 n 个点，输出每个点的“包含自身、满足 $x_j \leq x_i, y_j \leq y_i, z_j \leq z_i$ 的点数”，顺序按输入编号输出。

输入

3

1

1 1 1

4

1 1 1

2 2 2

2 1 2

1 2 1

4

1 1 1

1 1 1

1 1 1

2 2 2

输出

1

1 4 2 2

3 3 3 4

第二组同时包含三个坐标方向上的相等关系；第三组专门检查完全相同点是否先合并并按 `cnt` 传播。若把比较条件写成严格小于，第二、三组都会错误。

CDQ 优化 DP 的识别句式是“转移来自满足下标/坐标偏序的前面状态”，把 `dp[j]` 看成事

件，在 CDQ 合并阶段把左半的贡献加入数据结构，更新右半。若转移只依赖一维单调队列，不要为了 CDQ 增加复杂度。

7.15.6 可持久化字典树 (Persistent Trie)

当询问是“区间 $[l, r]$ 中与 x 异或最大/第 k 大”，前缀异或 Trie 的每个版本保存 $1..i$ 的所有数。区间查询用版本 $\text{root}[r]$ 减去 $\text{root}[l-1]$ 的子树大小。

```
const int LOG = 60; // 覆盖  $[0, 2^{61})$ ；若值域更小可以相应降低。
```

```
struct Node {
    int ch[2]{}; // 当前二进制位选择 0/1 后的儿子节点编号。
    int cnt = 0; // 该节点子树中数字的数量。
} tr[MAXNODE];
```

```
int tot = 0; // 已经使用的节点数，0 号节点表示空树。
int root[MAXN]; // root[i] 是前 i 个数构成的版本。
```

// 接口: `insert(old, bit, x)`: 从旧版本根复制路径，向持久化二进制 Trie 插入数值 x 并返回新根；会修改当前结构；多测时重新构造或显式清空成员。

// 参数: 旧版本 Trie 根节点编号 `old`；当前递归处理的二进制位编号 `bit`；待插入持久化二进制 Trie 的非负整数 x 。

```
int insert(int old, int bit, unsigned long long x) {
    // “持久化”的关键：复制旧根到新节点，只修改本次路径。
    // 变量: now=复制旧版本路径时新建的持久化二进制 Trie 节点编号 now。
    int now = ++tot;
    tr[now] = tr[old];
    ++tr[now].cnt;

    // bit<0 表示所有二进制位都处理完了。
    if (bit < 0) return now;

    // 变量: b=整数 x 在当前 bit 位上的二进制值 b。
    int b = (x >> bit) & 1ULL;
    // 旧版本的另一条分支仍然复用；当前分支递归创建新节点。
    tr[now].ch[b] = insert(tr[old].ch[b], bit - 1, x);
    return now;
}
```

// 接口: `max_xor(left_root, right_root, bit, x)`: 在两个前缀版本之差表示的区间内返回与 x 的最大异或值；返回类型 `unsigned long long`。

// 参数: 区间左端前一个前缀版本根 `left_root`；区间右端前缀版本根 `right_root`；当前处理的二进制位 `bit`；求最大异或值时使用的初始值 x 。

```
unsigned long long max_xor(int left_root, int right_root, int bit,
```

```

        unsigned long long x) {
// 两个版本相减: root[r] 的数量 - root[l-1] 的数量, 正好保留 [l,r]。
if (bit < 0) return 0;

// 为了让异或结果当前位为 1, 优先走和 x 当前位相反的分支。
// 变量: want=为使当前异或位取 1 而优先选择的相反二进制分支编号 want。
int want = ((x >> bit) & 1ULL) ^ 1;
int a = tr[tr[right_root].ch[want]].cnt
        - tr[tr[left_root].ch[want]].cnt;

if (a > 0) {
    // 目标分支在区间中存在, 贪心保留这一位的 1。
    return (1ULL << bit) | max_xor(tr[left_root].ch[want],
                                    tr[right_root].ch[want], bit - 1, x);
}

// 目标分支为空, 只能走相同位; 这一位贡献 0。
// 变量: same=与查询值 x 当前二进制位相同的 Trie 分支编号 same。
int same = want ^ 1;
return max_xor(tr[left_root].ch[same],
                tr[right_root].ch[same], bit - 1, x);
}

```

调用时 `root[i]=insert(root[i-1],LOG,a[i])`, 查询 `[l,r]` 用 `max_xor(root[l-1],root[r],LOG,x)`。注意节点数约为 $n \times (\text{LOG}+1)$, 不能只按 $4 \times n$ 开。练习可从数据结构题单中的可持久化 Trie 开始, 遇到“区间 + 异或最大”时不要用普通 Trie, 因为普通 Trie 不能删除左端前缀。

测试样例: 版本相减和区间端点

下面每组输入 `nq`、数组, 随后每个查询为 `lrx`, 输出区间 `[l,r]` 中某个数与 `x` 的最大异或值。

输入

```

3
1 2
0
1 1 0
1 1 7
4 4
1 2 4 8
1 4 0
1 4 7
2 3 3
3 3 0

```

```

3 2
5 5 1
1 2 1
2 3 7

```

输出

```

0
7
8
15
7
4
4
6

```

第一组检查单版本和零值；第二组检查 $[1, n]$ 、内部区间和单点；第三组检查重复值以及 $\text{root}[r] - \text{root}[l-1]$ 的左端点是否写错。

7.15.7 李超线段树：整条直线与线段插入

维护若干直线 $y=kx+b$ ，询问某个整数 x 的最大/最小值，且插入和询问在线时，用李超树。与凸包不同，直线可以任意交叉；每个节点只保存当前区间中在某些位置更优的直线。下面让斜率、截距与查询坐标使用 `long long`，求值和查询结果使用 `__int128`，并显式记录空直线，避免乘加溢出或哨兵与合法答案冲突。

```

struct Line {
    long long k = 0; // 直线斜率，表示  $y=k \star x+b$ 。
    long long b = 0; // 直线截距。
    bool empty = true; // 空节点显式标记，避免负无穷哨兵与合法值冲突。
    // 接口：Line(): 构造一条空直线；构造并初始化对象。
    Line() = default;
    // 接口：Line(k, b): 构造斜率 k、截距 b 的真实直线；构造并初始化对象。
    // 参数：直线斜率 k；直线截距 b。
    Line(long long k, long long b): k(k), b(b), empty(false) {}

    // 接口：get(x): 返回直线在横坐标 x 处的函数值；返回类型 __int128。
    // 参数：计算直线函数值的横坐标 x。
    __int128 get(long long x) const {
        return empty ? -((__int128)1 << 126) : (__int128)k * x + b;
    }
};

// 变量：tree=按线段树下标存储候选直线的李超树节点数组 tree。
Line tree[4 * MAXX];

```

```

// 接口: add_line(nw, p, l, r): 把候选直线 nw 插入整数横坐标闭区间 [l,r] 的李超树节点
// p; 会修改当前结构; 多测时重新构造或显式清空成员。
// 参数: 待插入李超树的直线 nw; 当前节点 u 的父节点 p; 闭区间左端点 l; 闭区间右端点 r。
void add_line(Line nw, int p, int l, int r) {
    // 节点 p 负责整数横坐标 [l,r], tree[p] 是当前候选直线。
    // 变量: m=当前递归区间 [l,r] 的中点 m。
    int m = l + (int)((long long)r - l) / 2);

    // 比较新旧直线在左端点、中点的优劣。
    // 变量: lef=新直线在区间左端是否更优 lef。
    bool lef = nw.get(l) > tree[p].get(l);
    // 变量: mid=当前区间中点 mid。
    bool mid = nw.get(m) > tree[p].get(m);

    // 中点处更优的直线必须留在当前节点; 被换下来的直线只可能在某一侧更优。
    if (mid) swap(nw, tree[p]);
    if (l == r) return;

    // 如果两条线在左端和中点的优劣不同, 交点在左半, 否则在右半。
    if (lef != mid) add_line(nw, p << 1, l, m);
    else add_line(nw, p << 1 | 1, m + 1, r);
}

// 接口: query(x, p, l, r): 返回横坐标 x 处沿李超树查询路径得到的最优直线函数值; 返回类型 __int128。
// 参数: 计算候选直线最优值的横坐标 x; 当前线段树节点编号 p (根通常为 1); 闭区间左端点 l; 闭区间右端点 r。
__int128 query(int x, int p, int l, int r) {
    // 答案必须比较沿根到 x 所在叶子的每一个节点保存的直线。
    // 变量: res=李超树当前节点直线在横坐标 x 处的候选最优值 res。
    __int128 res = tree[p].get(x);
    if (l == r) return res;
    // 变量: m=当前递归区间 [l,r] 的中点 m。
    int m = l + (int)((long long)r - l) / 2);
    if (x <= m) return max(res, query(x, p << 1, l, m));
    return max(res, query(x, p << 1 | 1, m + 1, r));
}

```

插入“只在 [ql,qr] 有效的线段”时, 把 add_line 外面再套区间递归, 只在覆盖节点调用 add_line。例题: P4097 李超线段树: 题目插入线段、询问横坐标 k 处最高的线段。坐标压缩不能随便做, 因为线性函数的比较依赖真实横坐标; 若题目横坐标本来是整数区间 [1, 39989], 直接按整数域开树即可。

测试样例：直线交叉、端点和空树

下面约定横坐标域为 $[1, X]$ ；操作 **1kb** 插入 $y=kx+b$ ，操作 **2x** 查询最大值。空树查询的输出按题目改成对应的负无穷标记。

输入

```
3
1 4
1 2 1
2 1
1 -1 5
2 1
5 5
1 1 0
1 -1 10
2 1
2 5
1 0 100
2 3
5 5
1 2 0
1 -2 10
2 1
2 5
2 3
```

输出

```
3
4
9
5
100
8
10
6
```

第二、三组必须在不同横坐标上选择不同的最优直线；如果只比较中点、不沿根到叶查询，或者把真实横坐标错误压成编号，通常会在这里出错。

7.15.8 区间最值势能线段树 (Segment Tree Beats)

当题目同时出现“区间取 $\min(x, \cdot)$ / 区间取 $\max(x, \cdot)$ ”与区间和，普通懒标记无法把所有元素统一修改，因为只有大于 x 的元素会变。Beats 维护：最大值 $mx1$ 、次大值 $mx2$ 、最大值个数 cnt_mx 、区间和；若 $mx2 < x < mx1$ ，说明只有最大值会下降，可以整段打标记。

```

struct BeatNode {
    __int128 sum; // 精确区间和；宽类型避免 long long 极值与区间长度相乘溢出。
    long long mx1; // 区间最大值。
    __int128 mx2; // 严格小于 mx1 的次大值；用更宽类型保存无冲突哨兵。
    int cnt;      // 最大值出现次数。
};

vector<BeatNode> tr; // 调用前建树并分配足够节点；多测时重新初始化。
// 变量：NEG=不存在次大值时使用的 __int128 负无穷哨兵 NEG。
const __int128 NEG = -((__int128)1 << 126);

// 接口：push_chmin(p, x)：把父节点的 chmin 约束下传到孩子；不返回值。
// 参数：当前线段树节点编号 p（根通常为 1）；节点最大值要压低到的上界 x。
void push_chmin(int p, long long x) {
    // 只有 x < mx1 时才有变化；调用者还必须保证 x > mx2。
    // 这样区间内只有“最大值这一类”会被修改，才能整段打标记。
    if (x >= tr[p].mx1) return;
    tr[p].sum -= ((__int128)tr[p].mx1 - x) * tr[p].cnt;
    tr[p].mx1 = x;
}

// 接口：push_up(p)：由两个孩子重新计算节点 p 的全部统计量；会修改当前结构；多测时重新构造或显式清空成员。
// 参数：当前线段树节点编号 p（根通常为 1）。
void push_up(int p) {
    // 合并两个儿子的 sum、最大值、次大值和最大值个数。
    tr[p].sum = tr[p<<1].sum + tr[p<<1|1].sum;
    tr[p].mx1 = max(tr[p<<1].mx1, tr[p<<1|1].mx1);
    tr[p].cnt = 0;
    tr[p].mx2 = NEG;
    // 变量：q=从容器 {p << 1, p << 1 | 1} 枚举的元素 q。
    for (int q : {p << 1, p << 1 | 1}) {
        if (tr[q].mx1 == tr[p].mx1) {
            tr[p].cnt += tr[q].cnt;
            tr[p].mx2 = max(tr[p].mx2, tr[q].mx2);
        } else {
            tr[p].mx2 = max(tr[p].mx2, ((__int128)tr[q].mx1);
        }
    }
}

// 接口：push_down(p)：把节点 p 的区间约束下传到两个孩子；会修改当前结构；多测时重新构造或显式清空成员。
// 参数：当前线段树节点编号 p（根通常为 1）。
void push_down(int p) {
    // 父节点的 mx1 可能已经被压低，儿子仍保留更大的旧最大值。
    // 把父亲的上界传给儿子，不能直接复制整个节点。

```

```

    if (tr[p<<1].mx1 > tr[p].mx1) push_chmin(p<<1, tr[p].mx1);
    if (tr[p<<1|1].mx1 > tr[p].mx1) push_chmin(p<<1|1, tr[p].mx1);
}
// 接口: range_chmin(p, l, r, ql, qr, x): 对闭区间 [ql,qr] 执行 a[i]=min(a[i],x)
//  ; 不返回值。
// 参数: 当前线段树节点编号 p (根通常为 1); 闭区间左端点 l; 闭区间右端点 r; 目标闭区间左
// 端 ql; 目标闭区间右端 qr; 目标区间元素要压低到的上界 x。
void range_chmin(int p, int l, int r, int ql, int qr, long long x) {
    // 没交集, 或当前最大值已经不超过 x: 无需处理。
    if (r < ql || qr < l || x >= tr[p].mx1) return;

    // x 严格大于次大值时, 只有最大值会下降, 可以 O(1) 修改当前节点。
    if (ql <= l && r <= qr && x > tr[p].mx2) {
        push_chmin(p, x);
        return;
    }

    // 否则区间内至少有两类值会被影响, 必须下传后递归到儿子。
    push_down(p); // 把父节点较小的 mx1 下传给孩子
    // 变量: m=当前递归区间 [l,r] 的中点 m。
    int m = (l + r) >> 1;
    range_chmin(p<<1, l, m, ql, qr, x);
    range_chmin(p<<1|1, m+1, r, ql, qr, x);
    push_up(p);
}

```

完整题例: P6242 线段树 3。如果还要区间取 `max`, 对称维护最小值 `mn1/mn2/cnt_mn`; 如果要区间加, 再额外维护 `add` 并在 `push_down` 中先传加法。这个算法的均摊复杂度依赖势能, 不能把 `push_chmin` 的条件删掉后“递归到底”, 否则会退化。

测试样例: 区间取最小值的整段标记与递归分支

下面约定操作 `1lrx` 是区间 `chmin`, 操作 `2lr` 查询区间和。

输入

```

3
1 4
5
2 1 1
1 1 1 5
1 1 1 3
2 1 1
4 4
1 3 3 7

```

```

2 1 4
1 1 4 5
2 1 4
1 2 3 2
2 2 3
5 5
10 1 8 8 3
1 1 5 8
2 1 5
1 1 5 5
2 1 5
1 2 4 4
2 2 4

```

输出

```

5
3
14
12
4
28
19
9

```

第二组的最大值有重复，第三组连续两次降低最大值后再对子区间降低，能同时检查 `mx1/mx2 /cnt`、`push_down` 和 “`x>mx2` 才能整段修改” 的条件。

7.15.9 可持久化并查集

题面出现 “第 k 个版本中连不连通” “回到历史版本再合并” 时，普通并查集不能回退。用可持久化线段树保存 `fa` 和秩/深度数组，每次只复制从根到叶子的 $O(\log n)$ 个节点。

```

struct PNode { int l, r, val; } tr[MAXNODE]; // 左右儿子版本节点及叶子存值。
// 变量: tot=当前已经创建的数据结构节点数量 tot。
int tot;
// 接口: build(l, r): 建立覆盖闭区间 [l,r] 的初始持久化线段树版本; 完成初始化; 多测时每组重新调用。
// 参数: 闭区间左端点 l; 闭区间右端点 r。
int build(int l, int r) {
    // build 建立版本 0: 每个位置的父亲初始是自己。
    // 变量: p=新建可持久化节点的编号 p。
    int p = ++tot;
    if (l == r) tr[p].val = l;
    else {

```

```

        // 变量: m=当前递归区间 [l,r] 的中点 m。
        int m = (l + r) >> 1;
        tr[p].l = build(l, m);
        tr[p].r = build(m + 1, r);
    }
    return p;
}

// 接口: update(old, l, r, pos, val): 从版本根 old 复制到 pos 的路径、把叶子父节点值
// 改为 val 并返回新版本根; 返回类型 int。
// 参数: 旧版本根节点编号 old; 闭区间左端点 l; 闭区间右端点 r; 持久化线段树中要修改的 0-
// based 位置 pos; 写入的新值 val。
int update(int old, int l, int r, int pos, int val) {
    // 只复制从根到 pos 的路径; 没有改动的子树直接复用 old 版本。
    // 变量: p=新建可持久化节点的编号 p。
    int p = ++tot; tr[p] = tr[old];
    if (l == r) { tr[p].val = val; return p; }
    // 变量: m=当前递归区间 [l,r] 的中点 m。
    int m = (l + r) >> 1;
    if (pos <= m) tr[p].l = update(tr[old].l, l, m, pos, val);
    else tr[p].r = update(tr[old].r, m + 1, r, pos, val);
    return p;
}

// 接口: query(p, l, r, pos): 返回持久化线段树版本 p 在位置 pos 保存的父节点值; 返回类
// 型 int。
// 参数: 当前线段树节点编号 p (根通常为 1); 闭区间左端点 l; 闭区间右端点 r; 持久化线段树
// 中要查询的 0-based 位置 pos。
int query(int p, int l, int r, int pos) {
    // 按 pos 在持久化线段树中向下找到 fa[pos]。
    if (l == r) return tr[p].val;
    // 变量: m=当前递归区间 [l,r] 的中点 m。
    int m = (l + r) >> 1;
    return pos <= m ? query(tr[p].l, l, m, pos)
        : query(tr[p].r, m + 1, r, pos);
}

// 接口: find_root(root, x): 在持久化并查集版本 root 中沿父指针返回元素 x 所属集合的代
// 表元; 返回类型 int。
// 参数: 当前版本根节点编号 root; 待查找并查集代表元的元素编号 x。
int find_root(int root, int x) {
    // 不能做路径压缩: 那会修改旧版本; 用递归沿父指针查根。
    // 变量: f=query 从版本 root 中读出的元素 x 当前父节点编号 f。
    int f = query(root, 1, n, x);
    return f == x ? x : find_root(root, f);
}

// 接口: merge_version(root, x, y): 在持久化并查集版本 root 上合并 x、y 所属集合; 简

```

化实现把 x 的代表元父指针设为 y 的代表元并返回新版本根；返回类型 `int`。

// 参数：当前版本根节点编号 `root`；待合并集合中的元素编号 x ；待合并集合中的元素编号 y 。

```
int merge_version(int root, int x, int y) {
    // root 是“要从哪个历史版本继续操作”，不是固定写 root[n-1]。
    x = find_root(root, x); y = find_root(root, y);
    if (x == y) return root;
    // 应按秩/按大小把较浅树挂到较深树上；秩也要持久化。
    // 真实题目还要维护 rank/size 的第二棵持久化线段树。
    return update(root, 1, n, x, y);
}
```

调用测试：历史版本不能被后续合并污染

下面不强行规定 P3402 的操作编号，直接按接口描述测试：版本 0 是三个孤立点。

```
版本 1: 从版本 0 合并 (1,2)
版本 2: 从版本 0 合并 (2,3)
查询版本 1: connected(1,2) = true
查询版本 1: connected(2,3) = false
查询版本 2: connected(1,2) = false
查询版本 2: connected(2,3) = true
```

如果版本一和版本二都变成同一棵并查集，说明更新时修改了旧版本节点；如果从版本二继续合并时版本一也改变，说明没有真正复制根到叶子的路径。

例题：P3402 可持久化并查集。每个版本 `root[i]` 从 `root[i-1]` 继承；操作 0 复制某版本，操作 1 合并，操作 2 查询连通性。上面为了突出结构只展示 `fa`，正式提交要再维护按秩合并的深度，否则最坏会退化；不做路径压缩是为了保持版本不被修改。

8 字符串算法

8.1 选择算法

需求	算法	复杂度
单模式匹配	KMP / Z	$O(n+m)$
单词/前缀计数	Trie	$O(L)$
多模式匹配	AC 自动机	$O(\text{text} + \text{总模式长})$
子串相等、二分答案	双哈希	期望 $O(1)$ 查询
最长回文子串	Manacher	$O(n)$
所有后缀排序/不同子串	后缀数组 + LCP	$O(n \log n)$
子串出现次数/自动机	SAM	$O(n)$

8.2 Knuth–Morris–Pratt 字符串匹配算法（KMP）

$\text{nxt}[i]$ 表示模式前缀 $p[0..i]$ 的最长真前后缀长度。匹配失败时跳到 $\text{nxt}[j-1]$ ，不回退文本指针；可求出现位置、最小周期、前缀函数树。

最小周期：若 $n \% (n - \text{pi}[n-1]) == 0$ ，周期为 $n - \text{pi}[n-1]$ ，否则为 n 。

```
// 接口: prefix_function(s): 返回 KMP 前缀函数 pi; pi[i] 是 s[0..i] 的最长真 border
// 长度; 返回类型 vector<int>。
// 参数: 待计算前缀函数的字符串 s。
vector<int> prefix_function(const string& s) {
    // 变量: pi=KMP 前缀函数 pi。
    vector<int> pi(s.size());
    // 变量: i=前缀函数正在计算 pi[i] 的字符串位置 i。
    for (int i = 1; i < (int)s.size(); ++i) {
        // 变量: j=前缀函数计算中当前可回退的已匹配前缀长度 j。
        int j = pi[i - 1];
        while (j && s[i] != s[j]) j = pi[j - 1];
        if (s[i] == s[j]) ++j;
        pi[i] = j;
    }
}
```

```

    return pi;
}
// 接口: kmp(text, pat): 返回模式串 pat 在 text 中所有 0-based 起点; 返回类型 vector<int>。
// 参数: 主串 text; 模式串 pat。
vector<int> kmp(const string& text, const string& pat) {
    // 变量: pi=模式串 pat 的 KMP 前缀函数数组 pi; pos=所有匹配起点的返回数组 pos。
    vector<int> pi = prefix_function(pat), pos;
    if (pat.empty()) return pos; // 本接口约定空模式不返回匹配位置。
    // 变量: j=当前已经匹配的模式串前缀长度 j。
    int j = 0;
    // 变量: i=KMP 正在扫描的主串字符位置 i。
    for (int i = 0; i < (int)text.size(); ++i) {
        while (j && text[i] != pat[j]) j = pi[j - 1];
        if (text[i] == pat[j]) ++j;
        if (j == (int)pat.size()) {
            pos.push_back(i - (int)pat.size() + 1);
            j = pi[j - 1];
        }
    }
    return pos;
}

```

8.3 Z 函数

$z[i]$ 是 $s[i..]$ 与 $s[0..]$ 的最长公共前缀, 本模板约定 $z[0]=n$ 。维护当前最右匹配段 $[l, r)$, 落在段内时先继承 $z[i-l]$ 。用 Z 函数做匹配时, 分隔符必须保证不在输入字符集中。

```

// 接口: z_function(s): 返回 Z 数组;  $z[i]$  是  $s$  与  $s[i..]$  的最长公共前缀长度; 返回类型 vector<int>。
// 参数: 待计算 Z 函数的字符串  $s$ 。
vector<int> z_function(const string& s) {
    // 变量: n=容器 s 的元素数量 n; l=当前区间左端点 l; r=当前区间右端点 r; z=Z 函数数组 z;  $z[i]$  是  $s$  与  $s[i..]$  的最长公共前缀。
    int n = s.size(), l = 0, r = 0; vector<int> z(n);
    if (n) z[0] = n;
    // 变量: i=Z 函数正在计算  $z[i]$  的后缀起点 i。
    for (int i = 1; i < n; ++i) {
        if (i < r) z[i] = min(r - i, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) ++z[i];
        if (i + z[i] > r) l = i, r = i + z[i];
    }
    return z;
}

```



```
}
```

8.4 字符串哈希

前缀哈希 $H[i+1]=H[i]*B+s[i]$ ，子串哈希： $H[r]-H[l]*powB[r-l]$ 。双模数或 64 位自然溢出降低碰撞；随机底数并不能提供形式化保证。比较子串、二分最长公共前缀、回文判定都可用。

```
// 变量: BASE=字符串哈希使用的固定底数 BASE。
const unsigned long long BASE = 911382323ull;
// 变量: h=字符串哈希的前缀哈希数组 h; pw=幂次预处理数组 pw。
vector<unsigned long long> h(n + 1), pw(n + 1, 1);
// 变量: i=字符串哈希正在扩展前缀哈希的字符位置 i。
for (int i = 0; i < n; ++i)
    h[i + 1] = h[i] * BASE + (unsigned char)s[i], pw[i + 1] = pw[i] * BASE;
// 接口: get_hash(l, r): 返回字符串 0-based 半开区间 [l,r) 的哈希值。
// 参数: 子串左端点 l; 子串右端点 r (不含)。
auto get_hash = [&](int l, int r) { return h[r] - h[l] * pw[r - l]; }; // [l,r)
```

8.5 Manacher

把每个字节转成非负整数，在字符之间插入 -1，并在两端放不同的负数哨兵，求每个中心向外的回文半径 $p[i]$ ；维护最右回文段 $(center, right)$ ，线性求最长回文。整数哨兵不会与任意输入字节冲突。

```
// 变量: t=插入分隔符和两端哨兵后的整数串 t。
vector<int> t = {-2, -1};
// 变量: c=当前输入字节 c。
for (unsigned char c : s) t.push_back(c), t.push_back(-1);
t.push_back(-3);
// 变量: p=Manacher 半径数组 p; center=Manacher 当前最右回文的中心 center; right=滑动窗口右端点 right。
vector<int> p(t.size()); int center = 0, right = 0;
// 变量: i=Manacher 扩展串中当前计算回文半径的中心位置 i。
for (int i = 1; i + 1 < (int)t.size(); ++i) {
    // 变量: mir=Manacher 中与 i 关于中心对称的位置 mir。
    int mir = 2 * center - i;
    if (i < right) p[i] = min(right - i, p[mir]);
    while (t[i + 1 + p[i]] == t[i - 1 - p[i]]) ++p[i];
    if (i + p[i] > right) center = i, right = i + p[i];
}
```

8.6 字典树与 Aho–Corasick 多模式匹配自动机（AC 自动机）

8.6.1 普通字符串字典树（Trie）

字符集固定时用数组保存儿子。下面的模板只接受小写字母；`terminal[u]` 统计完整字符串，`pass[u]` 统计以当前路径为前缀的字符串。重复插入会重复计数，单次插入或查询复杂度均为 $O(L)$ 。节点上限至少取所有不同前缀数加一，大数组对象应放在全局区；多测时重新构造或显式清空。

```
struct StringTrie {
    int ch[MAXN][26]{}; // ch[u][c]=节点 u 沿字符 c 的儿子；0 表示不存在。
    int pass[MAXN]{};    // pass[u]=经过节点 u 的已插入字符串数量。
    int terminal[MAXN]{}; // terminal[u]=恰好在节点 u 结尾的字符串数量。
    int tot = 0;         // 当前最后一个节点编号；根为 0，节点范围 [0,tot]。

    // 接口：insert(s)：插入一个仅含小写字母的字符串；重复字符串会重复计数。
    // 参数：待插入的字符串 s。
    void insert(const string& s) {
        // 变量：u=当前 Trie 节点编号。
        int u = 0;
        ++pass[u];
        // 变量：c=当前处理的字符。
        for (char c : s) {
            // 变量：x=字符 c 对应的 0-based 字母编号。
            int x = c - 'a';
            if (!ch[u][x]) ch[u][x] = ++tot;
            u = ch[u][x];
            ++pass[u];
        }
        ++terminal[u];
    }

    // 接口：count_word(s)：返回字符串 s 被插入的次数；不存在时返回 0。
    // 参数：待查询的完整字符串 s。
    int count_word(const string& s) const {
        // 变量：u=当前 Trie 节点编号。
        int u = 0;
        // 变量：c=当前处理的字符。
        for (char c : s) {
            // 变量：x=字符 c 对应的 0-based 字母编号。
            int x = c - 'a';
            if (!ch[u][x]) return 0;
            u = ch[u][x];
        }
    }
};
```

```

        return terminal[u];
    }

    // 接口: count_prefix(s): 返回以字符串 s 为前缀的已插入字符串数量。
    // 参数: 待查询的前缀字符串 s; 空串前缀返回已插入字符串总数。
    int count_prefix(const string& s) const {
        // 变量: u=当前 Trie 节点编号。
        int u = 0;
        // 变量: c=当前处理的字符。
        for (char c : s) {
            // 变量: x=字符 c 对应的 0-based 字母编号。
            int x = c - 'a';
            if (!ch[u][x]) return 0;
            u = ch[u][x];
        }
        return pass[u];
    }
};

```

8.6.2 Aho–Corasick 多模式匹配自动机

AC 自动机先把模式串插入 Trie，再用 BFS 建 fail：节点的 fail 指向最长真后缀状态，匹配时沿 fail 转移；把输出计数沿 fail 累加可统计所有模式出现次数。模式集合固定后一次建树、多次查询。

```

struct AC {
    int ch[MAXN][26]{}; // Trie 边; build 后也会补齐为自动机转移。
    int fail[MAXN]{};    // 最长真后缀节点。
    int out[MAXN]{};     // 模式串结束次数及 fail 链汇总次数。
    int tot = 0;         // 当前最后一个节点编号; 根为 0, 节点范围 [0,tot]。
    // 接口: insert(s): 把小写模式串 s 插入 Trie, 并在终止节点累计出现次数; 会修改当前
    // 结构; 多测时重新构造或显式清空成员。
    // 参数: 输入字符串 s。
    void insert(const string& s) {
        // 变量: u=字符串自动机从根开始游走的状态编号 u。
        int u = 0;
        // 变量: c=从容器 s 枚举的元素 c。
        for (char c : s) {
            // 变量: x=当前字符映射到的字母表编号 x。
            int x = c - 'a';
            if (!ch[u][x]) ch[u][x] = ++tot;
            u = ch[u][x];
        }
        // 重复模式串要累加, 而不是只记一个 bool。
    }
};

```

```

    ++out[u];
}
// 接口: build(): 用 BFS 建立 AC 自动机 fail 指针、补全转移并汇总输出计数; 完成初始化; 多测时每组重新调用。
void build() {
    // 变量: q=AC 自动机建立 fail 指针时按层遍历状态的 BFS 队列 q。
    queue<int> q;
    // 根节点的直接儿子的 fail 都是根。
    // 变量: c=当前枚举的 0-based 字母表编号 c。
    for (int c = 0; c < 26; ++c) if (ch[0][c]) q.push(ch[0][c]);
    while (!q.empty()) {
        // 变量: u=AC 自动机构建过程中当前出队并补全转移的状态编号 u。
        int u = q.front(); q.pop();
        // 如果要统计每个模式串, 推荐建 fail 树后再统一 DFS 汇总;
        // 这里直接累加适合只问“当前位置结束的所有模式数”。
        out[u] += out[fail[u]];
        // 变量: c=当前枚举的 0-based 字母表编号 c。
        for (int c = 0; c < 26; ++c) {
            // 变量: v=自动机状态 u 经字符 c 转移到的状态引用 v。
            int &v = ch[u][c];
            if (v) {
                // v 的 fail 是 fail[u] 沿同字符转移。
                fail[v] = ch[fail[u]][c], q.push(v);
            } else {
                // 补全转移, 扫描时不需要反复 while 回退。
                v = ch[fail[u]][c];
            }
        }
    }
}
// 接口: query(s): 扫描主串 s 并返回全部模式串出现次数之和; 返回类型 int。
// 参数: 输入字符串 s。
int query(const string& s) {
    // 变量: u=字符串自动机从根开始扫描的状态编号 u; ans=AC 自动机扫描主串时累计的全部模式匹配次数 ans。
    int u = 0, ans = 0;
    // 变量: c=从容器 s 枚举的元素 c。
    for (char c : s) {
        // u 表示当前文本前缀的最长 Trie 后缀。
        u = ch[u][c - 'a'];
        ans += out[u];
    }
    return ans;
}

```

```
};
```

8.7 后缀数组与最长公共前缀 (Suffix Array and Longest Common Prefix, LCP)

倍增排序：按 $(\text{rank}[i], \text{rank}[i+k])$ 对后缀排序，重复 $O(\log n)$ 次，复杂度 $O(n \log^2 n)$ （用计数排序可到 $O(n \log n)$ ）。Kasai 求相邻后缀 LCP，RMQ 查询任意两个后缀 LCP。常见应用：不同子串数 $n(n+1)/2 - \sum \text{LCP}$ 、最长重复子串、字典序第 k 个子串。

```
// 接口: suffix_array(s): 返回 sa; sa[i] 是字典序第 i 小后缀的 0-based 起点; 返回类型
//      vector<int>。
// 参数: 待构造后缀数组的字符串 s。
vector<int> suffix_array(string s) {
    // 加入最小哨兵, 保证所有真正后缀都排在哨兵之后。
    // 变量: n=容器 s 的元素数量 n。
    s.push_back(0); int n = s.size();
    // 变量: sa=后缀数组 sa; sa[i] 是第 i 小后缀起点; r=当前轮每个后缀长度 k 前缀的等价
    //      类排名数组 r; nr=按长度 2k 重新排序后得到的新排名数组 nr。
    vector<int> sa(n), r(n), nr(n);
    iota(sa.begin(), sa.end(), 0);
    // 变量: i=后缀数组初始化单字符排名的后缀起点 i。
    for (int i = 0; i < n; ++i) r[i] = (unsigned char)s[i];
    // 变量: k=后缀倍增中当前二元组每一段的长度 k; 本轮比较长度 2k 的前缀。
    for (int k = 1; k <= 1) {
        // 用长度 k 的前半排名和后半排名比较长度 2k 的前缀。
        // 接口: 比较器 lambda(i, j): 按长度 2k 的两段排名二元组比较后缀 i 与 j。
        // 参数: 第一个后缀起点 i; 第二个后缀起点 j。
        sort(sa.begin(), sa.end(), [&](int i, int j) {
            if (r[i] != r[j]) return r[i] < r[j];
            // 变量: ri=后缀 i 偏移 k 后第二段的等价类排名 ri; 越界时为 -1。
            int ri = i + k < n ? r[i + k] : -1;
            // 变量: rj=后缀 j 偏移 k 后第二段的等价类排名 rj; 越界时为 -1。
            int rj = j + k < n ? r[j + k] : -1;
            return ri < rj;
        });
        // 重新压缩排名; 二元组完全相同的后缀共享排名。
        nr[sa[0]] = 0;
        // 变量: i=后缀数组排序后用于比较相邻后缀并重编号的排名位置 i。
        for (int i = 1; i < n; ++i)
            nr[sa[i]] = nr[sa[i - 1]] +
                (r[sa[i - 1]] != r[sa[i]] ||
                 (sa[i - 1] + k < n ? r[sa[i - 1] + k] : -1) !=
                 (sa[i] + k < n ? r[sa[i] + k] : -1));
    }
}
```

```

        r.swap(nr);
        if (r[sa.back()] == n - 1) break;
    }
    sa.erase(sa.begin()); return sa;
}

```

8.8 后缀自动机 (Suffix Automaton, SAM)

在线加入字符，状态保存 `len`、`link`、转移；每个状态代表一组 `endpos` 相同的子串。状态数最多 $2n-1$ 。按 `len` 降序累加出现次数，可求每个子串出现频次、不同子串数 $\sum(\text{len}[v]-\text{len}[\text{link}[v]])$ 。转移字符集大时用 `map`，小字符集用数组。

```

struct SAM {
    struct Node {
        int next[26]{}; // 从该 endpos 等价类沿字符转移。
        int link = -1; // suffix link, 指向更短的等价类。
        int len = 0; // 该状态能代表的最长子串长度。
        int occ = 0; // 直接作为前缀末尾出现的次数，之后沿 link 汇总。
    } st[2 * MAXN];
    int last = 0; // 当前整个字符串对应的状态。
    int sz = 1; // 状态 0 是初始状态。
    // 接口: extend(c): 向当前字符串自动机加入一个字符并更新状态；会修改当前结构；多测时
    // 重新构造或显式清空成员。
    // 参数: 待加入自动机的字符 c。
    void extend(char c) {
        // cur 表示加入 c 后的新前缀及其所有后缀。
        // 变量: cur=加入新字符后创建的 SAM 状态编号 cur; p=SAM 扩展前原末状态的游走指
        // 针 p。
        int cur = sz++, p = last; st[cur].len = st[last].len + 1; st[cur].occ
        = 1;
        // 从 last 沿 suffix link 补齐缺失的 c 转移。
        while (p >= 0 && !st[p].next[c - 'a']) st[p].next[c - 'a'] = cur, p =
        st[p].link;
        if (p < 0) st[cur].link = 0;
        else {
            // 变量: q=SAM 状态 p 经当前字符转移到的既有状态 q。
            int q = st[p].next[c - 'a'];
            if (st[p].len + 1 == st[q].len) st[cur].link = q;
            else {
                // q 太长，复制出 clone 把长度截断到 st[p].len+1。
                // 变量: clone=后缀自动机为拆分转移而复制的克隆状态 clone。
                int clone = sz++; st[clone] = st[q]; st[clone].len = st[p].len
                + 1; st[clone].occ = 0;
            }
        }
    }
}

```

```

        // 把原来能经过 c 到 q 的一段 link 链改指 clone。
        while (p >= 0 && st[p].next[c - 'a'] == q) st[p].next[c - 'a']
= clone, p = st[p].link;
        st[q].link = st[cur].link = clone;
    }
}
last = cur;
}
};

```

8.9 回文自动机 (Palindromic Automaton, PAM, 了解)

每个节点代表一个不同回文串，fail 指向最长回文后缀；在线插入字符即可统计不同回文子串、每个前缀的回文后缀。代码比 Manacher 长，只有在“动态加入/回文后缀结构”时使用。

```

struct PAM {
    int next[MAXN][26]; // 回文串两端加字符后的转移。
    int fail[MAXN];      // 最长真回文后缀。
    int len[MAXN];       // 节点对应回文串长度。
    int s[MAXN];         // 已插入字符，寻找可扩展回文时要回看它。
    int occ[MAXN];       // 先记作最长回文后缀的次数，再沿 fail 汇总。
    int last, n, tot;    // last=当前最长回文后缀，n=长度，tot=节点数。
    // 接口：init(): 清空成员并建立本自动机所需的初始根节点；完成初始化；多测时每组重新调用。
    void init() {
        // 0 号根长度 0，1 号根长度 -1；两个根让边界转移统一。
        memset(next, 0, sizeof(next)); memset(occ, 0, sizeof(occ));
        n = 0; tot = 1; last = 0; s[0] = -1;
        len[0] = 0; len[1] = -1; fail[0] = fail[1] = 1;
    }
    // 接口：get_fail(u): 沿 fail 链找到能被当前字符继续扩展的节点；返回类型 int。
    // 参数：顶点编号 u。
    int get_fail(int u) {
        // 沿 fail 跳，直到 u 的两端可以接上新字符 s[n]。
        while (s[n - len[u] - 1] != s[n]) u = fail[u];
        return u;
    }
    // 接口：add(c): 向回文自动机追加字符，更新 last 并返回新的最长回文后缀状态编号；会修改当前结构；多测时重新构造或显式清空成员。
    // 参数：待加入自动机的字符 c。
    void add(char c) {
        // 先把新字符写入串，再寻找能够扩展的最长回文后缀。
        // 变量：u=沿 fail 链找到的可被当前字符扩展的回文状态 u。

```

```

s[++n] = c - 'a'; int u = get_fail(last);
if (!next[u][s[n]]) {
    // 这是一个此前未出现过的新回文节点。
    // 变量: v=本次新建的回文自动机状态编号 v。
    int v = ++tot; len[v] = len[u] + 2;
    // 新节点的 fail: 从 fail[u] 开始寻找也能被该字符扩展的节点。
    fail[v] = next[get_fail(fail[u])][s[n]];
    next[u][s[n]] = v;
}
// last 是当前前缀的最长回文后缀; 每个位置只直接计数一次。
last = next[u][s[n]]; ++occ[last];
}
// 接口: build(str): 清空回文自动机, 依次加入 str 的字符并沿 fail 汇总出现次数; 完成初始化; 多测时每组重新调用。
// 参数: 待构建自动机的完整字符串 str。
int build(const string& str) {
    init();
    // 变量: c=从容器 str 枚举的元素 c。
    for (char c : str) add(c);
    // fail 从长回文指向短回文, 必须按长度降序汇总;
    // 许多简化板子用编号倒序, 只在编号也随长度单调时才安全。
    // 变量: ord=按回文长度从大到小排列、用于沿 fail 链汇总出现次数的回文自动机状态编号 ord。
    vector<int> ord(max(0, tot - 1));
    iota(ord.begin(), ord.end(), 2); // 0/1 是两个虚拟根, 不参与统计。
    // 接口: 比较器 lambda(a, b): 按回文长度从大到小排列自动机状态, 供出现次数沿 fail 链汇总。
    // 参数: 第一个回文状态 a; 第二个回文状态 b。
    sort(ord.begin(), ord.end(), [&](int a, int b) { return len[a] > len[b]; });
    // 变量: v=按指定顺序枚举的自动机状态编号 v。
    for (int v : ord) occ[fail[v]] += occ[v];
    return tot - 1; // 不同非空回文子串数量
}
};

```

8.10 字符串常见组合

- 子串出现次数: KMP 的前缀函数树或 SAM 的 endpos 计数。
- 多个模式在文本中出现: AC 自动机; 需要撤销/动态模式时考虑 fail 树 + 树状数组。
- 比较循环字符串: 最小表示法 (Booth) $O(n)$ 。
- 最短覆盖/最小表示: 先判周期, 再用 KMP/哈希做边界验证。


```
// 接口: min_rotation(s): 返回最小循环表示的 0-based 起点; 返回类型 int。
// 参数: 输入字符串 s。
int min_rotation(const string& s) {
    // 变量: t=用于寻找最小循环表示的双倍字符串 t=s+s; i=最小循环表示算法当前第一个候选
    // 起点 i; j=最小循环表示算法当前第二个候选起点 j; n=容器 s 的元素数量 n。
    string t = s + s; int i = 0, j = 1, n = s.size();
    while (i < n && j < n) {
        // 变量: k=比较两个循环表示时已匹配的偏移长度 k。
        int k = 0;
        while (k < n && t[i + k] == t[j + k]) ++k;
        if (k == n) break;
        if (t[i + k] > t[j + k]) i += k + 1, i += (i == j);
        else j += k + 1, j += (i == j);
    }
    return min(i, j);
}
```

8.11 字符串高级算法：参数、调用与改造

本节紧跟前面的基础模板。先看每个接口传什么，再看哪些不变量不能动，最后看如何替换维护信息或转移。

8.11.1 Booth 最小表示法

循环字符串的最小表示不是把字符串复制后排序所有起点。Booth 双指针在 $s+s$ 上比较两个候选起点，淘汰整段，复杂度 $O(n)$ 。

```
// 接口: booth(s): 返回字符串字典序最小循环表示的 0-based 起点; 返回类型 int。
// 参数: 输入字符串 s。
int booth(const string& s) {
    // t=s+s, 把所有循环移位统一成 t 中长度 n 的子串。
    // 变量: t=用于寻找最小循环表示的双倍字符串 t=s+s。
    string t = s + s;
    // 变量: n=容器 s 的元素数量 n; i=最小循环表示算法当前第一个候选起点 i; j=最小循环表示
    // 算法当前第二个候选起点 j; k=比较两个循环表示时已匹配的偏移长度 k。
    int n = s.size(), i = 0, j = 1, k = 0;
    while (i < n && j < n && k < n) {
        // k 表示两个候选起点已经匹配的长度。
        if (t[i+k] == t[j+k]) { ++k; continue; }

        // 第一个不同字符决定哪个起点更大; 较大的候选可以淘汰一整段。
        if (t[i+k] > t[j+k]) i = i + k + 1;
    }
}
```

```

    else j = j + k + 1;

    // 两个指针不能相等，否则候选数会少一个。
    if (i == j) ++j;
    k = 0;
}

// 最小表示的起点一定在 [0, n-1] 中。
return min(i, j);
}

```

测试样例：最小表示的重复字符、周期和首尾交叉

下面每组输入一个字符串，输出它的字典序最小循环移位。

输入

4

a

aaaa

baca

caba

输出

a

aaaa

abac

abac

前两组检查 $i==j$ 和重复字符；后两组的最优起点不在原串开头，能抓住循环串复制后排序时的下标错误。

例题：P1368 工艺。把输入串交给 **booth**，从返回下标开始输出 n 个字符。题目求的是循环移位，不是反转后的最小串；若题面允许反转，要额外对反串再调用一次。

8.11.2 回文自动机 (Palindromic Automaton)

PAM 的每个节点代表一个不同的回文串；**fail** 指向去掉两端后最长的回文后缀。插入字符时沿 **fail** 找到可以向两端扩展的节点。节点数最多 $n+2$ ，所以它适合“所有回文子串”的统计。

```

struct PAM {
    static const int SIG = 26; // 字符集大小；本模板要求输入字符在 'a'..'z'。
    int next[MAXN][SIG]; // next[u][c]: 回文串 u 两端加字符 c 后的节点。
    int fail[MAXN]; // fail[u]: u 的最长真回文后缀。
};

```

```

int len[MAXN];          // len[u]: 节点 u 代表的回文串长度。
int s[MAXN];            // 已插入的字符, get_fail 要访问历史字符。
long long occ[MAXN];    // 先统计“作为最长回文后缀”的次数, 再沿 fail 汇总。
int last, tot, n;       // last=当前前缀的最长回文后缀, tot=节点总数, n=串长。

```

// 接口: `init()`: 清空成员并建立本自动机所需的初始根节点; 完成初始化; 多测时每组重新调用。

```

void init() {
    // 0 号根表示长度 -1 的虚拟回文, 1 号根表示长度 0 的空串。
    memset(next, 0, sizeof(next));
    memset(occ, 0, sizeof(occ));
    n = 0; tot = 1; last = 1;
    len[0] = -1; fail[0] = 0;
    len[1] = 0; fail[1] = 0;
}

```

// 接口: `get_fail(x)`: 沿 `fail` 链找到能被当前字符继续扩展的节点; 返回类型 `int`。

// 参数: 开始沿 `fail` 链检查的自动机状态编号 `x`。

```

int get_fail(int x) {
    // 从当前回文后缀沿 fail 向上跳, 直到它能和新字符首尾匹配。
    while (n - 1 - len[x] < 0 || s[n - 1 - len[x]] != s[n])
        x = fail[x];
    return x;
}

```

// 接口: `add(ch)`: 向回文自动机追加字符, 更新 `last` 并返回新的最长回文后缀状态编号; 会修改当前结构; 多测时重新构造或显式清空成员。

// 参数: 待加入字符 `ch`。

```

int add(char ch) {
    // 新字符暂存在 s[n], 此时 n 还是旧长度。
    s[n] = ch - 'a';
    // 变量: cur=沿 fail 链找到的、可被新字符扩展的最长回文后缀状态 cur; c=本次加入回文自动机的字符编号 c。

```

```

    int cur = get_fail(last), c = s[n];
    if (!next[cur][c]) {
        // 当前回文后缀加上两端字符, 产生一个此前没有出现过的新回文。
        // 变量: now=本次新建的回文自动机状态编号 now。
        int now = ++tot;
        len[now] = len[cur] + 2;

        // 长度 1 的回文 fail 到空串根; 其它节点继续从 fail[cur] 找。
        fail[now] = (len[now] == 1) ? 1 : next[get_fail(fail[cur])][c];
        next[cur][c] = now;
    }
}

```

// 新前缀的最长回文后缀就是这条转移到达的节点。

```

        last = next[cur][c]; ++occ[last]; ++n;
        return last;
    }
    // 接口: count_occurrence(): 按回文长度从大到小把 occ 沿 fail 链累加, 原地得到每个回文状态的出现次数; 无返回值。
    void count_occurrence() {
        // fail 一定从长串指向短串, 所以必须按 len 从大到小传递。
        // 节点编号不保证按长度排序, 不能盲写 for(i=tot;i>=1;--i)。
        // 变量: ord=按回文长度从大到小排列、用于沿 fail 链汇总 occ 的回文自动机状态编号 ord。
        vector<int> ord(max(0, tot - 1));
        iota(ord.begin(), ord.end(), 2); // 0/1 是两个虚拟根, 不参与统计。
        // 接口: 比较器 lambda(a, b): 按回文长度从大到小排列自动机状态, 供出现次数沿 fail 链汇总。
        // 参数: 第一个回文状态 a; 第二个回文状态 b。
        sort(ord.begin(), ord.end(), [&](int a, int b) {
            return len[a] > len[b];
        });
        // 变量: u=按指定顺序枚举的自动机状态编号 u。
        for (int u : ord) occ[fail[u]] += occ[u];
    }
};

```

测试样例: 回文自动机的根、重复回文和多测清空

下面每组输入一个小写字符串, 输出不同的非空回文子串数量, 也就是 $\text{tot}-1$ 。

```

输入
4
a
aaaa
abacaba
ababa

输出
1
4
7
5

```

aaaa 的不同回文串是 a,aa,aaa,aaaa; abacaba 的七个不同回文串可用来检查 fail 链。连续多组调用前必须执行 `init()`, 否则第二组的节点和出现次数会污染第三组。

例题: P5496 【模板】回文自动机 (PAM)。题目要求每个位置结尾的回文串数量; 插入第 i

个字符后，最长回文后缀是 `last`，所有回文后缀沿 `fail` 链连接，因此可以用 `dp[i]=dp[i-len[last]]+1` 的题目特定统计，或沿 `fail` 链统计。若只问不同回文子串数量，答案是 `tot-1`；若问出现次数，要先 `count_occurrence()`。

8.11.3 后缀数组、最长公共前缀、后缀自动机与 Aho–Corasick 失配树

后缀数组适合“后缀排序、子串字典序、多个后缀的最长公共前缀”；LCP 用 Kasai 求相邻后缀公共前缀，再用 RMQ 回答任意两个后缀的 LCP。题目出现“第 k 个不同子串”时，SAM 往往更直接：每个状态记录 `len`、`link` 和转移，按 DAG DP 统计从状态出发的不同串数。

```
struct SAM {
    int next[MAXN][26]; // 状态转移；字符集大时改成 map/unordered_map。
    int link[MAXN];      // 后缀链接：指向最长的更短 endpos 等价类。
    int len[MAXN];       // 该状态能表示的最长子串长度。
    int last = 1;        // 整个当前前缀对应的状态。
    int tot = 1;         // 1 号状态是初始状态；0 号转移为空。

    // 接口：extend(c)：向当前字符串自动机加入一个字符并更新状态；会修改当前结构；多测时
    // 重新构造或显式清空成员。
    // 参数：待加入自动机的 0-based 字母表编号 c。
    void extend(int c) {
        // 新状态 cur 代表“以新字符结尾”的所有后缀。
        // 变量：cur=加入新字符后创建的 SAM 状态编号 cur。
        int cur = ++tot; len[cur] = len[last] + 1;
        // 变量：p=SAM 扩展前原来末状态的游走指针 p。
        int p = last;

        // 沿 suffix link 补上以前不存在的 c 转移。
        while (p && !next[p][c]) next[p][c] = cur, p = link[p];
        if (!p) link[cur] = 1;
        else {
            // 变量：q=SAM 状态 p 经当前字符转移到的既有状态 q。
            int q = next[p][c];
            if (len[p] + 1 == len[q]) link[cur] = q;
            else {
                // q 的长度太长，不能直接作为 cur 的 link。
                // clone 复制 q 的转移，但把最长长度截为 len[p]+1。
                // 变量：clone=后缀自动机为拆分转移而复制的克隆状态 clone。
                int clone = ++tot;
                memcpy(next[clone], next[q], sizeof next[q]);
                len[clone] = len[p] + 1; link[clone] = link[q];

                // 把原来指向 q 的、位于同一等价区间的转移改指 clone。
                while (p && next[p][c] == q)
```

```

        next[p][c] = clone, p = link[p];
        link[q] = link[cur] = clone;
    }
}
last = cur;
}
};

```

调用测试：后缀自动机的空串、重复字符和克隆状态

当前代码片段只负责 **extend**，所以先测试最基本的不变量：初始状态编号为 **1**，字符串全相同时每个字符新增一个状态；如果要测试出现次数，还必须把 **occ** 统计和链接树排序一并接上。

调用输入

```

s = ""
s = "a"
s = "aaaa"
s = "ab"

```

应得到的状态总数 **tot**

```

1
2
5
3

```

第一组检查 **init**；第二、三组检查 **last** 和长度；第四组检查不同字符转移。真正测试克隆分支时，应使用一个会产生重复后缀等价类的字符串，并检查所有被旧状态指向的转移是否一起改到 **clone**。

例题：P3804 【模板】后缀自动机（SAM）问出现次数最多的子串：按 **len** 从大到小把 **occ[u]** 加到 **occ[link[u]]**，答案是 $\max(\text{len}[u] \times \text{occ}[u])$ 。若是第 **k** 个子串，先在状态 DAG 上做 $\text{dp}[u] = \sum(1 + \text{dp}[v])$ ，再按字符顺序跳转并扣减 $1 + \text{dp}[v]$ 。

AC 自动机把多个模式串建 Trie，再按 BFS 建 **fail**；把 **fail** 指针反向成树，就能把“文本中经过节点”转成 **fail** 子树求和。例题可用 P3796 AC 自动机加强版：插入模式串，扫描文本，沿 **fail** 链累计出现次数，最后找最大次数的模式串。查询中如果有“一个模式串是另一个模式串后缀”，不要只看 Trie 节点本身，必须沿 **fail** 树汇总。

哈希二分 LCP 的模板：预处理双模/64 位前缀哈希，比较 $\text{hash}(i, \text{len}) == \text{hash}(j, \text{len})$ ，二分最长长度。例题：P3370 【模板】字符串哈希。如果是高风险哈希题，使用双模或 **uint64_t** 随机基，不能把单模碰撞当成严格证明。

8.11.4 后缀数组与 LCP

后缀数组的倍增法按“前半排名、后半排名”排序。工程上可以使用 `sort` 版先保证正确； n 达到 $2e5$ 以上再换基数排序优化。

```
// 接口: suffix_array(s): 返回 sa; sa[i] 是字典序第 i 小后缀的 0-based 起点; 返回类型
//      vector<int>。
// 参数: 待构造后缀数组的字符串 s。
vector<int> suffix_array(const string& s) {
    // 变量: n=容器 s 的元素数量 n。
    int n = s.size();
    if (n == 0) return {};
    // 变量: sa=后缀数组 sa; sa[i] 是第 i 小后缀起点; rk=rk[i] 表示后缀 s[i..] 按当前
    //      倍增长度得到的等价类排名; tmp=本轮按长度 2k 的排名二元组排序后写入的新等价类排名数组
    //      tmp。
    vector<int> sa(n), rk(n), tmp(n);
    iota(sa.begin(), sa.end(), 0);
    // rk[i] 是长度 1 的后缀 s[i..] 的排名; 不同字符最好先压成连续整数。
    // 变量: i=后缀数组初始化单字符排名的后缀起点 i。
    for (int i = 0; i < n; ++i) rk[i] = s[i];
    // 变量: k=后缀倍增中当前二元组每一段的长度 k; 本轮比较长度 2k 的前缀。
    for (int k = 1; k <= 1) {
        // 现在按长度 2k 的二元组 (前 k 长排名, 后 k 长排名) 排序。
        // 接口: 比较器 lambda(i, j): 按长度 2k 的两段排名二元组比较后缀 i 与 j。
        // 参数: 第一个后缀起点 i; 第二个后缀起点 j。
        sort(sa.begin(), sa.end(), [&](int i, int j) {
            if (rk[i] != rk[j]) return rk[i] < rk[j];
            // 变量: ri=后缀 i 偏移 k 后第二段的排名 ri; 越界时为 -1。
            int ri = i + k < n ? rk[i+k] : -1;
            // 变量: rj=后缀 j 偏移 k 后第二段的排名 rj; 越界时为 -1。
            int rj = j + k < n ? rk[j+k] : -1;
            return ri < rj;
        });
        // 根据排好序的后缀重新编号; 相邻二元组相同才共享排名。
        tmp[sa[0]] = 0;
        // 变量: i=后缀数组排序后用于比较相邻后缀并重编号的排名位置 i。
        for (int i = 1; i < n; ++i) {
            // 变量: a=字典序前一个后缀的起点 a=sa[i-1]; b=字典序当前后缀的起点 b=sa[
            i]。
            int a = sa[i-1], b = sa[i];
            // 变量: diff=相邻后缀 a、b 的第一段长度 k 排名是否不同的标记 diff。
            bool diff = rk[a] != rk[b];
            // 变量: ra=后缀 a 偏移 k 后第二段的排名 ra; 越界时为 -1。
            int ra = a+k < n ? rk[a+k] : -1;
            // 变量: rb=后缀 b 偏移 k 后第二段的排名 rb; 越界时为 -1。
            int rb = b+k < n ? rk[b+k] : -1;
            tmp[sa[i]] = diff || ra != rb ? tmp[sa[i-1]] + 1 : tmp[sa[i-1]];
        }
    }
    return sa;
}
```

```

        int rb = b+k < n ? rk[b+k] : -1;
        tmp[b] = tmp[a] + (diff || ra != rb);
    }
    rk.swap(tmp);
    // 所有排名都不同后, sa 已经是最终后缀数组。
    if (rk[sa.back()] == n - 1) break;
}
return sa;
}

// 接口: kasai(s, sa): 由 s 和 sa 返回相邻后缀 LCP 数组; 返回类型 vector<int>。
// 参数: 后缀数组对应的原字符串 s; 原字符串的后缀数组 sa。
vector<int> kasai(const string& s, const vector<int>& sa) {
    // 变量: n=容器 s 的元素数量 n; rank=每个后缀起点在后缀数组 sa 中的排名 rank; lcp=
    相邻后缀最长公共前缀数组 lcp。
    int n = s.size(); vector<int> rank(n), lcp(n);
    // 变量: i=后缀数组 sa 中当前排名位置 i。
    for (int i = 0; i < n; ++i) rank[sa[i]] = i;
    // 变量: i=Kasai 算法当前计算 LCP 的后缀起点 i。
    for (int i = 0, h = 0; i < n; ++i) {
        // 变量: r=后缀 i 在后缀数组 sa 中的排名 r。
        int r = rank[i];
        if (r == 0) continue;
        // 变量: j=后缀数组中排名紧邻后缀 i 之前的后缀起点 j。
        int j = sa[r-1];
        while (i+h < n && j+h < n && s[i+h] == s[j+h]) ++h;
        lcp[r] = h;
        if (h) --h;
    }
    return lcp; // lcp[r] = sa[r] 与 sa[r-1] 的 LCP
}

```

测试样例：后缀排序和 LCP 的经典字符串

输入字符串

banana

应得到后缀数组

5 3 1 0 4 2

应得到 lcp 数组

0 1 3 0 0 2

banana 同时含有重复前缀和不同首字符,能检查倍增第二关键字的越界排名;lcp[2]=3 是 ana

与 anana 的公共前缀，能抓住 Kasai 的 h 是否正确复用。

例题：P3809 【模板】后缀排序。若要任意两个后缀的 LCP，把 lcp 做 RMQ；若要第 k 个不同子串，按后缀顺序累加 $n - \text{sa}[i] - \text{lcp}[i]$ ，第一次超过 k 的后缀就是答案所在位置。

8.11.5 AC 自动机与 fail 树

```
struct AC {
    int ch[MAXN][26]{}; // build 后不存在的转移也会补成 fail 转移。
    int fail[MAXN]{};    // 当前节点的最长真后缀节点。
    int out[MAXN]{};     // 节点本身及 fail 链上的模式串计数。
    int tot = 0;         // 当前最后一个节点编号；根为 0，节点范围 [0,tot]。
    vector<int> tree[MAXN]; // fail 指针反图；用于子树汇总。
    // 接口：insert(s)：把小写模式串 s 插入 Trie，并在终止节点累计出现次数；会修改当前
    // 结构；多测时重新构造或显式清空成员。
    // 参数：输入字符串 s。
    int insert(const string& s) {
        // 变量：u=字符串自动机从根开始游走的状态编号 u。
        int u = 0;
        // 变量：cc=从模式串 s 依次读取的字符 cc。
        for (char cc : s) {
            // 变量：c=当前文本字符映射到的字母表编号 c。
            int c = cc - 'a';
            if (!ch[u][c]) ch[u][c] = ++tot;
            u = ch[u][c];
        }
        // 返回结束节点，题目要分别输出时保存这个编号。
        return u;
    }
    // 接口：build()：用 BFS 建立 AC 自动机 fail 指针、补全转移并汇总输出计数；完成初
    // 始化；多测时每组重新调用。
    void build() {
        // 变量：q=AC 自动机建立 fail 指针时按层遍历状态的 BFS 队列 q。
        queue<int> q;
        // 根的 fail 是自己；根的直接儿子 fail 都是根。
        // 变量：c=当前枚举的 0-based 字母表编号 c。
        for (int c = 0; c < 26; ++c)
            if (ch[0][c]) q.push(ch[0][c]);
        while (!q.empty()) {
            // 变量：u=AC 自动机构建过程中当前出队并补全转移的状态编号 u。
            int u = q.front(); q.pop();
            // 变量：c=当前枚举的 0-based 字母表编号 c。
            for (int c = 0; c < 26; ++c) {
                if (ch[u][c]) {
                    // 子节点的 fail：沿 u 的 fail 走同一个字符。

```

```

        fail[ch[u][c]] = ch[fail[u]][c];
        q.push(ch[u][c]);
    } else {
        // 补全自动机转移，扫描文本时就不需要 while 回退。
        ch[u][c] = ch[fail[u]][c];
    }
}

// fail[u] 是 u 的父亲，之后可以把文本经过次数沿 fail 树向上累加。
// 变量: u=当前加入 fail 树的 AC 自动机状态编号 u。
for (int u = 1; u <= tot; ++u) tree[fail[u]].push_back(u);
}

// 接口: scan(text): 扫描文本 text，并把每次到达自动机状态的次数原地累加到 occ；无返回值。
// 参数: 主串 text。
void scan(const string& text) {
    // 变量: u=字符串自动机从根开始游走的状态编号 u。
    int u = 0;
    // 变量: cc=从主串 text 依次读取并送入 AC 自动机的字符 cc。
    for (char cc : text) {
        // 每读一个字符，u 就是当前前缀的最长 Trie 后缀状态。
        u = ch[u][cc - 'a'];
        ++out[u];
    }
}

// 接口: dfs_fail(u): 沿 fail 树汇总模式串出现次数；不返回值。
// 参数: 顶点编号 u。
void dfs_fail(int u) {
    // 变量: v=从 tree[u] 枚举的相邻顶点 v。
    for (int v : tree[u]) {
        dfs_fail(v);
        // v 的所有出现也意味着 fail[v] 对应的模式串出现。
        out[u] += out[v];
    }
}

};

```

测试样例：AC 自动机的后缀模式和重复出现

模式串依次为 a、ab、bab，文本为 abab。扫描后沿 fail 树汇总，三个模式的出现次数应为：

```

a    -> 2
ab   -> 2
bab  -> 1

```

ab 是 **a** 的延长，**bab** 又是文本中较长的后缀模式；如果只读取当前 Trie 节点的 **out** 而不沿 fail 树汇总，**a** 的次数通常会少算。

例题：P3808 【模板】AC 自动机（简单版）。每个模式串的结束节点记录 **id**；扫描文本后，调用 **dfs_fail(0)**，该节点的 **out** 就是模式串作为后缀出现的总次数。简单版只需问总出现次数，若要每个模式串分别输出，就保存插入时的节点编号。

9 图论

9.1 图的表示与基础遍历

稀疏图用邻接表，边结构保存 `to, weight, id`；需要删除/反向边时给每条有向边保存反向边下标。无向边应添加两条边。网格可把 (x, y) 映射为 $x \times m + y$ ，或直接四方向 BFS。

DFS 可求连通块、染色、子树大小、DFS 序、树高；BFS 求无权最短路和层数。递归 DFS 在 $n \approx 2e5$ 时可能爆栈，改手写栈或提高栈空间。

```
// 接口: bfs_dist(g, s): 返回源点 s 到每个顶点的最少边数；不可达为 -1。
// 参数: 0-based 无权图邻接表 g；合法源点 s。
vector<int> bfs_dist(const vector<vector<int>>& g, int s) {
    // 变量: dist=源点到各顶点的最少边数，-1 表示不可达。
    vector<int> dist(g.size(), -1);
    // 变量: q=已确定最短距离、等待扩展的顶点队列。
    queue<int> q;
    dist[s] = 0; q.push(s);
    while (!q.empty()) {
        // 变量: u=本轮从队首取出的顶点。
        int u = q.front(); q.pop();
        // 变量: v=u 的相邻顶点。
        for (int v : g[u]) if (dist[v] == -1)
            dist[v] = dist[u] + 1, q.push(v);
    }
    return dist;
}
```

9.1.1 多源 BFS

多源 BFS 用来求每个点到“最近源点”的无权最短距离。与单源 BFS 的区别只有初始化：把所有源点的距离设为 0，并在搜索开始前全部入队；后面的扩展过程完全相同。每个点第一次被访问时得到的就是它到源点集合的最短距离，复杂度为 $O(V+E)$ 。

```
// 接口: multi_source_bfs(g, sources): 返回每个顶点到最近源点的最少边数；不可达为 -1
// 参数: 0-based 无权图邻接表 g；允许重复的合法源点列表 sources。
```

```

vector<int> multi_source_bfs(const vector<vector<int>>& g,
                           const vector<int>& sources) {
    // 变量: dist=各顶点到最近源点的最少边数, -1 表示不可达。
    vector<int> dist(g.size(), -1);
    // 变量: q=已确定最近源距离、等待扩展的顶点队列。
    queue<int> q;
    // 变量: s=当前初始化的源点编号。
    for (int s : sources) {
        if (dist[s] != -1) continue;
        dist[s] = 0; q.push(s);
    }
    while (!q.empty()) {
        // 变量: u=本轮从队首取出的顶点。
        int u = q.front(); q.pop();
        // 变量: v=u 的相邻顶点。
        for (int v : g[u]) {
            if (dist[v] != -1) continue;
            dist[v] = dist[u] + 1;
            q.push(v);
        }
    }
    return dist;
}

```

9.2 有向无环图 (Directed Acyclic Graph, DAG) 与拓扑排序

Kahn: 把入度为 0 的点入队, 删除出边; 处理点数小于 v 说明有环。DFS 三色标记也能判环。DAG 上按拓扑序做最长路/最短路、路径计数、DP; 多源最短路可把所有源同时入队。

关键路径: $\text{earliest}[v] = \max(\text{earliest}[v], \text{earliest}[u] + w)$ 。若要恢复方案, 保存前驱; 同一最优值的多条路径计数要注意模数。

```

// 变量: q=Kahn 拓扑排序中保存当前入度为 0 顶点的队列 q。
queue<int> q;
// 变量: u=初始化 Kahn 队列时当前检查入度的顶点编号 u。
for (int u = 0; u < n; ++u) if (!indeg[u]) q.push(u);
// 变量: topo=拓扑序列 topo。
vector<int> topo;
while (!q.empty()) {
    // 变量: u=Kahn 拓扑排序当前出队并删除出边的顶点编号 u。
    int u = q.front(); q.pop(); topo.push_back(u);
    // 变量: v=当前边的终点 v; w=当前边权 w。
    for (auto [v, w] : g[u]) if (--indeg[v] == 0) q.push(v);
}

```

```
if ((int)topo.size() != n) throw runtime_error("cycle");
```

9.3 最短路

条件	算法	复杂度
无权	BFS	$O(V+E)$
边权 0/1	0-1 BFS	$O(V+E)$
非负权	Dijkstra + 堆	$O((V+E)\log V)$
允许负边、单源	Bellman-Ford	$O(VE)$
DAG	拓扑 DP	$O(V+E)$
全源、 $V \leq 500$	Floyd	$O(V^3)$

Dijkstra 的核心前提是所有边权非负；堆中允许重复状态，弹出时若距离不是当前 `dist[u]` 就跳过。多源时将所有起点距离设为 0。

负环检测：Bellman-Ford 第 V 轮仍有更新即存在从源可达负环；SPFA 可做队列优化但最坏仍为 $O(VE)$ ，不要无条件依赖。

差分约束：不等式 $x_v \leq x_u + w$ 建 $u \rightarrow v$ 权 w ，求最短路；若存在负环则无解。求最大下界时改写方向/用最长路。

```
using P = pair<long long, int>;
// 数值域：边权必须非负；INF 是“不参与松弛”的距离上界，所有需要区分的有限最短路必须严格
// 小于 INF。
// 变量：dist=当前维护的距离 dist。
vector<long long> dist(n, INF);
// 变量：pq=当前优先队列 pq；堆顶含义见比较器。
priority_queue<P, vector<P>, greater<P>> pq;
dist[s] = 0; pq.push({0, s});
while (!pq.empty()) {
    auto [du, u] = pq.top(); pq.pop();
    if (du != dist[u]) continue;
    // 变量：v=当前边的终点 v；w=当前边权 w。
    for (auto [v, w] : g[u]) {
        // 变量：candidate=宽整数计算的候选距离；大于等于 INF 时按本接口的不可区分上界
        // 处理。
        __int128 candidate = (__int128)du + w;
        if (candidate >= dist[v]) continue;
        dist[v] = (long long)candidate;
        pq.push({dist[v], v});
    }
}
```

```
// 变量: q=0-1 BFS 中按新距离优先级保存待扩展顶点的双端队列 q。
deque<int> q;
// 变量: d=从指定源点到各顶点的距离数组 d。
vector<int> d(n, INF); d[s] = 0; q.push_front(s);
while (!q.empty()) {
    // 变量: u=0-1 BFS 当前从队首取出并松弛出边的顶点编号 u。
    int u = q.front(); q.pop_front();
    // 变量: v=从 zero_one_graph[u] 枚举的相邻顶点 v; w=连接该顶点的边权 w。
    for (auto [v, w] : zero_one_graph[u]) if (d[v] > d[u] + w) {
        d[v] = d[u] + w;
        if (w == 0) q.push_front(v); else q.push_back(v);
    }
}
```

9.4 最小生成树

9.4.1 Kruskal

按边权排序，用 DSU 合并不同连通块；恰好选 $V-1$ 条边则图连通。复杂度 $O(E \log E)$ 。可用于“最小化最大边”“第二小生成树”的基础。

```
struct MstEdge {
    int u, v;          // 无向边的两个 1-based 端点。
    long long w;       // 边权；允许为负。
    // 接口: operator<(o): 按边权升序比较两条边，供 Kruskal 排序；当当前边权 w 小于 o.w 时返回 true。
    // 参数: 与当前边比较权值的另一条最小生成树候选边 o。
    bool operator<(const MstEdge &o) const { return w < o.w; }
};

struct KruskalDSU {
    vector<int> p, sz; // p[x]=父节点; sz[root]=集合大小; 点编号 1..n。
    // 接口: KruskalDSU(n): 构造管理点 1..n 的 Kruskal 专用并查集；构造并初始化对象；每组测试建议重新构造。
    // 参数: Kruskal 并查集管理的顶点数 n。
    KruskalDSU(int n) : p(n + 1), sz(n + 1, 1) {
        iota(p.begin(), p.end(), 0);
    }
    // 接口: find(x): 返回 x 当前所在并查集的代表元；返回类型 int。
    // 参数: 待查询代表元的元素编号 x。
    int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); }
    // 接口: unite(x, y): 合并两个元素所在集合；原本同集合时返回 false；实际合并两个集合时返回 true。
    // 参数: 第一个待合并元素编号 x；第二个待合并元素编号 y。
```

```

    bool unite(int x, int y) {
        x = find(x); y = find(y);
        if (x == y) return false;
        if (sz[x] < sz[y]) swap(x, y);
        p[y] = x; sz[x] += sz[y];
        return true;
    }
};

struct MstResult {
    long long cost;           // 已选边总权值；不连通时是最小生成森林权值。
    int used;                 // 实际选择的边数。
    bool connected;          // 是否选满 n-1 条边，即原图是否连通。
    vector<MstEdge> chosen;   // 按 Kruskal 选择顺序保存的边。
};

// 接口: kruskal(n, edges): 返回无向图最小生成森林是否连通及总权值；返回类型 MstResult
// 参数: 无向图顶点数 n；无向带权边集合 edges。
MstResult kruskal(int n, vector<MstEdge> edges) {
    sort(edges.begin(), edges.end());
    // 变量: dsu=Kruskal 扫描边时维护连通块的并查集对象 dsu。
    KruskalDSU dsu(n);
    // 变量: ans=Kruskal 当前生成森林的总权、已选边数、连通状态和边集 ans。
    MstResult ans{0, 0, n <= 1, {}};
    // 变量: e=从容器 edges 枚举的元素 e。
    for (auto e : edges) if (dsu.unite(e.u, e.v)) {
        ans.cost += e.w;
        ++ans.used;
        ans.chosen.push_back(e);
        if (ans.used == n - 1) break;
    }
    ans.connected = (n <= 1 || ans.used == n - 1);
    return ans; // 不连通时 chosen 是最小生成森林
}

```

返回值同时保留总权值、选边数、连通性和所选边。若只需要权值可删除 **chosen**；图不连通时不能把当前 **cost** 当作生成树答案。代码块内的 **KruskalDSU** 是自足版本，若已复制并查集主模板可直接替换。

9.4.2 Prim

从任意点出发，每次选未加入点中连接代价最小者，堆优化 $O(E \log V)$ ；稠密图可用朴素 $O(V^2)$ 。
MST 性质：若一条非树边加入形成环，环上最大边被替换可得到候选；用于瓶颈路径、边权敏感性、树上倍增求环上最大值。


```

// 接口: prim(g): 返回图是否连通及最小生成树总权值; 返回类型 pair<bool, long long>。
// 参数: Prim 使用的 0-based 加权无向图邻接表 g; 每项为 (终点, 边权)。
pair<bool, long long> prim(
    const vector<vector<pair<int, long long>>> &g) {
    // 变量: n=容器 g 的元素数量 n。
    int n = g.size();
    if (n == 0) return {true, 0};
    // 变量: INF64=long long 不可达哨兵 INF64。
    const long long INF64 = numeric_limits<long long>::max();
    // 变量: best=目前找到的最优值 best。
    vector<long long> best(n, INF64);
    // 变量: used=记录每个对象是否已被使用的标记 used。
    vector<char> used(n);
    using State = pair<long long, int>;
    // 变量: pq=当前优先队列 pq; 堆顶含义见比较器。
    priority_queue<State, vector<State>, greater<State>> pq;
    best[0] = 0; pq.push({0, 0});
    // 变量: cost=当前累计费用 cost。
    long long cost = 0;
    // 变量: cnt=Prim 已经加入生成树的顶点数量 cnt。
    int cnt = 0;
    while (!pq.empty()) {
        auto [w, u] = pq.top(); pq.pop();
        if (used[u]) continue;
        used[u] = true; cost += w; ++cnt;
        // 变量: v=从 g[u] 枚举的相邻顶点 v; c=连接该顶点的边权 w。
        for (auto [v, c] : g[u])
            if (!used[v] && c < best[v])
                best[v] = c, pq.push({c, v});
    }
    return {cnt == n, cost};
}

```

9.4.3 Kruskal 重构树

将边按权值从小到大加入。每次连接两个连通块时新建一个父节点，其点权等于当前边权，两个连通块的根作为儿子。原图有 n 个点时至多建立 $2n-1$ 个点；祖先点权单调不降。

两点首次连通的边权阈值等于它们在重构树中 LCA 的点权，也等于两点间最小瓶颈路的答案。对顶点 u 和阈值 lim 向上跳到点权不超过 lim 的最高祖先，即得到只保留权值不超过 lim 的边时， u 所在连通块；其 sz 是连通块大小。

```

struct KruskalTree {
    struct Edge { int u, v; long long w; }; // 原图边: 1-based 端点与边权。

```

```

int n, tot, LOG;           // n=原点数; tot=重构树总点数; LOG=倍增层数。
vector<int> dsu, dep, sz;   // dep=重构树深度; sz[u]=u 子树中的原图点数。
vector<long long> val;     // val[u]=合并节点对应边权; 原图点权为负无穷。
vector<vector<int>> up;     // up[k][u]=u 的 2^k 级祖先。
vector<vector<int>> child;  // 重构树孩子表; 边从合并节点指向两个分量。

// 接口: find(x): 返回 x 当前所在并查集的代表元; 返回类型 int。
// 参数: 待查询代表元的元素编号 x。
int find(int x) {
    return dsu[x] == x ? x : dsu[x] = find(dsu[x]);
}

// 接口: build(n_, edges): 按边权建立 Kruskal 重构树并预处理深度、子树大小和倍增祖先; 完成初始化; 多测时每组重新调用。
// 参数: 原图点数 n_; 输入边集合 edges。
void build(int n_, vector<Edge> edges) { // 顶点编号 1..n
    n = n_; tot = n;
    // 变量: cap=Kruskal 重构树为原点和合并点预留的数组容量 cap。
    int cap = 2 * n + 5;
    LOG = 1;
    while ((1LL << LOG) <= 2 * n) ++LOG;
    dsu.resize(cap); iota(dsu.begin(), dsu.end(), 0);
    dep.assign(cap, 0); sz.assign(cap, 0);
    val.assign(cap, numeric_limits<long long>::lowest());
    up.assign(LOG, vector<int>(cap));
    child.assign(cap, {});
    // 变量: i=Kruskal 重构树初始化子树大小的原图顶点编号 i。
    for (int i = 1; i <= n; ++i) sz[i] = 1;
    sort(edges.begin(), edges.end(),
        // 接口: 匿名 lambda(a, b): 按边权从小到大排列 Kruskal 候选边。
        // 参数: 第一条候选边 a; 第二条候选边 b。
        [](const Edge &a, const Edge &b) { return a.w < b.w; });
    // 变量: e=从容器 edges 枚举的元素 e。
    for (auto e : edges) {
        // 变量: x=Kruskal 当前边一端所在连通块的代表元 x; y=Kruskal 当前边一端所在连通块的代表元 y。
        int x = find(e.u), y = find(e.v);
        if (x == y) continue;
        ++tot; val[tot] = e.w;
        sz[tot] = sz[x] + sz[y];
        child[tot] = {x, y};
        up[0][x] = up[0][y] = tot;
        dsu[x] = dsu[y] = tot;
        dsu[tot] = tot;
    }
}

```

```

// 变量: st=遍历 Kruskal 重构森林并计算深度的顶点栈 st。
vector<int> st;
// 变量: u=当前检查是否为 Kruskal 重构森林根的节点编号 u。
for (int u = 1; u <= tot; ++u) if (dsu[u] == u) {
    dep[u] = 1; st.push_back(u); // 不连通图得到森林
}
while (!st.empty()) {
    // 变量: u=当前从 Kruskal 重构森林遍历栈弹出的节点编号 u。
    int u = st.back(); st.pop_back();
    // 变量: v=从 child[u] 枚举的相邻顶点 v。
    for (int v : child[u]) dep[v] = dep[u] + 1, st.push_back(v);
}
// 变量: j=Kruskal 重构树正在预处理的  $2^j$  级祖先层编号 j。
for (int j = 1; j < LOG; ++j)
    // 变量: u=当前填写倍增祖先表的 Kruskal 重构树节点编号 u。
    for (int u = 1; u <= tot; ++u)
        up[j][u] = up[j - 1][up[j - 1][u]];
}
// 接口: lca(u, v): 返回树上 u,v 的最近公共祖先; 返回类型 int。
// 参数: 当前顶点编号 u; 与 u 配对的顶点编号 v。
int lca(int u, int v) const {
    if (dep[u] < dep[v]) swap(u, v);
    // 变量: d=把较深顶点提升到同深度所需的层数差 d。
    int d = dep[u] - dep[v];
    // 变量: j=按深度差 d 的置位情况提升顶点 u 时检查的倍增层编号 j。
    for (int j = 0; j < LOG; ++j) if (d >> j & 1) u = up[j][u];
    if (u == v) return u;
    // 变量: j=Kruskal 重构树 LCA 中从高到低比较祖先的倍增层编号 j。
    for (int j = LOG - 1; j >= 0; --j)
        if (up[j][u] != up[j][v])
            u = up[j][u], v = up[j][v];
    return up[0][u];
}
// nullopt 表示两点在原图中不连通; u==v 时瓶颈记为 -INF。
// 接口: bottleneck(u, v): 返回 u,v 路径上的瓶颈边权; 不连通时返回空; 返回类型
optional<long long>。
// 参数: 当前顶点编号 u; 与 u 配对的顶点编号 v。
optional<long long> bottleneck(int u, int v) {
    if (find(u) != find(v)) return nullopt;
    return val[lca(u, v)];
}
// 接口: component_node(u, lim): 返回 Kruskal 重构树中阈值 lim 对应的祖先节点;
返回类型 int。
// 参数: 当前顶点编号 u; 允许的阈值 lim。

```

```

int component_node(int u, long long lim) const {
    // 变量: j=按阈值 lim 向上跳转时从高到低检查的倍增层编号 j。
    for (int j = LOG - 1; j >= 0; --j) {
        // 变量: a=顶点 u 的 2^j 级祖先 a。
        int a = up[j][u];
        if (a && val[a] <= lim) u = a;
    }
    return u;
}

// 接口: component_size(u, lim): 返回阈值 lim 下点 u 所在连通块大小; 返回类型 int。
// 参数: 当前顶点编号 u; 允许的阈值 lim。
int component_size(int u, long long lim) const {
    return sz[component_node(u, lim)];
}
};

```

若题目按边权从大到小合并（例如询问边权至少为某值的连通性），反向排序并把祖先跳跃条件同步改成 $\text{val}[a] \geq \text{lim}$ 。多测时每次重新调用 `build`；同权边顺序只影响树形，不影响阈值答案。

9.5 强连通分量（Strongly Connected Components, SCC）与缩点

Tarjan: DFS 维护 `dfn/low` 与栈， $\text{low}[u] == \text{dfn}[u]$ 时弹出一个 SCC， $O(V+E)$ 。Kosaraju: 正图后序 + 反图 DFS，代码更直观。SCC 缩成 DAG 后做 DP、计数、可达性。缩点后不要把同一分量内的边当作 DAG 边。

```

// 变量: timer=强连通分量 Tarjan DFS 使用的递增时间戳 timer; dfn=DFS 首次访问时间戳 dfn; low=Tarjan 最低可达时间戳 low; stk=尚未归入强连通分量的顶点栈 stk; in=顶点当前是否在 Tarjan 栈内的标记 in; comp=每个顶点所属强连通分量编号 comp; cc=当前已经找到的强连通分量数量 cc。
int timer = 0; vector<int> dfn(n), low(n), stk, in(n), comp(n); int cc = 0;
// 接口: tarjan(u): 从 u 开始求强连通分量的 Tarjan DFS; 不返回值。
// 参数: 当前顶点编号 u。
void tarjan(int u) {
    dfn[u] = low[u] = ++timer; stk.push_back(u); in[u] = 1;
    // 变量: v=当前边的终点 v。
    for (int v : g[u]) {
        if (!dfn[v]) tarjan(v), low[u] = min(low[u], low[v]);
        else if (in[v]) low[u] = min(low[u], dfn[v]);
    }
    if (low[u] != dfn[u]) return;
    ++cc;
    while (true) {

```

```

// 变量: v=Tarjan 当前弹出并归入强连通分量的顶点 v。
int v = stk.back(); stk.pop_back(); in[v] = 0; comp[v] = cc;
if (v == u) break;
}
}

```

9.6 割点、桥、双连通

无向图 Tarjan: $\text{low}[v] \geq \text{dfn}[u]$ 时, u 是割点分隔子树; $\text{low}[v] > \text{dfn}[u]$ 时 (u, v) 是桥。根节点成为割点的条件是 DFS 子树数大于 1。边有重边时必须用边 id 判断父边, 不能只判断父节点。

点双/边双连通可在 Tarjan 过程中用边栈或 DSU 缩桥。桥树上路径问题常转为树上 LCA/差分。

```

// 变量: timer=无向图桥 Tarjan DFS 使用的递增时间戳 timer; dfn=DFS 首次访问时间戳 dfn
; low=Tarjan 最低可达时间戳 low; bridges=已经找到的桥边端点集合 bridges。
int timer = 0; vector<int> dfn(n), low(n); vector<pair<int, int>> bridges;
// 接口: dfs(u, parent_edge): 从 u 继续无向图 Tarjan, 填写 dfn/low 并收集桥; 不返回值。
// 参数: 当前顶点编号 u; DFS 进入 u 所用的无向边编号 parent_edge。
void dfs(int u, int parent_edge) {
    dfn[u] = low[u] = ++timer;
    // 变量: v=当前边的终点 v; id=当前无向边的唯一编号 id, 用于跳过 DFS 父边。
    for (auto [v, id] : g[u]) {
        if (id == parent_edge) continue;
        if (!dfn[v]) {
            dfs(v, id); low[u] = min(low[u], low[v]);
            if (low[v] > dfn[u]) bridges.push_back({u, v});
        } else low[u] = min(low[u], dfn[v]);
    }
}

```

9.7 二分图

9.7.1 染色判定

BFS/DFS 交替染色; 发现相邻同色则不是二分图。每个连通块可独立选择两种颜色, 计数时乘法合并。

```

// 变量: color=二分图 BFS 中每个顶点的 0/1 颜色数组 color; -1 表示尚未染色。
vector<int> color(n, -1);
// 变量: ok=当前条件是否仍成立 ok。

```

```

bool ok = true;
// 变量: s=当前新连通分量开始二分图染色的起点 s。
for (int s = 0; s < n; ++s) if (color[s] < 0) {
    // 变量: q=当前连通块二分图染色所用的 BFS 顶点队列 q。
    queue<int> q; q.push(s); color[s] = 0;
    while (!q.empty()) {
        // 变量: u=二分图染色 BFS 当前出队并检查邻点的顶点编号 u。
        int u = q.front(); q.pop();
        // 变量: v=当前边的终点 v。
        for (int v : g[u]) {
            if (color[v] < 0) color[v] = color[u] ^ 1, q.push(v);
            else if (color[v] == color[u]) ok = false;
        }
    }
}
}

```

9.7.2 最大匹配

- 匈牙利 DFS: 适合 $V, E \leq$ 几千, 复杂度 $O(VE)$ 。
- Hopcroft-Karp: 分层增广, $O(E\sqrt{V})$, 适合更大二分图。
- 二分图最小点覆盖 = 最大匹配 (König); 最大独立集 = V -最大匹配。

```

// 接口: max_bipartite_matching(g, right_n): 返回二分图最大匹配边数。
// 参数: g[u] 给出左部点 u 的右部邻点; 右部点数 right_n, 编号为 [0, right_n)。
int max_bipartite_matching(const vector<vector<int>>& g, int right_n) {
    // 变量: matchR=右部点匹配到的左部点; vis=本轮增广访问时间戳。
    vector<int> matchR(right_n, -1), vis(right_n);
    // 变量: stamp=当前左部点增广搜索使用的访问时间戳。
    int stamp = 0;
    // 接口: 匿名 lambda(u): 从左部点 u 尝试寻找一条增广路; 返回是否增广成功。
    function<bool(int)> aug = [&](int u) {
        // 变量: v=当前尝试匹配的右部点。
        for (int v : g[u]) if (vis[v] != stamp) {
            vis[v] = stamp;
            if (matchR[v] == -1 || aug(matchR[v]))
                return matchR[v] = u, true;
        }
        return false;
    };
    // 变量: matching=已经找到的匹配边数。
    int matching = 0;
    // 变量: u=当前发起增广搜索的左部点。
    for (int u = 0; u < (int)g.size(); ++u)
        ++stamp, matching += aug(u);
}

```

```
    return matching;
}
```

9.7.3 带权匹配

KM 算法求完备二分图最大权匹配，通常约 $O(n^3)$ ；需要处理负权时整体平移或按模板维护顶标。

9.8 网络流

9.8.1 建模

容量表示资源上限，源点连供应端，需求端连汇点。拆点处理点容量；上下界流增加下界需求并用超级源汇检查可行性。

9.8.2 Dinic

BFS 建层次图，DFS 在层次递增的图上发送阻塞流；一般题目规模足够使用，二分图匹配也可套用。每条边要维护反向边 `cap`。

```
struct Edge { int to, rev; long long cap; }; // 终点、反向边下标、当前残量容量。
// 变量: fg=Dinic 残量网络邻接表 fg。
vector<vector<Edge>> fg;
// 变量: level=Dinic 层次图中的顶点层数 level; it=Dinic 中每个顶点下一条待尝试边的当前弧下标 it。
vector<int> level, it;
// 接口: add_edge(u, v, c): 向 Dinic 残量网络加入容量 c 的边及零容量反向边；会修改当前结构；多测时重新构造或显式清空成员。
// 参数: 当前顶点编号 u; 与 u 配对的顶点编号 v; Dinic 正向边容量 c。
void add_edge(int u, int v, long long c) {
    // 变量: a=正向残量边 a; b=配套的反向残量边 b。
    Edge a{v, (int)fg[v].size(), c}, b{u, (int)fg[u].size(), 0};
    fg[u].push_back(a); fg[v].push_back(b);
}
// 接口: bfs(s, t): 从源点 s 建立 Dinic 层次图；当残量网络中汇点 t 可达时返回 true。
// 参数: 层次图的源点编号 s; 层次图的汇点编号 t。
bool bfs(int s, int t) {
    // 变量: q=Dinic 建立层次图时保存待扩展顶点的 BFS 队列 q。
    fill(level.begin(), level.end(), -1); queue<int> q;
    level[s] = 0; q.push(s);
    // 变量: u=Dinic 层次图 BFS 当前出队并扫描残量边的顶点编号 u。
    while (!q.empty()) { int u = q.front(); q.pop();
```

```

        for (auto &e : fg[u]) if (e.cap && level[e.to] < 0)
            level[e.to] = level[u] + 1, q.push(e.to);
    }
    return level[t] >= 0;
}
// 接口: dfs(u, t, f): 在 Dinic 层次图上从 u 向汇点 t 推送至多 f 的阻塞流; 返回类型 long long。
// 参数: 当前顶点编号 u; 增广目标汇点编号 t; 本次 DFS 允许继续发送的流量 f。
long long dfs(int u, int t, long long f) {
    if (u == t) return f;
    for (int &i = it[u]; i < (int)fg[u].size(); ++i) {
        // 变量: e=当前遍历的边对象 e。
        Edge &e = fg[u][i];
        if (e.cap && level[e.to] == level[u] + 1) {
            // 变量: got=本次增广实际发送的流量 got。
            long long got = dfs(e.to, t, min(f, e.cap));
            if (got) { e.cap -= got; fg[e.to][e.rev].cap += got; return got; }
        }
    }
    return 0;
}

```

最大流最小割：最大流值等于最小割容量；割边可由残量网络中从源可达的点集确定。最小割常用于“选/不选且有冲突代价”的闭合图建模。

9.8.3 费用流

增广最短路（势能 + Dijkstra 或 SPFA）反复找最小费用路径；适合规模较小的带费用匹配/运输，注意负边与势能初始化。

```

struct MCEdge { int to, rev, cap, cost; }; // 终点、反向边下标、残量容量、单位费用。
// 变量: mg=最小费用流残量网络邻接表 mg。
vector<vector<MCEdge>> mg(n);
// 接口: add_edge(u, v, cap, cost): 向最小费用流残量网络加入容量 cap、费用 cost 的边及其反向边；会修改当前结构；多测时重新构造或显式清空成员。
// 参数: 当前顶点编号 u; 与 u 配对的顶点编号 v; 边容量 cap; 每单位流费用 cost。
void add_edge(int u, int v, int cap, int cost) {
    // 变量: a=正向残量边 a。
    MCEdge a{v, (int)mg[v].size(), cap, cost};
    // 变量: b=与正向费用流边配对的反向残量边 b。
    MCEdge b{u, (int)mg[u].size(), 0, -cost};
    mg[u].push_back(a); mg[v].push_back(b);
}

```



```

// 接口: min_cost_flow(s, t, need): 发送至多 need 单位流, 返回 (实际流量, 最小费用);
// 返回类型 pair<int, long long>。
// 参数: 费用流源点编号 s; 费用流汇点编号 t; 希望发送的流量 need。
pair<int, long long> min_cost_flow(int s, int t, int need) {
    // 变量: flow=当前已经发送的流量 flow; cost=当前累计费用 cost。
    int flow = 0; long long cost = 0;
    // 变量: pot=费用流顶点势能数组 pot; dist=当前维护的距离 dist。
    vector<long long> pot(n), dist(n);
    // 变量: pv=最短增广路中顶点的前驱点 pv; pe=最短增广路中使用的前驱边下标 pe。
    vector<int> pv(n), pe(n);
    while (flow < need) {
        fill(dist.begin(), dist.end(), (long long)4e18);
        priority_queue<pair<long long, int>, vector<pair<long long, int>>,
            greater<>> pq;
        dist[s] = 0; pq.push({0, s});
        while (!pq.empty()) {
            auto [du, u] = pq.top(); pq.pop();
            if (du != dist[u]) continue;
            // 变量: i=最小费用流中当前检查的顶点 u 出边下标 i。
            for (int i = 0; i < (int)mg[u].size(); ++i) {
                // 变量: e=当前遍历的边对象 e。
                auto const &e = mg[u][i];
                if (!e.cap) continue;
                // 变量: nd=候选新距离 nd。
                long long nd = du + e.cost + pot[u] - pot[e.to];
                if (nd < dist[e.to])
                    dist[e.to] = nd, pv[e.to] = u, pe[e.to] = i,
                    pq.push({nd, e.to});
            }
        }
        if (dist[t] == (long long)4e18) break;
        // 变量: v=当前检查势能更新的费用流顶点 v。
        for (int v = 0; v < n; ++v) if (dist[v] < (long long)4e18) pot[v] +=
dist[v];
        // 变量: add=当前操作要叠加的常数增量 add。
        int add = need - flow;
        // 变量: v=沿最短增广路从汇点回溯到源点的当前顶点 v。
        for (int v = t; v != s; v = pv[v]) add = min(add, mg[pv[v]][pe[v]].cap);
        // 变量: v=沿最短增广路从汇点回溯到源点的当前顶点 v。
        for (int v = t; v != s; v = pv[v]) {
            // 变量: e=最短增广路上从 pv[v] 指向 v 的残量边引用 e。
            auto &e = mg[pv[v]][pe[v]];
            e.cap -= add; mg[v][e.rev].cap += add; cost += 1LL * add * e.cost

```

```

    ;
    }
    flow += add;
}
return {flow, cost};
}

```

9.9 树上算法

9.9.1 DFS 序与子树

$\text{tin}[u]$ 、 $\text{tout}[u]$ 使子树变成连续区间 $[\text{tin}[u], \text{tout}[u]]$ 。子树加/和直接用树状数组或线段树；路径则还需 LCA/重链。

```

// 变量: timer=DFS 使用的递增时间戳 timer; tin=DFS 进入每个节点的时间戳 tin; tout=DFS
// 离开每个节点的时间戳 tout; depth=DFS 树中每个顶点的深度数组 depth; euler=欧拉路径
// 最终顶点序列 euler。
int timer = 0; vector<int> tin(n), tout(n), depth(n), euler(n + 1);
// 接口: dfs(u, p): 为以 u 为根的子树填写 DFS 序、深度和子树区间; 不返回值。
// 参数: 当前顶点编号 u; 当前节点 u 的父节点 p。
void dfs(int u, int p) {
    tin[u] = ++timer; euler[timer] = u;
    // 变量: v=从 tree[u] 枚举的相邻顶点 v。
    for (int v : tree[u]) if (v != p)
        depth[v] = depth[u] + 1, dfs(v, u);
    tout[u] = timer;
}

```

9.9.2 最近公共祖先 (Lowest Common Ancestor, LCA) 倍增

预处理 $\text{up}[u][j]$ 与深度，查询时先抬高深度差，再从大到小跳；预处理 $O(V \log V)$ ，查询 $O(\log V)$ 。同时可维护路径最小/最大/和。

```

// 变量: LOG=倍增表层数 LOG, 需满足  $2^{\text{LOG}}$  大于最大规模。
const int LOG = 20;
int up[MAXN][LOG]; // up[u][k]=u 的  $2^k$  级祖先; 不存在时为 0。
int dep[MAXN]; // dep[u]=u 的深度; 根深度约定为 0。
// 接口: init_lca(u, p): 从 u 开始 DFS, 原地填写各顶点深度 dep 与倍增祖先表 up; 无返回值。
// 参数: 当前节点 u; u 的父节点 p, 根节点的父节点传 0。
void init_lca(int u, int p) {
    up[u][0] = p;
    // 变量: j=为顶点 u 递推 up[u][j] 时当前填写的倍增层编号 j。
}

```

```

    for (int j = 1; j < LOG; ++j) up[u][j] = up[up[u][j - 1]][j - 1];
    // 变量: v=从 tree[u] 枚举的相邻顶点 v。
    for (int v : tree[u]) if (v != p) dep[v] = dep[u] + 1, init_lca(v, u);
}
// 接口: lca(u, v): 返回树上 u,v 的最近公共祖先; 返回类型 int。
// 参数: 当前顶点编号 u; 与 u 配对的顶点编号 v。
int lca(int u, int v) {
    if (dep[u] < dep[v]) swap(u, v);
    // 变量: d=把较深顶点提升到同深度所需的层数差 d。
    int d = dep[u] - dep[v];
    // 变量: j=按深度差 d 的置位情况提升顶点 u 时检查的倍增层编号 j。
    for (int j = 0; j < LOG; ++j) if (d >> j & 1) u = up[u][j];
    if (u == v) return u;
    // 变量: j=LCA 查询中从高到低比较 u、v 祖先的倍增层编号 j。
    for (int j = LOG - 1; j >= 0; --j)
        if (up[u][j] != up[v][j]) u = up[u][j], v = up[v][j];
    return up[u][0];
}

```

9.9.3 树上差分

点路径 (u,v) 加一: $d[u]++$, $d[v]++$, $d[lca]--$, $d[parent(lca)]--$; 边路径把 $d[lca]-=2$ 。后序累加得到每点/边被经过次数。

```

// 接口: add_path(u, v): 给树上 u-v 路径做差分标记, 之后必须自底向上 collect; 会修改当前结构; 多测时重新构造或显式清空成员。
// 参数: 当前顶点编号 u; 与 u 配对的顶点编号 v。
void add_path(int u, int v) {
    // 变量: w=树上路径 u-v 的最近公共祖先节点 w。
    int w = lca(u, v);
    ++diff[u]; ++diff[v]; --diff[w];
    if (up[w][0]) --diff[up[w][0]];
}
// 接口: collect(u, p): 后序累加树上路径差分, 把孩子贡献汇总到父节点; 不返回值。
// 参数: 当前顶点编号 u; 当前节点 u 的父节点 p。
void collect(int u, int p) {
    // 变量: v=从 tree[u] 枚举的相邻顶点 v。
    for (int v : tree[u]) if (v != p)
        collect(v, u), diff[u] += diff[v];
}

```

9.9.4 直径与重心

树直径：从任一点 BFS/DFS 找最远 s ，再从 s 找最远 t ； $\text{dist}(s,t)$ 为直径。带权树同理（边权非负）。重心：删除后最大连通块最小的点，可由子树大小判断。

```
// 接口: farthest(s): 返回从起点出发最远的 (距离, 顶点); 返回类型 pair<int, long long>。
// 参数: 树上最远点搜索的起点 s。
pair<int, long long> farthest(int s) {
    // 变量: d=从起点 s 到各树顶点的距离数组 d; q=树上最远点搜索中保存待扩展顶点的 BFS 队列 q。
    vector<long long> d(n, -1); queue<int> q;
    d[s] = 0; q.push(s);
    // 变量: best=目前找到的最优值 best。
    int best = s;
    // 变量: u=树上最远点 BFS 当前出队并扩展邻边的顶点编号 u。
    while (!q.empty()) { int u = q.front(); q.pop();
        if (d[u] > d[best]) best = u;
        // 变量: v=从 weighted_tree[u] 枚举的相邻顶点 v; w=连接该顶点的边权 w。
        for (auto [v, w] : weighted_tree[u]) if (d[v] < 0)
            d[v] = d[u] + w, q.push(v);
    }
    return {best, d[best]};
}
```

9.9.5 树形 DP

先子树后父亲。常见状态：选/不选（独立集）、子树颜色、背包合并、最大匹配、距离和。换根 DP：第一次 DFS 求根答案，第二次用父节点答案推出子节点答案。

```
// 接口: tree_dp(u, p): 自底向上计算以 u 为根的树形 DP 状态; 不返回值。
// 参数: 当前顶点编号 u; 当前节点 u 的父节点 p。
void tree_dp(int u, int p) {
    dp[u][0] = 0; dp[u][1] = weight[u];
    // 变量: v=从 tree[u] 枚举的相邻顶点 v。
    for (int v : tree[u]) if (v != p) {
        tree_dp(v, u);
        dp[u][0] += max(dp[v][0], dp[v][1]);
        dp[u][1] += dp[v][0];
    }
}
```

9.9.6 重链剖分 (Heavy-Light Decomposition, HLD)

把每条重链映射到数组，路径拆成 $O(\log V)$ 个连续区间，配合线段树实现路径/子树修改查询，复杂度 $O(\log^2 V)$ 。先求 `size, heavy`，再求 `top, dfn`。

```
// 接口: dfs1(u, p): 计算 HLD 所需的父亲、深度、子树大小和重儿子; 不返回值。
// 参数: 当前顶点编号 u; 当前节点 u 的父节点 p。
void dfs1(int u, int p) {
    parent[u] = p; size[u] = 1; heavy[u] = 0;
    // 变量: v=从 tree[u] 枚举的相邻顶点 v。
    for (int v : tree[u]) if (v != p) {
        dep[v] = dep[u] + 1; dfs1(v, u); size[u] += size[v];
        if (!heavy[u] || size[v] > size[heavy[u]]) heavy[u] = v;
    }
}

// 接口: dfs2(u, topf): 沿重链优先编号并填写链顶 top; 不返回值。
// 参数: 当前顶点编号 u; 当前重链链顶 topf。
void dfs2(int u, int topf) {
    top[u] = topf; dfn[u] = ++timer;
    if (heavy[u]) dfs2(heavy[u], topf);
    // 变量: v=从 tree[u] 枚举的相邻顶点 v。
    for (int v : tree[u]) if (v != parent[u] && v != heavy[u]) dfs2(v, v);
}

// 接口: path_sum(u, v): 返回树上 u 到 v 路径的权值和; 返回类型 long long。
// 参数: 当前顶点编号 u; 与 u 配对的顶点编号 v。
long long path_sum(int u, int v) {
    // 变量: ans=重链剖分查询路径 u-v 时累计的各链段区间和 ans。
    long long ans = 0;
    while (top[u] != top[v]) {
        if (dep[top[u]] < dep[top[v]]) swap(u, v);
        ans += seg.query(dfn[top[u]], dfn[u]); u = parent[top[u]];
    }
    if (dep[u] > dep[v]) swap(u, v);
    return ans + seg.query(dfn[u], dfn[v]);
}
```

9.9.7 点分治 (入门)

以重心分治树，统计跨重心的路径信息，递归处理子树；每层总计 $O(V)$ ，深度 $O(\log V)$ ，常见复杂度 $O(V \log V)$ 。适用于树上距离小于/等于 k 的点对计数、距离最小值。

```
// 接口: get_size(u, p): 统计当前未删除连通块的子树大小; 返回类型 int。
// 参数: 当前顶点编号 u; 当前节点 u 的父节点 p。
int get_size(int u, int p) {
    sz[u] = 1;
```

```

// 变量: v=从 tree[u] 枚举的相邻顶点 v。
for (int v : tree[u]) if (v != p && !removed[v]) sz[u] += get_size(v, u);
return sz[u];
}
// 接口: get_centroid(u, p, total): 返回当前连通块的重心; 返回类型 int。
// 参数: 当前顶点编号 u; 当前节点 u 的父节点 p; 当前连通块总大小 total。
int get_centroid(int u, int p, int total) {
    // 变量: v=从 tree[u] 枚举的相邻顶点 v。
    for (int v : tree[u]) if (v != p && !removed[v])
        if (sz[v] * 2 > total) return get_centroid(v, u, total);
    return u;
}
// 接口: divide(entry): 找出 entry 所在未删除连通块的重心、标记 removed 并递归分解各剩
// 余子块; 无返回值。
// 参数: 当前连通块入口 entry。
void divide(int entry) {
    // 变量: c=当前未删除连通块的重心顶点 c。
    int c = get_centroid(entry, 0, get_size(entry, 0));
    removed[c] = true;
    // 统计经过 c 的贡献
    // 变量: v=从 tree[c] 枚举的相邻顶点 v。
    for (int v : tree[c]) if (!removed[v]) divide(v);
}

```

9.9.8 树上启发式合并 (Disjoint Set Union on Tree, DSU on Tree)

先求子树大小和重儿子; 轻儿子递归后清空, 重儿子保留统计结果, 再把所有轻子树合并进来。每个节点被清除/加入的次数受重轻性质控制, 总复杂度通常为 $O(V \log V)$ 。

```

vector<int> sz(n + 1), heavy(n + 1); // 子树大小、重儿子。
vector<int> color(n + 1), freq(MAXC); // 颜色和当前统计集合中的频次。
// 变量: answer=当前维护的答案 answer。
vector<int> answer(n + 1);
int distinct = 0; // 示例维护量: 当前集合中出现过的颜色数。

// 接口: dfs_size(u, p): 计算以 u 为根的子树大小 sz, 并选出子树大小最大的重儿子 heavy
// ; 不返回值。
// 参数: 当前顶点编号 u; 当前节点 u 的父节点 p。
void dfs_size(int u, int p) {
    // 第一次 DFS 只决定轻/重儿子, 不修改答案统计。
    sz[u] = 1;
    // 变量: v=从 tree[u] 枚举的相邻顶点 v。
    for (int v : tree[u]) if (v != p) {
        dfs_size(v, u); sz[u] += sz[v];
    }
}

```

```

        if (!heavy[u] || sz[v] > sz[heavy[u]]) heavy[u] = v;
    }
}
// 接口: add_subtree(u, p, delta): 遍历 u 的整棵子树且不回到父节点 p, 按 delta=1 加入颜色频次或按 delta=-1 移除颜色频次; 会修改当前统计。
// 参数: 当前顶点编号 u; 当前节点 u 的父节点 p; 本次增量 delta。
void add_subtree(int u, int p, int delta) {
    // delta=+1 是加入, delta=-1 是清空。
    // 这里的“答案维护器”可以替换成最大频率、频率平方和等任何可增删统计。
    if (delta == 1 && freq[color[u]]++ == 0) ++distinct;
    if (delta == -1 && --freq[color[u]] == 0) --distinct;
    // 必须递归整棵子树; 只处理 u 会漏掉大量颜色。
    // 变量: v=从 tree[u] 枚举的相邻顶点 v。
    for (int v : tree[u]) if (v != p) add_subtree(v, u, delta);
}
// 接口: dsu_on_tree(u, p, keep): 按 keep 参数执行 DSU on Tree; keep=false 时清除贡献; 不返回值。
// 参数: 当前顶点编号 u; 当前节点 u 的父节点 p; 是否保留 u 子树统计贡献 keep。
void dsu_on_tree(int u, int p, bool keep) {
    // 轻儿子先算并清空: 它们只贡献一次, 避免统计互相污染。
    // 变量: v=从 tree[u] 枚举的相邻顶点 v。
    for (int v : tree[u]) if (v != p && v != heavy[u]) dsu_on_tree(v, u, false);
    // 重儿子保留答案, 后面把轻子树和 u 合并进这份大集合。
    if (heavy[u]) dsu_on_tree(heavy[u], u, true);
    // 把所有轻儿子整棵加入重儿子的统计集合。
    // 变量: v=从 tree[u] 枚举的相邻顶点 v。
    for (int v : tree[u]) if (v != p && v != heavy[u]) add_subtree(v, u, 1);
    // 最后加入 u 自己, 此时集合恰好是 u 的整棵子树。
    if (freq[color[u]]++ == 0) ++distinct;
    answer[u] = distinct; // 换成题目所需的当前子树统计量
    if (!keep) {
        // u 是父节点的轻儿子时, 当前统计不会被父节点复用, 整棵删除。
        add_subtree(u, p, -1);
    }
}

```

9.9.9 长链剖分

长链剖分按“向下最长深度”而不是子树大小选重儿子, 适合树上深度/祖先方向的 DP 优化和长链合并。它和 HLD 的选重标准不同, 不能混用复杂度证明。

```

// 变量: parent2=长链剖分中每个顶点的父节点数组 parent2; depth2=长链剖分中每个顶点的深度数组 depth2; heavy_len=从每个点向下的最长链长度 heavy_len。

```

```

vector<int> parent2(n + 1), depth2(n + 1), heavy_len(n + 1);
// 变量: heavy2=每个点的最长链儿子 heavy2; top2=每个点所属最长链链顶 top2。
vector<int> heavy2(n + 1), top2(n + 1);
// 接口: long_chain_dfs1(u, p): 计算子树最大深度并选择最长链儿子; 不返回值。
// 参数: 当前顶点编号 u; 当前节点 u 的父节点 p。
void long_chain_dfs1(int u, int p) {
    // heavy_len[u] 是从 u 向下的最长链长度, 不是子树大小。
    parent2[u] = p; heavy_len[u] = 1;
    // 变量: v=从 tree[u] 枚举的相邻顶点 v。
    for (int v : tree[u]) if (v != p) {
        depth2[v] = depth2[u] + 1;
        long_chain_dfs1(v, u);
        // 选择向下最深的儿子作为长链儿子。
        if (!heavy2[u] || heavy_len[v] > heavy_len[heavy2[u]]) heavy2[u] = v;
    }
    if (heavy2[u]) heavy_len[u] = heavy_len[heavy2[u]] + 1;
}
// 接口: long_chain_dfs2(u, topf): 沿最长链儿子优先递归并填写每个顶点所属链顶 top2; 不返回值。
// 参数: 当前顶点编号 u; 当前重链链顶 topf。
void long_chain_dfs2(int u, int topf) {
    // 长儿子沿用同一个链顶, 其他儿子各自开新链。
    top2[u] = topf;
    if (heavy2[u]) long_chain_dfs2(heavy2[u], topf);
    // 变量: v=从 tree[u] 枚举的相邻顶点 v。
    for (int v : tree[u])
        if (v != parent2[u] && v != heavy2[u]) long_chain_dfs2(v, v);
}

```

9.9.10 Link-Cut Tree 动态树 (LCT)

LCT 用 splay 维护动态森林, 支持 link/cut、换根和路径聚合。下面模板维护路径和; 将 pull 中的 sum 换成结构体即可维护最大值、异或等可合并信息。

```

struct LCT {
    struct Node {
        int ch[2]{}; // splay 左右儿子。
        int fa{}; // 辅助树父亲; 不等同于原树父亲。
        long long val{}; // 点权。
        long long sum{}; // 当前 splay 子树的维护信息。
        bool rev{}; // 是否有待下传的换根翻转标记。
    };
    vector<Node> t; // t[1..n] 对应原树点; t[0] 是空哨兵且所有字段为 0。
    // 接口: LCT(n): 构造含 n 个 1-based 点的 Link-Cut Tree; 构造并初始化对象; 每组测试

```


建议重新构造。

// 参数: Link-Cut Tree 管理的 1-based 顶点数量 n 。

```
LCT(int n): t(n + 1) {}
```

// 接口: `is_root(x)`: 判断 x 是否是当前 `splay` 辅助树的根; x 没有辅助树父节点或不作为父节点的直接儿子时返回 `true`。

// 参数: Link-Cut Tree 中的原树顶点编号 x 。

```
bool is_root(int x) const {
```

// x 没有 `splay` 父亲, 或不是父亲的直接儿子时, 是辅助树根。

// 变量: $f=t[x].fa$ 保存的节点 x 当前辅助树父节点编号 f 。

```
int f = t[x].fa;
```

```
return !f || (t[f].ch[0] != x && t[f].ch[1] != x);
```

```
}
```

// 接口: `pull_sum(x)`: 由两个 `splay` 儿子和自身点权重算 $t[x].sum$; 不返回值。

// 参数: Link-Cut Tree 中的原树顶点编号 x 。

```
void pull_sum(int x) {
```

// 修改儿子后必须重新计算聚合信息。

```
t[x].sum = t[x].val + t[t[x].ch[0]].sum + t[t[x].ch[1]].sum;
```

```
}
```

// 接口: `apply_rev(x)`: 交换 x 的左右儿子并翻转路径反转标记; 不返回值。

// 参数: Link-Cut Tree 中的原树顶点编号 x 。

```
void apply_rev(int x) {
```

```
if (!x) return;
```

// 翻转只交换辅助树儿子; 原树路径方向由 `makeroot` 实现。

```
swap(t[x].ch[0], t[x].ch[1]); t[x].rev ^= 1;
```

```
}
```

// 接口: `push(x)`: 把 LCT 辅助树节点 x 的路径反转标记下传给两个儿子; 会修改当前结构; 多测时重新构造或显式清空成员。

// 参数: Link-Cut Tree 中的原树顶点编号 x 。

```
void push(int x) {
```

// 旋转前必须把从辅助树根到 x 的所有 `rev` 标记按顺序下传。

```
if (t[x].rev) apply_rev(t[x].ch[0]), apply_rev(t[x].ch[1]), t[x].rev = false;
```

```
}
```

// 接口: `rotate(x)`: 在辅助 `splay` 中把 x 向上旋转一层; 会修改当前结构; 多测时重新构造或显式清空成员。

// 参数: Link-Cut Tree 中的原树顶点编号 x 。

```
void rotate(int x) {
```

// 一次旋转把 x 提升; b 是 x 原来靠旋转方向的另一棵子树。

// 变量: $y=LCT$ 旋转中节点 x 的父节点 y ; $z=LCT$ 旋转中节点 y 的父节点 z ; dx =节点 x 是 y 的右儿子时为 1 的方向标记 dx ; b =旋转后要接到 y 下方的子树根 b 。

```
int y = t[x].fa, z = t[y].fa, dx = (t[y].ch[1] == x), b = t[x].ch[dx ^ 1];
```

```
if (!is_root(y)) t[z].ch[t[z].ch[1] == y] = x;
```

```
t[x].fa = z; t[x].ch[dx ^ 1] = y; t[y].fa = x; t[y].ch[dx] = b;
```

```

    if (b) t[b].fa = y;
    pull_sum(y); pull_sum(x);
}
// 接口: splay(x): 把 x 旋到其当前辅助树根, 并先下传祖先标记; 会修改当前结构; 多测时
// 重新构造或显式清空成员。
// 参数: Link-Cut Tree 中的原树顶点编号 x。
void splay(int x) {
    // 先收集祖先, 再从上到下 push, 确保旋转时方向标记正确。
    // 变量: st=splay 前按自底向上顺序收集的辅助树祖先栈 st。
    vector<int> st{ x };
    // 变量: y=从 x 沿辅助树父链上溯并收集祖先的当前节点 y。
    for (int y = x; !is_root(y); y = t[y].fa) st.push_back(t[y].fa);
    while (!st.empty()) push(st.back()), st.pop_back();
    while (!is_root(x)) {
        // 变量: y=LCT 旋转中节点 x 的父节点 y; z=LCT 旋转中节点 y 的父节点 z。
        int y = t[x].fa, z = t[y].fa;
        if (!is_root(y)) ((t[y].ch[0] == x) ^ (t[z].ch[0] == y)) ? rotate(
x) : rotate(y);
        rotate(x);
    }
}
// 接口: access(x): 打通原树根到 x 的首选路径, 结束时 x 为辅助树根; 会修改当前结构;
// 多测时重新构造或显式清空成员。
// 参数: Link-Cut Tree 中的原树顶点编号 x。
void access(int x) {
    // 依次把 x 到原树根的 preferred path 变成一条辅助树路径。
    // y 是上一轮处理的节点, 挂到 x 的右儿子。
    // 变量: y=access 中上一轮已处理的首选路径根 y。
    for (int y = 0; x; x = t[y = x].fa) splay(x), t[x].ch[1] = y, pull_sum
(x);
}
// 接口: makeroot(x): 把 x 设为其动态树的根; 会修改当前结构; 多测时重新构造或显式清
// 空成员。
// 参数: Link-Cut Tree 中的原树顶点编号 x。
void makeroot(int x) {
    // access+splay 后 x 表示整条根到 x 的路径, 翻转它即可让 x 成为原树根。
    access(x); splay(x); apply_rev(x);
}
// 接口: findroot(x): 返回动态树中 x 所在树的根; 返回类型 int。
// 参数: Link-Cut Tree 中的原树顶点编号 x。
int findroot(int x) {
    // 暴露 x 后不断向左走, 左端点就是所在原树的根。
    access(x); splay(x);
    while (t[x].ch[0]) push(x), x = t[x].ch[0];
}

```

```

    splay(x); return x;
}
// 接口: split(x, y): 通过 makeroot(x)、access(y)、splay(y) 暴露原树路径 x..y
// 使该路径由 y 的辅助树维护; 无返回值。
// 参数: 待暴露路径的第一个原树顶点 x; 待暴露路径的第二个原树顶点 y。
void split(int x, int y) {
    // 之后 y 的 splay 子树正好维护原树路径 x..y。
    makeroot(x); access(y); splay(y);
}
// 接口: link(x, y): 连接原本不连通的 x,y; 会修改当前结构; 多测时重新构造或显式清空
// 成员。
// 参数: Link-Cut Tree 中的原树顶点编号 x; Link-Cut Tree 中的原树顶点编号 y。
void link(int x, int y) {
    // 先换根保证 x 没有原树父亲, 再确认两点不连通。
    makeroot(x); if (findroot(y) != x) t[x].fa = y;
}
// 接口: cut(x, y): 删除动态树中的边 (x,y); 会修改当前结构; 多测时重新构造或显式清空
// 成员。
// 参数: Link-Cut Tree 中的原树顶点编号 x; Link-Cut Tree 中的原树顶点编号 y。
void cut(int x, int y) {
    // 暴露边 x-y; 合法形态要求 y 的左儿子是 x 且 x 没有右儿子。
    split(x, y);
    if (t[y].ch[0] == x && !t[x].ch[1]) t[y].ch[0] = t[x].fa = 0, pull_sum
(y);
}
// 接口: path_sum(x, y): 返回树上 u 到 v 路径的权值和; 返回类型 long long。
// 参数: Link-Cut Tree 中的原树顶点编号 x; Link-Cut Tree 中的原树顶点编号 y。
long long path_sum(int x, int y) {
    split(x, y);
    return t[y].sum;
}
};

```

9.10 欧拉路、欧拉回路与哈密顿误区

无向图存在欧拉回路当且仅当非零度点连通且所有度为偶; 欧拉路径有且仅有 0 或 2 个奇度点。有向图对应入度 = 出度, 路径起点出度比入度多 1、终点相反。Hierholzer 用栈在线性时间构造。哈密顿路径一般是 NP-hard, 不要把两者混淆。

```

// 变量: ptr=每个顶点下一条尚未扫描的邻接边位置 ptr。
vector<int> ptr(n);
// 接口: hierholzer(start): 从 start 构造欧拉路径并返回正向顶点序列; 返回类型 vector<
int>。

```

```
// 参数：欧拉路径的起点 start。
vector<int> hierholzer(int start) {
    // 变量：st=当前 DFS 路径栈 st; euler=按退栈顺序得到的逆序欧拉路径 euler。
    vector<int> st{start}, euler;
    while (!st.empty()) {
        // 变量：u=当前路径栈顶顶点 u。
        int u = st.back();
        while (ptr[u] < (int)g[u].size() &&
               used[g[u][ptr[u]].second])
            ++ptr[u];
        if (ptr[u] == (int)g[u].size()) {
            euler.push_back(u); st.pop_back();
        } else {
            auto [v, id] = g[u][ptr[u]++];
            used[id] = true; st.push_back(v);
        }
    }
    reverse(euler.begin(), euler.end());
    return euler;
}
```

9.11 图论组合套路

- “删掉一些边使图满足条件”常考虑 MST、割、桥、最小割。
- “两点之间经过某类边最少”常做分层图/状态图。
- “最多经过 k 次特殊边”建 k+1 层节点。
- “必须恰好经过/使用”常用流、DP 或矩阵，不要只跑最短路。
- DAG + 计数要同时维护 ways[v]，相同最优值时累加、更新更优时覆盖。

10 树上算法与高级结构

10.1 树上高级算法：参数、调用与改造

本节紧跟前面的基础模板。先看每个接口传什么，再看哪些不变量不能动，最后看如何替换维护信息或转移。

10.1.1 树上启发式合并 (Disjoint Set Union on Tree)

识别信号：树不修改，每个节点有颜色/权值，每个询问问“子树中出现次数最多的颜色”“不同颜色数”“频率平方和”等。先求重儿子，保留重儿子的贡献，轻儿子算完后清空，复杂度通常 $O(n \log n)$ 。

```
vector<vector<int>> g;           // 1-based 无向树邻接表；每组测试重新建图。
vector<int> sz;                 // sz[u]=以 u 为根的子树大小。
vector<int> son;                // son[u]=u 的重儿子；没有儿子时为 -1/0（按初始化）。
vector<int> color;              // color[u]=点 u 的颜色，使用前需离散化到 cnt 范围。
vector<long long> ans;          // ans[u]=以 u 为根的子树询问答案；可容纳频次平方和。
vector<int> cnt;                // cnt[c]=当前保留节点中颜色 c 的出现次数。
long long cur_answer = 0; // 示例：维护所有颜色频次的平方和。

// 接口：dfs_size(u, fa)：计算以 u 为根的子树大小 sz，并选出子树大小最大的重儿子 son；
// 不返回值。
// 参数：当前顶点编号 u；父节点 fa。
void dfs_size(int u, int fa) {
    // sz 和 son 只计算一次；son[u] 是子树最大的儿子。
    sz[u] = 1;
    // 变量：v=当前边的终点 v。
    for (int v : g[u]) if (v != fa) {
        dfs_size(v, u);
        sz[u] += sz[v];
        if (sz[v] > sz[son[u]]) son[u] = v;
    }
}

// 接口：add_node(u, delta)：把顶点 u 的颜色计数增加 delta；不返回值。
// 参数：顶点编号 u；颜色出现次数的增量 delta，加入为 1，移除为 -1。
```

```

void add_node(int u, int delta) {
    // 先删除颜色旧贡献，再改变频次，最后加入新贡献。
    // 这里选 freq^2 作为演示；换题时只改这段“单点统计器”。
    cur_answer -= 1LL * cnt[color[u]] * cnt[color[u]];
    cnt[color[u]] += delta;
    cur_answer += 1LL * cnt[color[u]] * cnt[color[u]];
}

// 接口: add_subtree(u, fa, delta): 遍历 u 的整棵子树且不回到父节点 fa, 按 delta 把
// 各顶点贡献加入或移出统计；会修改当前统计。
// 参数: 当前顶点编号 u; 父节点 fa; 本次增量 delta。
void add_subtree(int u, int fa, int delta) {
    add_node(u, delta);

    // 加/删的是整棵子树，不是只有根 u。
    // 变量: v=当前边的终点 v。
    for (int v : g[u])
        if (v != fa) add_subtree(v, u, delta);
}

// 接口: dsu(u, fa, keep): 按 keep 参数执行 DSU on Tree; keep=false 时清除 u 子树贡
// 献；不返回值。
// 参数: 当前顶点编号 u; 父节点 fa; 是否保留 u 子树统计贡献 keep。
void dsu(int u, int fa, bool keep) {
    // 轻儿子先算完并清空，避免它们的统计污染当前节点。
    // 变量: v=当前边的终点 v。
    for (int v : g[u])
        if (v != fa && v != son[u]) dsu(v, u, false);

    // 重儿子的统计保留，之后所有轻子树都合并到这份大答案中。
    if (son[u]) dsu(son[u], u, true);

    // 把轻儿子整棵子树逐个加入。
    // 变量: v=当前边的终点 v。
    for (int v : g[u])
        if (v != fa && v != son[u]) add_subtree(v, u, +1);
    // 重子树已经保留、轻子树刚刚加入，此处只加入 u 自己。
    add_node(u, +1);

    // 当前集合恰好是 u 的整棵子树，读取题目答案。
    ans[u] = cur_answer; // 换成题目要求的统计量

    if (!keep) {
        // 父节点不需要这棵轻子树的贡献，因此整棵删除。
        add_subtree(u, fa, -1);
    }
}

```

```

    }
}

```

例题:CF600E Lomsat gelral。题目问每个子树中出现次数最多的颜色之和。上面代码中的 `cur_answer` 要改为“维护当前最大频次 `mx`，每次 `cnt[color]` 从旧频次变为新频次时更新 `sum_color[mx]`”；算法框架完全不变。不要把 `add_subtree` 写成只加节点 `u`，轻儿子整棵子树必须加入。

测试样例：树上启发式合并必须统计整棵子树

为了直接验证上面示例中的 `freq^2` 统计器，下面输出每个节点子树内各颜色频次平方和。输入每组为 `n`、颜色、`n-1` 条无向边。

```

输入
3
1
5
3
1 2 1
1 2
1 3
5
1 2 2 3 2
1 2
1 3
3 4
3 5

输出
1
5 1 1
11 1 5 1 1

```

第二组能抓住“只加入节点 `u`、没有加入轻儿子整棵子树”的错误；第三组同时包含重儿子和轻儿子，检查清空轻子树后父节点统计是否恢复正确。

10.1.2 长链剖分

识别信号：树上大量询问“第 k 个祖先/深度为某值的节点”， k 很大，普通倍增内存或常数不理想。长链剖分按“向下最深”的儿子选重儿子，并把同一条长链连续存储，常配合 $O(1)$ 的第 k 级祖先。

```

// 接口: dfs_len(u, fa): 计算 u 子树最大深度并选出最长链儿子; 不返回值。
// 参数: 当前顶点编号 u; 父节点 fa。
void dfs_len(int u, int fa) {

```

```

// up[u] 是父亲; len[u] 是从 u 向下走的最长链节点数。
up[u] = fa; len[u] = 1; son[u] = 0;
// 变量: v=当前边的终点 v。
for (int v : g[u]) if (v != fa) {
    dfs_len(v, u);
    // 长链剖分按“深度最长”选重儿子, 不是按子树大小。
    if (len[v] + 1 > len[u])
        len[u] = len[v] + 1, son[u] = v;
}
}
// 接口: dfs_chain(u, top): 沿最长链优先递归并填写每个点的链顶 top; 不返回值。
// 参数: 当前顶点编号 u; 顶点 u 所属长链的链顶编号 top。
void dfs_chain(int u, int top) {
    // 同一条长链上的节点共享链顶 top, 并连续编号。
    chain_top[u] = top;
    chain_pos[u] = ++timer;

    // 先沿长儿子走, 保证一条长链连续。
    if (son[u]) dfs_chain(son[u], top);

    // 其他儿子都是新长链的起点。
    // 变量: v=当前边的终点 v。
    for (int v : g[u])
        if (v != up[u] && v != son[u]) dfs_chain(v, v);
}

```

例题: P5903 树上 K 级祖先。读入查询 (x, k) 后, 如果 $k > \text{depth}[x]$ 输出 0; 否则沿长链的 `chain_top` 和预处理的链数组跳转。这个题还特别容易爆栈, $n=5e5$ 时要按题目环境决定是否使用非递归 DFS 或栈扩展。

测试样例: 长链、星形树和超过根深度

每组输入 nq , 随后给出节点 $2..n$ 的父亲, 再给出 q 个 (x, k) ; 不存在的祖先输出 0。

输入

```

3
1 3
1 0
1 1
1 0
4 4
1 2 3
4 0
4 1

```



```

4 3
4 4
5 4
1 1 1 1
2 1
2 2
5 0
1 0

```

输出

```

1
0
1
4
3
1
0
1
0
5
1

```

第一组检查单点和 $k=0$ ；第二组是最长链，第三组是星形树。 k 超过深度时必须输出 0 ，不能访问未初始化的倍增表。

10.1.3 点分治与点分树

静态树上“距离为 k 的点数/距离小于等于 k 的点权和”是点分治；有在线修改则把每个节点到各层重心的距离放入重心树容器，形成点分树。

```

// 接口: collect(u, fa, d, ds): 收集未删除子树中各节点到当前点分治重心的距离；不返回值。
// 参数: 当前顶点编号 u; 父节点 fa; 距离 d; 接收当前子树距离的输出数组 ds。
void collect(int u, int fa, int d, vector<int>& ds) {
    // ds 保存“当前重心 c 到该分支中所有节点的距离”。
    // 点分治的每层都只收集未被 removed 的节点。
    ds.push_back(d);
    // 变量: v=当前边的终点 v。
    for (int v : g[u])
        if (v != fa && !removed[v]) collect(v, u, d + 1, ds);
}
// 接口: decompose(entry): 找出 entry 所在未删除连通块的重心并递归分解各子块；不返回值。
// 参数: 当前连通块入口 entry。

```

```

void decompose(int entry) {
    // get_centroid 必须只在当前未删除连通块中找重心。
    int c = get_centroid(entry); // 按子树大小找重心
    removed[c] = true;

    // 变量: v=从 g[c] 枚举的相邻顶点 v。
    for (int v : g[c]) if (!removed[v]) {
        // 变量: ds=当前收集到的距离列表 ds。
        vector<int> ds;
        collect(v, c, 1, ds);

        // 把该分支的距离加入 c 的排序数组; 查询时二分。
        // 同时还要把这些距离按“属于哪一个 c 的儿子分支”保存,
        // 查询时减掉同一分支的重复贡献(容斥)。
    }

    // 删除 c 后, 剩下的是若干个互不相连的子问题。
    // 变量: v=从 g[c] 枚举的相邻顶点 v。
    for (int v : g[c]) if (!removed[v]) decompose(v);
}

```

例题可以从 P6329 点分树 | 震波开始: 修改一个点的权值时, 沿重心父链向上, 把该点到每个重心的距离对应的贡献更新; 查询重心时, 对每层距离数组二分, 并减去“同一子树方向”的重复贡献。点分树模板比普通点分治多一层“重心祖先链”, 现场不要混用两个模板。

说明: 当前代码只展示了点分治的分解和距离收集框架, 没有给出完整的心查找、查询容器、修改和答案接口。因此这里不伪造“输入/标准输出”样例; 等这些接口补全后, 测试必须至少包含单点、链、星形树、半径为零和跨重心重复计数五类情况。

10.1.4 伸展树、文艺平衡树与 Link-Cut Tree 动态树

Splay 适合维护序列: 节点有左右儿子、父亲、子树大小和翻转标记; 把第 k 个节点旋到根, 就能做区间翻转、插入、删除。文艺平衡树的区间反转调用流程是: 把 $l-1$ 旋到根, 把 $r+1$ 旋到根的右儿子, 目标区间就是右儿子的左子树, 打 **rev** 标记。

LCT 适合动态森林的路径操作, 核心接口是 `makeroot(x); access(y); splay(y)`, 此时 y 的 splay 子树就是原树路径 $x..y$ 。在已有 LCT 模板中, 题目操作的对应关系通常是:

```

// 路径查询 x..y:
// makeroot(x) 把 x 变成原树根; split(x,y) 再 access(y),
// 此时 y 的辅助树包含原树路径 x..y, 维护值在 tr[y] 中汇总。
makeroot(x);
split(x, y);

```

```

answer = tr[y].sum;

// 连边：先确认两点不在同一棵树，否则会形成环。
makeroot(x);
if (findroot(y) != x) tr[x].fa = y;

// 断边：把边 x-y 暴露出来后，理想形态是 y 的左儿子恰为 x，
// 且 x 没有右儿子；这两个条件都满足才能安全删除。
makeroot(x);
access(y);
splay(y);
if (ch[y][0] == x && ch[x][1] == 0)
    ch[y][0] = fa[x] = 0;

```

例题：P3690 动态树要求路径异或、修改点权、连边和断边。先用已有板子完成 `is_root/rotate/splay/access/makeroot/findroot/split/link/cut`，再把 `sum` 的合并从加法换成异或。LCT 第一次现场学习风险很高，遇到它时优先复制完整模板，只修改“维护信息”和输入操作映射。

测试样例：动态树的路径、修改、连边和断边

下面采用 P3690 操作编号：0xy 查询路径异或，1xy 连边，2xy 断边，3xv 修改点权。

输入

```

3
1 3
5
0 1 1
3 1 7
0 1 1
3 7
1 2 4
1 1 2
1 2 3
0 1 3
3 2 5
0 1 3
2 2 3
0 1 2
4 7
1 1 1 1
1 1 2
1 3 4
0 1 2

```

```
1 2 3
0 1 4
2 2 3
0 1 2
```

输出

```
5
7
7
0
4
0
0
0
```

第二组检查路径合并和单点修改;第三组先建立两棵树,再连成一棵,最后断开,能抓住 **makeroot** /**access/splay** 顺序和断边条件错误。实际提交前还要根据题目保证避免查询不连通节点。

11 动态规划

11.1 DP 四问法

1. 状态表示什么最小信息？
2. 最后一步/最后一个决策是什么？
3. 转移是否覆盖且不重复所有情况？
4. 初值、遍历顺序、答案位置是什么？

先写朴素正确版本，再按状态数量、转移数量、内存逐层优化。**dp** 数组含义要写在代码注释里。

11.2 线性 DP

- LIS: $O(n \log n)$ 维护每个长度的最小结尾。
- 严格递增用 `lower_bound`，非降用 `upper_bound`。
- 最长公共子序列: `dp[i][j]` 看前缀；滚动到一维时保存左上角旧值。
- 最大子段和: `bestEnding = max(a[i], bestEnding + a[i])`，也可用前缀最小值。
- 计数 DP: 同一状态的方案相加，注意顺序是否导致重复计数。

```
// 变量: tail=LIS 各长度对应的最小结尾值数组 tail。
vector<int> tail;
// 变量: x=从 a 枚举的元素值 x。
for (int x : a) {
    auto it = lower_bound(tail.begin(), tail.end(), x); // 非降改 upper_bound
    if (it == tail.end()) tail.push_back(x);
    else *it = x;
}
// 变量: lis=当前最长递增子序列长度 lis。
int lis = tail.size();

// 变量: bestEnding=必须以当前位置结尾的精确最大子段和; maxSubarray=扫描至今的精确最大子段和。
__int128 bestEnding = a[0], maxSubarray = a[0];
// 变量: i=最大子段和扫描中当前作为结尾的数组位置 i。
```

```

for (int i = 1; i < n; ++i) {
    bestEnding = max<__int128>(a[i], bestEnding + a[i]);
    maxSubarray = max(maxSubarray, bestEnding);
}

// 变量: lcs=lcs[i][j] 保存 a 前 i 个字符与 b 前 j 个字符的最长公共子序列长度。
vector<vector<int>> lcs(n + 1, vector<int>(m + 1));
// 变量: i=LCS 状态 lcs[i][j] 对应的字符串 a 前缀长度 i。
for (int i = 1; i <= n; ++i)
    // 变量: j=LCS 状态 lcs[i][j] 对应的字符串 b 前缀长度 j。
    for (int j = 1; j <= m; ++j)
        lcs[i][j] = (a[i - 1] == b[j - 1])
            ? lcs[i - 1][j - 1] + 1
            : max(lcs[i - 1][j], lcs[i][j - 1]);

```

11.3 背包

类型	状态/遍历
0/1 背包	$dp[j] = \max(dp[j], dp[j - w] + v)$, j 从大到小
完全背包	同式, j 从小到大
多重背包	二进制拆分或单调队列优化
分组背包	每组只能选一个, 组内倒序并用上一层状态
依赖背包	树形 DP/附件分组
计数背包	把 $\max$ 换成加法, 模数下处理重复顺序

容量 $W \leq 1e5$ 时常用 $O(nW)$; 物品价值小则把“最小重量”作为状态; $n \leq 40$ 可折半搜索。

```

// 接口: zero_one_knapsack(items, W): 返回每个容量上限下的 0/1 背包最大价值。
// 参数: W>=0; 每件物品重量为正且价值为 int。正重量保证可选件数不超过 W, long long 足以
// 容纳最优价值。
vector<long long> zero_one_knapsack(const vector<pair<int, int>>& items, int W) {
    // 变量: dp=容量不超过下标时的 0/1 背包最大价值。
    vector<long long> dp(W + 1, 0);
    // 变量: w=当前物品重量; v=当前物品价值。
    for (auto [w, v] : items)
        // 变量: j=倒序枚举的背包容量。
        for (int j = W; j >= w; --j)
            dp[j] = max(dp[j], dp[j - w] + v);
    return dp;
}

```

```
// 接口: complete_knapsack(items, W): 返回每个容量上限下的完全背包最大价值。
// 参数: W>=0; 每件物品重量为正且价值为 int。最多选择 W 件, long long 足以容纳最优价值。
vector<long long> complete_knapsack(const vector<pair<int, int>>& items, int W) {
    // 变量: dp=容量不超过下标时的完全背包最大价值。
    vector<long long> dp(W + 1, 0);
    // 变量: w=当前物品重量; v=当前物品价值。
    for (auto [w, v] : items)
        // 变量: j=正序枚举的背包容量。
        for (int j = w; j <= W; ++j)
            dp[j] = max(dp[j], dp[j - w] + v);
    return dp;
}
```

11.4 区间 DP

按区间长度递增, 枚举分割点: $dp[l][r] = \min/\max(dp[l][k] + dp[k+1][r] + cost)$ 。典型: 石子合并、矩阵链乘、括号匹配、回文分割。若代价满足四边形不等式/决策单调性, 可用四边形不等式优化, 但先验证条件。

```
// 变量: len=区间 DP 当前枚举的区间长度 len。
for (int len = 2; len <= n; ++len)
    // 变量: l=当前区间左端点 l。
    for (int l = 1; l + len - 1 <= n; ++l) {
        // 变量: r=当前区间右端点 r。
        int r = l + len - 1;
        dp[l][r] = INF;
        // 变量: k=当前循环枚举的整数下标 k。
        for (int k = l; k < r; ++k)
            dp[l][r] = min(dp[l][r], dp[l][k] + dp[k + 1][r] + cost(l, r));
    }
```

11.5 树形 DP 与换根 DP

树上没有环, 后序即可保证子树状态已知。独立集: $f[u][0] = \sum \max(f[v][0], f[v][1])$, $f[u][1] = w[u] + \sum f[v][0]$ 。换根时先从任意根求子树贡献, 再将父亲的答案传给儿子, 常见于“每个点到其他点距离和”。

```
// 接口: reroot1(u, p): 第一次 DFS, 计算每个子树的向下 DP; 不返回值。
// 参数: 当前顶点编号 u; 当前节点 u 的父节点 p。
void reroot1(int u, int p) {
```

```

    sz[u] = 1; down[u] = 0;
    // 变量: v=从 tree[u] 枚举的相邻顶点 v。
    for (int v : tree[u]) if (v != p)
        reroot1(v, u), sz[u] += sz[v], down[u] += down[v] + sz[v];
}
// 接口: reroot2(u, p): 第二次 DFS, 把父侧贡献传给儿子并得到全树答案; 不返回值。
// 参数: 当前顶点编号 u; 当前节点 u 的父节点 p。
void reroot2(int u, int p) {
    // 变量: v=从 tree[u] 枚举的相邻顶点 v。
    for (int v : tree[u]) if (v != p) {
        ans[v] = ans[u] - sz[v] + (n - sz[v]);
        reroot2(v, u);
    }
}

```

11.6 有向无环图动态规划与路径计数 (Directed Acyclic Graph Dynamic Programming)

拓扑序内转移; 最长路用 $-\text{INF}$ 初始化, 最短路用 INF 。若有多个源, 把所有源初始化; 要恢复路径保存 `pre`。在模数下统计方案时不要把不可达状态的 0 与真实方案混淆。

```

// 变量: u=从容器 topo 枚举的元素 u。
for (int u : topo) if (dp[u] != NEG)
    // 变量: v=当前边的终点 v; w=当前边权 w。
    for (auto [v, w] : dag[u]) {
        // 变量: candidate=经 u 到达 v 的候选最长路长度 candidate。
        long long candidate = dp[u] + w;
        if (candidate > dp[v]) {
            dp[v] = candidate;
            ways[v] = ways[u];
        } else if (candidate == dp[v]) {
            ways[v] = (ways[v] + ways[u]) % MOD;
        }
    }
}

```

11.7 状态压缩动态规划

`mask` 表示已选集合, 转移 `mask|(1<<i)`。 $n \leq 20$ 的旅行商、集合覆盖、配对、棋盘放置都适用。优化: 预处理 `popcount`、子集和、可行转移; 枚举子集总复杂度 $O(3^n)$ 。

```

// 变量: dp=dp[mask][u] 表示从顶点 0 出发、恰好访问 mask 中顶点且停在 u 的最短路径代
    价。

```



```

vector<vector<long long>> dp(1 << n, vector<long long>(n, INF));
dp[1][0] = 0;
// 数值域: 边权非负; INF 是不参与转移的上界, 所有需要区分的有限路径代价必须严格小于 INF。
// 变量: mask=用二进制位编码的当前状态 mask。
for (int mask = 1; mask < (1 << n); ++mask)
    // 变量: u=状态压缩 DP 当前路径末端的顶点 u。
    for (int u = 0; u < n; ++u) if (mask >> u & 1)
        // 变量: v=状态压缩 DP 当前尝试加入集合的顶点 v。
        for (int v = 0; v < n; ++v) if (!(mask >> v & 1) && dp[mask][u] < INF)
        {
            // 变量: candidate=宽整数计算的候选路径代价; 达到 INF 后按本接口的上界处理。
            __int128 candidate = (__int128)dp[mask][u] + w[u][v];
            if (candidate < dp[mask | (1 << v)][v])
                dp[mask | (1 << v)][v] = (long long)candidate;
        }

```

11.8 数位动态规划 (Digit Dynamic Programming)

按高位到低位处理, 状态通常为 (pos, tight, started, remainder/计数)。tight 表示前缀是否贴合上界, started 区分前导零。求 [L, R] 用 F(R) - F(L-1)。先写“不限制上界”的转移, 再加入 tight。

```

long long memo[20][2][2][11]; // -1=未计算; 末维 rem 是模 11 余数 [0,10]。
string digits; // 上界的十进制表示; pos 从 0 扫到 digits.size()-1。
// 接口: solve_digit(pos, tight, started, rem): 返回给定数位 DP 状态的合法后缀方案数; 返回类型 long long。
// 参数: 当前处理到的十进制数位下标 pos; 前缀是否仍贴住上界 tight; 是否已经放置非前导零数字 started; 当前余数状态 rem。
long long solve_digit(int pos, int tight, int started, int rem) {
    if (pos == (int)digits.size()) return started && rem == 0;
    // 变量: res=当前 DP 状态对应的记忆化答案引用 res。
    long long &res = memo[pos][tight][started][rem];
    if (!tight && res != -1) return res;
    res = 0;
    // 变量: lim=数位 pos 可枚举数字的上界 lim; tight 为真时取 digits[pos], 否则为 9。
    int lim = tight ? digits[pos] - '0' : 9;
    // 变量: d=数位 DP 当前枚举的十进制数字 d。
    for (int d = 0; d <= lim; ++d)
        res += solve_digit(pos + 1, tight && d == lim, started || d,
                           (rem * 10 + d) % mod);
    return res;
}

```

```
}
```

11.9 概率动态规划与期望动态规划 (Probability and Expected-Value Dynamic Programming)

状态转移可能含当前状态自身：整理成 $E = (1 + \text{other}) / (1 - p_{\text{self}})$ 。马尔可夫链有限状态可高斯消元；若转移无环则按拓扑/记忆化直接递推。概率取模时乘逆元，浮点题要保留足够精度。

```
// 无环状态图上的期望：先按题意保证 next[u] 都比 u 更靠后
// 变量：E=E[u] 表示从无环状态图的状态 u 出发直到终止所需的期望步数。
vector<double> E(n, 0.0);
// 变量：u=按逆拓扑序计算期望的当前状态 u。
for (int u = n - 1; u >= 0; --u) {
    if (transitions[u].empty()) continue; // 终止态没有后续操作，剩余代价为 0。
    E[u] = 1.0;
    // 变量：v=转移到的后继状态编号 v；p=采用该转移的概率 p。
    for (auto [v, p] : transitions[u]) E[u] += p * E[v];
}
```

11.10 记忆化搜索

当状态图是 DAG 或状态数量可控时，用 DFS + cache 比显式排序更自然。必须保证每个状态的所有后继已计算，含环问题不能直接记忆化递归。

```
int memo[MAXN]; // -1=状态尚未计算；否则为 solve(u) 的缓存答案。
// 接口：solve(u)：记忆化返回从 DAG 状态 u 出发可取得的最大后继收益；返回类型 int。
// 参数：顶点编号 u。
int solve(int u) {
    // 变量：res=当前 DP 状态对应的记忆化答案引用 res。
    int &res = memo[u];
    if (res != -1) return res;
    res = 0;
    // 变量：v=从 next[u] 枚举的后继状态 v。
    for (int v : next[u]) res = max(res, solve(v) + gain[u][v]);
    return res;
}
```

11.11 动态规划优化 (Dynamic Programming Optimization)

11.11.1 滚动数组

只依赖上一层时把二维压一维；确认本轮不会读到已更新的状态，遍历方向是关键。

```
// 变量: prev=滚动数组中保存的上一 DP 层 prev; cur=滚动数组中正在计算的当前 DP 层 cur
。
vector<long long> prev(m + 1), cur(m + 1);
// 变量: i=滚动数组中正在计算的当前 DP 层编号 i。
for (int i = 1; i <= n; ++i) {
    cur[0] = 0;
    // 变量: j=滚动数组中正在计算的当前 DP 状态下标 j。
    for (int j = 1; j <= m; ++j)
        cur[j] = transition(prev[j], cur[j - 1], a[i], b[j]);
    swap(prev, cur);
}
```

11.11.2 单调队列优化

形如 $dp[i] = base(i) + \min/\max(dp[j] + g(j))$ 且 j 的合法范围滑动时，维护候选队列。若 g 可分离且窗口固定，复杂度从 $O(nk)$ 降到 $O(n)$ 。

```
// 接口: window_min_dp(base, k): 在 __int128 中精确计算  $dp[i] = base[i] + \min(dp[j])$ 
, 其中  $i - k \leq j < i$ ;  $dp[0] = base[0]$ 。
// 参数: 每个位置的附加代价 base; 正窗口宽度 k。
vector<__int128> window_min_dp(const vector<long long>& base, int k) {
    // 变量: n=DP 状态总数。
    int n = (int)base.size();
    if (n == 0) return {};
    // 变量: dp=窗口最小值优化后的状态数组。
    vector<__int128> dp(n);
    // 变量: q=按 dp 值递增维护的候选下标队列。
    deque<int> q;
    dp[0] = base[0]; q.push_back(0);
    // 变量: i=当前计算的 DP 状态下标。
    for (int i = 1; i < n; ++i) {
        while (!q.empty() && q.front() < i - k) q.pop_front();
        dp[i] = base[i] + dp[q.front()];
        while (!q.empty() && dp[q.back()] >= dp[i]) q.pop_back();
        q.push_back(i);
    }
    return dp;
}
```

11.11.3 斜率优化 (Convex Hull Trick, CHT)

转移能化为 $dp[i] = x_i * k_j + b_j$ ，把每个 j 看作直线，按斜率/查询单调性维护凸包。斜率不单调时用 Li Chao 树；确认除法比较不溢出，必要时用 `__int128`。

```
struct Line {
    long long k, b; // 候选直线  $y=k*x+b$  的斜率与截距。
    // 接口: get(x): 在 __int128 中精确返回该直线在横坐标  $x$  处的函数值  $k*x+b$ 。
    // 参数: 任意 long long 查询横坐标  $x$ 。
    __int128 get(long long x) const { return (__int128)k * x + b; }
};
// 变量: hull=按有效顺序维护的凸包候选队列 hull。
deque<Line> hull;
// 接口: bad(a, b, c): 判断中间直线  $b$  是否因交点顺序而永远不会成为最优; 可删除  $b$  时返回 true。
// 参数: 按斜率顺序相邻的候选直线  $a$ 、 $b$ 、 $c$ ; 要求两个交叉乘积可由 signed __int128 表示, 差值会先提升再相减。
bool bad(Line a, Line b, Line c) {
    return ((__int128)b.b - a.b) * ((__int128)b.k - c.k)
        >= ((__int128)c.b - b.b) * ((__int128)a.k - b.k);
}
// 接口: add_line(x): 按单调斜率加入候选直线, 并从队尾删除永不最优的直线; 会修改当前结构; 多测时重新构造或显式清空成员。
// 参数: 待插入的候选直线  $x$ 。
void add_line(Line x) {
    while (hull.size() >= 2 && bad(hull[hull.size() - 2], hull.back(), x))
        hull.pop_back();
    hull.push_back(x);
}
```

11.11.4 分治优化

若最优决策点单调, $dp[i][j]$ 在分治递归中只枚举决策区间的一部分, 常见 $O(kn \log n)$ 。先证明四边形不等式或决策单调性。

```
// 接口: solve(l, r, optl, opttr): 在决策范围 [optl, opttr] 内计算 DP 下标区间 [l, r] 并递归缩小最优决策范围; 无返回值。
// 参数: 闭区间左端点  $l$ ; 闭区间右端点  $r$ ; 真实最优决策允许范围左端  $optl$ ; 真实最优决策允许范围右端  $opttr$ 。
void solve(int l, int r, int optl, int opttr) {
    if (l > r) return;
    // 变量: mid=当前区间中点 mid; best=目前找到的最优值 best。
    int mid = (l + r) / 2, best = optl;
    // 变量: k=当前 DP 状态枚举的候选决策下标  $k$ 。
    for (int k = optl; k <= min(mid, opttr); ++k)
```

```

    if (cost(k, mid) < cost(best, mid)) best = k;
    dp[mid] = cost(best, mid);
    solve(l, mid - 1, optl, best);
    solve(mid + 1, r, best, opttr);
}

```

11.12 组合游戏动态规划 (Combinatorial Game Dynamic Programming)

无环游戏状态标记胜负：存在一个后继为必败则当前必胜；所有后继必胜则当前必败。将多个独立游戏的 Grundy 值 xor。棋盘/取石子题先找周期或 SG。

// 接口：grundy(x)：返回状态参数对应的 Sprague-Grundy 值并记忆化；返回类型 int。

// 参数：待求 SG 值的游戏状态 x。

```

int grundy(int x) {
    if (memo[x] != -1) return memo[x];
    // 变量：seen=记录当前状态各后继 SG 值是否出现的布尔表 seen。
    bool seen[MAXG] = {};
    // 变量：y=状态 x 可到达的后继状态 y。
    for (int y : moves[x]) seen[grundy(y)] = true;
    // 变量：g=当前有向图的邻接表 g。
    int g = 0; while (seen[g]) ++g;
    return memo[x] = g;
}

```

11.13 常见 DP 误区

- 用“最后一个元素”定义状态，却漏掉了影响未来的最小信息。
- 把排列计数当组合计数，循环顺序写反。
- 区间 DP 长度顺序错误。
- 负权最大化仍用 0 初始化，导致不可达状态污染答案。
- 取模减法忘记加模；概率 DP 把浮点误差当精确相等。

11.14 动态规划进阶：参数、调用与改造

本节紧跟前面的基础模板。先看每个接口传什么，再看哪些不变量不能动，最后看如何替换维护信息或转移。

11.14.1 复杂数位动态规划与自动机数位动态规划

数位 DP 处理 $0 \dots N$ 中满足数字条件的数。状态通常是 $(pos, tight, started, extra)$ ；上下界用 $count(R) - count(L-1)$ 。若有“不能出现某模式/相邻数字限制”，把 $extra$ 换成自动机节点。

```
long long memo[20][2][2][STATE]; // -1=未计算；末维 st 为题目/自动机状态 [0, STATE)
// 接口: dfs(pos, tight, started, st): 返回给定数位 DP 状态的合法后缀方案数；返回类型 long long。
// 参数: 当前处理到的十进制数位下标 pos; 前缀是否仍贴住上界 tight; 是否已经放置非前导零数字 started; 处理当前前缀后所在的数字自动机状态 st, 范围 [0, STATE)。
long long dfs(int pos, bool tight, bool started, int st) {
    // pos: 正在处理第 pos 位; tight: 前缀是否仍等于上界;
    // started: 是否已经出现过非前导零; st: 额外限制的自动机状态。
    if (pos == len) return valid(st, started);
    // 只有 tight=false 的状态可以复用; tight=true 还依赖上界前缀。
    if (!tight && memo[pos][started][0][st] != -1)
        return memo[pos][started][0][st];
    // 变量: res=当前数位 DP 状态累计的合法后缀方案数 res。
    long long res = 0;
    // 变量: up=当前数位 pos 可枚举数字的上界 up; 贴住原数时为 digit[pos], 否则为 9。
    int up = tight ? digit[pos] : 9;
    // 变量: d=数位 DP 当前枚举的十进制数字 d。
    for (int d = 0; d <= up; ++d) {
        // 前导零通常不应触发“数字出现/相邻数字”等条件。
        // 变量: ns=转移后的 started 状态 ns。
        bool ns = started || d != 0;
        // 尚未开始时是否把 0 送入自动机, 必须按题意决定; 这里留在起点 0。
        // 变量: nst=在自动机状态 st 后放置数字 d 得到的下一自动机状态 nst; 仍处于前导零时保持起点 0。
        int nst = ns ? go[st][d] : 0;
        if (ns && nst == forbidden) continue;
        // 注意必须与真实上界位 digit[pos] 比较, 不能写成 d==up。
        res += dfs(pos + 1, tight && d == digit[pos], ns, nst);
    }
    if (!tight) memo[pos][started][0][st] = res;
    return res;
}
```

测试样例：数位 DP 的前导零和上下界差分

这里固定 $valid$ 为“数字十进制表示中不出现数字 4”，并把 0 计入合法数。每组输入 LR ，输出 $[L, R]$ 中合法整数个数。

输入

4
0 0
0 4
0 10
0 99

输出

1
4
10
81

$[0, 4]$ 的合法数是 $0, 1, 2, 3$; $[0, 99]$ 的答案是 $9 \times 9 = 81$ 。如果把前导零当成真实数字 4 的相邻字符，或者把 `tight && d == digit[pos]` 错写成 `d == up`，这些边界会出错。

这里 `up` 只表示当前位允许枚举到哪里；`tight` 更新必须比较真实上界位 `digit[pos]`。练习：P2602 数字计数；若题目问每位出现次数，把返回值从一个数改为“方案数 + 各位贡献”的结构体。

11.14.2 所有子集和动态规划、轮廓线动态规划与状态压缩

SOS DP 用来计算所有子集/超集的和：

```
// 数值域：0 ≤ K < 31，且每个子集总和都必须可由 f 的元素类型表示；否则把 f 改为 __int128。
// 变量：bit=当前处理的二进制位 bit。
for (int bit = 0; bit < K; ++bit)
    // 变量：mask=用二进制位编码的当前状态 mask。
    for (int mask = 0; mask < (1 << K); ++mask)
        if (mask >> bit & 1)
            // 把“去掉 bit 后的子集”贡献给当前 mask。
            // 处理完这一层后，f[mask] 已包含所有只在低 bit 位不同的子集。
            f[mask] += f[mask ^ (1 << bit)];
```

测试样例：SOS 动态规划的子集和方向

下面约定 $K=2$ ，初始 $f[\text{mask}]$ 是恰好等于 mask 的权值，输出所有子集和 $\text{sum}[\text{sub}]$ 。

输入

2
1
5 7
2
1 2 3 4

输出

5 12

1 3 4 10

第二组中 $\text{sum}[3]$ 必须是 $1+2+3+4=10$ ；如果把 $\text{mask}^{(1 \ll \text{bit})}$ 写成加上不存在的超集，或循环顺序和转移方向不匹配，会在 $\text{sum}[1]$ 、 $\text{sum}[2]$ 同时出错。

若网格宽度 $m \leq 10/12$ ，每一列的填法只影响相邻列，维护 $\text{dp}[\text{col}][\text{mask}]$ ；这就是轮廓线 DP。先预处理从 mask 到 next_mask 的合法转移，再做列 DP。棋盘放置题可练 P2704 炮兵阵地。状态中相邻两行都可能影响当前行时，不能只保留一行。

11.14.3 单调队列、斜率优化与李超树优化动态规划

若转移有 $\text{dp}[i] = \min(\text{dp}[j] + w(j, i))$ ，且 j 的优劣具有区间单调性，先尝试单调队列；若整理后是直线最值，才上凸包/李超树。

```
// 变量: q=按未来转移优劣维护候选 DP 下标的单调双端队列 q。
deque<int> q;
q.push_back(0); // 0 是已计算的初始候选; 调用前先设置 dp[0]。
// 变量: i=滑动窗口右端新加入单调队列的数组位置 i。
for (int i = 1; i < n; ++i) {
    // 过期候选先删除; 窗口条件由题目决定。
    while (!q.empty() && q.front() < i - window) q.pop_front();

    // 队首是当前窗口中最优候选, value(j, i) 是题目转移代价。
    dp[i] = value(q.front(), i);

    // 新候选 i 若在未来不可能优于队尾, 就淘汰队尾。
    // next_x 和 value 的单调性必须由题目转移证明, 不能盲套。
    while (!q.empty() && value(i, next_x(q.back()))
        <= value(q.back(), next_x(q.back()))) q.pop_back();
    q.push_back(i);
}
```

```
// 把 dp[i] 整理成“查询某条候选直线在 x[i] 处的最值”时使用。
struct Line {
    long long k, b; // 候选直线  $y=k \star x+b$  的斜率与截距。
    // 接口: value(x): 在 __int128 中精确返回直线  $k \star x+b$  在横坐标 x 处的值。
    // 参数: 任意 long long 查询横坐标 x。
    __int128 value(long long x) const { return (__int128)k * x + b; }
};
```

```
// 只有在斜率按单调顺序加入、查询 x 也按单调顺序时,
// 才能用普通 deque; 任意插入/任意查询请改用前面的李超线段树。
```



```

// 变量: hull=按有效顺序维护的凸包候选队列 hull。
deque<Line> hull;
// 接口: useless(a, b, c): 判断中间候选直线 b 是否可从凸包队列删除; b 永远不会最优时返回 true。
// 参数: 按斜率顺序相邻的候选直线 a、b、c; 要求两个交叉乘积可由 signed __int128 表示, 差值会先提升再相减。
bool useless(const Line& a, const Line& b, const Line& c) {
    // 交点顺序不再递增时, 中间直线永远不会成为最优。
    // 用 __int128 防止交叉相乘时溢出。
    return ((__int128)b.b - a.b) * ((__int128)b.k - c.k)
        >= ((__int128)c.b - b.b) * ((__int128)a.k - b.k);
}

// 接口: add_line(line): 按单调斜率加入候选直线, 并从队尾删除永不最优的直线; 会修改当前结构; 多测时重新构造或显式清空成员。
// 参数: 待插入的候选直线 line。
void add_line(Line line) {
    // 斜率顺序必须与 useless 的不等式方向一致; 换成求最大值时通常要反号。
    while (hull.size() >= 2 &&
        useless(hull[hull.size() - 2], hull.back(), line))
        hull.pop_back();
    hull.push_back(line);
}

```

调用测试: 斜率优化只能在顺序条件成立时使用

对下面三条直线做最小值查询:

加入顺序: $y=0$, $y=x$, $y=2x-1$

查询顺序: $x=0, 1, 3$

应得到: $0, 0, 1$

再把查询顺序改成 $x=3, 0, 1$ 。如果模板仍然使用只支持单调查询的双端队列, 必须重新检查题目是否真的满足单调性; 不能把这组非单调调用当成普通凸包优化的合法输入。

斜率优化的关键不是代码, 而是把转移整理为 $y=kx+b$, 并证明斜率和查询点单调; 若斜率不单调或要在线插入, 直接使用前面的李超线段树。例题: P5785 任务安排; 先把平方项展开, 再判断是否能维护直线。

11.14.4 分治优化、四边形不等式、树形背包与 Steiner 树状压动态规划

调用测试：分治优化的决策范围必须覆盖真实最优点

选一个可以暴力验证的代价： $\text{cost}(j,i)=(\text{prefix}[i]-\text{prefix}[j])^2$ ，对 $\text{prefix}=[0,1,3,6]$ 计算每一层的最优决策，并与 `divide_conquer_dp` 的结果逐项比较。重点不是某个固定答案，而是确认：

测试一： $\text{optL}=0, \text{optR}=i$ ，结果应与完整枚举一致。

测试二：把 optL/optR 缩到不包含真实最优 j 的范围，程序必须被判定为调用错误，不能把错误范围当成算法结论。

没有单调性证明时，不能因为测试一通过就直接把完整枚举范围缩成 $\text{optL}..\text{optR}$ 。

分治优化要求最优决策点满足单调性： $\text{opt}[l][r-1] \leq \text{opt}[l][r]$ 。模板递归 `solve(L,R,optL,optR)`，只在允许区间枚举决策点；没有证明时不能硬套。四边形不等式和决策单调性是证明工具，不是看到区间 DP 就默认成立。

```
// 典型形式: dp[layer][i] = min_j(dp[layer-1][j] + cost(j,i))。
// optL/optR 是当前区间内“最优决策 j”可能出现的范围。
// 接口: divide_conquer_dp(L, R, optL, optR): 计算 DP 下标闭区间 [L,R]，决策只搜索 [optL,optR]，并把结果写入全局 dp；无返回值。
// 参数: 闭区间左端点 L；闭区间右端点 R；真实最优决策允许范围左端 optL；真实最优决策允许范围右端 optR。
void divide_conquer_dp(int L, int R, int optL, int optR) {
    if (L > R) return;
    // 变量: mid=当前区间中点 mid。
    int mid = (L + R) >> 1;
    // 变量: best=目前找到的最优值 best。
    long long best = INF;
    // 变量: best_j=当前 DP 状态取得最优值的决策下标 best_j。
    int best_j = -1;

    // 没有单调性证明时不能把枚举范围缩成 optL..optR;
    // 这份模板的正确性前提就是 opt[mid] 位于该范围内。
    // 变量: j=分治 DP 在允许区间 [optL,optR] 内枚举的候选决策点 j。
    for (int j = optL; j <= min(optR, mid); ++j) {
        // 变量: cand=当前枚举得到的候选答案 cand。
        long long cand = old_dp[j] + cost(j, mid);
        if (cand < best) best = cand, best_j = j;
    }
    new_dp[mid] = best;

    // 由 opt[l] <= opt[mid] <= opt[r]，左右区间可以分别缩小。
    divide_conquer_dp(L, mid - 1, optL, best_j);
    divide_conquer_dp(mid + 1, R, best_j, optR);
}
```

```
}
```

树形背包的状态是“子树里选了 j 个”，合并子树时做背包卷积；先处理小子树，循环边界必须使用当前已合并大小。Steiner 树状压 DP 适用于关键点数 $k \leq 10/15$ 的“连通所有关键点最小代价”：先对每个 **mask** 做子集合并，再以 Dijkstra 在图上松弛。若关键点数超过二十，通常要重新分析题意。

```
// Steiner 树状态: dp[mask][v] = 连接 mask 中关键点, 并以 v 作为当前末端的最小代价。
// 变量: mask=用二进制位编码的当前状态 mask。
for (int mask = 1; mask < (1 << K); ++mask) {
    // 把 mask 拆成两个非空子集, 在同一个 v 汇合。
    // 变量: sub=当前枚举的子掩码 sub。
    for (int sub = (mask - 1) & mask; sub; sub = (sub - 1) & mask) {
        // 变量: other=mask 中除 sub 之外的互补子集 other。
        int other = mask ^ sub;
        if (sub > other) continue; // 对称拆分只做一次, 常数更小。
        // 变量: v=Steiner 树状态压缩 DP 当前汇合的图顶点 v。
        for (int v = 0; v < n; ++v)
            dp[mask][v] = min(dp[mask][v],
                               dp[sub][v] + dp[other][v]);
    }

    // 合并后, 末端 v 还可以沿图上的边移动;
    // 用 Dijkstra 在“固定 mask 的状态层”上继续松弛。
    dijkstra_on_state(mask);
}
```

动态 DP/矩阵线段树在本章 2.13 已给出调用思路；概率 DP 遇到自环时，把方程写成 $f = p \times f + \text{rest}$ ，得到 $f = \text{rest} / (1 - p)$ ；多个状态互相转移才需要高斯消元。不要在有自环时直接按拓扑序 DP。

11.14.5 动态动态规划与矩阵线段树

当树/序列上的一个点被修改后，答案仍然可以由局部状态转移合并，但普通 DP 不能重新算全局，就把每个位置的转移写成矩阵，线段树维护矩阵乘积。

```
struct Mat {
    long long a[2][2]; // a[i][j]: 第 i 个状态对第 j 个状态的线性系数。
};

// 接口: mul(A, B): 返回两个转移矩阵的乘积; 返回类型 Mat。
// 参数: 矩阵乘法的第一个输入矩阵 A; 矩阵乘法的第二个输入矩阵 B。
Mat mul(Mat A, Mat B) {
    Mat C{}; // 必须清零; 矩阵乘法是累加, 不是覆盖。
    // 变量: i=2x2 转移矩阵的行下标 i。
    for (int i = 0; i < 2; ++i)
```

```

// 变量: k=二阶转移矩阵乘法的公共维下标 k。
for (int k = 0; k < 2; ++k)
    // 变量: j=2×2 转移矩阵的列下标 j。
    for (int j = 0; j < 2; ++j)
        // C = A * B: 先应用 B, 再应用 A (按列向量约定)。
        C.a[i][j] = (C.a[i][j] + A.a[i][k] * B.a[k][j]) % MOD;

return C;
}

// 接口: identity(): 返回当前状态维数的单位转移矩阵; 返回类型 Mat。
Mat identity() {
    // 变量: I=当前二维状态空间的单位转移矩阵 I。
    Mat I{};
    I.a[0][0] = I.a[1][1] = 1;
    return I;
}

// 矩阵线段树的核心合并: 左区间的转移先于右区间的转移。
// 如果状态向量按行向量存储, 合并顺序会反过来, 必须从建模时保持一致。
// 接口: merge_segment(left, right): 按从左到右的应用顺序合并两个区间转移矩阵; 返回类型 Mat。
// 参数: 左区间转移矩阵 left; 右区间转移矩阵 right。
Mat merge_segment(Mat left, Mat right) {
    return mul(right, left);
}

```

现场映射方法: 先在纸上写出“左边 DP 状态向右移动一格如何变成新状态”, 把这一步写成矩阵; 叶子存单点矩阵, 线段树父节点存 $\text{left} * \text{right}$ 。矩阵乘法不可交换, 区间方向不能写反。它是 C 级内容, 不建议只看一遍标题就现场发明。

11.14.6 概率动态规划的自环方程

有自环的状态不能按拓扑序处理。例如从状态 i 以概率 p 留在 i , 以概率 q 转到已知状态, 期望满足:

$$E_i = pE_i + qE_j + c \implies E_i = \frac{qE_j + c}{1 - p}.$$

多个状态互相转移时, 把每个状态写成一列:

$$E_i - \sum_j p_{ij} E_j = c_i,$$

然后用高斯消元。 $1-p$ 很小时误差会放大, 题目若要求模意义下概率, 要先确认所有分母可

逆；题目若允许无限期望，要识别并输出题目规定的无穷值。

测试样例：概率自环的除法和无限期望

把“每次转移产生的固定代价”也写进方程，测试不能只测没有自环的普通转移：

调用 1: $E = 0.5 E + 1$

期望: $E = 2$

调用 2: $E = 0.5 E + 0.5 \times 4 + 1$

期望: $E = 6$

调用 3: $E = 1.0 E + 1$

期望: 不存在有限值；不能直接计算 $(1-p)$ 的逆。

调用 4:

$$E1 = 0.5 E1 + 0.5 E2 + 1$$

$$E2 = 0.25 E1 + 0.75 E2 + 2$$

期望: 先移项得到

$$0.5 E1 - 0.5 E2 = 1$$

$$-0.25 E1 + 0.25 E2 = 2,$$

两行线性相关但常数不相容，因此该系统无有限解。

前三组分别抓“漏除以 $1-p$ ”“把常数项放错侧”和“ $p=1$ 除零”。第四组抓高斯消元中的主元选择、无解判断与浮点误差。实际题目若保证期望存在，第四组应改成相容系统；测试代码本身仍应保留无解分支，避免模板在异常数据上产生假答案。

12 博弈专题

本章只放竞赛中最常见的博弈模板。先判断游戏是否为无偏、有限、无环状态；能拆成独立子游戏时用 SG，双方都在完整状态上决策时用 Minimax。所有“必胜/必败”结论都要先确认终止规则和是否允许重复状态。

12.1 Nim、Bash 与反常 Nim 取石子游戏

标准 Nim 每次从一堆中取任意正数个，所有堆异或非零时先手必胜。Bash 游戏每次最多取 k 个，单堆 n 的必败位置是 $n \bmod (k+1) == 0$ 。

```
// 接口: nim_first_win(piles): 返回标准 Nim 当前局面是否先手必胜；全部堆石子数异或非零时返回 true。
// 参数: 各堆石子数 piles。
bool nim_first_win(const vector<int>& piles) {
    int x = 0; // 整个无偏 Nim 游戏的 SG 异或和。
    // 变量: v=从 piles 枚举的单堆石子数 v。
    for (int v : piles) x ^= v;
    return x != 0;
}

// 接口: bash_first_win(n, k): 返回 Bash 取石子局面是否先手必胜；n 不是 k+1 的倍数时返回 true。
// 参数: Bash 游戏当前石子数 n；每步最多取走的石子数 k。
bool bash_first_win(int n, int k) {
    // 一堆石子每次取 1..k；模 k+1 为 0 的位置是必败态。
    return n % (k + 1) != 0;
}
```

反常 Nim（最后取走者输）：若所有堆大小都为 1，先手在堆数为偶数时必胜；否则仍看异或和是否非零。

```
// 接口: misere_nim_first_win(piles): 返回反常 Nim 当前局面是否先手必胜；按全 1 堆特判或普通 Nim 异或条件判胜。
// 参数: 各堆石子数 piles。
bool misere_nim_first_win(const vector<int>& piles) {
    // 只有“所有非空堆都是 1”时，最后一步输的规则会改变公式。
}
```

```

// 变量: all_one=所有堆是否都只有 1 个石子的标记 all_one。
bool all_one = true;
// 变量: x=反常 Nim 全部堆石子数的异或和 x; ones=石子数恰为 1 的堆数量 ones。
int x = 0, ones = 0;
// 变量: v=从 piles 枚举的单堆石子数 v。
for (int v : piles) {
    if (v > 0) all_one &= (v == 1); // 空堆不参与“所有非空堆均为 1”的判断。
    x ^= v;
    ones += (v == 1);
}
if (all_one) return ones % 2 == 0;
return x != 0;
}

```

12.2 通用 Sprague–Grundy 函数、最小未出现值与有向无环图博弈

无偏博弈的 Grundy 值满足 $sg[u] = \text{mex}(\{sg[v]\})$ ；独立子游戏的总值是异或。若状态图是 DAG，可以记忆化 DFS；若只是判断胜负，则“存在必败后继”为必胜，否则为必败。

```

// 接口: mex(values): 返回集合中没有出现的最小非负整数；返回类型 int。
// 参数: 要求 mex 的非负整数集合 values。
int mex(vector<int> values) {
    // 排序去重后，从 0 开始找第一个断点；适合后继数量不太大的单次计算。
    sort(values.begin(), values.end());
    values.erase(unique(values.begin(), values.end()), values.end());
    // 变量: g=当前有向图的邻接表 g。
    int g = 0;
    // 变量: x=参与 mex 计算的候选非负整数 x。
    for (int x : values) {
        if (x == g) ++g;
        else if (x > g) break;
    }
    return g;
}

vector<int> sg;           // sg[u]=状态 u 的 Sprague–Grundy 值。
vector<char> sg_vis;      // sg_vis[u]=1 表示 sg[u] 已完成记忆化计算。
// 接口: grundy(u): 返回状态参数对应的 Sprague–Grundy 值并记忆化；返回类型 int。
// 参数: 顶点编号 u。
int grundy(int u) {
    // DAG 上记忆化求后继 SG；有环时必须换成专门的环博弈分析。
    if (sg_vis[u]) return sg[u];
    sg_vis[u] = true;
}

```

```

// 变量: nxt=当前一步可到达的后继状态 nxt。
vector<int> nxt;
// 变量: v=当前边的终点 v。
for (int v : dag[u]) nxt.push_back(grundy(v));
return sg[u] = mex(nxt);
}

// 接口: first_win(starts): 返回若干独立子游戏 SG 值异或和是否非零; 异或和非零时返回 true。
// 参数: 若干独立子游戏的起始状态 starts。
bool first_win(const vector<int>& starts) {
    // 只有子游戏互不影响时才能异或; 共享资源的游戏不能套这里。
    // 变量: total=starts 中全部独立子游戏 SG 值的累计异或和 total。
    int total = 0;
    // 变量: s=从容器 starts 枚举的元素 s。
    for (int s : starts) total ^= grundy(s);
    return total != 0;
}

vector<int8_t> win; // 0: 未求, 1: 必胜, -1: 必败
// 接口: dag_game(u): 返回 DAG 状态 u 的 SG 值; 返回类型 int。
// 参数: 顶点编号 u。
int dag_game(int u) {
    // 先暂定为必败, 发现一个必败后继就改成必胜。
    if (win[u]) return win[u];
    win[u] = -1;
    // 变量: v=当前边的终点 v。
    for (int v : dag[u])
        if (dag_game(v) == -1) return win[u] = 1;
    return win[u];
}

```

12.3 Nim 变体: 取法集合与 Sprague–Grundy 周期

如果单堆允许取的数量集合固定, 先计算一维 SG; $sg[n]$ 出现周期后可以直接按周期回答超大 n 。多个互相独立的堆仍然异或。

```

// 接口: solve_subtraction(n, moves): 返回单堆剩余 n 个石子时、每步取数来自 moves 的减法游戏 SG 值; 返回类型 int。
// 参数: 待求 SG 值的石子数 n; 每步允许取走的石子数集合 moves。
int solve_subtraction(int n, const vector<int>& moves) {
    // 变量: sg=Sprague–Grundy 值数组 sg。
    vector<int> sg(n + 1);
    // 变量: seen=为当前减法游戏状态标记后继 SG 值的时间戳数组 seen。

```



```

vector<int> seen(moves.size() + 2);
// 变量: x=当前计算 SG 值的石子数 x。
for (int x = 1; x <= n; ++x) {
    // 变量: values=当前需要求 mex 的后继 SG 值集合 values。
    vector<int> values;
    // 变量: take=当前尝试取走的石子数量 take。
    for (int take : moves) if (take <= x)
        values.push_back(sg[x - take]);
    sg[x] = mex(values);
}
return sg[n];
}

```

12.4 极大极小搜索（Minimax）与 Alpha-Beta 剪枝

双方零和、轮流行动且状态树有限时，Minimax 取当前方能保证的最优值。Alpha-Beta 剪枝维护祖先已知的下界 **alpha** 与上界 **beta**；排序优先搜索好着法会显著提高剪枝率。

```

// 接口: minimax(state, depth, alpha, beta, maximizing): 返回当前博弈状态在双方最优
// 策略下的估值; 返回类型 int。
// 参数: 当前博弈状态 state; 剩余搜索深度 depth; 当前已知下界 alpha; 当前已知上界 beta
// ; 当前层是否轮到极大化一方 maximizing。
int minimax(State& state, int depth, int alpha, int beta, bool maximizing) {
    // evaluate 始终站在固定一方视角; max 方取最大, min 方取最小。
    // alpha 是 max 已知下界, beta 是 min 已知上界。
    if (depth == 0 || terminal(state)) return evaluate(state);
    if (maximizing) {
        // 变量: best=目前找到的最优值 best。
        int best = INT_MIN;
        for (Move mv : ordered_moves(state)) {
            // apply/undo 必须成对, 否则下一分支会继承上一分支的局面。
            apply(state, mv);
            best = max(best, minimax(state, depth - 1, alpha, beta, false));
            undo(state, mv);
            alpha = max(alpha, best);
            // beta cutoff: min 方已有更小上界, 后面的分支不会被选择。
            if (alpha >= beta) break; // beta cutoff
        }
        return best;
    }
    // 变量: best=目前找到的最优值 best。
    int best = INT_MAX;
    for (Move mv : ordered_moves(state)) {

```

```

    apply(state, mv);
    best = min(best, minimax(state, depth - 1, alpha, beta, true));
    undo(state, mv);
    beta = min(beta, best);
    // alpha cutoff: max 方已有更大下界，后面的分支不会被选择。
    if (alpha >= beta) break; // alpha cutoff
}
return best;
}

```

实际写棋类题时，`State/Move/apply/undo/terminal/evaluate` 替换为题目状态；有深度限制时应加入深度奖励、置换表和走法排序，避免把“未搜到底”的局面误判成终局。

12.5 博弈题检查清单

- 终止状态是“不能走”输，还是“最后一步”输？这决定普通/反常规则。
- 是否无偏？若双方可用的操作不同，不能直接套 SG xor。
- 状态是否可拆成独立和？有共享资源时不能强行异或。
- 状态图是否有环？有环时需要周期、记忆化博弈或胜负状态方程。
- 题目只问胜负用 Win/Lose，问组合价值或多堆合并才引入 Grundy。

12.6 博弈论：参数、调用与改造

本节紧跟前面的基础模板。先看每个接口传什么，再看哪些不变量不能动，最后看如何替换维护信息或转移。

12.6.1 必胜态/必败态动态规划 (Win/Lose Dynamic Programming)

定义 `win[u]`：轮到当前玩家在状态 `u` 操作时，当前玩家是否能赢。

$$win(u) = \text{true} \iff \exists v \in next(u), win(v) = \text{false}.$$

没有合法操作的状态是必败态。对 DAG 从终点反推，复杂度是 $O(V + E)$ 。

```

// g[u]: 从状态 u 可以一步走到的所有状态。
// 这里假设状态图是 DAG；如果有环，不能直接用这份递归模板。
// 变量: g=当前有向图的邻接表 g。
vector<vector<int>> g;

// win[u] 的三种取值:
// 0 = 还没有计算;

```

```

// 1 = 必胜态：轮到当前玩家时存在一步走到必败态；
// -1 = 必败态：所有合法走法都会把对手送入必胜态。
// 变量：win=状态是否必胜的记忆化数组 win。
vector<int8_t> win;

// 接口：solve_win(u)：返回 DAG 状态 u：1 必胜，-1 必败；返回类型 int。
// 参数：顶点编号 u。
int solve_win(int u) {
    // 记忆化：同一个状态可能从很多地方到达，不能重复搜索。
    if (win[u] != 0) return win[u];

    // 先假设 u 是必败态。
    // 如果后面找到一个必败后继，再把它改成必胜态并立即返回。
    win[u] = -1;
    // 变量：v=当前边的终点 v。
    for (int v : g[u]) {
        // 如果能把对手送到必败态，当前状态就是必胜态。
        if (solve_win(v) == -1)
            return win[u] = 1;
    }

    // 没找到必败后继，说明所有走法都让对手赢；没有走法时也会走到这里。
    return win[u];
}

```

测试样例：必胜态/必败态的多测和状态清空

这里规定每组输入为 nms ，接着输入 m 条有向边 uv ，从状态 s 开始；没有合法操作输出 **Lose**，能走到必败状态输出 **Win**。图保证无环。

输入

```

4
1 0 1
2 1 1 2
4 4 1 1 2 1 3 2 4 3 4
5 5 1 1 2 1 3 2 4 3 4 4 5

```

输出

```

Lose
Win
Lose
Win

```

第一组是终止态，第二组是一步必胜；第三组的两个后继都是必胜态，所以起点必败；第四组

多加一层，检查结果是否按后继层数交替。四组连续运行还能检查 `g`、`win` 是否按当前 `n` 重新初始化。

例题：P2197 [模板] Nim 游戏。标准 Nim 的状态是所有堆大小组成的元组，直接写全状态会爆炸；理论告诉我们它等价于一个数 $a[1]^{\wedge} \dots^{\wedge} a[n]$ 。当异或和为 0 时输出 **No**，否则输出 **Yes**。这道题的价值是练习“先识别状态压缩后的不变量”，不是练习 DFS。

12.6.2 有向无环图上的 Sprague–Grundy 函数与最小未出现值

单个无偏游戏状态的 Grundy 值为：

$$sg(u) = mex\{sg(v) \mid u \rightarrow v\}.$$

`mex` 是没有出现在集合中的最小非负整数。终止状态没有后继，因此 `sg=0`。

```
// 计算 mex (minimum excluded value)：集合中没有出现的最小非负整数。
// seen[x] == tag 表示“本次 mex 计算看到了 x”。
// 每次只递增 tag，就不需要把整个 seen 数组清零，适合大量重复调用。
// 接口：mex_by_stamp(values, seen, tag)：用时间戳数组返回 values 的 mex，避免每次
// 清空 seen；返回类型 int。
// 参数：要求 mex 的非负整数集合 values；mex 时间戳数组 seen；当前 mex 时间戳引用 tag
// 。
int mex_by_stamp(const vector<int>& values, vector<int>& seen, int& tag) {
    ++tag; // 开启一次全新的标记批次。
    // 变量：x=参与 mex 计算的候选非负整数 x。
    for (int x : values) {
        // SG 值一定非负；超出 seen 范围的值不会影响较小 mex。
        if (0 <= x && x < (int)seen.size()) seen[x] = tag;
    }

    // 变量：g=当前有向图的邻接表 g。
    int g = 0;
    // 从 0 开始找第一个没有被本批次标记的数。
    while (g < (int)seen.size() && seen[g] == tag) ++g;
    return g;
}

// dag[u]：有向无环图中从 u 出发的一步转移。
// 变量：dag=DAG 状态转移邻接表 dag。
vector<vector<int>>> dag;
vector<int> sg; // sg[u]：状态 u 的 Sprague–Grundy 值。
vector<char> vis; // 是否已经完成记忆化搜索。
vector<int> mark; // mex 的时间戳数组，大小要覆盖可能的 SG 值。
int mark_tag = 0; // 当前 mex 查询的时间戳；每次查询前自增。
```

```
// 接口: grundy(u): 返回状态参数对应的 Sprague-Grundy 值并记忆化; 返回类型 int。
// 参数: 顶点编号 u。
int grundy(int u) {
    // DAG 中递归一定会走向更靠后的状态; vis 防止重复计算。
    if (vis[u]) return sg[u];
    vis[u] = 1;

    // 变量: values=当前需要求 mex 的后继 SG 值集合 values。
    vector<int> values;
    // 变量: v=当前边的终点 v。
    for (int v : dag[u]) {
        // 先递归求所有后继状态的 SG 值。
        values.push_back(grundy(v));
    }

    // 当前状态的 SG 值只由“所有一步可达状态的 SG 值”决定。
    return sg[u] = mex_by_stamp(values, mark, mark_tag);
}
```

在实际题目中, **mark** 至少开到“可能出现的最大 **SG** 值 + 1”。如果不确定, 简单 **sort+unique** 的 **mex** 更安全。状态数很大时, 不能把整个状态对象直接当数组下标, 要用哈希记忆化。

小例子: 若 0 无后继, 1→0, 2→0,1, 则 **sg**[0]=0, **sg**[1]=**mex**{0}=1, **sg**[2]=**mex**{0,1}=2。两个独立游戏的值为 **sg**[x]^**sg**[y], 不为 0 就是先手胜。

测试样例: mex 的空集合、单后继和时间戳复用

这里规定每组输入为 **Nkmoves...**, 输出 **sg**[**N**]。四组连续执行, 专门检查 **seen** 的时间戳是否污染下一组。

输入

```
4
0 1 1
1 1 1
4 2 1 2
10 3 1 3 4
```

输出

```
0
1
1
1
```

N=0 必须直接得到 0; 取法 {1,2} 时序列从 0,1,2,0,1 开始, **sg**[4]=1。最后一组用于检查不能把上一组的 **seen** 标记当成当前组的结果。

12.6.3 多个独立子游戏为什么异或

若一次操作只能改变一个子游戏，而且各子游戏互不共享资源，那么整个局面是游戏和：

$$SG(G_1 + G_2 + \cdots + G_k) = SG(G_1) \oplus SG(G_2) \oplus \cdots \oplus SG(G_k).$$

“一堆石子”就是一个子游戏；“棋盘上多个互不相连、棋子不能跨组件移动的区域”也常常可以拆成多个子游戏。反例是：一次操作必须同时选择两堆，或某个操作会改变另一堆的合法操作，此时不能异或。

12.6.4 普通 Nim：不只判断，还要恢复必胜操作

令 $all = a[0] \oplus \cdots \oplus a[n-1]$ 。若 $all \neq 0$ ，找到最高位 b ，它在 all 中为 1。必然存在某个堆 $a[i]$ 的第 b 位为 1。令

$$target = a[i] \oplus all,$$

则 $target < a[i]$ ，把这堆从 $a[i]$ 减到 $target$ ，新异或和就为 0。

```
// 返回要操作的堆下标和操作后的大小；无必胜操作返回 {-1,-1}
// 接口: nim_move(a): 返回一个必胜 Nim 操作 (堆下标, 操作后石子数), 无则 (-1,0); 返回
// 类型 pair<int, unsigned long long>。
// 参数: 标准 Nim 各堆石子数 a。
pair<int, unsigned long long>
nim_move(const vector<unsigned long long>& a) {
    // all 是整个 Nim 局面的 SG 值，也就是所有堆大小的异或和。
    // 变量: all=标准 Nim 当前局面所有堆石子数的异或和 all。
    unsigned long long all = 0;
    // 变量: x=从 a 枚举的元素值 x。
    for (auto x : a) all ^= x;

    // 异或和为 0 的局面是必败态，不存在一步必胜操作。
    if (all == 0) return {-1, 0};

    // 变量: i=Nim 当前尝试执行必胜移动的石子堆下标 i。
    for (int i = 0; i < (int)a.size(); ++i) {
        // 只改变第 i 堆后，希望新异或和变成 0，目标大小必须是 a[i]^all。
        // 变量: target=使操作后 Nim 异或和为 0 时第 i 堆应剩余的石子数 target。
        unsigned long long target = a[i] ^ all;

        // 合法操作要求只能减少石子；这个判断同时保证目标确实更小。
        if (target < a[i]) return {i, target};
    }
}
```

```
return {-1, 0}; // 理论上不会到这里
}
```

演示：[3,4,5] 的异或为 2。对 3： $3^2=1<3$ ，所以把第一堆从 3 变成 1；新局面 [1,4,5] 的异或为 0。如果题目要求输出“取走多少”，输出 $a[i]-target$ ，不要输出 $target$ 。

测试样例：普通 Nim 必胜操作恢复

下面约定输出为“从 1 开始编号的堆下标、修改后的堆大小”；没有必胜操作输出 00。

输入

```
4
1
0
2
1 1
3
3 4 5
4
1 2 4 8
```

输出

```
0 0
0 0
1 1
4 7
```

前两组是异或和为 0 的必败态；第三组检查示例中的 $3 \rightarrow 1$ ；第四组检查最高位选择，不能只输出目标值而不输出堆下标。

12.6.5 Bash、反常 Nim 与阶梯 Nim

Bash 游戏。单堆有 n 个石子，每次取 $1..k$ 个，必败态是 $n \bmod (k+1) == 0$ 。原因是长度为 $k+1$ 的每一轮，后手可以补到 $k+1$ 。

```
// 接口: bash_win(n, k): 判断每次可取 1..k 个时 n 个石子的局面是否先手胜; n 不是 k+1
// 的倍数时返回 true。
// 参数: Bash 游戏当前石子数 n; 每步最多取走的石子数 k。
bool bash_win(long long n, long long k) {
    return n % (k + 1) != 0;
}
```

反常 Nim。最后一步的人输。若所有非空堆都为 1，轮流拿走一堆，堆数为偶数时先手胜；只要存在大于 1 的堆，结论仍然看普通 Nim 的异或和。

```
// 接口: misere_nim_win(a): 判断反常 Nim (取走最后石子者输) 是否先手胜; 全为 1 时堆数为
// 偶数, 否则总异或非 0 时返回 true。
// 参数: 反常 Nim 各堆石子数 a。
bool misere_nim_win(const vector<int>& a) {
    // 反常 Nim 只有在“所有非空堆大小都是 1”时与普通 Nim 不同。
    // 变量: all_one=所有堆是否都只有 1 个石子的标记 all_one。
    bool all_one = true;
    int x = 0;          // 普通 Nim 的异或和。
    int cnt = 0;        // 非空堆数量。
    // 变量: v=从 a 枚举的元素值 v。
    for (int v : a) {
        if (v != 1) all_one = false;
        if (v) ++cnt;
        x ^= v;
    }

    // 每次只能拿走一整堆 1, 最后拿的人输:
    // 非空堆为偶数时, 先手可以把奇数局面留给后手。
    if (all_one) return cnt % 2 == 0;

    // 只要有一堆大于 1, 就回到普通 Nim 的异或判定。
    return x != 0;
}
```

阶梯 Nim 的约定。位置 0 是地面, 石子在 $i \geq 1$ 的台阶上; 每次从台阶 i 向 $i-1$ 移任意正数, 移到地面的石子不再参与游戏。此时只异或奇数编号台阶:

```
// 接口: staircase_nim_win(a): 判断阶梯 Nim 局面是否先手胜; 奇数层石子数的异或和非 0
// 时返回 true。
// 参数: 阶梯 Nim 每层石子数 a。
bool staircase_nim_win(const vector<long long>& a) {
    long long x = 0; // 阶梯 Nim 化成的等价 Nim 异或和。
    // 按“0 是地面、石子从 i 移到 i-1”的约定, 只异或奇数层。
    // 变量: i=Nim 当前尝试执行必胜移动的石子堆下标 i。
    for (int i = 1; i < (int)a.size(); i += 2) x ^= a[i];
    return x != 0;
}
```

演示 $a[0..5]=[0,3,1,4,2,5]$: 奇数台阶为 3,4,5, 异或是 2。按普通 Nim 规则把台阶 1 的 3 变为 1, 即移 2 个到地面, 奇数台阶异或变为 0。题目若把“底层”编号成 1, 先把下标约定翻译清楚再套公式; 阶梯 Nim 最常见的错就是奇偶方向弄反。

测试样例：Bash、反常 Nim、阶梯 Nim 的分界

三段函数分别测试。输出 **Win/Lose**，不能把三种游戏共用同一个公式。

Bash 输入

```
4
0 3
1 1
4 3
5 3
```

Bash 输出

```
Lose
Win
Lose
Win
```

反常 **Nim** 输入

```
4
1 1
2 1 1
1 2
2 2 2
```

反常 **Nim** 输出

```
Lose
Win
Win
Lose
```

阶梯 **Nim** 输入

```
4
2 0 0
2 0 1
4 0 2 1 2
5 0 3 0 4 1
```

阶梯 **Nim** 输出

```
Lose
Win
Lose
Win
```

反常 Nim 的前两组专门检查“所有非空堆都是 1”的特殊分支；阶梯 Nim 的第三组两个奇数层异或为零，能抓住奇偶层写反的问题。

12.6.6 Sprague–Grundy 函数打表、周期与大范围取石子

固定取法集合 `moves` 时，先求：

```
// 接口: subtraction_sg(N, moves): 返回减法游戏 0..N 的 SG 数组; 返回类型 vector<int>。
// 参数: 要预处理的单堆石子数 N; 每步允许取走的石子数集合 moves。
vector<int> subtraction_sg(int N, const vector<int>& moves) {
    // sg[x]: 还剩 x 个石子时的 SG 值; sg[0] = 0 (不能操作)。
    // 变量: sg=Sprague–Grundy 值数组 sg。
    vector<int> sg(N + 1);
    // 一次状态最多有 moves.size() 个后继, 所以 mex 不会大于 moves.size()。
    // 变量: seen=为当前减法游戏状态标记后继 SG 值的时间戳数组 seen。
    vector<int> seen(moves.size() + 5);
    // 变量: x=当前计算 SG 值的石子数 x。
    for (int x = 1; x <= N; ++x) {
        // 用 x 作为时间戳: 本轮标记只写 seen, 不需要清空数组。
        // 变量: tag=复用 seen 数组时标记当前石子数 x 这一轮的时间戳 tag。
        int tag = x;
        // 变量: take=当前尝试取走的石子数量 take。
        for (int take : moves) {
            // 只有拿得走时, x-take 才是合法后继。
            if (take <= x && sg[x - take] < (int)seen.size())
                seen[sg[x - take]] = tag;
        }

        // 变量: g=当前状态 x 的后继 SG 集合中最小未出现的非负整数 g。
        int g = 0;
        while (g < (int)seen.size() && seen[g] == tag) ++g;
        sg[x] = g;
    }
    return sg;
}
```

发现周期不能只看“最后几个数像了”。可靠流程是：先打到足够大 N ，枚举候选周期 p ，检查一段连续窗口是否满足 $sg[i] == sg[i-p]$ ，再用更大的窗口或数学证明确认。模板如下：

```
// 接口: find_period(sg, min_start, min_len): 在 SG 序列中寻找可验证周期并返回周期长度; 找不到时返回 -1。
// 参数: 已计算的 SG 数列 sg; 允许作为周期起点的最小下标 min_start; 除两段完整周期外还要验证的最少位置数 min_len。
int find_period(const vector<int>& sg, int min_start = 0,
               int min_len = 10) {
    // 变量: n=容器 sg 的元素数量 n。
    int n = (int)sg.size();
```

```

// p 是候选周期。至少需要两段长度为 p 的数据进行比较。
// 变量: p=当前验证的候选周期长度 p。
for (int p = 1; p * 2 <= n; ++p) {
    // st 是周期开始位置: 不要默认周期从 0 开始, 很多题有前导非周期段。
    // 变量: st=当前候选周期开始的 0-based 位置 st。
    for (int st = min_start; st + 2 * p + min_len <= n; ++st) {
        // 变量: ok=当前条件是否仍成立 ok。
        bool ok = true;

        // 检查从 st+p 开始的所有位置是否与 p 前的位置相同。
        // 变量: i=SG 周期验证中与位置 st 对应的后续位置 i。
        for (int i = st + p; i < n; ++i)
            if (sg[i] != sg[i - p]) { ok = false; break; }
        if (ok) return p;
    }
}
return -1;
}

```

若题目给 $n \leq 10^{18}$, 打表只是猜周期; 必须确认周期确实适用于题目的完整取法, 并把前导非周期段单独处理: $n < \text{start}$ 查表, 否则查 $\text{start} + (n - \text{start}) \% \text{period}$ 。

测试样例: 周期不能只看一小段

取法集合为 $\{1, 2\}$ 时, sg 从 0 开始按 0, 1, 2 周期重复。先打表到 $N=20$, 再调用 `find_period`, 应得到周期 3。

调用输入

$N = 20, \text{moves} = \{1, 2\}$

应得到

$sg[0..8] = 0 \ 1 \ 2 \ 0 \ 1 \ 2 \ 0 \ 1 \ 2$

`find_period(sg) = 3`

如果只比较最后几个值, 容易把前导段误当周期; 如果 `find_period` 返回 1 或 2, 优先检查检查窗口和周期起点。

12.6.7 图游戏建模、状压博弈和记忆化

“棋子在图上移动, 不能走到某些点”通常是图游戏。若图无环, 节点本身就是状态; 若有多个棋子, 只有当棋子之间完全不影响时才拆成 SG 异或。

当 $n \leq 20$, 状态压缩通常是 `mask`: 位为 1 表示未用/仍在场, 转移枚举一个合法位置并清位。

// 变量: n =当前测试用例读入的元素个数 n 。

```

int n;
// 变量: memo=记忆化缓存 memo; 哨兵值表示尚未计算。
unordered_map<int, char> memo;

// 接口: dfs_mask(mask): 返回状压状态 mask 是否为先手必胜态并记忆化; 存在一个必败后继时
// 返回 true。
// 参数: 当前状压状态 mask。
bool dfs_mask(int mask) {
    // mask 的每一位代表一个“仍然可用”的对象。
    // 如果状态还有 last、资源数等信息, 必须一并编码进 key。
    // 变量: it=容器 memo 中当前查找结果的迭代器 it。
    auto it = memo.find(mask);
    if (it != memo.end()) return it->second;

    // 变量: i=状压博弈状态 mask 中当前尝试选择的对象位 i。
    for (int i = 0; i < n; ++i) {
        // 第 i 位为 0, 说明对象 i 已经被使用, 不能再次选择。
        if (!(mask >> i & 1)) continue;

        // legal 由题目定义, 例如不能与上一次选择相邻、不能违反图边限制等。
        if (!legal(mask, i)) continue;

        // 只要存在一个后继是必败态, 当前状态就是必胜态。
        if (!dfs_mask(mask ^ (1 << i)))
            return memo[mask] = 1;
    }

    // 没有任何一步能把对手送入必败态。
    return memo[mask] = 0;
}

```

如果状态中还有 `last`、资源数 `r` 或轮到谁, 就把它们一起放进 `key`, 例如 `key=(mask<<5)|last`; 不能只哈希 `mask` 而丢掉会影响后续转移的信息。状态压缩博弈的练习可从洛谷博弈题单里的 Nim 变体开始, 再做带图/带限制的题。

调用测试: 先固定 `legal`, 再测试记忆化

原代码中的 `legal(mask,i)` 是题目相关接口, 不能凭空编一个题目输入。这里先用最小调用测试: 规定任何仍在 `mask` 中的元素都合法, 每次只能拿走一个元素; 因此剩余元素个数为奇数时先手胜。

```

调用输入 (n=3)
mask = 0, 1, 3, 7

```

应得到

Lose

Win

Lose

Win

四次调用之间必须清空 `memo` 或重新建立测试状态。正式题目若还有 `last`、资源数或当前玩家，必须把它们加入状态后再重新设计样例；只测试 `mask` 不能证明完整状态压缩正确。

12.6.8 极大极小搜索（Minimax）与 Alpha-Beta 剪枝

Minimax 适用于：双方目标相反、操作可能不同、不能拆成独立子游戏，且状态树能在给定深度内搜索。`evaluate` 必须站在固定一方视角，例如“对先手越大越好”；轮到另一方时取 `min`。

```
// 接口: alpha_beta(s, depth, alpha, beta, maxing): 返回带 alpha-beta 剪枝的极大极
// 小估值; 返回类型 int。
// 参数: 当前搜索状态 s; 剩余搜索深度 depth; 当前已知下界 alpha; 当前已知上界 beta; 当前
// 层是否轮到极大化一方 maxing。
int alpha_beta(State& s, int depth, int alpha, int beta, bool maxing) {
    // State: 完整局面; depth: 还允许搜索多少层;
    // alpha: 极大方当前至少能保证的分数;
    // beta: 极小方当前至多允许极大方得到的分数。
    // evaluate 必须始终从同一方（例如先手）视角返回分数。
    if (depth == 0 || terminal(s)) return evaluate(s);

    // 好着法优先搜索会更早提高 alpha / 降低 beta, 剪枝效果更好。
    auto moves = ordered_moves(s); // 吃子、将军等好着法先搜
    if (maxing) {
        // 变量: value=极大化层当前已搜索着法中的最大局面估值 value。
        int value = INT_MIN;
        for (Move mv : moves) {
            // apply 和 undo 必须严格成对; 否则后续分支会读到被污染的棋盘。
            apply(s, mv);
            value = max(value, alpha_beta(s, depth - 1,
                                         alpha, beta, false));
            undo(s, mv);

            // 极大方已经知道至少能得到 value, 因此 alpha 可以上调。
            alpha = max(alpha, value);

            // beta <= alpha 时, 极小方不会允许进入这个分支, 后面的着法无需搜索。
            if (alpha >= beta) break;
        }
    }
    return value;
}
```

```

    }
    // 变量: value=极小化层当前已搜索着法中的最小局面估值 value。
    int value = INT_MAX;
    for (Move mv : moves) {
        apply(s, mv);
        value = min(value, alpha_beta(s, depth - 1,
                                     alpha, beta, true));
        undo(s, mv);

        // 极小方希望分数更小, 因此 beta 只能向下收紧。
        beta = min(beta, value);
        if (alpha >= beta) break;
    }
    return value;
}

```

例题式映射: 井字棋中 **State** 是 3x3 棋盘加当前玩家, **Move** 是空格编号, **apply/undo** 落子/撤销, **terminal** 检查三连或棋盘已满, **evaluate** 对胜负返回 +100/-100/0。如果题目要求输出具体第一步, 外层枚举每个 **Move**, 对每个子局面调用 Alpha-Beta, 保存得分最高的那一步。深度搜索不到终局时, 必须用启发式评估, 并给将要赢的状态加深度奖励, 否则可能“宁愿晚赢”。

调用测试说明: Alpha-Beta 代码中的 **State**、**Move**、**apply**、**undo**、**terminal** 和 **evaluate** 都没有固定定义, 不能伪造统一的输入输出。实际接入井字棋时, 至少手测三个局面: 当前已经三连的终止局面、一步可以获胜的局面、必须先堵住对手的局面; 并逐一检查每个分支的 **apply/undo** 后棋盘是否恢复。

12.6.9 状态去重、哈希与有环博弈

棋类搜索中同一局面可能由不同顺序到达。可以把规范化后的状态放入置换表:

```

struct StateHash {
    // 接口: operator(): 返回键值的 size_t 哈希值, 供 unordered_map/unordered_set
    // 去重; 返回类型 size_t。
    size_t operator()(const State& s) const {
        // 变量: h=状态哈希的累计值 h。
        size_t h = 0;
        // 变量: x=从 s.board 枚举的元素值 x。
        for (int x : s.board)
            h = h * 1000003ULL + (unsigned)x + 1;
        h = h * 2 + s.turn;
        return h;
    }
};

```

```
// 变量: table=状态到记忆化答案的哈希表 table。  
unordered_map<State, int, StateHash> table;
```

若图上有环，普通 DFS 的“访问过就返回”是不对的，因为“正在递归路径上”和“已经确定为必胜/必败”不是一回事。无平局、且题目保证最终会结束时可用三色 DFS；允许双方无限循环时，要单独定义平局的收益，不能强行套 Win/Lose。

调用测试说明：**StateHash** 的样例必须和具体 **State** 一起写。至少构造两个“棋盘内容相同但轮到不同玩家”的状态，确认哈希表键不同；再用两个不同走法到达的同一规范局面，确认查表命中同一个状态。不能只打印哈希值判断正确，因为哈希相等不是状态相等。

13 计算几何与计算技巧

13.1 浮点规范

整数坐标优先用 `longlong` 叉积,乘法可能到 `1e18` 时用 `__int128`。实数几何统一 `longdouble`, 比较用 `sgn(x)=(x>eps)-(x<-eps)`, 不要直接 `==`。

```
using Real = long double;
// 变量: EPS=浮点比较容差, 绝对值不超过 EPS 视为 0。
const Real EPS = 1e-12L;
struct Point {
    Real x, y; // 二维点/向量坐标; 所有浮点比较统一经过 sgn/EPS。
    // 接口: operator+(b): 返回两个点/向量的坐标和; 返回类型 Point。
    // 参数: 加到当前点或向量上的向量 b。
    Point operator+(Point b) const { return {x + b.x, y + b.y}; }
    // 接口: operator-(b): 返回两个点/向量的坐标差; 返回类型 Point。
    // 参数: 从当前点或向量中减去的向量 b。
    Point operator-(Point b) const { return {x - b.x, y - b.y}; }
    // 接口: operator*(k): 返回当前二维点向量乘实数系数 k 的数乘结果; 返回类型 Point。
    // 参数: 点向量数乘的实数系数 k。
    Point operator*(Real k) const { return {x * k, y * k}; }
};
// 接口: cross(a, b): 返回二维向量 a,b 的叉积; 返回类型 Real。
// 参数: 叉积左向量 a; 叉积右向量 b。
Real cross(Point a, Point b) { return a.x * b.y - a.y * b.x; }
// 接口: dot(a, b): 返回二维向量 a,b 的点积; 返回类型 Real。
// 参数: 点积左向量 a; 点积右向量 b。
Real dot(Point a, Point b) { return a.x * b.x + a.y * b.y; }
// 接口: sgn(x): 按 EPS 返回 x 的符号: 正为 1, 负为 -1, 近零为 0; 返回类型 int。
// 参数: 待按 EPS 判断符号的实数 x。
int sgn(Real x) { return (x > EPS) - (x < -EPS); }
```


13.2 向量与叉积

$\text{cross}(a,b)=a.x \times b.y - a.y \times b.x$: 正为逆时针, 负为顺时针, 0 共线。点 p 在有向线段 ab 左侧当且仅当 $\text{cross}(b-a, p-a) > 0$ 。

点积判断投影/夹角: $\text{dot}(a,b) > 0$ 锐角, < 0 钝角; 距离平方尽量不开放平方根。

```
// 接口: dist2(a, b): 返回点 a,b 的欧氏距离平方; 返回类型 Real。
// 参数: 距离平方的第一个端点 a; 距离平方的第二个端点 b。
// 变量: d=从点 b 指向点 a 的差向量 d。
Real dist2(Point a, Point b) { Point d = a - b; return dot(d, d); }
// 接口: left_of(a, b, p): 判断 p 是否严格位于有向直线 a->b 左侧; 叉积 cross(b-a, p-a) > EPS 时返回 true。
// 参数: 有向直线起点 a; 有向直线终点 b; 待判断相对有向直线位置的点 p。
bool left_of(Point a, Point b, Point p) { return sgn(cross(b - a, p - a)) > 0; }
```

13.3 直线、线段与交点

线段相交: 先做包围盒快速排除, 再判断四个方向叉积; 共线时检查投影是否重叠。求交点用参数方程: $a+t(b-a)$, 分母是两方向向量叉积; 平行/重合要单独处理。

点到直线距离: $\text{abs}(\text{cross}(b-a, p-a))/|b-a|$; 点到线段要先看投影是否落在段内, 否则取端点距离。

```
// 接口: on_segment(a, b, p): 判断点 p 是否在线段 ab 上 (含端点); p 与 ab 共线且位于两端点之间时返回 true。
// 参数: 闭线段第一个端点 a; 闭线段第二个端点 b; 待判断是否在线段 ab 上的点 p。
bool on_segment(Point a, Point b, Point p) {
    return sgn(cross(b - a, p - a)) == 0 && sgn(dot(p - a, p - b)) <= 0;
}
// 接口: intersect(a, b, c, d): 判断闭线段 ab 与 cd 是否相交 (含端点); 两闭线段存在公共点时返回 true。
// 参数: 第一条闭线段的起点 a; 第一条闭线段的终点 b; 第二条闭线段的起点 c; 第二条闭线段的终点 d。
bool intersect(Point a, Point b, Point c, Point d) {
    // 变量: c1=点 c 相对有向线段 a->b 的方向叉积 c1; c2=点 d 相对有向线段 a->b 的方向叉积 c2。
    Real c1 = cross(b - a, c - a), c2 = cross(b - a, d - a);
    // 变量: c3=点 a 相对有向线段 c->d 的方向叉积 c3; c4=点 b 相对有向线段 c->d 的方向叉积 c4。
    Real c3 = cross(d - c, a - c), c4 = cross(d - c, b - c);
    if (sgn(c1) == 0 && on_segment(a, b, c)) return true;
    if (sgn(c2) == 0 && on_segment(a, b, d)) return true;
    return sgn(c1) * sgn(c2) < 0 && sgn(c3) * sgn(c4) < 0;
}
```

```
}
```

13.3.1 圆几何

直线与圆交点先投影到直线，再根据圆心到直线的距离判断交点数；两圆交点用圆心连线方向分解。判别相切时仍使用 EPS。

```
struct Circle { Point o; Real r; }; // o=圆心, r=非负半径。
// 接口: line_circle(a, b, c): 返回直线与圆的全部交点 (0/1/2 个); 返回类型 vector<
    Point>。
// 参数: 直线上的第一个点 a; 直线上的第二个点 b; 待求交的圆 c。
vector<Point> line_circle(Point a, Point b, Circle c) {
    // 变量: d=从点 a 指向点 b 的方向向量 d。
    Point d = b - a;
    // 变量: dd=直线方向向量 d 的长度平方 dd; t=圆心 c.o 在线段 a-b 所在直线上的投影参
    数 t。
    Real dd = dot(d, d), t = dot(c.o - a, d) / dd;
    // 变量: foot=点在直线上的垂足 foot。
    Point foot = a + d * t;
    // 变量: h2=圆交计算中从圆心连线到交点的垂直距离平方 h2。
    Real h2 = c.r * c.r - dist2(foot, c.o);
    if (h2 < -EPS) return {};
    if (h2 <= EPS) return {foot};
    // 变量: unit=单位方向向量 unit。
    Point unit = d * (sqrtl(h2 / dd));
    return {foot - unit, foot + unit};
}

// 接口: circle_circle(a, b): 返回两圆全部交点 (0/1/2 个; 重合圆需单独处理); 返回类型
    vector<Point>。
// 参数: 第一个输入圆 a; 第二个输入圆 b。
vector<Point> circle_circle(Circle a, Circle b) {
    // 变量: d=从圆 a 圆心指向圆 b 圆心的向量 d; dd=两圆圆心距离的平方 dd; dis=两圆圆心
    之间的实际距离 dis。
    Point d = b.o - a.o; Real dd = dot(d, d), dis = sqrtl(dd);
    if (dis < EPS || dis > a.r + b.r + EPS || dis < fabsl(a.r - b.r) - EPS)
    return {};
    // 变量: x=两圆公共弦中点沿圆心连线离圆 a 圆心的距离 x。
    Real x = (a.r * a.r - b.r * b.r + dd) / (2 * dis);
    // 变量: h2=圆交计算中从圆心连线到交点的垂直距离平方 h2。
    Real h2 = max((Real)0, a.r * a.r - x * x);
    // 变量: base=两圆公共弦中点 base。
    Point base = a.o + d * (x / dis);
    // 变量: perp=与当前方向垂直的单位向量 perp。
```

```

    Point perp{-d.y / dis, d.x / dis};
    if (h2 <= EPS) return {base};
    // 变量: off=从中点到交点的垂直偏移 off。
    Point off = perp * sqrtl(h2);
    return {base - off, base + off};
}

```

13.4 多边形

- 有向面积: $\Sigma \text{cross}(p[i], p[(i+1)\%n])/2$ 。
- 点在多边形: 射线法 (奇偶交点) 或绕数法; 边界先单独判定。
- 凸多边形内点可二分扇区, 查询 $O(\log n)$; 一般多边形射线法 $O(n)$ 。
- Pick 定理: 格点多边形 $A=I+B/2-1$, 其中 $B=\Sigma \text{gcd}(|dx|, |dy|)$ 。

```

// 接口: area2(p): 返回多边形有向面积的两倍; 返回类型 Real。
// 参数: 按边界顺序给出的多边形顶点 p。
Real area2(const vector<Point>& p) {
    // 变量: s=多边形有向面积两倍的叉积累加值 s。
    Real s = 0;
    // 变量: i=多边形面积叉积和中当前有向边起点的顶点下标 i。
    for (int i = 0; i < (int)p.size(); ++i)
        s += cross(p[i], p[(i + 1) % p.size()]);
    return s;
}

```

13.5 凸包 (Andrew)

点按 (x, y) 排序去重; 维护下凸/上凸, 若 $\text{cross} \leq 0$ 弹出可得到不保留共线点的凸包, 若要保留边界改为 < 0 。复杂度 $O(n \log n)$ 。周长、面积、最远点对都以凸包为输入。

```

// 接口: convex_hull(p): 返回去重后的凸包顶点 (逆时针, 不重复首点); 返回类型 vector<Point>。
// 参数: 输入点 p。
vector<Point> convex_hull(vector<Point> p) {
    // 接口: 比较器 lambda(a, b): 按 x 升序、再按 y 升序排列输入点。
    // 参数: 第一个候选点 a; 第二个候选点 b。
    sort(p.begin(), p.end(), [](Point a, Point b) {
        return a.x < b.x || (a.x == b.x && a.y < b.y);
    });
    // 接口: 比较器 lambda(a, b): 判断两个点 a、b 的坐标是否完全相同, 供凸包输入去重。
    // 参数: 第一个候选点 a; 第二个候选点 b。
    p.erase(unique(p.begin(), p.end(), [](Point a, Point b) {

```

```

        return a.x == b.x && a.y == b.y;
    }}, p.end());
// 变量: h=Andrew 算法逐步维护的凸包顶点栈 h。
vector<Point> h;
for (Point x : p) {
    while (h.size() >= 2 && cross(h.back() - h[h.size() - 2], x - h.back())
    ) <= 0) h.pop_back();
    h.push_back(x);
}
// 变量: lower=Andrew 算法按横坐标递增构造的凸包下链顶点序列 lower。
int lower = h.size();
// 变量: i=Andrew 算法自右向左构造凸包上链的点下标 i。
for (int i = (int)p.size() - 2; i >= 0; --i) {
    while ((int)h.size() > lower && cross(h.back() - h[h.size() - 2], p[i]
    - h.back()) <= 0) h.pop_back();
    h.push_back(p[i]);
}
if (h.size() > 1) h.pop_back();
return h;
}

```

13.6 旋转卡壳

在凸包上用一个单调指针寻找对踵点，求直径（最远点对） $O(h)$ ；也可求最大三角形面积、两凸多边形距离。指针前进依据叉积面积是否增大，不要用浮点角度比较。

```

// 接口: diameter2(p): 返回凸多边形直径的平方; 返回类型 long double。
// 参数: 按逆时针顺序给出的凸多边形顶点 p。
long double diameter2(const vector<Point>& p) {
    // 变量: n=容器 p 的元素数量 n; j=旋转卡壳中沿凸包单调前进的对踵点下标 j; ans=旋转卡
    // 壳当前找到的最大点对距离平方 ans。
    int n = p.size(), j = 1; long double ans = 0;
    if (n <= 1) return 0;
    // 变量: i=旋转卡壳当前枚举的凸包边起点 i。
    for (int i = 0; i < n; ++i) {
        // 变量: ni=凸包顶点 i 的下一个循环下标 ni。
        int ni = (i + 1) % n;
        while (fabsl(cross(p[ni] - p[i], p[(j + 1) % n] - p[i])) >
            fabsl(cross(p[ni] - p[i], p[j] - p[i]))) j = (j + 1) % n;
        ans = max(ans, max(dist2(p[i], p[j]), dist2(p[ni], p[j])));
    }
    return ans;
}

```

13.7 扫描线

矩形并面积：按 x 端点事件排序， y 区间用线段树维护覆盖长度，当前面积增量为 $\text{covered_y} * \Delta x$ 。坐标压缩要保留相邻坐标差，节点代表小区间而非点。

```
struct Event { long long x, y1, y2; int delta; }; // x 扫描坐标、y 区间、覆盖增量
±1。
// 接口：sweep_area(events, covered_length, update, y_id): 按 x 扫描事件；面积可由
long long 表示时返回该值，否则返回 nullopt；空事件返回 0。
// 参数：covered_length 返回可转为 __int128 的当前非负 y 覆盖长度；update 接收离散 y
区间与增量；y_id 返回 y 的离散下标。
template<class CoveredLength, class Update, class YId>
optional<long long> sweep_area(vector<Event> events, CoveredLength
covered_length,
                                Update update, YId y_id) {
    if (events.empty()) return 0;
    // 接口：比较器 lambda(a, b): 按事件横坐标从小到大排序；返回 a 是否应排在 b 前。
    sort(events.begin(), events.end(), [](auto a, auto b) { return a.x < b.x;
});
    // 变量：area=已经扫过的精确覆盖面积；last_x=上一批事件的横坐标。
    __int128 area = 0, last_x = events[0].x;
    // 变量：max_area=接口允许返回的最大面积。
    const __int128 max_area = numeric_limits<long long>::max();
    // 变量：e=当前处理的矩形竖边事件。
    for (auto e : events) {
        // 变量：covered=当前非负覆盖长度。
        __int128 covered = covered_length();
        // 变量：width=本次推进的非负宽度；delta_area=推进到 e.x 新增的精确面积。
        __int128 width = (__int128)e.x - last_x;
        if (covered < 0) throw std::invalid_argument("negative covered length"
);
        if (covered != 0 && width > max_area / covered)
            return nullopt;
        // 变量：delta_area=推进到 e.x 新增且已确认不超过 long long 上限的面积。
        __int128 delta_area = covered * width;
        if (delta_area > max_area - area) return nullopt;
        area += delta_area;
        update(y_id(e.y1), y_id(e.y2) - 1, e.delta);
        last_x = e.x;
    }
    return (long long)area;
}
```

13.8 最近点对

分治按 x 排序, 递归求左右最小距离, 只检查中线附近按 y 排序的常数个点, 复杂度 $O(n\log n)$ 。若只需竞赛常见规模, 网格哈希也可用, 但要证明桶大小与最小距离的关系。

```
// 接口: closest(p, l, r): 返回 0-based 半开区间 [l,r) 内最近点对距离; 返回类型 long double。
// 参数: 按 x 坐标维护的点数组 p; 待处理 0-based 半开区间 [l,r) 的左端点 l; 待处理 0-based 半开区间 [l,r) 的右端点 r。
long double closest(vector<Point>& p, int l, int r) {
    if (r - l <= 3) {
        // 变量: ans=最近点对当前基础区间内的最小距离平方 ans。
        long double ans = 1e100L;
        // 变量: i=最近点对当前半开区间 [l,r) 内枚举的点下标 i。
        for (int i = l; i < r; ++i) for (int j = l; j < i; ++j) ans = min(ans, dist2(p[i], p[j]));
        // 接口: 比较器 lambda(a, b): 按 y 坐标升序排列当前最近点对半开区间中的点。
        // 参数: 第一个候选点 a; 第二个候选点 b。
        sort(p.begin() + l, p.begin() + r, [](Point a, Point b) { return a.y < b.y; });
        return ans;
    }
    // 变量: m=当前递归区间 [l,r) 的中点 m; mid=当前区间中点 mid。
    int m = (l + r) / 2; Real mid = p[m].x;
    // 变量: ans=最近点对左右子区间返回值中的较小距离平方 ans。
    Real ans = min(closest(p, l, m), closest(p, m, r));
    // 两半递归结束后各自按 y 有序, 合并后整段按 y 有序。
    // 接口: 匿名 lambda(a, b): 供最近点对按纵坐标归并; 返回 a 是否应排在 b 前。
    // 参数: 纵坐标归并的第一个候选点 a; 纵坐标归并的第二个候选点 b。
    inplace_merge(p.begin() + l, p.begin() + m, p.begin() + r,
        [](Point a, Point b) { return a.y < b.y; });
    // 变量: strip=与分界线横向距离平方小于当前答案的候选点带 strip。
    vector<Point> strip;
    // 变量: i=最近点对当前半开区间 [l,r) 内枚举的点下标 i。
    for (int i = l; i < r; ++i) if ((p[i].x - mid) * (p[i].x - mid) < ans) strip.push_back(p[i]);
    // 变量: i=最近点对候选带中作为当前基准点的下标 i。
    for (int i = 0; i < (int)strip.size(); ++i)
        // 变量: j=最近点对候选带中位于 i 之后、纵坐标差仍可能改进答案的点下标 j。
        for (int j = i + 1; j < (int)strip.size() && strip[j].y - strip[i].y < sqrtl(ans); ++j)
            ans = min(ans, dist2(strip[i], strip[j]));
    return ans;
}
```

13.9 半平面交 (Half-Plane Intersection, HPI, 了解)

把每个半平面写成有向直线左侧，按极角排序并用双端队列维护交集；处理平行情形和角度相等是难点。若题目只问凸多边形裁剪，优先用逐边 Sutherland–Hodgman。

```
// 接口: clip(poly, a, b): 用有向直线 a->b 的左半平面裁剪凸多边形 poly; 返回类型
//      vector<Point>。
// 参数: 待裁剪的凸多边形顶点 poly; 裁剪半平面边界的起点 a; 裁剪半平面边界的终点 b。
vector<Point> clip(const vector<Point>& poly, Point a, Point b) {
    // 保留有向边 a->b 的左侧; cp/cq 是端点相对切线的叉积符号。
    // 变量: out=半平面裁剪后保留下来的凸多边形顶点序列 out。
    vector<Point> out;
    // 变量: i=凸多边形裁剪中当前有向边起点的顶点下标 i。
    for (int i = 0; i < (int)poly.size(); ++i) {
        // 变量: p=待裁剪多边形当前边的起点 p; q=待裁剪多边形当前边的终点 q。
        Point p = poly[i], q = poly[(i + 1) % poly.size()];
        // 变量: cp=点 p 相对裁剪有向边 a->b 的带符号叉积 cp; cq=点 q 相对裁剪有向边
        //      a->b 的带符号叉积 cq。
        Real cp = cross(b - a, p - a), cq = cross(b - a, q - a);
        // p 在合法侧就保留 p。
        if (cp >= -EPS) out.push_back(p);
        // 一进一出时, 边 pq 与切线有交点, 按线性插值求交。
        if ((cp >= 0) != (cq >= 0)) {
            // 变量: t=线段 p-q 与裁剪边界交点的线性插值参数 t。
            Real t = cp / (cp - cq);
            out.push_back(p + (q - p) * t);
        }
    }
    return out;
}
```

下面是“有向直线左侧为合法半平面”的真正半平面交模板；返回空集表示交集无界或为空时需结合题目另行处理，常规竞赛题通常会额外给一个大矩形作边界。

```
struct HalfPlane { Point p, v; Real ang; }; // 边界 p+t*v, 保留其左侧; ang=方向极
//      角。
// 接口: line_inter(a, b): 返回两条非平行直线的交点; 返回类型 Point。
// 参数: 第一条有向边界直线 a; 第二条有向边界直线 b。
Point line_inter(HalfPlane a, HalfPlane b) {
    // a.p+t*a.v 与 b.p+s*b.v 的交点参数。
    // 变量: t=两直线交点在直线 a 参数方程中的参数 t。
    Real t = cross(b.p - a.p, b.v) / cross(a.v, b.v);
    return a.p + a.v * t;
}
// 接口: outside(h, p): 判断点 p 是否有向半平面的外侧; p 位于边界右侧时返回 true。
```



```

// 参数：当前半平面 h；输入点 p。
bool outside(HalfPlane h, Point p) {
    // 有向直线左侧合法；叉积小于 0 表示点在右侧，应该删掉。
    return sgn(cross(h.v, p - h.p)) < 0;
}

// 接口：half_plane_intersection(h)：返回半平面交得到的凸多边形；返回类型 vector<
    Point>。
// 参数：当前半平面 h。
vector<Point> half_plane_intersection(vector<HalfPlane> h) {
    // 极角排序让平面边界按方向排列，双端队列只需维护首尾交点。
    for (auto &x : h) x.ang = atan2l(x.v.y, x.v.x);
    // 接口：比较器 lambda(a, b)：按半平面边界方向角从小到大排序。
    // 参数：第一个候选半平面 a；第二个候选半平面 b。
    sort(h.begin(), h.end(), [](const HalfPlane& a, const HalfPlane& b) {
        return a.ang < b.ang;
    });
    // 变量：q=半平面交中按方向维护有效边界的双端队列 q。
    deque<HalfPlane> q;
    // 变量：cur=按方向角顺序处理的当前候选半平面 cur。
    for (auto cur : h) {
        // 新半平面会淘汰队尾/队首不在新半平面内的交点。
        while (q.size() >= 2 && outside(cur, line_inter(q[q.size() - 2], q.
            back())))) q.pop_back();
        while (q.size() >= 2 && outside(cur, line_inter(q[0], q[1]))) q.
            pop_front();
        if (!q.empty() && sgn(cross(q.back().v, cur.v)) == 0) {
            // 同向平行线只保留更严格的一条，反向平行通常意味着交集为空。
            if (outside(cur, q.back().p)) q.back() = cur;
        } else q.push_back(cur);
    }
    while (q.size() >= 3 && outside(q.front(), line_inter(q[q.size() - 2], q.
        back())))) q.pop_back();
    while (q.size() >= 3 && outside(q.back(), line_inter(q[0], q[1]))) q.
        pop_front();
    if (q.size() < 3) return {};
    // 变量：poly=当前凸多边形顶点序列 poly。
    vector<Point> poly;
    // 变量：i=半平面交结果中当前相邻有效边界的下标 i。
    for (int i = 0; i < (int)q.size(); ++i)
        poly.push_back(line_inter(q[i], q[(i + 1) % q.size()]));
    return poly;
}

```


13.10 几何中的随机化与哈希技巧

- 随机 `shuffle` + 快速选择求期望线性第 k 小。
- 64 位随机哈希表示集合，支持多重集合差异的高概率比较；严谨题目用排序/Trie/计数。
- Treap 随机优先级避免退化；不要把 `rand()` 作为高质量随机源，使用 `mt19937_64`。

```
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
shuffle(a.begin(), a.end(), rng);
nth_element(a.begin(), a.begin() + k, a.end());
// 变量: answer=当前维护的答案 answer。
int answer = a[k];
uint64_t random_hash = rng();
```

13.11 几何中的位集优化

`std::bitset` 将 64/机器字并行，适合传递闭包（布尔矩阵）、集合交并、子集计数；`bitset` 大小需编译期常量，动态场景使用 `vector<unsignedlonglong>`。

```
// 变量: N=当前代码允许使用的最大下标上界 N。
const int N = 2000;
// 接口: geometry_bitset_closure(reach, n): 原地计算前 n 个顶点的传递闭包。
void geometry_bitset_closure(array<bitset<N>, N>& reach, int n) {
    // 变量: k=当前允许作为中转点的顶点。
    for (int k = 0; k < n; ++k)
        // 变量: i=当前起点。
        for (int i = 0; i < n; ++i)
            if (reach[i][k]) reach[i] |= reach[k];
}
```

13.12 几何快速小技巧

- `std::gcd` (C++17); `std::lcm` 仅在结果可由返回类型表示时使用，否则改用下方可检测溢出的 `safe_lcm`。
- `__builtin_ctzll` 只对非零数调用。
- 排序后用 `unique` 去重，差异数组用 `adjacent_difference`。
- 大量模乘、叉积、斜率比较时先判断上界，必要时统一 `__int128`。

```
// 接口: abs128(x): 在 __int128 中计算任意 long long 的绝对值，包括 LLONG_MIN。
// 参数: 任意 long long 整数 x。
__int128 abs128(long long x) {
    return x >= 0 ? (__int128)x : -(__int128)x;
```

```

}

// 接口: gcd128(a, b): 返回两个非负 __int128 整数的最大公约数。
// 参数: 非负整数 a、b。
__int128 gcd128(__int128 a, __int128 b) {
    while (b != 0) {
        // 变量: r=a 除以 b 后的非负余数。
        __int128 r = a % b;
        a = b; b = r;
    }
    return a;
}

// 接口: normalize_direction(dx, dy): 把任意非零 long long 方向约为互质且符号唯一的
//      __int128 二元组。
// 参数: 方向分量 dx、dy; 二者不能同时为 0。宽返回类型可表示 LLONG_MIN 取反后的 2^63。
pair<__int128, __int128> normalize_direction(long long dx, long long dy) {
    if (dx == 0 && dy == 0) throw invalid_argument("zero direction");
    // 变量: x、y=在宽整数域中的方向分量; g=两分量绝对值的最大公约数。
    __int128 x = dx, y = dy, g = gcd128(abs128(dx), abs128(dy));
    x /= g; y /= g;
    if (x < 0 || (x == 0 && y < 0)) x = -x, y = -y;
    return {x, y};
}

// 接口: safe_lcm(a, b): 非负最小公倍数可由 long long 表示时返回该值, 溢出时返回
//      nullopt; 0 与任意数返回 0。
// 参数: 任意 long long 整数 a、b, 包括 LLONG_MIN 与 LLONG_MAX。
optional<long long> safe_lcm(long long a, long long b) {
    if (a == 0 || b == 0) return 0;
    // 变量: x、y=在宽整数域中的绝对值; g=最大公约数; value=精确最小公倍数。
    __int128 x = abs128(a), y = abs128(b);
    // 变量: g=x 与 y 的最大公约数; value=宽整数域中的精确最小公倍数。
    __int128 g = gcd128(x, y), value = x / g * y;
    if (value > numeric_limits<long long>::max()) return nullopt;
    return (long long)value;
}

```

13.13 计算几何高级算法：参数、调用与改造

本节紧跟前面的基础模板。先看每个接口传什么，再看哪些不变量不能动，最后看如何替换维护信息或转移。

13.13.1 极角排序与凸包

极角排序常用于按方向扫描点或半平面。不要直接 `atan2` 排序，优先用半平面标记加叉积：

```
// 接口: half(a): 返回向量所在半平面编号, 供无 atan2 极角排序; 返回类型 int。
// 参数: 待分类到上下半平面的向量 a。
int half(Point a) { return (a.y > 0 || (a.y == 0 && a.x >= 0)) ? 0 : 1; }
// 接口: polar_cmp(a, b): 按极角比较两个向量, 供 sort 使用; a 的极角应排在 b 前时返回 true。
// 参数: 第一个待比较极角的向量 a; 第二个待比较极角的向量 b。
bool polar_cmp(Point a, Point b) {
    if (half(a) != half(b)) return half(a) < half(b);
    return cross(a, b) > 0;
}
```

凸包单调链中, 若题目要求保留边上的点用 `cross<0`, 只保留顶点通常用 `cross<=0` 弹出。例题可配合 P1452 Beauty Contest G: 先求凸包, 再用旋转卡壳求直径平方。

13.13.2 旋转卡壳、凸多边形内点与最近点对

凸包直径的卡壳指针只向前走, 面积/距离比较是单调的; 不要对每个点重新扫描所有点。最近点对分治按 `x` 排序, 递归解决左右, 再只检查按 `y` 排序的窄条带, 复杂度 $O(n \log n)$ 。

```
// 接口: closest(l, r): 返回 0-based 半开区间 [l,r) 内最近点对距离; 返回类型 long double。
// 参数: 待处理 0-based 半开区间 [l,r) 的左端点 l; 待处理 0-based 半开区间 [l,r) 的右端点 r。
long double closest(int l, int r) {
    if (r - l <= 1) return INF;
    // 变量: m=当前递归区间 [l,r] 的中点 m。
    int m = (l + r) >> 1;
    // 变量: midx=当前区间中央横坐标 midx。
    long double midx = p[m].x;
    // 变量: ans=最近点对左右子区间返回值中的较小距离平方 ans。
    long double ans = min(closest(l, m), closest(m, r));
    // 变量: strip=最近点对分治中靠近中线、仍可能改进答案的候选点带 strip。
    vector<Point> strip;
    // 变量: i=最近点对当前半开区间 [l,r) 内枚举的点下标 i。
    for (int i = l; i < r; ++i)
        if (fabsl(p[i].x - midx) * fabsl(p[i].x - midx) < ans)
            strip.push_back(p[i]);
    sort(strip.begin(), strip.end(), by_y);
    // 变量: i=最近点对候选带中作为当前基准点的下标 i。
    for (int i = 0; i < (int)strip.size(); ++i)
        // 变量: j=最近点对候选带中位于 i 之后、纵坐标差仍可能改进答案的点下标 j。
```

```

        for (int j = i + 1; j < (int)strip.size() &&
            (strip[j].y - strip[i].y) * (strip[j].y - strip[i].y) < ans; ++j)
            ans = min(ans, dist2(strip[i], strip[j]));
    return ans;
}

```

测试样例：最近点对的单点、重合点和跨分治边界

下面每组输入 n 个点，输出最小平方距离。题目若要求输出实际距离，再在最后开平方。

输入

```

4
2
0 0
3 4
2
1 1
1 1
4
0 0
0 3
4 0
4 3
5
0 0
10 10
5 5
5 6
20 0

```

输出

```

25
0
9
1

```

第二组检查重复点必须直接得到 **0**；第三组有多条同样的最短边；第四组的答案来自中间两个点，能检查合并后的 **strip** 是否按 **y** 排序以及带状区域边界是否写错。

例题：P1429 平面最近点对（加强版）。按 **x** 排序后调用分治；若保留平方距离，比较时避免不必要的 **sqrt**，最后输出时再开根号。

凸多边形点内判定可以先叉积判断扇区，再对扇区二分；切割则保留直线左侧点并对每条边和切线求交。题面已保证凸多边形时不要用通用半平面交替代简单二分。

13.13.3 半平面交 (Half-Plane Intersection)

把每个不等式写成“有向直线左侧保留”，按极角排序，用双端队列维护当前交集。关键接口是 `outside(line, point)` 和相邻直线求交；平行线要保留更严格的一条。

```
struct HalfPlane { Point a, v; }; // a + t*v, 保留左侧
// 接口: outside(h, p): 判断点 p 是否有向半平面的外侧; p 位于边界右侧时返回 true。
// 参数: 当前半平面 h; 输入点 p。
bool outside(const HalfPlane& h, Point p) {
    // cross(v, p-a) < 0 表示 p 在有向直线右侧, 即不满足半平面。
    return cross(h.v, p - h.a) < -EPS;
}
// 接口: intersection(a, b): 返回两条边界直线的交点; 返回类型 Point。
// 参数: 第一条有向半平面边界 a; 第二条有向半平面边界 b。
Point intersection(HalfPlane a, HalfPlane b) {
    // 令 a.a + t*a.v = b.a + s*b.v, 叉乘消去 s 得 t。
    // 调用前必须保证两条直线不平行, 否则分母为 0。
    // 变量: t=两直线交点在直线 a 参数方程中的参数 t。
    long double t = cross(b.v, b.a - a.a) / cross(b.v, a.v);
    return a.a + a.v * t;
}
```

调用测试：半平面交的方向、平行边和空交集

当前片段只展示了 `outside` 和两条直线求交，完整的双端队列流程没有写出，因此不伪造完整输入输出。补齐 HPI 后必须至少验证：

测试一： $x \geq 0$ 、 $x \leq 2$ 、 $y \geq 0$ 、 $y \leq 2$ ，面积应为 4。

测试二：在测试一中再加入 $x \leq 1$ ，面积应为 2。

测试三：同时加入 $x \leq 0$ 和 $x \geq 1$ ，交集为空，面积应为 0。

其中测试一检查有向边左侧约定，测试二检查同向平行边保留更严格者，测试三检查双端队列清空和空交集判断。

例题：P4196 半平面交 / 凸多边形。每个输入凸多边形的每条边都转成“边的左侧”，所有多边形的边合在一起做 HPI，最后对双端队列中的相邻交点求面积。若交集为空，输出 0；平行同向边必须只留下更靠内的那条。

13.13.4 圆几何与扫描线

点到圆、圆与圆位置关系先比较距离平方和半径平方，避免无意义开根号；两圆相交时，沿圆心连线求基点，再沿垂直方向偏移。线圆交点用参数方程代入二次方程。

```
// 接口: circle_circle(a, r, b, R): 返回两圆全部交点 (0/1/2 个; 重合圆需单独处理);
// 返回类型 vector<Point>。
```

```

// 参数：第一个输入圆的圆心 a；第一个圆的非负半径 r；第二个输入圆的圆心 b；第二个圆的非负半径 R。
vector<Point> circle_circle(Point a, long double r,
                           Point b, long double R) {
    // d 是圆心向量，len 是圆心距；先判位置关系，再做开根号。
    // 变量：d=从点 a 指向点 b 的方向向量 d。
    Point d = b - a;
    // 变量：len=两个输入圆心之间的距离 len。
    long double len = hypotl(d.x, d.y);
    if (len < EPS) return {}; // 同心圆无有限个交点，统一交给调用者单独分类。
    if (len > r + R + EPS || len < fabsl(r - R) - EPS) return {};

    // 两圆公共弦所在直线到圆心 a 的距离方向投影为 x。
    // 变量：x=公共弦中点沿圆心连线离圆 a 圆心的距离 x。
    long double x = (r*r - R*R + len*len) / (2*len);

    // h2 是公共交点相对基点的垂直距离平方，浮点误差可能让它略小于 0。
    // 变量：h2=圆交计算中从圆心连线到交点的垂直距离平方 h2。
    long double h2 = max((long double)0, r*r - x*x);
    // 变量：e=从圆心 a 指向圆心 b 的单位方向向量 e；base=两圆公共弦中点 base。
    Point e = d * (1 / len), base = a + e * x;
    // 变量：per=与圆心连线单位方向 e 垂直的单位向量 per。
    Point per = {-e.y, e.x};

    // h2=0 是相切；否则关于圆心连线对称地产生两个交点。
    if (h2 < EPS) return {base};
    // 变量：h=圆交点相对圆心连线的垂直偏移长度 h。
    long double h = sqrtl(h2);
    return {base + per*h, base - per*h};
}

```

最小圆覆盖的现场策略是随机打乱点，增量维护当前圆：发现点在圆外，就以它为边界重新枚举之前点；双层/三层枚举保证期望 $O(n)$ 。例题：P1742 最小圆覆盖。

测试样例：圆相交的无交、相切和双交点

下面调用 `circle_circle`，只统计交点个数；重合圆必须在调用前单独分类，不能让 `len=0` 进入除法。

输入

```

4
0 0 1 5 0 1
0 0 1 2 0 1
0 0 1 1 0 1

```

```
0 0 1 0 3 1
```

输出

```
0
```

```
1
```

```
2
```

```
0
```

四组分别是外离、外切、两点相交和内离。第二组必须允许 $h2=0$ 返回一个点；如果浮点误差使 $h2$ 略小于零而没有截成 0，相切样例会出现异常。

矩形并面积用扫描线 + 线段树维护 x 轴覆盖长度；每个事件是 $(y, x1, x2, delta)$ ，按 y 排序，先用上一层覆盖长度乘高度差，再更新 $[x1, x2]$ 。例题：P5490 扫描线。坐标离散化后叶子代表相邻坐标段，不是原始坐标点。

13.13.5 圆与直线、最小圆覆盖

直线写成 $p+t \cdot v$ ，代入圆方程后得到关于 t 的二次方程；判别式小于 0 无交点，等于 0 相切，大于 0 两交点。

```
// 接口: line_circle(p, v, o, r): 返回直线与圆的全部交点 (0/1/2 个); 返回类型 vector<Point>。
// 参数: 直线经过的基点 p; 直线方向向量 v; 圆心 o; 圆的非负半径 r。
vector<Point> line_circle(Point p, Point v, Point o, long double r) {
    // 直线参数式: P(t)=p+t*v; 如果题目给的是线段, 最后还要检查 t in [0,1]。
    // 变量: q=从圆心 o 指向直线基点 p 的位移向量 q。
    Point q = p - o;
    // 变量: A=直线参数方程代入圆方程后的二次项系数 A; B=直线参数方程代入圆方程后的一次项系数 B。
    long double A = dot(v, v), B = 2 * dot(q, v);
    // 变量: C=直线参数方程代入圆方程后的常数项 C。
    long double C = dot(q, q) - r * r;
    // 变量: D=直线与圆求交二次方程的判别式 D。
    long double D = B * B - 4 * A * C;
    // 判别式决定交点个数; 因为有误差, 接近 0 时按相切处理。
    if (D < -EPS) return {};
    D = max((long double)0, D);
    // 变量: t1=第一个参数方程解 t1。
    long double t1 = (-B - sqrtl(D)) / (2 * A);
    // 变量: t2=直线与圆第二个交点对应的参数方程根 t2。
    long double t2 = (-B + sqrtl(D)) / (2 * A);
    if (D < EPS) return {p + v * t1};
    return {p + v * t1, p + v * t2};
}
```

```
// 接口: min_circle(p): 返回覆盖所有输入点的最小圆; 返回类型 Circle。
// 参数: 输入点 p。
Circle min_circle(vector<Point> p) {
    // 随机打乱是期望线性复杂度的关键; 不要用固定输入顺序硬套均摊结论。
    shuffle(p.begin(), p.end(), mt19937(chrono::steady_clock::now().
time_since_epoch().count()));
    // 变量: c=随机增量算法当前维护的最小覆盖圆 c。
    Circle c{{0, 0}, -1};
    // 变量: i=最小覆盖圆随机增量中当前检查的点下标 i。
    for (int i = 0; i < (int)p.size(); ++i) if (!inside(c, p[i])) {
        // p[i] 在当前圆外, 新的最小圆必须把它放在边界上。
        c = {p[i], 0};
        // 变量: j=最小覆盖圆随机增量中检查当前圆是否覆盖的先前点下标 j。
        for (int j = 0; j < i; ++j) if (!inside(c, p[j])) {
            // 两个边界点确定直径圆; 若还不够, 再由三个点确定外接圆。
            c = circle_diameter(p[i], p[j]);
            // 变量: k=最小圆增量构造中早于 j 的历史点下标 k。
            for (int k = 0; k < j; ++k) if (!inside(c, p[k]))
                c = circle_three_points(p[i], p[j], p[k]);
        }
    }
    return c;
}
```

例题仍可用 P1742 最小圆覆盖。`inside` 要允许 $c.r < 0$ 作为空圆; 三点共线时不能直接除以零, 应退化为三点中最远的两点直径圆。几何模板里的 `Point/Circle/dot/cross` 名称必须统一, 不能把 `Point::dot` 和全局 `dot` 重复定义成不同类型。

测试样例: 直线圆交、最小圆覆盖的退化点集

先测 `line_circle` 的判别式分支。以下都把圆设为圆心 $(0,0)$ 、半径 1, 直线方向为 $(1,0)$:

调用 1: $p=(0,0), v=(1,0)$
 结果: 2 个交点, 参数 $t=-1,1$

调用 2: $p=(0,1), v=(1,0)$
 结果: 1 个交点, 参数 $t=0$

调用 3: $p=(0,2), v=(1,0)$
 结果: 0 个交点

调用 4: $p=(2,0), v=(1,0)$
 结果: 无限直线仍有 2 个交点, 参数 $t=-3,-1$;
 若题目要的是线段, 必须额外检查 t 是否在 $[0,1]$ 。

再测 `min_circle`。输出使用误差 $1e-9$ 比较，不能因为随机打乱点的顺序不同而要求浮点输出逐位相同：

调用 1: [(0,0)]

期望：圆心 (0,0)，半径 0

调用 2: [(0,0),(2,0)]

期望：圆心 (1,0)，半径 1

调用 3: [(0,0),(1,0),(3,0)]

期望：圆心 (1.5,0)，半径 1.5

调用 4: [(0,0),(0,2),(2,0),(2,2)]

期望：圆心 (1,1)，半径 $\sqrt{2}$

第一组专门抓 $D < 0 / D = 0 / D > 0$ 判断和“直线误当线段”；第二组专门抓空圆半径、两点直径、三点共线除零，以及三点外接圆的分支。

13.13.6 扫描线求矩形并面积

把每个矩形 $[x_1, x_2] \times [y_1, y_2]$ 变成两个事件：在 y_1 加入，在 y_2 删除。线段树维护当前 x 轴覆盖长度；`cover > 0` 时整段都被覆盖，否则由左右儿子合并。

```
struct Event { long long y, x1, x2; int delta; }; // y 扫描坐标、x 区间、覆盖增量 ±1。
struct CoverNode {
    int tag = 0; // 当前 x 小区间被完整覆盖了多少层。
    __int128 len = 0; // 该节点代表范围内至少被覆盖一次的精确长度。
};
// 变量：xs=离散化后排序去重的坐标 xs。
vector<long long> xs;
// 变量：seg=扫描线维护的线段树 seg。
CoverNode seg[MAXN * 4];
// 接口：pull(p, l, r)：按覆盖计数和两个孩子重算扫描线节点 p 表示的实际覆盖长度；会修改当前结构；多测时重新构造或显式清空成员。
// 参数：当前覆盖长度线段树节点编号 p（根为 1）；节点 p 表示的离散下标半开区间 [l,r) 左端点 l；节点 p 表示的离散下标半开区间 [l,r) 右端点 r。
void pull(int p, int l, int r) {
    // xs[l..r] 是坐标点，节点覆盖的是 [xs[l], xs[r]]。
    if (seg[p].tag) seg[p].len = (__int128)xs[r] - xs[l];
    else if (r - l == 1) seg[p].len = 0;
    else seg[p].len = seg[p<<1].len + seg[p<<1|1].len;
}
// 接口：modify(p, l, r, ql, qr, d)：给扫描线离散下标半开区间 [ql,qr) 增加覆盖计数 d；会修改当前结构；多测时重新构造或显式清空成员。
```

```

// 参数：当前覆盖长度线段树节点编号 p（根为 1）；节点 p 表示的离散下标半开区间 [l,r) 左
// 端点 l；节点 p 表示的离散下标半开区间 [l,r) 右端点 r；目标离散下标半开区间 [ql,qr)
// 左端点 ql；目标离散下标半开区间 [ql,qr) 右端点 qr；本次加入或移除的覆盖计数增量 d。
void modify(int p, int l, int r, int ql, int qr, int d) {
    // 更新的是离散下标半开区间 [ql,qr)，不是点 [ql,qr]。
    if (qr <= l || r <= ql) return;
    if (ql <= l && r <= qr) { seg[p].tag += d; pull(p,l,r); return; }
    // 变量：m=当前递归区间 [l,r] 的中点 m。
    int m = (l + r) >> 1;
    modify(p<<1,l,m,ql,qr,d);
    modify(p<<1|1,m,r,ql,qr,d);
    pull(p,l,r);
}

// 接口：union_area(e)：矩形并面积可由 long long 表示时返回该值，否则返回 nullopt；空
// 事件返回 0。
// 参数：矩形横边事件 e；xs 需预先收集所有横坐标、排序并去重。
optional<long long> union_area(vector<Event> e) {
    if (e.empty() || xs.size() < 2) return 0;
    // 事件按 y 扫描；在处理当前 y 前，seg[1].len 对应上一段高度。
    // 接口：比较器 lambda(a, b)：按矩形扫描线事件的 y 坐标升序排列。
    // 参数：第一个扫描线事件 a；第二个扫描线事件 b。
    sort(e.begin(), e.end(), [](auto& a, auto& b) { return a.y < b.y; });
    // 变量：ans=扫描线按高度条带累计的矩形并面积 ans；last_y=扫描线上一个事件纵坐标
    last_y。
    __int128 ans = 0, last_y = e[0].y;
    // 变量：max_area=接口允许返回的最大面积。
    const __int128 max_area = numeric_limits<long long>::max();
    // 变量：i=扫描线中当前 y 坐标事件组的首事件下标 i。
    for (int i = 0; i < (int)e.size(); i) {
        // 先按 long long 返回上限做除法检查，避免极端坐标的宽×高本身溢出 __int128。
        // 变量：height=当前事件与上一批事件之间的非负扫描高度。
        __int128 height = (__int128)e[i].y - last_y;
        if (seg[1].len != 0 && height > max_area / seg[1].len)
            return nullopt;
        // 变量：delta_area=本段新增且已确认不超过 long long 上限的面积。
        __int128 delta_area = seg[1].len * height;
        if (delta_area > max_area - ans) return nullopt;
        ans += delta_area;
        // 变量：j=扫描线中从 i 开始寻找同一 y 坐标事件组末尾的下标 j。
        int j = i;
        while (j < (int)e.size() && e[j].y == e[i].y) {
            // 同一条扫描线上的事件必须先全部更新，再进入下一段高度。
            // 变量：l=扫描线事件左端横坐标 x1 的离散下标 l。
            int l = lower_bound(xs.begin(), xs.end(), e[j].x1) - xs.begin();

```

```

        // 变量: r=扫描线事件右端横坐标 x2 的离散下标 r; 更新半开区间 [l,r)。
        int r = lower_bound(xs.begin(), xs.end(), e[j].x2) - xs.begin();
        modify(1, 0, (int)xs.size()-1, l, r, e[j].delta);
        ++j;
    }
    last_y = e[i].y; i = j;
}
return (long long)ans;
}

```

例题: P5490 【模板】扫描线。xs 必须去重排序, 线段树叶子代表 xs[i] 到 xs[i+1] 的小区间; 所以树的右边界是 xs.size()-1, 更新是半开区间 [l,r)。把离散点当成叶子会多算或少算一格。

测试样例: 矩形并面积的相邻、重叠、嵌套

下面采用“输入矩形左下角和右上角”的调用格式。每个矩形对应半开区域 $[x_1, x_2) \times [y_1, y_2)$; 若你的题目允许任意顺序输入, 先规范化端点。

调用 1: 1 个矩形

[(0,0,2,1)]

期望面积: 2

调用 2: 两个只接边、不重叠的矩形

[(0,0,1,1),(1,0,2,1)]

期望面积: 2

调用 3: 两个有重叠的矩形

[(0,0,2,2),(1,1,3,3)]

期望面积: $4+4-1=7$

调用 4: 一个矩形完全包含另一个矩形

[(0,0,4,4),(1,1,3,3)]

期望面积: 16

第二组检查相邻坐标是否被错误重复计算; 第三组检查覆盖层数从 2 降回 1 时是否错误清空长度; 第四组检查 cover>0 时是否直接取整段长度。每个调用都必须清空 xs、事件数组、线段树 tag/len, 否则把四组放进同一个 t 中会暴露多测污染。

13.14 超过十行模板的针对性自测

这一节专门给前面基础章节中的大板子做“抄完立即验”的小测试。每一组都按总模板包装:

```

// 变量: t=当前程序要执行的测试用例数量 t。

```

```
int t = 1;
cin >> t;
while (t--) solve();
```

若某个板子本身只是函数或数据结构，没有统一题面输入，就写成“调用测试”；这不是少写样例，而是避免把不存在的接口硬编成错误题面。真正提交时，把调用测试里的对象和操作翻译到题目自己的 `solve()` 即可。所有多测都要在 `solve()` 开头重建图、清空节点池、重置时间戳和懒标记。

13.14.1 基础、枚举、差分与二分

折半搜索：空半边、没有可行解、恰好命中上界

代码维护的是“子集和不超过 `limit` 的最大值”，四个测试应分别得到：

```
T=4
1) a=[], limit=0                -> 0
2) a=[7], limit=6               -> 0
3) a=[3,5,6], limit=8          -> 8
4) a=[8,1,4,4], limit=9        -> 9
```

第三组抓两半枚举后的二分恰好命中，第四组抓重复值和“不能贪心取当前最大数”。第一组必须让 `gen` 返回只含 0 的数组，不能访问空半边。

一维差分：单点、整段、重叠更新和负数更新

每组先给初始数组，再执行所有区间加，输出最终数组：

```
T=4
1) [5]; add(1,1,+3)             -> [8]
2) [0,0,0,0]; add(1,4,+2)       -> [2,2,2,2]
3) [1,2,3,4]; add(2,3,+5), add(1,4,-1) -> [0,6,7,3]
4) [5,5,5]; add(1,2,-5), add(2,2,+10) -> [0,10,5]
```

第三、四组必须检查 `diff[r+1]` 的位置没有越界，并且多次更新的顺序是可叠加的；把 `r+1` 写成 `r` 会在第二组整段更新时立刻暴露。

二分答案：答案在端点、每组一段和多组切分

对“把正数数组分成至多 `k` 段，最小化最大段和”测试：

```
T=4
1) [5], k=1                     -> 5
2) [1,2,3], k=3                 -> 3
3) [7,2,5,10], k=2             -> 14
```

4) $[4, 4, 4, 4]$, $k=2$ $\rightarrow 8$

第二组抓 $k=n$ ，第三组抓不能只按平均值切分，第四组抓多个相同最大值。 l 必须从数组最大值开始，不能从 0 开始；`check` 中遇到单个 $x > limit$ 要立即失败。

差分约束：无约束、可行环、负环和多条链

边 (u, v, w) 表示 $x[v] \leq x[u] + w$ ，只检查是否有解：

$T=4$

- | | |
|---|----------------------|
| 1) $n=1$, 0 条边 | $\rightarrow$ 有解 |
| 2) $n=2$: $1 \rightarrow 2(5)$, $2 \rightarrow 1(5)$ | $\rightarrow$ 有解 |
| 3) $n=2$: $1 \rightarrow 2(-1)$, $2 \rightarrow 1(-1)$ | $\rightarrow$ 无解（负环） |
| 4) $n=3$: $1 \rightarrow 2(0)$, $2 \rightarrow 3(0)$, $3 \rightarrow 1(1)$ | $\rightarrow$ 有解 |

第三组必须触发第 v 轮更新计数；第四组说明“有环”不等于“无解”，只有负环才无解。多测时 `pushed/in_queue/distance` 不清空，会让后一组错误继承负环状态。

13.14.2 数学、筛法、生成函数与树计数

任意精度整数：符号、整除、超 128 位与模幂

- | | |
|-------------------------|---|
| 1) $10^{50} + 12345$ | $\rightarrow$ 十进制输入输出不丢位 |
| 2) a 和 $-a$ 分别除以 97 | $\rightarrow$ 都满足 $a = (a/b) \times b + a \% b$ |
| 3) 10^{50} | $\rightarrow$ 明显超过 <code>__int128</code> |
| 4) $(-2)^5 \bmod 13$ | $\rightarrow 7$ |

第三组抓误用 `__int128`；第四组抓负底数未先归一化。若题目要求余数位于 $[0, b)$ ，不能直接把负数的 $a \% b$ 当答案。

试除分解、线性筛和欧拉函数：1、质数、平方数、重复质因子

质因数分解 $T=4$:

- | | |
|---------|------------------------------|
| 1) 1 | $\rightarrow$ 空分解 |
| 2) 2 | $\rightarrow 2^1$ |
| 3) 36 | $\rightarrow 2^2 \times 3^2$ |
| 4) 97 | $\rightarrow 97^1$ |

欧拉函数 $T=4$:

- | | |
|---------------|------------------|
| 1) $\phi(1)$ | $\rightarrow 1$ |
| 2) $\phi(2)$ | $\rightarrow 1$ |
| 3) $\phi(12)$ | $\rightarrow 4$ |
| 4) $\phi(36)$ | $\rightarrow 12$ |

筛法则单独检查 $N=1, 2, 10, 30$: $N=1$ 不应访问质数数组越界; $N=10$ 的质数是 $2, 3, 5, 7$, 对应 $\text{phi}(1..10)=1, 1, 2, 2, 4, 2, 6, 4, 6, 4$. $p==\text{lp}[i]$ 的分支必须让 $\text{mu}[i \times p]=0$ 并停止继续枚举。

整除分块: 商相同区间和边界 n/i

计算 $\text{sum}(\text{floor}(n/i), i=1..n)$:

```
n=1 -> 1
n=2 -> 2+1=3
n=5 -> 5+2+1+1+1=10
n=10 -> 10+5+3+2+2+1+1+1+1+1=27
```

四组都要让 $r=n/(n/l)$, 不能写成 $r=n/l$; 最后一个商为 1 的长区间是最容易漏掉的部分。

非质数模的同余合并: 互质、相容非互质、相同模数和无解

调用 `merge_congruence(a1, m1, a2, m2)`:

```
1) x=1 (mod 3), x=4 (mod 5)      -> x=4, mod 15
2) x=1 (mod 4), x=3 (mod 6)      -> x=9, mod 12
3) x=2 (mod 4), x=2 (mod 6)      -> x=2, mod 12
4) x=0 (mod 2), x=1 (mod 4)      -> 无解
```

第二、三组抓 $g=\text{gcd}(m1, m2)$ 后的最小公倍数; 第四组抓 $d\%g!=0$ 。合并后的 a 必须归一到 $[0, m)$, 中间乘法用 `__int128`, 否则大模数样例会溢出。

异或前缀和与普通小卷积: 周期四、单位元和重复系数

```
xor_prefix(0)=0, xor_prefix(1)=1, xor_prefix(2)=3, xor_prefix(3)=0
```

```
[1] * [1]          -> [1]
[1,1] * [1,1]      -> [1,2,1]
[1,2,3] * [4,5]    -> [4,13,22,15]
```

第一行抓 $n\&3==0$, 不能把 0 和 4 的分支混淆; 卷积第三组同时检查 $c[i+j]$ 下标和重复贡献累加。

Prüfer 序列与矩阵树定理: 最小树、星形树、路径树和不连通图

Prüfer 解码:

```
1) code=[]          -> n=2, 边为 1-2
2) code=[1]         -> n=3, 边为 2-1、1-3
3) code=[1,1]       -> n=4, 得到以 1 为中心的星形树
```

Prüfer 编码:

4) 路径 1-2-3-4 $\rightarrow$ [2,3] (使用最小编号叶子)

矩阵树的删除一行一列后的行列式:

- | | |
|--|-----------------|
| 1) 单边图 1-2 的 1×1 矩阵 [1] | $\rightarrow 1$ |
| 2) 三角形的矩阵 $\begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix}$ | $\rightarrow 3$ |
| 3) 四点路径的三对角余子式 | $\rightarrow 1$ |
| 4) 不连通图的零矩阵 | $\rightarrow 0$ |

第一组抓 $n-2=0$ 的空序列; 第四组抓叶子优先队列和 $\text{pivot}==n$ 的行列式零分支。矩阵树代码若模数不是质数, 不能直接使用费马逆元, 这四组默认模数为质数。

13.14.3 数据结构: 下标、懒标记、版本与回滚

并查集、树状数组和普通线段树: 单点、整段、重复操作、多测清空

并查集 $T=4$:

- | | |
|--|-------------------------------------|
| 1) $n=1$, <code>query(1,1)</code> | $\rightarrow \text{true}$ |
| 2) $n=3$, <code>union(1,2)</code> , <code>query(1,3)</code> | $\rightarrow \text{false}$ |
| 3) 接上 <code>union(2,3)</code> , <code>query(1,3)</code> | $\rightarrow \text{true}$ |
| 4) 再次 <code>union(1,3)</code> | $\rightarrow \text{false}$ (不能重复计数) |

树状数组 $T=4$:

- | | |
|--|-----------------|
| 1) $n=1$, <code>add(1,5)</code> , <code>sum(1)</code> | $\rightarrow 5$ |
| 2) $n=4$, <code>add(1,4,2)</code> , <code>range(1,4)</code> | $\rightarrow 8$ |
| 3) [1,2,3,4] 通过单点加构造后 <code>range(2,3)</code> | $\rightarrow 5$ |
| 4) 先 <code>add(4,-3)</code> , 再 <code>query(1,4)</code> , 必须包含最后一个下标 | |

普通“区间加、区间和”线段树再按下列四组手算最终查询:

- | | |
|--|------------------|
| 1) [5]: <code>query(1,1)</code> | $\rightarrow 5$ |
| 2) [0,0,0,0]: <code>add(1,4,2)</code> , <code>query(1,4)</code> | $\rightarrow 8$ |
| 3) [1,2,3,4]: <code>add(2,3,5)</code> , <code>query(1,4)</code> | $\rightarrow 20$ |
| 4) [1,1,1]: <code>add(1,3,2)</code> , <code>add(2,2,-1)</code> , <code>query(1,3)</code> | $\rightarrow 8$ |

第四组必须经过“整段懒标记后访问子区间”的 `push`; 把 `push_down` 忘掉时通常只会在这组错。

线段树上二分与稀疏表: 第一个位置、找不到、重复最值

对 `first_at_least`:

- | | |
|--------------------|-----------------|
| 1) $a=[0]$, $x=0$ | $\rightarrow 1$ |
|--------------------|-----------------|

```

2) a=[1,3,5], x=4          -> 3
3) a=[1,3,5], x=6          -> -1
4) a=[0,0,0,7], x=7        -> 4

```

对静态区间最小值：

```

1) [5], query(1,1)          -> 5
2) [3,1,2,1], query(1,4)    -> 1
3) [3,1,2,1], query(2,3)    -> 1
4) [7,7,7], query(1,3)      -> 7

```

第二、三组同时检查 $k = \text{floor}(\log_2(\text{len}))$ 和右端点 $r - (1 \ll k) + 1$ ；稀疏表只能用于幂等运算，不能把这份代码改成区间和。

二进制字典树、主席树和块状数组：零值、重复值、跨边界

二进制字典树（先插入，再查询最大异或）：

```

1) 插入 [0], query(0)          -> 0
2) 插入 [1,2,4], query(3)      -> 7
3) 插入 [5,5], query(5)        -> 0
4) 插入 [8,15], query(7)       -> 15

```

主席树静态区间第 k 小：

```

1) [5], query(1,1,1)          -> 5
2) [1,2,3], query(1,3,1), query(1,3,3) -> 1,3
3) [4,4,1], query(1,3,2)      -> 4
4) [10,0,7,3], query(2,4,2)    -> 3

```

块状数组使用 0-based 下标测试 $\text{add}(l, r, v)$ ：

```

1) n=1, add(0,0,5)            -> [5]
2) n=4, add(0,3,1)            -> [1,1,1,1]
3) n=5, add(1,3,2)            -> [0,2,2,2,0]
4) n=6, add(0,5,3), add(2,4,-1) -> [3,3,2,2,2,3]

```

主席树第二、三组抓版本相减的左右端点；块状数组第三、四组抓“同一块”和“跨多个整块”两个分支。

Treap 与可回滚并查集：空树、重复键、快照恢复

Treap 不固定随机优先级，不能比较树形，只比较中序序列和子树大小：

```

1) 空树 split(key=0)          -> 左右均空
2) 插入 [5], split(4)          -> 左空，右为 [5]

```


3) 插入 $[2,2,7]$ ，中序必须保留两个 2 $\rightarrow [2,2,7]$
 4) `split(5)` 后 `merge(left,right)` $\rightarrow$ 恢复原中序和 `size`

可回滚并查集：

初始 1,2,3 独立；`s0=snapshot()`
`union(1,2)`；`s1=snapshot()`
`union(2,3)`；此时 1 与 3 连通
`rollback(s1)`；此时 1 与 2 连通、1 与 3 不连通
`rollback(s0)`；三个点再次全部独立

这里专门抓 `rollback` 时恢复的是“合并前的大集合大小”，以及空合并不应压入历史栈；如果把路径压缩加回去，回滚会失效。

13.14.4 图论：路径、连通性、流和树上边界

拓扑排序：单点、链、分支和有向环

1) $V=1, E=0$ $\rightarrow$ 拓扑序 $[1]$
 2) $1 \rightarrow 2 \rightarrow 3$ $\rightarrow [1,2,3]$
 3) $1 \rightarrow 3, 2 \rightarrow 3$ $\rightarrow 1/2$ 任一先出，3 必须最后
 4) $1 \rightarrow 2, 2 \rightarrow 1$ $\rightarrow$ 检测为环，不能输出完整序列

第三组不能把拓扑序误认为唯一；第四组检查 `topo.size()!=n`，并且每个 `solve()` 都要重新计算入度。

Dijkstra 与 0-1 BFS：重复堆项、不可达点和零边

Dijkstra 的四组边（源点均为 1）：

1) $V=1, E=0$ $\rightarrow \text{dist}[1]=0$
 2) $1 \rightarrow 2(5), 1 \rightarrow 3(1), 3 \rightarrow 2(1)$ $\rightarrow \text{dist}[2]=2$
 3) $1 \rightarrow 2(4)$ ，点 3 无入边 $\rightarrow \text{dist}[3]=\text{INF}$
 4) $1 \rightarrow 2(0), 2 \rightarrow 3(0), 1 \rightarrow 3(5)$ $\rightarrow \text{dist}[3]=0$

0-1 BFS 用同样的方向边测试：

1) 单点 $\rightarrow 0$
 2) $1 \rightarrow 2(0), 2 \rightarrow 3(1)$ $\rightarrow \text{dist}[3]=1$
 3) $1 \rightarrow 2(1), 1 \rightarrow 3(0), 3 \rightarrow 2(0)$ $\rightarrow \text{dist}[2]=0$
 4) 点 4 不可达 $\rightarrow \text{dist}[4]=\text{INF}$

第三组抓 $w=0$ 必须放队首；Dijkstra 第二组抓旧堆项，弹出时不能再次松弛旧距离。

最小生成树：单点、重边、环和不连通图

1) $V=1, E=0$	-> $\text{cost}=0, \text{connected}=\text{true}$
2) 三角形边权 1,2,3	-> $\text{cost}=3$
3) 两点有两条边权 5,1	-> $\text{cost}=1$
4) $V=3$, 仅有边 1-2	-> $\text{connected}=\text{false}$

第三组检查排序和重边；第四组检查不连通图不能误报生成树。Kruskal 和 Prim 在四组上的结果应完全一致。

强连通分量、割点和桥：空边、全环、重边

强连通分量：

1) $V=1, E=0$	-> 1 个分量
2) $1 \rightarrow 2, 2 \rightarrow 3$	-> 3 个分量
3) $1 \rightarrow 2, 2 \rightarrow 3, 3 \rightarrow 1$	-> 1 个分量
4) $1 \leftarrow 2, 2 \rightarrow 3, 3 \leftarrow 4$	-> $\{1,2\}, \{3,4\}$ 两个分量

无向桥：

1) 单点	-> 0 条桥
2) 链 1-2-3	-> 2 条桥
3) 三角形	-> 0 条桥
4) 两条平行边 1-2, 再加 2-3	-> 只有 2-3 是桥

第四组必须使用边编号跳过父边；若只写 $v==\text{parent}$ ，平行边会被错误当成桥。SCC 第四组抓缩点后保留两个分量而不是把分量内边当成有向无环图边。

二分图染色与最大匹配：孤立点、增广链、奇环

染色：

1) $V=1, E=0$	-> 是二分图
2) 一条边 1-2	-> 是
3) 路径 1-2-3-4	-> 是
4) 三角形 1-2-3-1	-> 不是

最大匹配（左侧点数=右侧点数）：

1) 无边	-> 0
2) 只有 1 条边	-> 1
3) 边 $1-\{1,2\}, 2-\{2\}$	-> 2（必须回溯增广）
4) 完全二分图 $K_{3,3}$	-> 3

第三组是 vis 时间戳最关键的样例；每个左点开始增广前必须换新时间戳，不能每次都清空错误的那一侧或沿用旧访问标记。

最大流与最小费用最大流：零容量、瓶颈、拆路和需求不足

Dinic:

- | | |
|-------------------------------|------|
| 1) s, t 之间没有边 | -> 0 |
| 2) 唯一边 $s \rightarrow t$ 容量 5 | -> 5 |
| 3) 两条互不相交路径, 容量分别 3、2 | -> 5 |
| 4) 两条路径共享中间边容量 1 | -> 1 |

每次 BFS 后 it 必须清零; 第三、四组检查反向边和残量网络。费用流 `min_cost_flow(s, t, need)`:

- | | |
|------------------------------|-------------------|
| 1) $need=0$ | -> (0, 0) |
| 2) 唯一路径容量 3、单位费用 5, $need=2$ | -> (2, 10) |
| 3) 两条路径费用 2 和 8, $need=3$ | -> (3, 10) |
| 4) 总容量 1 但 $need=2$ | -> flow=1, 不能伪造满流 |

若题目允许负费用边, 再加一组 $s \rightarrow a(-5), a \rightarrow t(0), s \rightarrow t(0)$, 最优费用应为 -5; 这组用于检查初始势能是否正确, 不能把“所有费用非负”这个前提偷偷当成算法的一部分。

最近公共祖先、树上差分、直径和树形动态规划

用同一棵树 1-2, 1-3, 3-4, 3-5:

最近公共祖先: $lca(1, 1)=1, lca(2, 4)=1, lca(4, 5)=3, lca(1, 5)=1$
 树上路径加: 路径 (4, 5) 加 1 后, 3、4、5 各增加 1, 1、2 不增加
 直径: 端点可为 2、4/5, 长度为 3

再单独测: 单点树直径为 0, 链 1-2-3-4 直径为 3, 星形树 1-{2, 3, 4} 直径为 2。树形最大独立集用点权:

- | | |
|----------------------|-------|
| 1) 单点权 5 | -> 5 |
| 2) 边 1-2, 权 5, 7 | -> 7 |
| 3) 链权 1, 10, 1 | -> 10 |
| 4) 星形中心权 10, 三个叶子权 6 | -> 18 |

第一组抓根节点和 $lca(u, u)$; 路径差分要区分“点路径”和“边路径”, 不能把 lca 的减法公式混用。

深度优先序、重链剖分与长链剖分: 链、星、跨链路径

树 A: 1-2-3-4

- 子树(1) 应覆盖全部 DFS 序
- 子树(3) 只覆盖 {3, 4}
- HLD `path_sum(2, 4)` 必须包含 2、3、4
- 长链根 1 的长儿子必须是 2, 而不是按“子树大小”重新选择

树 B: 1- $\{2,3,4\}$

5) HLD `path_sum(2,3)` 必须拆成两条轻链并经过 1

6) 长链长度相同的儿子只要求结果正确，不能依赖某个固定 `tie-break`

这组不比较 `dfn` 的具体编号，因为同深度遍历顺序可以不同；只检查子树区间连续、路径和正确、长链选择标准确实是“最大向下深度”。

点分治代码若只保留重心分解而没有 `query/modify/answer`，只能做调用测试：单点、四点链、四点星、半径 0、两个子树各有一个距离相同的点。重点是验证重心被标记后不会再次进入同一分治层，并且跨重心的点对只计一次。

13.14.5 动态规划和字符串

线性动态规划、树形动态规划和数位动态规划

线性动态规划这一段同时测试“最长严格递增子序列”和“最大子段和”两段代码，输出写成 (LIS长度, 最大子段和)：

1) <code>a=[5]</code>	$\rightarrow (1, 5)$
2) <code>a=[-5]</code>	$\rightarrow (1, -5)$
3) <code>a=[3, 1, 2]</code>	$\rightarrow (2, 6)$
4) <code>a=[5, 1, 5, 1]</code>	$\rightarrow (2, 12)$

第二组抓最大子段和不能把答案初值固定成 0；第三组抓 `lower_bound` 与“不能把 LIS 当成最长连续段”。树形 DP 用前面的四组独立集权值；换根 DP 再测单点、链、星和两个相同最大值，重点检查第二次 DFS 使用的是“父答案减去当前子树贡献”，不能把子树答案重复加回。

数位 DP 统计区间内不含数字 4 的数（包含 0）：

T=4	
1) <code>[0, 0]</code>	$\rightarrow 1$
2) <code>[0, 4]</code>	$\rightarrow 4$
3) <code>[0, 10]</code>	$\rightarrow 10$
4) <code>[0, 99]</code>	$\rightarrow 81$

第二组抓数字 4 的排除和前导零；第四组抓 `count(R)-count(L-1)`，以及 `tight` 只在仍贴着上界时为真。

Knuth-Morris-Pratt 字符串匹配：首位、重叠和无匹配

T=4（文本，模式，第一次出现位置/重叠出现次数）

1) <code>a</code>	, <code>a</code>	$\rightarrow 0 / 1$
2) <code>aaaaa</code>	, <code>aaa</code>	$\rightarrow 0 / 3$

```
3) abc      , d      -> -1 / 0
4) abababa , aba     -> 0 / 3
```

第二、四组必须允许重叠匹配；把匹配成功后 j 直接清零会少算。模式为空串是否允许必须由题面规定，模板不要擅自把它当普通非空模式。

AC 自动机、后缀数组、后缀自动机和回文自动机的四组详细样例已经紧跟补充实现页；再加一组统一多测：**a、aaaa、abacaba、abab**。每组都必须重新 `init`，否则上一组的 Trie 节点和 fail 指针会污染下一组。后缀数组比较后缀时要检查空后缀，回文自动机要保留长度 -1 的虚根。

13.14.6 进阶专题：整体二分、时间分治、可满足性与匹配

下面这些模板的代码依赖题目状态，采用调用级测试，但每组仍然给出可核对的目标。

整体二分与线段树分治时间

整体二分（第 k 小）：

```
1) 数组 [5], k=1          -> 5
2) [1,2,3], k=1          -> 1
3) [1,2,3], k=3          -> 3
4) [4,4,1], k=2          -> 4
```

线段树分治时间（边存在区间）：

```
1) 单条边全程存在，询问其两端          -> 连通
2) 边只在时间 [2,3] 存在，询问 t=1      -> 不连通
3) 两条边的存在区间只在一个时刻重合    -> 该时刻连通
4) 删除不存在的边                      -> 必须按题面判非法，不能修改回滚栈
```

整体二分中答案区间必须覆盖真实值域；时间分治中叶节点才回答询问，回到父节点时必须回滚到进入节点前的快照。

二可满足性算法与约束状态图

子句格式为 $(x_i \text{ 或 } x_j)$ ：

```
1) 0 个变量、0 个子句          -> 可满足
2) (x1 或 x1)                  -> x1=true
3) (x1 或 x1) 且 (!x1 或 !x1)   -> 不可满足
4) (x1 或 x2)、(!x1 或 x2)、(x1 或 !x2) -> 可满足，唯一要求 x1=x2=true
```

第三组抓同一变量和反变量是否落在同一个强连通分量；第四组抓蕴含边方向。状态图搜索若没有定义“重复状态”和“终止状态”，不能只套 DFS，应分别测试空状态、一步可胜、重复到旧状态和无路可走四类调用。

快速傅里叶变换/数论变换与 Kuhn–Munkres 加权匹配

卷积用单位多项式、重复系数和非二次长度三组；结果应与前面生成函数中的三个小多项式测试完全一致，另外再加：

$[1, 0, 0, 1] \star [1, 1] \rightarrow [1, 1, 0, 1, 1]$

它专门抓变换长度是否至少为 `a.size()+b.size()-1`，以及逆变换除以长度是否在模意义下使用逆元。

Kuhn–Munkres 最大权完备匹配：

1) <code>[[5]]</code>	$\rightarrow 5$
2) <code>[[1, 2], [3, 4]]</code>	$\rightarrow 5$
3) <code>[[-1, -2], [-3, -4]]</code>	$\rightarrow -5$
4) 对角为 10、非对角为 1 的 3x3 矩阵	$\rightarrow 30$

第三组不能把答案初值设成 0；如果模板只支持非负权，必须先整体平移并在最后还原。

笛卡尔树、虚树、Steiner 树和高斯消元

最小笛卡尔树调用：

1) <code>[1]</code>	$\rightarrow$ 根为 1
2) <code>[1, 2, 3]</code>	$\rightarrow$ 根为 1，右链
3) <code>[3, 2, 1]</code>	$\rightarrow$ 根为 1，左链
4) <code>[2, 1, 3]</code>	$\rightarrow$ 根为 1，左右子树分别为 2、3

虚树四组关键点：单个关键点、关键点包含根、两个叶子、关键点的最近公共祖先不是原关键点。输出只检查保留了所有关键点及其必要的 LCA，不能把整棵原树都复制进虚树。

Steiner 树先用三点图手算： $1-2=1, 2-3=1, 1-3=3$ ，终端 $\{1, 3\}$ 的最小连通代价为 2；再测单终端代价 0、重复终端不能重复计费、不可达终端返回题面规定的无解。高斯消元则复用补充页的“唯一解、无解、多解”三组，另加交换行的主元为 0 的矩阵，确认会换到下面的非零行。

这些自测不是随机数据集：每组都至少命中一个边界分支和一个正常分支。需要做更大压力测试时，保留同一批核心模式，用生成器重复 t 次；不要把 $t=2345678$ 的数百万行文本直接塞进 PDF，现场测试应生成后通过标准输入喂给程序。

14 完整例题、接口注释与可执行测试

本章把前面各章的“调用测试”落到可执行程序。测试文件不依赖 PDF 中偶然出现的全局变量，均从 `muban.cpp` 的 C++17 地基开始追加；每个接口在代码旁写明输入含义、返回值、初始化和多测清空要求。运行某个测试文件时，所有断言通过才打印 `PASS`，断言失败会直接终止并给出行号。

14.1 统一卡片接口注释

每个板子接入题目时，先在 `solve()` 中完成以下四件事：

1. 读入题目数据，明确下标是 0-based 还是 1-based；
2. 调用初始化接口（例如 `build/init/clear`），清空节点池、时间戳、懒标记和多测答案；
3. 调用主接口，记录返回值或读取对象的查询接口；
4. 输出题目要求的答案，不把上一次测试的全局状态带入下一组。

代码中的接口注释统一采用下面的格式；复制模板时只需要把题目变量映射到这些字段：

```
// 接口：当前例题所调用函数的完整签名、参数含义与下标范围。  
// 前置：需要 build/init 的数据、值域限制和不变量。  
// 返回：答案含义；无解、不连通或空结构如何表示。  
// 清空：多测必须重置哪些数组、节点池、时间戳和懒标记。
```

若某个代码块没有自己的题面，使用测试文件中的调用测试；调用测试不是伪样例，而是直接对接口执行操作并用断言检查答案。正式题例则给出输入输出，便于把调用方式改写进题目的 `solve()`。

14.2 测试矩阵与正式题例

领域	覆盖接口	测试文件（均在正式题例 / 边界 tests/）	
基础、数学、数据结构	折半搜索、差分、二分答案、滑动窗口、单调结构、快速幂、扩展欧几里得、筛法、组合数恒等式、树状数组、线段树、01-Trie、回滚并查集、LIS	core.cpp	空集合、单点、组合数越界、整段更新、重复值、负更新、无可行解；可对照 P1177、P3368、P3372。
图论与树	BFS、多源 BFS、0-1 BFS、Dijkstra、拓扑排序、二分图染色、SCC、桥、Kuhn、Dinic、LCA	graph.cpp	重复源点、不可达点、零边、环、平行边、增广链、反向残量、根节点 LCA；可对照 P4779、P3387、P3388、P3376。
字符串	KMP、Z 函数、Manacher、Booth、AC、SAM、PAM、后缀数组与 LCP	strings.cpp	空串、重叠匹配、后缀模式、克隆状态、重复回文；可对照 P3375、P3796、P3804、P5496。
最小生成树与重构树	Kruskal、Prim、阈值连通块、瓶颈路	mst.cpp	单点、重边、环、不连通、同权边、跨森林查询；可对照 P3366。
动态规划与几何	LIS、LCS、01/完全背包、区间/数位 DP、Nim、叉积、凸包、面积、线段相交	dp_geometry.cpp	重复值、容量为零、空区间、数字 4、重复点、共线点、端点相交；可对照 P1020、P1048、P1880、P2742。
进阶数学与数据结构	高精度整数、线性基、扩展 CRT、矩阵快速幂、高斯消元、主席树、李超树	advanced.cpp	正负数除法与余数、超 128 位整数、负底数模幂、相关向量、非互质同余、无解方程、重复值、空直线集和交叉直线。

表中的测试文件均位于仓库根目录的 tests/；完整文件名和编译命令见仓库 README.md。

14.3 基础与数学：完整小题例

下面的题例与 refactored_core.cpp 中的断言一一对应。把每行调用放入 solve() 即可得到正式题解；返回值含义和边界处理写在函数上方的接口注释中。

14.3.1 子集和、区间加与二分答案

题例 A：给 n 个非负数和上界 `limit`，求不超过 `limit` 的最大子集和。

输入

```
4
0 0
1 6
3 8 3 5 6
4 9 8 1 4 4
```

输出

```
0
0
8
9
```

题例 B: 把正数数组分成至多 k 段, 最小化最大段和。

输入

```
2
4 2
7 2 5 10
4 3
1 2 3 4
```

输出

```
14
4
```

14.3.2 树状数组、线段树与回滚并查集

题例: 初始数组 $[1, 2, 3, 4]$, 执行区间加和区间和查询。

输入

```
4 4
1 2 3 4
Q 1 4
A 2 3 5
Q 1 4
Q 2 3
```

输出

```
10
20
15
```

边界调用:

Fenwick(1) 只更新 1; 线段树整段懒标记后查询子区间;
rollback(snapshot) 后, 合并前的集合大小和连通性必须恢复。

14.3.3 高精度整数四则运算

读入两个十进制整数 a, b （保证 $b \neq 0$ ），依次输出和、差、积、向零截断的商与余数：

输入

123456789012345678901234567890 97

输出

123456789012345678901234567987

123456789012345678901234567793

11975308534197530853419753085330

1272750402189130710322005854

52

负数还要检查恒等式 $a = (a/b) * b + a \% b$ ；需要数学意义的非负余数时，再执行 $r = (r+b) \% b$ （前提是 $b > 0$ ）。

14.4 图论与树：完整小题例

14.4.1 最短路、拓扑与最大流

题例：有向非负权图，从 1 号点求最短路。

输入

4 5

1 2 5

1 3 1

3 2 1

2 4 1

3 4 4

输出

0 2 1 3

题例：网络流的两条不相交路径。

输入

4 4

1 2 3

1 3 2

2 4 3

3 4 2

输出

5

不可达点输出 INF；拓扑排序若处理点数小于 n 必须报告有环；Dinic 每次分层后重置 `it`，增广时同步修改反向边。

14.4.2 SCC、桥、匹配与 LCA

调用测试：

SCC: 1->2, 2->3, 3->1, 3->4	-> 2 个分量
桥: 1-2 的平行边 + 2-3	-> 只有 2-3 是桥
Kuhn: 左 1->{1,2}, 左 2->{2}	-> 匹配数 2
LCA: 1-{2,3}, 3-{4,5}	-> lca(4,5)=3

14.5 字符串、动态规划与几何：完整小题例

字符串：模式串 [a, ab, bab]，文本 abab

总出现次数应为 5；逐模式次数应为 [2, 2, 1]。

动态规划：物品重量 [2,3,4]、价值 [3,4,5]、容量 5

01 背包答案为 7；容量 0 时答案为 0。

几何：正方形 (0,0),(1,0),(1,1),(0,1)

凸包顶点数为 4，面积为 1；两条对角线相交，平行的两条单位线段不相交。

14.6 一键验收

在仓库根目录执行下列命令；每一行都必须输出 PASS，任一断言失败都应先修复模板或测试，而不是删掉断言：

```
g++ -std=c++17 -O2 -Wall tests/refactored_core.cpp
./a.out
g++ -std=c++17 -O2 -Wall tests/refactored_graph.cpp
./a.out
g++ -std=c++17 -O2 -Wall tests/refactored_strings.cpp
./a.out
g++ -std=c++17 -O2 -Wall tests/refactored_mst.cpp
./a.out
g++ -std=c++17 -O2 -Wall tests/refactored_dp_geometry.cpp
./a.out
g++ -std=c++17 -O2 -Wall tests/refactored_advanced.cpp
./a.out
```

这六个文件构成大版本当前的可执行冒烟集；章节中的边界清单仍然是扩展题目时的回归用例，新增变体必须先加入对应测试，再把接口注释和例题补到卡片。

15 对拍

对拍 (Stress Testing) 是竞赛中查错的核心手段：用随机数据生成器产生大量测试数据，分别用暴力程序和待测程序运行，比较输出是否一致。一旦发现不一致，即可定位 bug。对拍三件套：

- **gen.cpp**: 随机数据生成器，输出一组合法的输入数据。
- **brute.cpp**: 暴力解法（正确但慢），作为对照标准。
- **sol.cpp**: 你的正式解法（快但可能有 bug）。

15.1 对拍脚本

15.1.1 Linux / macOS (Bash)

```
#!/bin/bash
# 编译三个程序
g++ -O2 -o gen gen.cpp
g++ -O2 -o brute brute.cpp
g++ -O2 -o sol sol.cpp

for ((i = 1; ; i++)); do
    # 生成随机数据（用循环变量 i 做种子）
    ./gen $i > in.txt
    # 分别运行暴力和正解
    ./brute < in.txt > out_brute.txt
    ./sol < in.txt > out_sol.txt
    # 比较输出
    if diff out_brute.txt out_sol.txt > /dev/null 2>&1; then
        echo "Test $i: AC"
    else
        echo "Test $i: WA !!!"
        echo "--- Input ---"
        cat in.txt
        echo "--- Brute ---"
        cat out_brute.txt
    fi
done
```

```
        echo "--- Sol ---"
        cat out_sol.txt
        break
    fi
done
```

15.1.2 Windows (BAT)

```
@echo off
g++ -O2 -o gen.exe gen.cpp
g++ -O2 -o brute.exe brute.cpp
g++ -O2 -o sol.exe sol.cpp

set i=0
:loop
set /a i+=1
gen.exe %i% > in.txt
brute.exe < in.txt > out_brute.txt
sol.exe < in.txt > out_sol.txt
fc out_brute.txt out_sol.txt > nul
if errorlevel 1 (
    echo Test %i%: WA !!!
    echo --- Input ---
    type in.txt
    echo --- Brute ---
    type out_brute.txt
    echo --- Sol ---
    type out_sol.txt
    pause
    goto :eof
)
echo Test %i%: AC
goto loop
```

- 编译时加- O2 让暴力跑得更快，能测更多组。
- 如果正解有可能输出多种正确答案（如最优方案不唯一），不能直接 diff，需要写 checker 来判断正确性。
- 如果对拍跑很久都没问题，考虑增大数据范围或换生成策略（边界、特殊图等）。

15.2 随机数据生成器 (gen.cpp)

生成器的核心原则：

- 用命令行参数作为随机种子，保证每次运行的数据不同，且可复现。
- 生成的数据必须满足题目的输入格式和约束条件。
- 对拍时先用小数据 ($n \leq 10$)，确认暴力能跑出来。

```
#include <bits/stdc++.h>
using namespace std;

mt19937 rng; // 梅森旋转随机数引擎

// 生成 [lo, hi] 范围内的随机整数
// 接口: randint(lo, hi): 返回闭区间 [l,r] 内均匀随机整数; 返回类型 int。
// 参数: 均匀随机整数闭区间下界 lo; 均匀随机整数闭区间上界 hi。
int randint(int lo, int hi) {
    return uniform_int_distribution<int>(lo, hi)(rng);
}

// 生成 [lo, hi] 范围内的随机 long long
// 接口: randll(lo, hi): 返回闭区间 [l,r] 内均匀随机 long long; 返回类型 long long。
// 参数: 均匀随机 long long 闭区间下界 lo; 均匀随机 long long 闭区间上界 hi。
long long randll(long long lo, long long hi) {
    return uniform_int_distribution<long long>(lo, hi)(rng);
}

// 生成长度为 len 的随机小写字母字符串
// 接口: randstr(len): 生成并返回长度为 len 的均匀随机小写字母字符串; 返回类型 string。
// 参数: 目标字符串长度 len。
string randstr(int len) {
    // 变量: s=正在逐字符构造并由 randstr 返回的随机小写字母字符串 s。
    string s(len, ' ');
    for (char& c : s) c = 'a' + randint(0, 25);
    return s;
}

// 生成 n 个节点的随机树 (输出 n-1 条边)
// 方法: 每个新节点随机连接到已有节点
// 接口: rand_tree(n): 生成 n 个点的随机树边集; 返回类型 vector<pair<int,int>>。
// 参数: 随机树的顶点数 n。
vector<pair<int,int>> rand_tree(int n) {
    // 变量: edges=当前随机树已经生成的无向边集合 edges。
    vector<pair<int,int>> edges;
```

```

// 变量: i=随机树生成中当前新加入的顶点编号 i。
for (int i = 2; i <= n; i++) {
    // 变量: fa=当前节点的父节点 fa。
    int fa = randint(1, i - 1);
    edges.push_back({fa, i});
}
// 打乱边的顺序和方向
shuffle(edges.begin(), edges.end(), rng);
for (auto& [u, v] : edges)
    if (randint(0, 1)) swap(u, v);
return edges;
}

// 生成 n 个节点 m 条边的随机简单图 (无重边无自环)
// 接口: rand_graph(n, m): 生成 n 点 m 边随机简单图; 返回类型 vector<pair<int,int>>。
// 参数: 随机图的顶点数 n; 随机图的边数 m。
vector<pair<int,int>> rand_graph(int n, int m) {
    // 变量: used=记录每个对象是否已被使用的标记 used。
    set<pair<int,int>> used;
    // 变量: edges=当前随机简单图已经生成的无向边集合 edges。
    vector<pair<int,int>> edges;
    while ((int)edges.size() < m) {
        // 变量: u=随机简单图候选边的第一个端点 u; v=随机简单图候选边的第二个端点 v。
        int u = randint(1, n), v = randint(1, n);
        if (u == v) continue;
        if (u > v) swap(u, v);
        if (used.count({u, v})) continue;
        used.insert({u, v});
        edges.push_back({u, v});
    }
    shuffle(edges.begin(), edges.end(), rng);
    for (auto& [u, v] : edges)
        if (randint(0, 1)) swap(u, v);
    return edges;
}

// 生成 n 个不重复的随机整数 (值域 [lo, hi])
// 接口: rand_distinct(n, lo, hi): 生成范围内互不相同的随机整数; 返回类型 vector<int>。
// 参数: 需要生成的整数个数 n; 随机取值闭区间下界 lo; 随机取值闭区间上界 hi。
vector<int> rand_distinct(int n, int lo, int hi) {
    // 变量: used=记录每个对象是否已被使用的标记 used。
    set<int> used;
    // 变量: res=尚未重复采样得到的随机整数结果 res。
    vector<int> res;

```

```
while ((int)res.size() < n) {
    // 变量: x=当前采样并检查去重的随机整数 x。
    int x = randint(lo, hi);
    if (!used.count(x)) {
        used.insert(x);
        res.push_back(x);
    }
}
return res;
}

// 生成随机排列 [1, 2, ..., n]
// 接口: rand_perm(n): 生成 1..n 的随机排列; 返回类型 vector<int>。
// 参数: 随机排列的长度 n。
vector<int> rand_perm(int n) {
    // 变量: p=待随机打乱为 1..n 排列的数组 p。
    vector<int> p(n);
    iota(p.begin(), p.end(), 1);
    shuffle(p.begin(), p.end(), rng);
    return p;
}

// 接口: main(argc, argv): 用 argv[1] 初始化随机种子, 生成并输出一组小规模随机数组; 入口函数; 每组测试前按注释清空全局状态。
// 参数: 命令行参数个数 argc; 命令行参数数组 argv。
int main(int argc, char* argv[]) {
    // 用命令行参数做种子
    rng.seed(atoi(argv[1]));

    // ===== 根据题目修改以下内容 =====
    int n = randint(1, 10); // 对拍时用小范围
    cout << n << "\n";
    // 变量: i=随机数据生成器当前输出的数组位置 i。
    for (int i = 0; i < n; i++) {
        cout << randint(1, 100);
        if (i + 1 < n) cout << " ";
    }
    cout << "\n";

    return 0;
}
```


- `mt19937` 比 `rand()` 质量更好且可移植。用 `mt19937_64` 生成 64 位随机数。
- `uniform_int_distribution<int>(lo, hi)` 生成 $[lo, hi]$ 闭区间均匀分布。
- 生成随机树时，如果需要链状树（极端数据），让每个节点连接 $i - 1$ ；如果需要菊花图，让所有节点连接节点 1。
- 随机图的边数 m 不能超过 $\frac{n(n-1)}{2}$ ，否则死循环。

15.3 暴力解法（brute.cpp）

暴力程序的原则：正确性第一，效率无所谓。能用 $O(n^3)$ 就用 $O(n^3)$ ，能枚举就枚举，绝不优化。

```
#include <bits/stdc++.h>
using namespace std;
#define int long long
#define endl '\n'

// 接口：solve()：读取一个整数数组，暴力枚举全部子数组并输出满足题目条件的计数；无返回值。
void solve() {
    // 变量：n=输入数组的元素个数 n。
    int n;
    cin >> n;
    // 变量：a=暴力解法读入的长度为 n 的整数数组 a。
    vector<int> a(n);
    // 变量：i=当前从输入读取数组元素的下标 i。
    for (int i = 0; i < n; i++) cin >> a[i];

    // ===== 暴力做法（保证正确，不管复杂度）=====
    // 例：暴力枚举所有子集 / 全排列 /  $O(n^2)$  枚举
    // 变量：ans=暴力枚举中累计满足题目条件的方案数 ans。
    int ans = 0;
    // 变量：i=暴力枚举子数组的起点 i。
    for (int i = 0; i < n; i++)
        // 变量：j=暴力枚举中从起点 i 向右扩展的子数组终点 j。
        for (int j = i; j < n; j++) {
            // ...
        }

    cout << ans << endl;
}
```

```
// 接口: main(): 开启快速 I/O, 并按测试数 t 调用暴力解 solve; 入口函数; 每组测试前按注释清  
空全局状态。  
signed main() {  
    ios::sync_with_stdio(0);  
    cin.tie(0);  
    // 变量: t=当前程序要执行的测试用例数量 t。  
    int t = 1;  
    // cin >> t; // 根据题目决定是否多测  
    while (t--) solve();  
    return 0;  
}
```

15.4 交互题对拍

交互题无法用管道对拍, 需要写一个 interactor 同时充当评测端和数据生成器。

```
// interactor.cpp —— 模拟交互评测  
// 编译后用管道连接:  
// Linux: mkfifo pipe; (./interactor < pipe | ./sol > pipe)  
// 或者直接写成单文件 interactor, 用 popen 启动 sol  
#include <bits/stdc++.h>  
using namespace std;  
  
// 接口: main(): 生成隐藏答案, 读取选手猜测并回复比较结果, 直至猜中或超过询问上限; 入口函  
数; 每组测试前按注释清空全局状态。  
int main() {  
    mt19937 rng(42);  
    // 生成隐藏数据  
    // 变量: secret=交互题中仅供本地模拟的隐藏答案 secret。  
    int secret = uniform_int_distribution<int>(1, 1000)(rng);  
  
    // 模拟交互过程  
    // 变量: queries=输入询问集合 queries。  
    int queries = 0;  
    while (true) {  
        // 变量: guess=本轮向交互器提交的猜测值 guess。  
        int guess;  
        // 读取选手的输出 (选手的 cout 是这里的 cin)  
        if (!(cin >> guess)) break;  
        queries++;  
        if (guess == secret) {  
            cout << "=" << endl; // 正确  
            break;  
        }  
    }  
}
```

```
    } else if (guess < secret) {  
        cout << "<" << endl; // 太小  
    } else {  
        cout << ">" << endl; // 太大  
    }  
}  
// 检查询问次数是否超限  
cerr << "Queries used: " << queries << endl;  
return 0;  
}
```

- 对拍时 gen 的数据范围要小！暴力程序通常是 $O(n^2)$ 或 $O(2^n)$ ， n 太大跑不完。一般 $n \leq 10$ 起步，跑上万组。
- 每次对拍脚本传入不同的种子 `./ gen $i`，确保数据多样性。找到 WA 后用同一个种子可以复现。
- 注意行末空格和换行的差异可能导致 diff 报 WA，可以在对比前做 trim。
- 如果正解涉及浮点数输出，diff 无法判断精度误差，需要写 checker 比较相对/绝对误差。
- 生成的输入必须严格满足题目约束（如连通图、无重边、值域范围等），否则暴力和正解可能行为都不确定。

16 竞赛环境速查

16.1 Windows 命令行（CMD / PowerShell）

16.1.1 编译与运行

```
:: 基础编译
g++ -o sol.exe sol.cpp

:: 推荐编译选项（竞赛常用）
g++ -O2 -std=c++17 -o sol.exe sol.cpp

:: 开所有警告 + 调试信息（调试时用）
g++ -Wall -Wextra -Wshadow -g -fsanitize=address,undefined -o sol.exe sol.cpp

:: 编译并立即运行
g++ -O2 -o sol.exe sol.cpp && sol.exe

:: 从文件读入、输出到文件
sol.exe < in.txt
sol.exe < in.txt > out.txt

:: 同时输出到屏幕和文件（PowerShell）
.\sol.exe < in.txt | Tee-Object out.txt

:: 计时运行（PowerShell）
Measure-Command { .\sol.exe < in.txt > out.txt }
```

- `-fsanitize=address,undefined` 可以检测数组越界、未定义行为等，调试利器！但会显著变慢，提交前务必去掉。
- `-O2` 和 `-fsanitize` 不要同时用，优化可能把 sanitizer 的检测优化掉。
- Windows 下 PowerShell 和 CMD 的重定向语法一样，但管道行为有差异。推荐用 PowerShell。

16.1.2 文件操作

```
:: 查看文件内容
type in.txt                :: CMD
cat in.txt                 :: PowerShell (Get-Content 的别名)

:: 比较两个文件
fc out_brute.txt out_sol.txt :: CMD (按行比较)
diff out1.txt out2.txt      :: PowerShell (Compare-Object 的别名, 需 Git)

:: 复制 / 移动 / 删除
copy a.txt b.txt
move a.txt b.txt
del a.txt

:: 创建目录
mkdir contest

:: 列出当前目录文件
dir                        :: CMD
ls                         :: PowerShell
```

16.1.3 进程与调试

```
:: 强制终止程序 (死循环时用)
Ctrl + C                :: 终端中直接按

:: 查看正在运行的进程
tasklist | findstr sol.exe

:: 强制杀死进程
taskkill /F /IM sol.exe

:: 查看程序返回值 (上一条命令的退出码)
echo %errorlevel%        :: CMD
echo $LASTEXITCODE       :: PowerShell
```

16.1.4 常用技巧

```
:: 快速生成测试数据并对拍 (单行命令)
g++ -O2 -o gen.exe gen.cpp && g++ -O2 -o brute.exe brute.cpp && g++ -O2 -o sol.exe sol.
cpp
```

```

:: 批量编译当前目录所有 cpp 文件 (PowerShell)
Get-ChildItem *.cpp | ForEach-Object {
    g++ -O2 -o ($_.BaseName + ".exe") $_.FullName
}

:: 清理编译产物 (PowerShell)
Remove-Item *.exe, *.o, in.txt, out*.txt -ErrorAction SilentlyContinue

:: 查看 g++ 版本
g++ --version

:: 检查是否安装了某工具
where g++
where python

```

- 文件重定向 < in.txt > out.txt 在比赛中比在代码里写 freopen 更灵活，不用改代码。
- 如果需要在代码内重定向（某些 OJ 要求），用：


```
freopen("input.txt", "r", stdin);
freopen("output.txt", "w", stdout);
```
- PowerShell 中 Measure-Command 可以精确测量运行时间，判断是否会 TLE。

16.2 VS Code 快捷键速查

以下快捷键基于 Windows 默认键位。macOS 把 Ctrl 换成 Cmd。

16.2.1 文件与编辑

Ctrl + N	新建文件
Ctrl + O	打开文件
Ctrl + S	保存
Ctrl + Shift + S	另存为
Ctrl + W	关闭当前标签
Ctrl + Tab	切换已打开的文件
Ctrl + Z	撤销
Ctrl + Shift + Z	重做（或 Ctrl + Y）
Ctrl + X	剪切整行（光标在行上，不选中也可以）
Ctrl + C	复制整行（同上）
Ctrl + Shift + K	删除整行
Alt + Up/Down	上下移动当前行

Shift + Alt + Up/Down	向上/下复制当前行
Ctrl + D	选中当前单词，再按选下一个相同的
Ctrl + Shift + L	选中所有相同的单词（批量重命名变量超好用）

16.2.2 查找与替换

Ctrl + F	查找
Ctrl + H	查找并替换
Ctrl + Shift + F	全局搜索（跨文件）
Ctrl + G	跳转到指定行号
F12	跳转到定义
Alt + F12	速览定义（弹窗）
Ctrl + Shift + O	跳转到当前文件的符号（函数名等）

16.2.3 多光标与选择

Alt + Click	添加多光标
Ctrl + Alt + Up/Down	上下添加光标（竖向多行编辑）
Ctrl + Shift + L	选中所有匹配项并添加光标
Ctrl + L	选中当前行
Shift + Alt + 拖拽	列选择（矩形选择）
Ctrl + Shift + [/]	折叠/展开代码块

16.2.4 终端

Ctrl + `	打开/关闭集成终端
Ctrl + Shift + `	新建终端
Ctrl + Shift + 5	拆分终端（左右并排）
Ctrl + PageUp/Down	切换终端标签

- 在终端中按 `Up` 键可以翻出上一条命令，快速重复编译运行。
- 可以配置 VS Code 的 `tasks.json` 实现一键编译运行（见下方配置）。

16.2.5 调试

F5	启动调试
Shift + F5	停止调试
F9	切换断点
F10	单步跳过（Step Over）

F11	单步进入 (Step Into)
Shift + F11	单步跳出 (Step Out)
Ctrl + Shift + D	打开调试面板

16.2.6 其他实用操作

Ctrl + Shift + P	命令面板 (万能入口)
Ctrl + ,	打开设置
Ctrl + B	切换侧边栏显示/隐藏
Ctrl + J	切换底部面板显示/隐藏
Ctrl + K, Ctrl + S	查看所有快捷键
Ctrl + /	注释/取消注释当前行
Ctrl + Shift + A	块注释 (/* ... */)
Ctrl + K, Ctrl + F	格式化选中代码
Shift + Alt + F	格式化整个文件
Ctrl + Shift + V	预览 Markdown

16.3 VS Code 竞赛配置

16.3.1 一键编译运行 (tasks.json)

在 `.vscode/tasks.json` 中配置, 按 `Ctrl+Shift+B` 即可一键编译运行当前文件。

```
{
  "version": "2.0.0",
  "tasks": [
    {
      "label": "compile and run",
      "type": "shell",
      "command": "g++",
      "args": [
        "-O2", "-std=c++17",
        "-Wall", "-Wextra",
        "-o", "${fileDirname}/${fileBasenameNoExtension}.exe",
        "${file}"
      ],
      "group": {
        "kind": "build",
        "isDefault": true
      },
      "problemMatcher": "$gcc"
    }
  ]
}
```



```
]
}
```

16.3.2 调试配置 (launch.json)

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Debug C++",
      "type": "cppdbg",
      "request": "launch",
      "program": "${fileDirname}/${fileBasenameNoExtension}.exe",
      "args": [],
      "cwd": "${fileDirname}",
      "stopAtEntry": false,
      "externalConsole": true,
      "MIMode": "gdb",
      "miDebuggerPath": "gdb",
      "preLaunchTask": "compile and run"
    }
  ]
}
```

16.3.3 推荐插件

- **C/C++ (Microsoft)**: 语法高亮、智能补全、调试支持。
- **Code Runner**: 右键一键运行代码，支持输入重定向。
- **Competitive Programming Helper (cph)**: 自动抓取样例、一键测试，支持 Codeforces/AtCoder 等。
- **Error Lens**: 将编译错误/警告直接显示在代码行内，快速定位问题。
- **Bracket Pair Colorizer** (VS Code 已内置): 括号彩色匹配，嵌套多层时一目了然。

16.3.4 推荐用户设置 (settings.json 片段)

```
{
  // 自动保存
  "files.autoSave": "afterDelay",
  "files.autoSaveDelay": 1000,
  // 字体推荐等宽字体
```

```
"editor.fontFamily": "Consolas, 'Courier New', monospace",
"editor.fontSize": 15,
// 显示行号
"editor.lineNumbers": "on",
// Tab 转空格, 宽度 4
"editor.tabSize": 4,
"editor.insertSpaces": true,
// 括号匹配高亮
"editor.bracketPairColorization.enabled": true,
// 终端默认 PowerShell
"terminal.integrated.defaultProfile.windows": "PowerShell"
}
```

- 比赛时建议关闭自动补全的延迟弹出，避免打字时频繁弹出提示干扰思路：
`"editor.quickSuggestions": false`
- 使用 `Ctrl+Shift+P` 输入 “Snippets” 可以自定义代码片段，把常用模板设为一键插入。
- 如果终端太小，`Ctrl+`` 打开终端后，拖拽顶部边框可以调整高度。

17 题型/关键词 → 算法索引

本页专门给“现场查名字”使用。下表中的算法先写完整中文/英文名称，括号内才是题解和洛谷题面中常见的简称；不要只记简称。标注“作用”的条目为 2100 分及以上的重点内容。

17.1 一、博弈论

约分	完整名称（常见写法）	2100 分及以上：作用
1800–2000	必胜态/必败态动态规划 (Win/Lose Dynamic Programming)	—
1800–2100	有向无环图上的 Sprague–Grundy 函数与最小 未出现值 (Directed Acyclic Graph、SG、mex)	把无偏无环博弈状态转成整数，便于判断胜负或合并子游戏。
1800–2100	多个独立子博弈的 Sprague–Grundy 值异或	判断游戏能否拆成互不影响的部分，并用异或合并各部分。
1800–2000	Bash 取石子游戏及限制取石子变形	—
1800–2000	普通 Nim 游戏的必胜操作构造	—
1900–2100	反常 Nim 游戏 (Misère Nim)	—
1900–2200	阶梯 Nim 游戏 (Staircase Nim)	只对特定编号台阶取异或，处理石子只能向下移动的游戏。
1900–2200	图游戏建模 (Graph Game Modeling)	把棋子移动、可达性和取点过程转成状态图，再做胜负或 Grundy 分析。

约分	完整名称（常见写法）	2100 分及以上：作用
2000–2300	Sprague–Grundy 函数打表与周期发现	将超大取石子数量压缩到前导段和周期内，避免按数量线性计算。
2100–2400	大范围取石子游戏的 Sprague–Grundy 周期压缩	对超大状态值利用已证明的周期直接回答，常用于 n 达到 10^{18} 的题。
1900–2200	普通 Nim 游戏的必胜策略恢复	根据异或和构造具体取哪一堆、取多少个。
2000–2300	状态压缩博弈动态规划 (State-Compressed Game Dynamic Programming)	用位掩码保存少量棋子/资源状态，枚举合法操作并记忆化。
1800–2100	记忆化博弈搜索 (Memoized Game Search)	对有大量重复状态的博弈搜索去重，避免重复递归。
1900–2200	极大极小搜索 (Minimax Search)	双方操作不同或不能拆成独立子游戏时，在完整博弈树上求最优决策。
2000–2300	Alpha-Beta 剪枝 (Alpha-Beta Pruning)	在极大极小搜索中利用上下界跳过不可能改变答案的分支。
2100–2400	博弈树状态去重与状态哈希 (Transposition Table and State Hashing)	合并不同走法到达的同一局面，降低棋类搜索的状态数。
2200–2500	多阶段/带资源限制的组合游戏	处理多个阶段、共享资源和操作规则变化，不能直接套独立游戏异或。
2300–2500	有环图上的博弈状态分析 (Cyclic Graph Game Analysis)	区分必胜、必败和平局，处理会回到旧状态的图博弈。

查阅位置：博弈专题基础模板后紧跟的博弈论详解与自测样例。

17.2 二、高级数据结构

约分	完整名称（常见写法）	2100 分及以上：作用
1800–2100	高阶前缀和与多阶差分 (Higher-Order Prefix Sums and Differences)	—
1900–2200	多重懒标记线段树 (Segment Tree with Multiple Lazy Tags)	同时维护多种区间修改，并正确合成和下传标记。
1900–2200	仿射标记线段树 (Affine-Transformation Segment Tree)	统一处理区间乘法、区间加法和区间和，标记形式为 $x \rightarrow ax+b$ 。
1800–2100	线段树上的二分 (Segment Tree Binary Search)	在区间最大值、容量或非负前缀和上找第一个满足条件的位置。
1800–2100	莫队算法 (Mo’s Algorithm)	—
2200–2400	带修改莫队算法 (Mo’s Algorithm with Modifications)	离线处理区间询问和时间维修改，状态移动为左端点、右端点、修改时间。
2200–2400	回滚莫队算法 (Rollback Mo’s Algorithm)	在不支持普通删除或需要撤销的区间统计中，用日志回退维护状态。
2000–2300	三维偏序的 CDQ 分治 (Three-Dimensional Dominance CDQ Divide and Conquer)	把一个维度分治、一个维度排序、最后一维交给树状数组统计。
2200–2400	CDQ 分治优化动态规划 (CDQ Divide and Conquer Optimization)	让左半状态只贡献给右半状态，优化具有时间/坐标偏序的动态规划转移。
1900–2200	可持久化字典树 (Persistent Trie / Persistent Prefix Tree)	对历史版本和区间版本做异或最大值、第 k 大等查询。
2200–2400	可持久化并查集 (Persistent Disjoint Set Union)	保存每个时刻的连通关系，支持复制历史版本、合并和查询连通性。
1900–2200	树上启发式合并 (Disjoint Set Union on Tree, DSU on Tree)	在线性树上统计每个子树的颜色频次、众数或频率函数。
2200–2500	长链剖分 (Long-Chain Decomposition)	用最长向下链优化树上深度 DP、 k 级祖先和同深度信息合并。

约分	完整名称（常见写法）	2100 分及以上：作用
2100–2400	点分树/重心树（Centroid Tree）	把树上距离查询转成重心祖先链上的若干距离容器，支持在线修改和查询。
2100–2400	李超线段树（Li Chao Segment Tree）	动态维护直线集合，在指定横坐标查询最大值或最小值。
2200–2400	线段插入版李超线段树（Li Chao Segment Tree for Line Segments）	让每条直线只在一个横坐标区间有效，处理线段插入与在线最值查询。
2300–2500	区间最值势能线段树（Segment Tree Beats）	维护区间取最小/最大与区间和，利用最大值、次大值和势能保证均摊复杂度。
2200–2500	伸展树（Splay Tree）	通过旋转把访问节点移到根，维护动态序列、排名和区间翻转。
2200–2400	文艺平衡树/序列伸展树（Reversible Splay Tree）	用伸展树维护序列区间反转、插入、删除和区间查询。
2400–2500	Link-Cut Tree 动态树（Link-Cut Tree, LCT）	维护动态森林的连边、断边、换根和路径信息。
2400–2500	动态动态规划与矩阵线段树（Dynamic Dynamic Programming and Matrix Segment Tree）	把局部状态转移写成矩阵，用线段树合并并支持单点修改后的全局答案。

17.3 三、数学与组合计数

约分	完整名称（常见写法）	2100 分及以上：作用
基础–1800	任意精度整数（Big Integer）	纯 C++17 处理超过 <code>__int128</code> 的有符号整数四则运算、模快速幂和最大公约数。
1800–2200	高级容斥与倍数容斥（Advanced Inclusion–Exclusion）	—
1900–2200	卢卡斯定理（Lucas Theorem）	—

约分	完整名称（常见写法）	2100 分及以上：作用
1900–2200	扩展中国剩余定理 (Extended Chinese Remainder Theorem, 扩展 CRT)	合并模数不互质的同余方程，并处理解的存在性和最小非负解。
2200–2500	模合数的组合数 (Binomial Coefficients Modulo a Composite Number)	在不能直接使用阶乘逆元或 Lucas 定理时，按模数质因数分解计算组合数。
1800–2200	矩阵快速幂与线性递推 (Matrix Exponentiation and Linear Recurrence)	—
2000–2300	高斯消元求秩与解空间 (Gaussian Elimination for Rank and Solution Space)	判断方程组是否有解、自由元数量以及线性约束的解空间维数。
2100–2400	模意义高斯消元 (Gaussian Elimination over a Finite Field)	在模质数或可逆环上求线性方程组，常用于概率、计数和线性约束。
2100–2400	高斯消元求行列式 (Determinant by Gaussian Elimination)	计算矩阵行列式，作为矩阵树定理等组合计数的核心子程序。
2200–2500	莫比乌斯反演 (Möbius Inversion)	把“所有倍数/所有约数”的累计统计反推出“恰好倍数/恰好最大公约数”的统计。
2200–2400	原根判定与原根求解 (Primitive Root Testing and Construction)	构造模意义下生成整个乘法群的元素，为离散对数和周期问题提供坐标。
2200–2400	Baby-step Giant-step 离散对数算法 (BSGS)	在模数较小时求 $a^x=b$ 的离散对数，复杂度约为平方根级别。
2300–2500	扩展 Baby-step Giant-step 离散对数算法 (Extended BSGS)	处理底数与模数不互质的离散对数，先剥离公因子再进入 BSGS。
2300–2500	快速傅里叶变换与数论变换	在合适质数模数下快速计算多项式卷积、计数合并和字符串卷积。
2200–2400	普通生成函数基础 (Ordinary Generating Functions)	把计数递推和组合对象转成多项式乘法或系数提取问题。

约分	完整名称（常见写法）	2100 分及以上：作用
2300–2500	整数分拆生成函数 (Generating Functions for Integer Partitions)	计算整数拆分、硬币组合和无序选取的方案数。
2400–2500	集合分拆与排列计数生成函数 (Generating Functions for Set Partitions and Permutations)	对集合结构、排列结构和组件合并进行高阶计数。
2000–2300	Burnside 引理 (Burnside’s Lemma)	把等价染色的数量转成群元素固定点数量的平均值。
2200–2500	Pólya 计数定理 (Pólya Enumeration Theorem)	按置换的循环结构批量计算旋转、翻转对称下的染色方案。
2100–2400	Legendre 符号 (Legendre Symbol)	快速判断模奇质数下一个数是否为二次剩余。
2200–2500	Tonelli–Shanks 二次剩余算法 (Tonelli–Shanks Algorithm)	在奇质数模下求平方根，处理 $x^2 \equiv a \pmod p$ 的有根和无根情况。
2300–2500	Cipolla 二次剩余算法 (Cipolla’s Algorithm)	另一种模奇质数平方根算法，适合需要稳定实现的二次剩余题。
2000–2300	Prüfer 序列 (Prüfer Sequence)	把标号树编码成长度为 $n-2$ 的序列，并利用度数公式计数。
2200–2400	Prüfer 序列计数应用	由点度数、叶子数量和限制条件反推标号树数量。
2200–2500	矩阵树定理 (Matrix-Tree Theorem)	用拉普拉斯矩阵的余子式计算无向图生成树数量。
2300–2500	Lindström–Gessel–Viennot 引理 (LGV 引理)	用行列式计算不相交路径组的数量，处理格点路径和网络路径计数。
1800–2200	鸽巢原理构造 (Pigeonhole Principle Constructions)	—
1800–2200	线性递推的矩阵构造 (Matrix Construction for Linear Recurrences)	—

17.4 四、字符串算法

约分	完整名称（常见写法）	2100 分及以上：作用
1800–2000	Booth 循环字符串最小表示法（Booth’s Algorithm）	—
2000–2300	回文自动机（Palindromic Automaton, PAM）	维护每个位置的最长回文后缀和所有不同回文串。
2100–2300	回文自动机出现次数统计	沿回文自动机失配指针汇总每个回文串的出现次数。
2300–2500	回文自动机与动态规划	把回文后缀链转成分割、计数或最优值转移。
2100–2300	后缀数组、最长公共前缀与区间最小值查询（Suffix Array、Longest Common Prefix、Range Minimum Query）	对后缀排序后用最长公共前缀和区间最小值查询回答任意后缀/子串关系。
2200–2400	字典序第 k 个不同子串	用后缀数组或后缀自动机统计每个前缀分支的子串数量并恢复答案。
2200–2400	多字符串后缀数组（Generalized Suffix Array）	把多个字符串放在一起比较后缀，求跨字符串最长公共子串等。
2100–2300	后缀自动机的结束位置集合树（Suffix Automaton Endpos Tree）	按后缀链接树汇总出现次数、包含关系和子串贡献。
2000–2300	后缀自动机出现次数与不同子串统计（Suffix Automaton）	线性构建所有子串状态，统计出现次数、不同子串数量和最大贡献。
2300–2500	多字符串后缀自动机（Multiple-String Suffix Automaton）	在多个文本上合并统计每个状态的覆盖串数或共同子串。
2200–2400	后缀自动机求字典序第 k 个子串	在后缀自动机状态有向无环图上做计数并按字符恢复路径。
2200–2400	Aho–Corasick 多模式匹配自动机的失配树（Aho–Corasick Automaton Fail Tree）	将文本经过节点的次数沿失配树汇总到模式串，处理后缀包含关系。

约分	完整名称（常见写法）	2100 分及以上：作用
2200–2400	Aho–Corasick 自动机与树状数组	用失配树 DFS 序支持模式串动态开关、子树加和查询。
2300–2500	动态插入/删除模式串的 Aho–Corasick 自动机	将动态模式集合拆成若干静态自动机，支持在线增删和匹配统计。
2300–2500	二进制分组动态 Aho–Corasick 自动机	用二进制分组控制多个自动机数量，使动态操作达到对数级组数。
1800–2100	哈希二分最长公共前缀（Hash-Based Longest Common Prefix）	—

17.5 五、计算几何

约分	完整名称（常见写法）	2100 分及以上：作用
1800–2000	极角排序（Polar-Angle Sorting）	—
1900–2200	射线排序与半平面方向判定（Ray Sorting and Half-Plane Orientation Test）	用叉积和半平面标记稳定地排序方向、判断左右侧和旋转顺序。
2000–2200	旋转卡壳求凸包直径（Rotating Calipers for Convex Polygon Diameter）	在凸包上在线性时间内求最远点对或最大距离。
2100–2300	旋转卡壳求最大三角形（Rotating Calipers for Maximum-Area Triangle）	在凸多边形上优化三点枚举，求最大面积三角形。
1900–2200	凸多边形内点二分（Binary Point Location in a Convex Polygon）	利用凸性把点内判定从线性扫描降到对数复杂度。
2100–2400	凸多边形半平面切割（Convex Polygon Cutting）	用一条有向直线保留多边形一侧，连续切出新的凸多边形。
2300–2500	半平面交（Half-Plane Intersection, HPI）	求多个线性不等式的公共区域及其面积、顶点和空集情况。

约分	完整名称（常见写法）	2100 分及以上：作用
2000–2300	最近点对分治（Closest Pair of Points Divide and Conquer）	在平面点集中求最小欧氏距离，复杂度为 $O(n\log n)$ 。
1900–2200	圆与直线交点（Line-Circle Intersection）	—
2000–2200	两圆交点（Circle-Circle Intersection）	—
1800–2100	点到圆/圆与圆位置关系	—
2200–2500	两圆公切线（Common Tangents of Two Circles）	求两圆的外公切线、内公切线及切点，处理相切和包含边界。
2100–2300	最小圆覆盖（Minimum Enclosing Circle）	随机增量维护包含所有点的最小圆，期望线性时间完成。
2300–2500	多边形与圆面积交（Polygon-Circle Intersection Area）	将多边形边分解为三角形/圆弓，精确累加相交面积。
2000–2300	扫描线求矩形并面积（Sweep Line for Rectangle Union Area）	通过事件排序和线段树维护当前横向覆盖长度。
2200–2400	扫描线求覆盖次数面积（Sweep Line for Area Covered at Least k Times）	在线段树中维护覆盖层数，统计覆盖至少 k 次的面积。
1800–2100	几何去重与方向向量规范化（Geometric Deduplication and Normalized Direction Vectors）	—

17.6 六、动态规划进阶

约分	完整名称（常见写法）	2100 分及以上：作用
2300–2500	复杂数位动态规划（Advanced Digit Dynamic Programming）	处理上界、前导零、数字和、禁止模式等多个数位状态。

约分	完整名称（常见写法）	2100 分及以上：作用
2200–2400	数位动态规划与自动机 (Digit Dynamic Programming with Automaton)	用自动机状态记录模式匹配、相邻限制和数位约束。
1900–2200	带自环方程的概率动态规划 (Probability Dynamic Programming with Self-Loops)	把自环期望方程移项，或建立线性方程组后求解。
2200–2500	概率动态规划与高斯消元 (Probability Dynamic Programming with Gaussian Elimination)	处理多个状态相互转移的期望和概率方程。
1800–2200	子集枚举与所有子集和动态规划 (Subset Enumeration and Sum Over Subsets Dynamic Programming, SOS DP)	—
2100–2400	轮廓线动态规划 (Profile Dynamic Programming)	用一列/几列的状态描述窄网格的铺砖、放置和相邻约束。
1800–2100	单调队列优化动态规划 (Monotone Queue Optimization)	—
2100–2400	斜率优化/凸包技巧 (Convex Hull Trick, CHT)	把二次或交叉项转成直线最值，利用斜率和查询单调性降复杂度。
2200–2400	李超线段树优化动态规划 (Li Chao Segment Tree Optimization)	在线插入斜率不单调的转移直线并查询最优值。
2200–2400	分治优化动态规划 (Divide and Conquer Optimization)	在决策点单调时缩小每层转移枚举范围。
2300–2500	四边形不等式优化 (Quadrangle Inequality Optimization)	证明区间动态规划的决策单调性，从而减少转移枚举。
2300–2500	决策单调性证明 (Decision Monotonicity)	证明最优决策点不向左移动，为分治优化或单调优化提供依据。

约分	完整名称（常见写法）	2100 分及以上：作用
2400–2500	动态动态规划与矩阵线段树	用矩阵乘积维护局部转移，支持单点修改后的全局答案。
2400–2500	生成函数优化计数动态规划	把动态规划转成多项式运算，利用卷积或生成函数提速。
2300–2500	Steiner 树状压动态规划 (Steiner Tree Subset Dynamic Programming)	在关键点数量很少时求连接所有关键点的最小代价。
2100–2400	树形背包优化 (Tree Knapsack Optimization)	合并子树选择数量，处理树上资源分配和选点限制。
2300–2500	长链剖分优化树形动态规划 (Long-Chain Decomposition for Tree DP)	利用长链合并同深度状态，优化树形 DP 的空间或时间。

17.7 七、按题面快速查找

- “区间加/区间乘/区间和”：多重懒标记线段树或仿射标记线段树。
- “离线区间统计”：莫队算法；有修改时用带修改莫队算法；只能撤销时用回滚莫队算法。
- “三维偏序/时间和坐标双重限制”：三维偏序 CDQ 分治。
- “动态维护直线最值”：李超线段树；直线只在一段横坐标有效时用线段插入版李超线段树。
- “区间取最小/最大并查询区间和”：区间最值势能线段树。
- “树上子树颜色频次”：树上启发式合并；“动态树路径”：Link-Cut Tree 动态树。
- “模数不互质的多个同余式”：扩展中国剩余定理。
- “ n 很大但递推阶数很小”：矩阵快速幂与线性递推。
- “所有倍数统计/恰好最大公约数”：莫比乌斯反演。
- “ $a^x \equiv b \pmod p$ ”：Baby-step Giant-step 离散对数算法；底数与模数不互质时用扩展版本。
- “多项式卷积”：数论变换；“对称染色”：Burnside 引理或 Pólya 计数定理。
- “ $x^2 \equiv a \pmod p$ ”：Legendre 符号先判定，Tonelli–Shanks 或 Cipolla 算法求根。
- “不同子串/出现次数”：后缀自动机；“回文后缀链”：回文自动机；“多个模式串”：Aho–Corasick 多模式匹配自动机。
- “凸包最远点”：旋转卡壳；“多个线性不等式公共区域”：半平面交；“矩形并面积”：扫描线与线段树。
- “胜负态”：必胜态/必败态动态规划；“无偏且可拆分”：Sprague–Grundy 函数；“双方完整决策”：极大极小搜索与 Alpha-Beta 剪枝。

17.8 八、常见算法组合

1. 离散化 + 树状数组：排名、逆序、区间计数。
2. DFS 序 + 线段树：树上子树修改/查询。
3. 最近公共祖先 + 树上差分：大量路径经过次数。
4. 强连通分量缩点 + 有向无环图动态规划：有向图上的可达/最大权。
5. 二分答案 + 广度优先搜索/贪心/网络流：判定型最优化。
6. Kruskal 重构树 + 倍增：按边权阈值查询连通块信息。
7. Aho-Corasick 自动机 + 动态规划：避免模式或统计匹配串。
8. 可持久化线段树 + 最近公共祖先：树上路径第 k 小。
9. 扫描线 + 线段树 + 坐标压缩：矩形并、覆盖次数。
10. 回滚并查集 + 线段树时间分治：离线动态连通性。